

머신러닝 팀 프로젝트 통합 보고서

1팀 · 이혜현 · 이진희 · 김소현 · 전유경 · 권용우
2026년 9월

목차

서론	1
1. 산탄데르 고객만족 예측	3
0. 초록	3
1. 서론	3
2. 데이터셋 소개	4
3. 탐색적 데이터 분석(EDA)	6
4. 데이터 전처리	7
5. 분석방법(Analysis method)	11
6. 모델링	12
7. 결과	17
8. 종합 평가	18
9. 결론	19
10. 회고	19
11. 참고문헌	21
12. 부록: 함수코드	21
2. 신용카드 사기 탐지	27
0. 초록	30
1. 서론	31
2. 데이터셋 소개	32
3. 탐색적 데이터 분석(EDA)	35
4. 데이터 전처리	38
5. 분석방법(Analysis method)	40
6. 모델링	43
7. 결과	50
8. 종합 평가	54
9. 결론	55
10. 회고	57
11. 참고문헌	59
12. 부록: 함수 코드	60
3. Opinosis	63
0. 초록	66
1. 서론	67
2. 데이터 소개	68
3. 탐색적 데이터 분석 (EDA)	70
4. 데이터 전처리 및 벡터화	71

5. 분석방법 (Analysis Method)	78
6. 모델링	80
7. 결과	95
8. 종합 평가	98
9. 결론	99
10. 회고	100
11. 참고문헌	102
12. 부록: 함수코드	103
4. Mercari 가격 예측	105
0. 초록	105
1. 서론	106
2. 데이터 소개	106
3. 탐색적 데이터 분석(EDA)	108
4. 데이터 전처리	113
5. Feature Engineering	116
6. 분석 방법 및 모델링	120
7. 하이퍼파라미터 최적화	124
8. 모델 성능 비교 및 분석	127
9. 팀원별 실험 상세	131
10. 결론 및 회고	133
결론	135
후기	136

서론

이 보고서를 시작하기 전에, 우리 팀이 이번 프로젝트에서 무엇을 가장 중요하게 봤는지 먼저 밝히고자 한다.

머신러닝이란 수많은 데이터를 모델에 학습시켜, 새로운 데이터가 들어왔을 때 값을 예측하거나 범주를 분류하고, 나아가 데이터 안에 숨은 구조와 패턴까지 찾아내도록 하는 것이다. 그렇다면 이 과정에서 가장 중요하게 봐야 할 것은 무엇일까?

가장 먼저 떠오르는 것은 '얼마나 잘하는가'이다. 이는 문제에 맞는 다양한 평가지표를 통해 확인할 수 있다. 하지만 우리 팀은 이 수치를 넘어, 왜 우리가 이 프로젝트를 이렇게 진행하는가에 대해서도 고민하고 싶었다. 우리가 중요하게 본 것은 두 가지였다.

첫째, '현실성'이다.

우리는 만든 모델을 실무에서 실제로 쓸 수 있는지를 중요하게 봤다. 평가지표 점수만 쫓다 보면, 정작 그 모델을 현장에 가져다 쓸 수 있는지는 놓치기 쉽다. 예를 들어 학습 데이터에는 있지만 실제 상황에서는 확보할 수 없는 정보에 모델이 기대고 있다면, 점수가 아무리 높아도 그 모델은 현장에서 쓸 수 없다.

물론 상황에 따라 다른 부분을 일부 포기하더라도 특정 평가지표를 우선해야 할 때도 있다. 다만 이번 프로젝트에서는 하나의 지표에만 매몰되지 않고, 그 수치가 실제로 어떤 의미를 갖는지를 함께 보려 했다.

둘째, '과정과 검증'이다.

정답을 빠르게 찾는 것만이 목적이었다면, 캐글 상위 솔루션을 그대로 따라가거나 수업에서 배운 방식을 그대로 적용하는 것이 가장 빨랐을 것이다. 하지만 우리는 다음 질문에 더 집중하고 싶었다.

- 왜 이 데이터가 필요한가?
- 왜 이런 전처리를 해야 하는가?
- 왜 이 모델을 사용해야 하는가?
- 우리가 예상한 결과와 실제 결과는 일치하는가?

그래서 하나의 방법을 정답으로 정해두지 않고, 여러 전처리 방법과 모델을 직접 적용하며 실험했다. 예상과 다른 결과가 나오면 원인을 추측해 확인했고, 예상대로 나오더라도 데이터나 지표에 왜곡된 부분은 없는지 다시 점검했다. 이 과정을 기록하고 팀 안에서 의견을 나누며 더 나은 답을 찾아가는 것, 그것이 이번 팀 프로젝트의 중요한 의의라고 생각한다.

이 보고서가 다루는 것

이번 프로젝트에서 우리 팀은 성격이 다른 네 개의 캐글 데이터셋을 분석했다.

(1) 산탄데르 고객만족 예측 — 익명화된 수치형 피처를 이용해 고객 불만족 여부를 분류한다. 만족·불만족 클래스의 불균형을 고려해 정확도만으로 성능을 판단하지 않고 ROC-AUC를 주요 평가 기준으로 삼았다.

(2) 신용카드 사기 탐지 — 정상 거래에 비해 사기 거래가 극히 드문 불균형 데이터에서 사기 여부를 분류한다.

(3) Opinosis — 사용자 리뷰 텍스트의 특징을 추출하고 유사한 리뷰를 군집화하는 비지도 학습 문제다.

(4) Mercari 가격 예측 — 상품의 텍스트·범주 정보를 이용해 판매 가격을 예측하는 회귀 문제다.

각 데이터셋마다 '**계획** → **시행착오** → **결과**' 순서로 정리했고, 실패한 시도도 그대로 실었다. 모든 팀원이 각자 맡은 부분의 진행 과정과 거기서 얻은 피드백을 하나씩 담으려 했다.

시행착오를 보고서에 전부 담지는 못했다는 아쉬움은 있다. 그럼에도 이 보고서와 발표자료에 우리가 무엇을 두고 고민했고, 어떤 근거로 선택하고 고쳤는지 그 흔적이 최대한 드러나기를 바란다.

산탄데르 고객만족예측 분석보고서

0. 초록

본 보고서는 스페인 산탄데르 은행이 Kaggle에 공개한 *Santander Customer Satisfaction* 데이터셋(76,020건, 369개 익명화 피처)을 이용해 고객의 서비스 불만족 여부(TARGET)를 예측하는 이진 분류 프로젝트를 정리한 것이다. 불만족 고객이 전체의 3.96%에 불과한 심각한 클래스 불균형과, 결측치가 -999999, 9999999999 같은 비정상 값으로 위장되어 있는 데이터 특성상 정교한 전처리가 요구되었다. 팀원 5명이 Decision Tree, Gradient Boosting, XGBoost, LightGBM, Random Forest, Logistic Regression 6개 모델을 각자 독립적으로 실험하고, RandomizedSearchCV, GridSearchCV, Hyperopt (TPE) 등 서로 다른 하이퍼파라미터 최적화 기법을 적용해 비교했다. 그 결과 부스팅 계열(LightGBM·GBM·XGBoost)이 ROC-AUC 약 0.825~0.828 구간에서 사실상 동률의 최고 성능을 기록했으며, 그중 LightGBM(0-importance 피처 제거 + PCA 추가)이 **0.8277**로 프로젝트 전체 최고 기록을 세웠다. 분석 결과, 모델 선택보다 이상치 처리·파생 변수·피처 가지치기로 이어지는 **전처리 품질**이 성능 상향을 결정하는 더 큰 요인이었으며, 하이퍼파라미터 최적화의 효과는 모델 구조(단일 트리 vs 앙상블 vs 선형모델)에 따라 크게 달랐다는 결론을 얻었다.

1. 서론

1.1 연구 배경 및 필요성

Santander Customer Satisfaction은 스페인 산탄데르 은행이 Kaggle에 공개한 이진 분류 데이터셋으로, 익명화된 370여 개의 금융·거래 관련 피처를 이용해 고객이 은행 서비스에 **불만족(TARGET=1)** 할지 **만족(TARGET=0)** 할지를 예측하는 문제이다. 실제 은행 입장에서는 불만족 고객을 이탈(해지) 이전에 조기 식별해 선제적으로 대응하는 것이 핵심 목표이며, 이 때문에 단순 정확도보다 두 클래스를 얼마나 잘 분리하는지를 나타내는 **ROC-AUC**가 대회 및 본 프로젝트의 공식 평가지표로 사용된다.

데이터셋은 76,020건의 학습 샘플과 371개 컬럼(ID, TARGET 포함)으로 구성되어 있으며, 불만족 고객이 전체의 약 3.96%에 불과한 심각한 클래스 불균형을 가진다. 또한 컬럼명이 `var3`, `imp_op_var39_comer_ult1`, `saldo_medio_var5_hace2` 처럼 비식별화되어 있어 도메인 지식을 직접 적용하기 어렵고, 대신 결측치가 -999999, 9999999999 같은 비정상적인 큰 값으로 인코딩되어 있는 등 실무형 데이터 전처리 역량이 요구되는 데이터셋이다.

1.2 과제 목적

본 프로젝트는 팀원 각자가 서로 다른 전처리 전략과 모델(Decision Tree, GBM, XGBoost, LightGBM, Random Forest, Logistic Regression)을 담당해 실험한 뒤, 이를 하나의 파이프라인 관점에서 비교·정리하는 것을 목표로 한다. 절차는 (1) 데이터 이해 및 EDA, (2) 데이터 전처리, (3) 모델링 및 비교, (4) 하이퍼파라미터 최적화, (5) 성능평가 및 해석, (6) 결론 및 회고의 순서로 진행되었으며, 본 보고서 역시 이 흐름을 그대로 따른다.

2. 데이터셋 소개

2.1 데이터 출처 및 구성

항목	내용
데이터 출처	Kaggle <i>Santander Customer Satisfaction</i> (<code>train.csv</code> , <code>test.csv</code>)
학습 데이터 크기	76,020행 × 371열 (<code>ID</code> , <code>TARGET</code> 포함)
제출용 테스트 데이터 크기	75,818행 × 370열 (<code>TARGET</code> 미공개)
피쳐 수(<code>ID</code> · <code>TARGET</code> 제외)	369개, 전부 익명화된 수치형 변수
타깃 변수	<code>TARGET</code> — 0: 만족, 1: 불만족 (이진 분류)
클래스 분포	0: 73,012건(96.04%) / 1: 3,008건(3.96%) → 약 24.3 : 1 불균형
결측치(<code>NaN</code>)	0건 — 대신 <code>var3=-999999</code> , <code>delta_*=9999999999</code> 등 값으로 위장된 결측치/이상치 존재
평가지표	ROC-AUC

2.2 데이터 특성 및 구조 분석

`describe()` 기준으로 확인한 대표 변수들의 통계량은 다음과 같다 (전처리 전 원본 기준).

변수	추정 의미	count	mean	std	min	25%	50%	75%	max
<code>var3</code>	국적/ 지역 코드 추정	76,020	-1,523.20	39,033.46	-999,999	2.00	2.00	2.00	238.00
<code>var15</code>	연령 추정	76,020	33.21	12.96	5.00	23.00	28.00	40.00	105.00
<code>var38</code>	자산· 잔액 규모 추정	76,020	117,235.81	182,664.60	5,163.75	67,870.61	106,409.16	118,756.25	22,034,738.76
<code>imp_op_var39_comer_ult1</code>	최근 상거래 관련 금액	76,020	72.36	339.32	0.00	0.00	0.00	0.00	12,888.03

`var3` 은 25/50/75% 값이 모두 2 인데 `min` 만 `-999999` 인 것에서 결측치가 특정 상수값으로 인코딩되어 있음을 바로 알 수 있고, `var38` 은 왜도(skewness)가 **51.27**에 달할 정도로 극단적인 우측 편향 분포를 가진다. 369개 피쳐가 전부 수치형이면서도 스케일(범위)이 컬럼마다 수천 배씩 차이 나는 것도 이 데이터셋의 구조적 특징이며, 이 두 특징이 이후 전처리 방향(이상치 치환, 로그 변환)을 결정한 핵심 근거가 되었다.

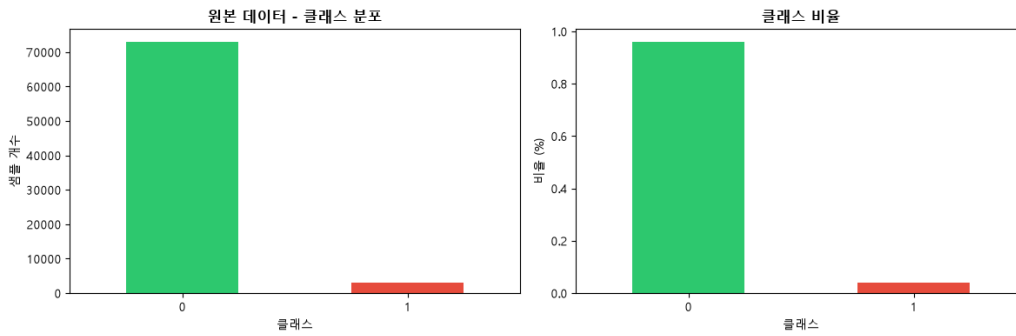


그림 1. TARGET 클래스 분포 — 원본 건수(좌)와 비율(우). 불만족 고객이 전체의 약 4%에 불과한 심각한 불균형을 보인다.

2.3 Feature 이름 특징

369개 피처는 비식별화되어 있지만, 컬럼명 접두사만으로도 대략적인 데이터 성격을 유추할 수 있다. 접두사별 개수를 집계하면 다음과 같다.

접두사	개수	추정 의미	예시
num_	143	거래/상품 보유 건수(count)	num_var4 , num_var5_0 , num_meses_var5_ult3
ind_	75	특정 상품·조건 보유 여부 지표(0/1 플래그)	ind_var1_0 , ind_var26 , ind_var39
saldo_	71	계좌·상품별 잔액(balance)	saldo_var30 , saldo_medio_var5_hace2
imp_	49	거래 금액(import/amount)	imp_op_var39_comer_ult1 , imp_ent_var16_ult1
delta_	26	두 시점(예: 1년 vs 3년) 사이 값의 변화량	delta_imp_aporar_var13_1y3
접두사 없는 개별 변수	5	국적(var3)·연령(var15)·자산규모(var38) 등 별도 체계로 명명된 변수	var3 , var15 , var21 , var36 , var38

_ult1 / _ult3 (최근 1개월/3개월), _hace2 (2기 전) 같은 접미사도 함께 나타나는데, 이는 은행 데이터에서 흔히 쓰이는 "관측 시점 (최근 N개월/N기 전)" 표기로 추정된다. 즉 완전히 익명화된 데이터라도 접두사·접미사 명명 규칙을 분석하면 "이 피처가 건수인지 금액인지 잔액인지 변화량인지"는 어느 정도 구분할 수 있으며, 이는 3.2절의 상관관계 해석과 3.3절 이상치 분석(어떤 접두사 계열에서 결측 위장 값이 나타나는지)의 근거로 활용되었다.

2.4 동일 값·중복 Feature

전처리 이전에 확인해야 할, "정보량이 없어 제거 대상이 되는 컬럼" 두 가지를 한 절로 정리한다. 실제 제거는 둘 다 4.4절(피처 정제)에서 함께 수행한다.

동일 값(상수) 컬럼 — 전체 369개 피처 중 **34개 컬럼**은 모든 76,020개 샘플에서 값이 단 하나뿐인 상수(constant) 컬럼이었다(예: 특정 상품을 가진 고객이 데이터셋 안에 아예 없어 해당 지표가 항상 0인 경우). 상수 컬럼은 분산이 0이므로 어떤 모델을 쓰든 예측에 전혀 기여할 수 없다.

중복 컬럼 — 상수 컬럼을 먼저 제거한 뒤 남은 335개 컬럼을 대상으로 전치(.T) 후 duplicated() 로 확인한 결과, **29개 컬럼**이 다른 컬럼과 76,020개 값이 완전히 동일한 중복 컬럼이었다(예: ind_var26_0 과 ind_var26 , num_var26_0 과 num_var26 처럼 _0 접미사가 붙은 버전과 안 붙은 버전이 사실상 같은 값을 가지는 경우가 다수). 중복 컬럼을 남겨두면 다중공선성만 키우고 정보량은 늘지 않는다.

2.5 ID 변수

ID 는 1~151,838 범위의 정수로, 76,020개 행 모두 유일값(중복 없음)이며 값이 단조 증가하는 것으로 보아 고객 계좌 개설 순서 또는 Kaggle 제출용 일련번호로 추정된다. 실제 서비스 만족도와는 인과적으로 무관한 단순 식별자이므로, 모든 팀원이 학습 피처에서 제외하고 예측 이후 제출(submission) 파일을 만들 때 인덱스 역할로만 사용했다.

2.6 타깃 변수 분포 분석

클래스 불균형이 매우 심하기 때문에 단순 정확도는 다수 클래스만 예측해도 96%에 도달하는 착시를 일으킨다. 따라서 팀 전체가 초기 EDA 단계부터 정확도 대신 ROC-AUC, 그리고 필요 시 Precision/Recall/F1을 함께 확인하는 방향으로 지표를 통일했다(그림 1).

3. 탐색적 데이터 분석(EDA)

3.1 기술통계 분석

2.2절의 `describe()` 결과에서 이미 확인했듯, `var3` 은 결측이 상수로 위장돼 있고 `var38` 은 왜도 51.27의 극단적 우측 편향을 보인다. 이 두 변수 외에도 `imp_ / saldo_` 계열 다수가 0이 대부분이고(75%값이 0) 소수의 큰 거래 금액만 꼬리처럼 늘어진 분포를 공통적으로 가져, 전체 369개 피처 상당수가 "0이 압도적으로 많고 우측으로 치우친" 준희소(quasi-sparse) 분포라는 공통 특성을 확인했다.

3.2 상관관계 기반 주요 변수 분석

TARGET과의 단순 상관계수를 몇몇 대표 변수에 대해 확인하면 다음과 같다.

변수	TARGET과의 상관계수
<code>num_meses_var5_ult3</code>	-0.1483
<code>var36</code>	+0.1029
<code>var15</code>	+0.1013
<code>num_var4</code>	-0.0802
<code>saldo_var30</code>	-0.0371
<code>saldo_medio_var5_hace2</code>	-0.0201
<code>var38</code>	-0.0195
<code>imp_op_var39_comer_ult1</code>	+0.0104

가장 큰 절댓값 상관을 보인 `num_meses_var5_ult3` (최근 3개월 내 `var5` 관련 활동 개월 수 추정)은 값이 클수록(최근 활동이 활발할수록) 오히려 불만족 확률이 낮아지는 방향(-)으로, "활발한 거래 이력 = 안정적 고객"이라는 해석과 부합한다. `var15` (연령)와 `var36` 은 양의 상관을 보여, 특정 연령대·상품 유형(`var36`)에서 불만족 비율이 상대적으로 높다는 것을 시사한다. 다만 개별 변수의 절대 상관계수는 모두 0.15 미만으로 크지 않아, 단일 변수만으로는 설명력이 부족하고 여러 변수의 조합(3.4절 이상치 처리 이후 파생 변수, PCA 성분 등)이 필요하다는 판단의 근거가 되었다. 실제로 5.4절 모델링에서도 이 변수들(`var15`, `var38`, `saldo_var30` 계열)이 XGBoost-GBM 모두에서 상위 중요 피처로 재확인된다(그림 7).

3.3 이상치 분석

`describe()` 의 `min / max` 극단값을 통해 이 데이터셋의 이상치가 무작위 노이즈가 아니라 **특정 값으로 위장된 결측치**라는 패턴을 확인했다.

- `var3 = -999999` : 정상 범위(대부분 2 근방)를 크게 벗어난 값으로, 결측을 나타내는 관례적 sentinel 값으로 추정된다(116건).
- `delta * 26개 컬럼의 9999999999` : 정상 값이 대부분 -1~1.5 범위인데, 이 값만 10자리의 명백히 비정상적인 대형값으로 나타나 역시 결측/해당없음의 sentinel로 판단했다.

이런 "값으로 위장된 결측치"는 `isnull().sum()` 으로는 전혀 잡히지 않기 때문에(2.1절에서 확인했듯 결측치 0건으로 표시됨), `describe()` 의 min/max를 직접 들여다보는 과정이 없었다면 놓쳤을 이상치다.

3.4 변수별 이상치 시각화

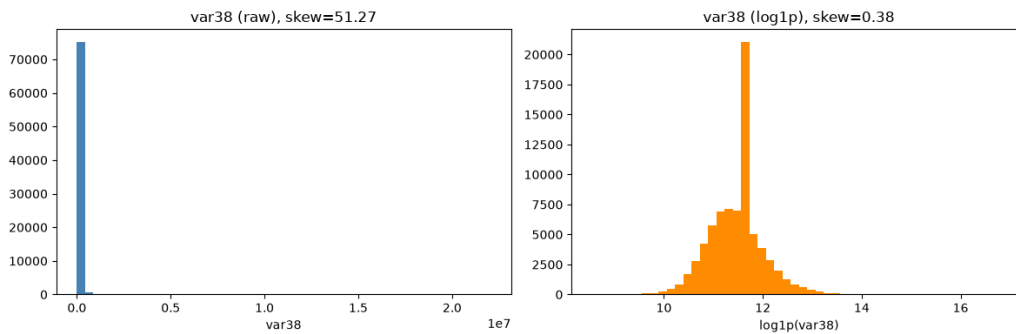


그림 2. var38 로그 변환 전(좌, skew=51.27) 후(우) 분포 비교.

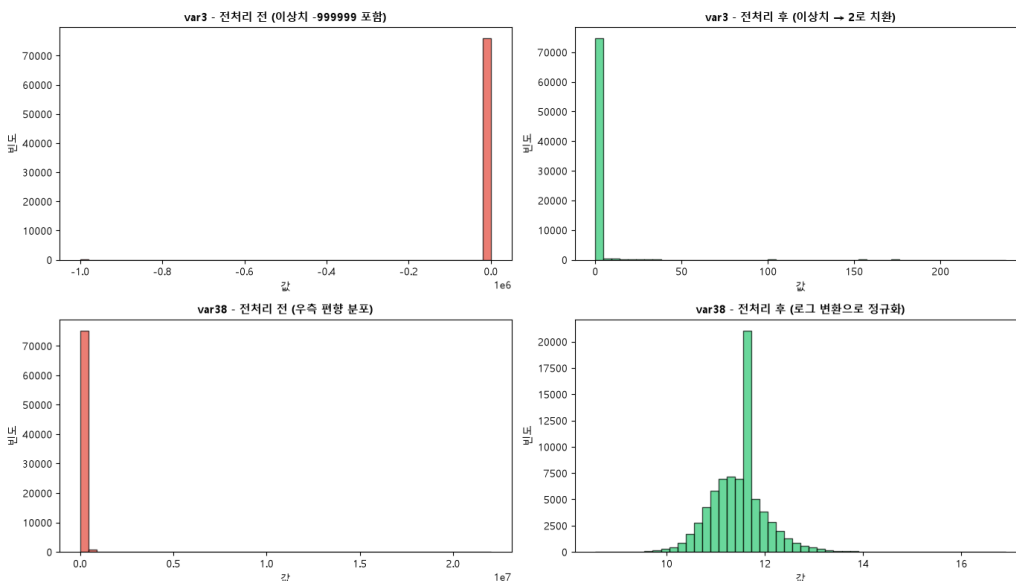


그림 3. var3 이상치 치환(상단), var38 로그 변환(하단) 전/후 분포 비교.

3.5 중복 값 제거

컬럼(변수) 차원의 중복은 2.4절에서 다뤘고, 여기서는 **행(row) 차원의 중복**을 확인한다. ID를 제외한 나머지 370개 컬럼(피처 + TARGET) 전부가 동일한 행이 있는지 `duplicated()`로 재검사한 결과, 총 5,791건이 하나 이상의 중복 그룹에 속했고 그중 4,807건 (전체의 **6.32%**)이 "첫 등장 이후의 여분(extra)" 중복 행으로 확인되었다. 중복 행 그룹 내 TARGET 분포(만족 5,536 : 불만족 255 ~ 95.6:4.4)는 전체 분포(96.04:3.96)와 크게 다르지 않아, 이 중복이 특정 클래스에 쏠려 성능을 왜곡시키는 정도는 아닌 것으로 판단된다. 다만 본 프로젝트의 실제 팀원 노트북들을 재확인한 결과 이 행 단위 중복은 어느 파이프라인에서도 명시적으로 제거되지 않았다 — 이는 8장 회고의 개선점으로 함께 기록한다.

3.6 특정 컬럼 로그 변환

`var38`은 왜도 51.27의 극단적 우측 편향 분포를 가져 `log1p` 변환으로 정규화했다(그림 2). 구체적인 변환 코드와, 이를 `var38` 한 컬럼을 넘어 왜도가 큰 다른 컬럼들로 확장 적용한 사례는 4.2절에서 다룬다.

4. 데이터 전처리

팀원 대부분이 아래와 같은 공통 전처리 파이프라인을 기반으로 각자의 모델에 맞게 일부 단계를 가감하는 방식으로 작업했다. 트리 계열(RandomForest 등)은 단조 변환에 불변하다는 이유로 로그 변환·PCA를 생략하는 등, **모델 특성에 맞춰 전처리를 다르게 적용**한 점이 본 프로젝트 전처리 전략의 특징이다.

4.1 이상치 처리

- **var3 결측 이상치:** -999999 로 인코딩된 116건(학습 기준)을 최빈값인 2 로 치환.
- **delta_* 계열 이상치:** 26개 delta_* 컬럼에 남아있던 999999999 (비정상 대형값)를 결측으로 간주해 NaN 처리 후 각 컬럼의 중앙값으로 대체(용우). 일부 모델(Decision Tree)에서는 값을 대체하는 대신 "몇 개의 delta_* 컬럼이 이상치였는가"를 별도 플래그 피쳐(delta_missing_cnt)로 남겨 정보 손실을 방지했다.

4.2 분포 변환

var38 은 왜도 51.27의 극단적 우측 편향 분포를 가져 log1p 변환으로 정규화했다. 아래는 로그 변환 전후 분포 비교이다(그림 2, 3 재참조).

Random Forest와 로지스틱 회귀를 모두 담당한 유경은 두 모델의 특성에 맞춰 로그 변환 적용 여부 자체를 다르게 가져갔다. 로지스틱 회귀에서는 var38 한 컬럼에만 로그 변환을 적용하는 것에서 나아가, **왜도 절댓값이 1을 초과하는 326개 컬럼 전체**에 부호를 보존하는 signed log1p ($\text{sign}(x) \cdot \log1p(|x|)$) 변환을 적용해 선형 결정 경계가 왜곡된 분포에 과도하게 휘돌리지 않도록 했다. 반면 Random Forest에서는 "트리 분할은 임계값 비교만 사용하므로 단조 변환이 성능에 영향을 주지 않는다"는 이유로 동일한 로그 변환을 의도적으로 생략했다 — 같은 사람이 담당 모델의 특성에 따라 전처리를 다르게 설계한 대표적인 사례다.

4.3 파생 변수 생성

행(row) 단위로 고객의 거래 패턴을 요약하는 파생 변수를 추가했다.

파생 변수	정의	의도
n0	고객별 값이 0인 피쳐 개수	계좌 활동성(활발/휴면) 대리 지표
row_std / row_mean / row_median	고객별 거래-잔액의 행 방향 통계량	거래 규모 및 변동성
row_max / row_min / row_range	고객별 최대/최소/범위	이상 거래 존재 여부
n_negative	값이 음수인 피쳐 개수	환불-손실성 거래 빈도
delta_missing_cnt	이상치(999999999)였던 delta_* 개수(Decision Tree 전용)	결측 패턴 자체를 신호로 활용
var38_is_missing	var38이 최빈값(=평균 대체 추정값)인지 플래그(Random Forest 전용)	잠재적 결측 대체 흔적 탐지

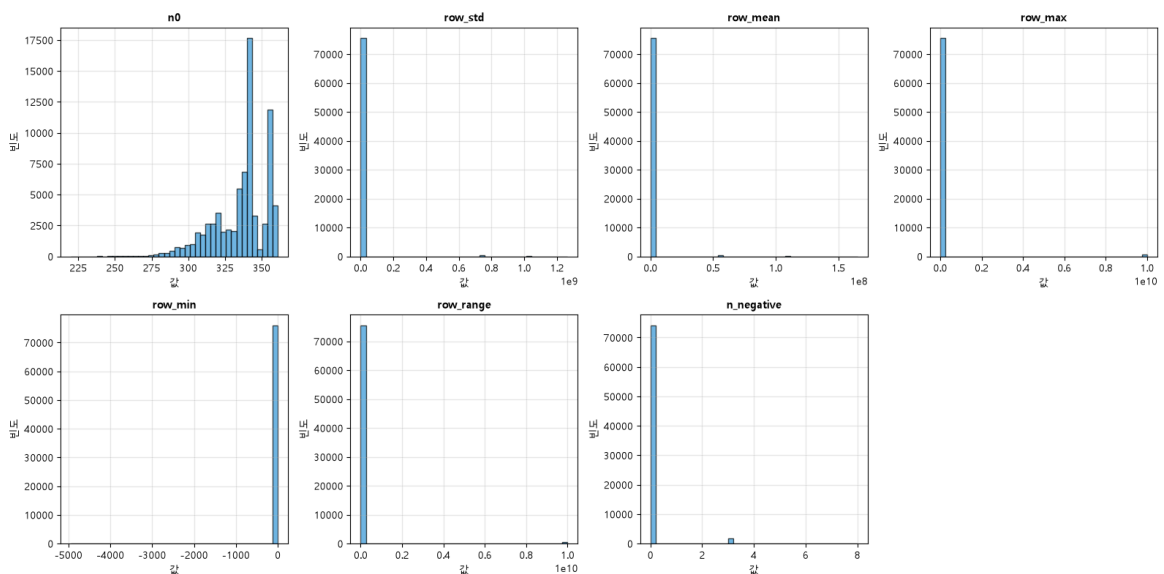


그림 4. 주요 파생 변수들의 분포.



그림 5. 파생 변수 간 상관관계 히트맵. row_mean-row_max-row_range처럼 "거래 규모"를 나타내는 변수들끼리는 상관이 높은 반면, n0(활동도)-n_negative(손실 빈도)는 비교적 독립적인 정보를 포착한다.

4.4 피처 정제 및 중요도 기반 가지치기

1. **상수 컬럼 제거:** 전체 샘플에서 값이 하나뿐인 컬럼 34개 제거 (2.4절, 모든 고객에게 동일한 값 → 정보량 없음).
2. **중복 컬럼 제거:** 다른 컬럼과 완전히 동일한 값을 가지는 컬럼 29개 제거 (2.4절, 다중공선성 방지).
3. **도메인 기반 수동 제거:** var6 계열 및 delta_imp_reemb_var13_1y3 등 노이즈로 판명된 delta 컬럼 4개, 총 9개 수동 제거.
4. **모델별 중요도 0 피처 사후 가지치기:** 각 모델을 한 번 학습시켜 feature importance가 0인(=한 번도 분기에 쓰이지 않은) 피처를 제거. 트리 계열 모델마다 "중요하지 않다"고 판단하는 피처 집합이 달라, **가지치기는 모델별로 독립적으로 수행**했다(예: XGBoost 기준 375→111개, GBM 기준 305→95개, RandomForest 기준 294→219개). 로지스틱 회귀는 "트리 importance=0이 선형 관계에서도 무의미하다는 보장이 없다"는 이유로 이 가지치기를 적용하지 않았다.

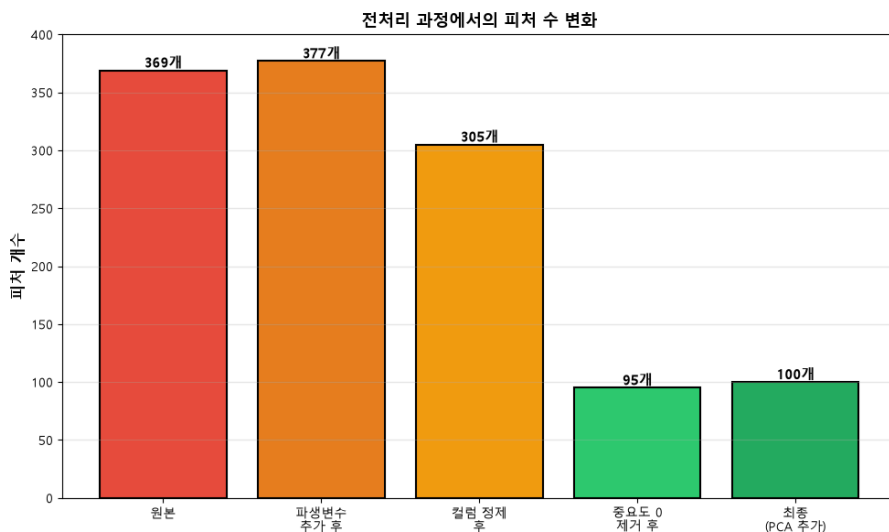


그림 6. GBM 파이프라인(혜현) 기준 전처리 단계별 피처 수 변화 — 원본 369개 → 파생변수 추가 377개 → 컬럼 정제 305개 → 중요도 0 제거 95개 → PCA 추가 100개.

가지치기의 효과는 성능 손실 없이 학습 속도를 크게 개선했다는 점에서 유효했다. 예를 들어 용우의 XGBoost 실험에서는 375개 → 111개로 피처를 70% 이상 줄였음에도 테스트 ROC-AUC는 0.8242로 **완전히 동일**했고, Decision Tree/Random Forest/GradientBoosting에서도 동일 경향(가지치기 후 오히려 소폭 상승)이 확인되었다.

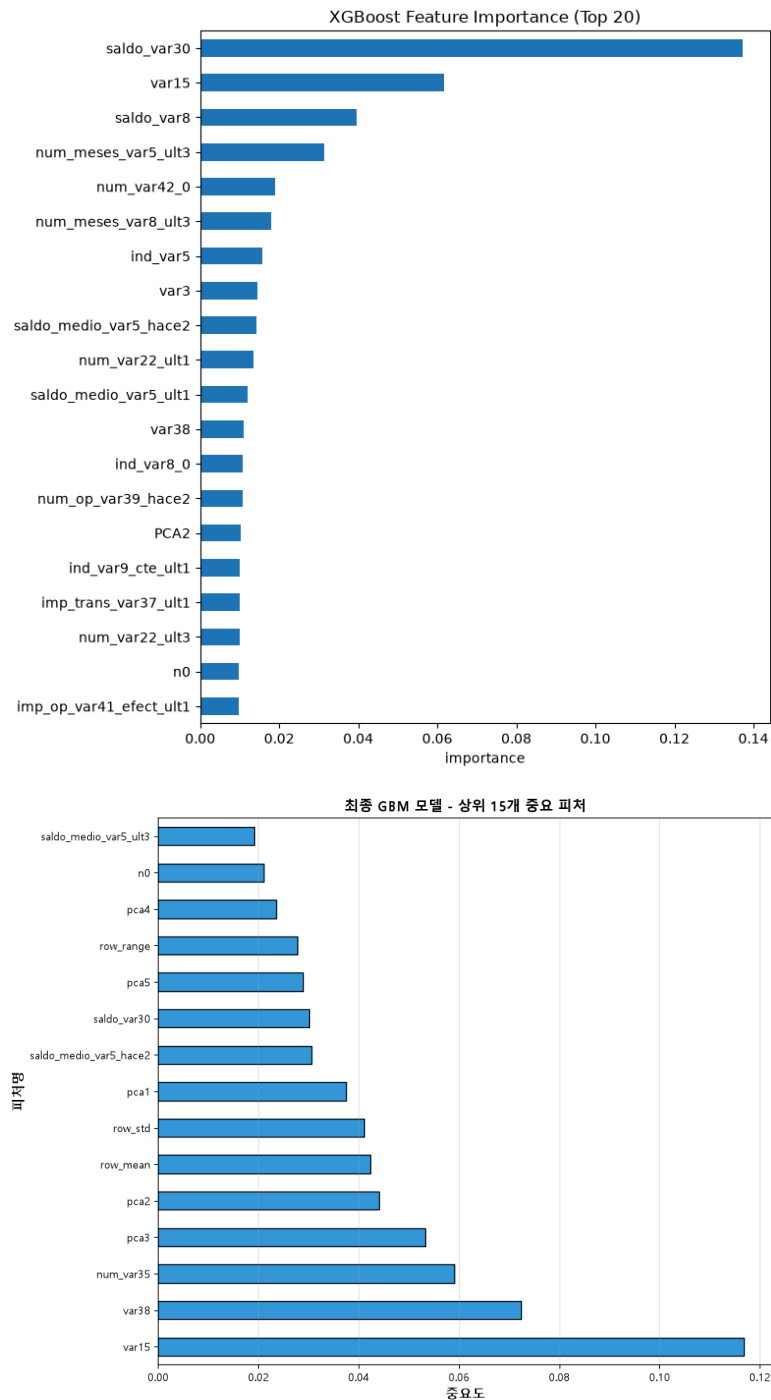


그림 7. XGBoost(좌) / GBM(우) 기준 상위 중요 피처. 두 모델 모두 saldo_var30, var15, num_var*, saldo_medio_var5 계열, PCA 성분들이 상위권에 공통적으로 등장한다.

4.5 스케일링 및 차원 축소

- 트리 계열 모델(Decision Tree, Random Forest, GBM, XGBoost, LightGBM)은 스케일에 영향을 받지 않으므로 원칙적으로 스케일링을 생략했다.
- 로지스틱 회귀는 스케일에 민감하므로 `StandardScaler` 를 반드시 적용했다(학습 세트로만 `fit`, 검증/테스트는 `transform` 만 적용해 데이터 누수를 차단).

- 대부분의 파이프라인에서 `StandardScaler` 이후 **PCA로 5개의 주성분(pca1~pca5)**을 추출해 원본 피처에 추가 컬럼으로 결합했다. 완전한 차원 축소가 아니라 "원본 피처 + 압축된 전역 패턴"을 함께 제공하는 방식이며, 실제로 XGBoost/GBM 모두 `pca` 계열 컬럼이 상위 중요 피처에 포함되어 유의미한 신호로 활용되었음이 확인되었다(그림 7).

4.6 데이터 분할

데이터 누수를 방지하기 위해 아래 원칙을 전 팀원이 공통으로 준수했다.

- 1차 분할:** `train_test_split(test_size=0.2, stratify=TARGET, random_state=0)` 으로 학습 세트(80%, 60,816건)와 **최종 테스트 세트**(20%, 15,204건)를 분리. 최종 테스트 세트는 모델-하이퍼파라미터 선택 과정에서는 절대 사용하지 않고, 마지막 성능 보고에만 1회 사용.
- 2차 분할:** 학습 세트를 다시 훈련용(70%)과 검증/조기종료용(30%)으로 분할하거나, 5-Fold(Stratified)K-Fold 교차검증으로 하이퍼파라미터를 탐색.
- 스케일러·PCA·중요도 기반 피처 선택 등 모든 변환은 **훈련 세트 기준으로만 fit** 하고 검증·테스트 세트에는 `transform` 만 적용.

5. 분석방법(Analysis method)

5.1 전처리 필요성 및 과정 개요

3장 EDA에서 확인한 세 가지 데이터 특성 — (1) 결측치 sentinel 값으로 위장돼 있다는 점, (2) 일부 변수의 극단적 왜도, (3) 상수/중복 컬럼과 행 단위 중복이 섞여 있다는 점 — 은 각각 4장의 이상치 치환, 로그 변환, 피처 정제 단계로 직결되었다. 즉 본 프로젝트의 전처리는 임의의 관용적 절차가 아니라, EDA에서 실측한 문제를 하나씩 대응해 설계한 결과다.

5.2 사용 모델 설명

report.md에서 정한 순서(Decision Tree → GBM → XGBoost → LightGBM → RandomForest → Logistic Regression)에 따라, 각 모델을 선택한 이유를 간단히 정리한다.

모델	유형	선택 이유
Decision Tree	단일 트리	규제·튜닝의 효과를 가장 극적으로 관찰할 수 있는 베이스라인
Gradient Boosting	순차적 부스팅 앙상블	scikit-learn 기본 구현으로 부스팅 계열의 표준 성능 기준점 제공
XGBoost	정규화 부스팅 앙상블	결측치 자체 처리, 정규화 항(reg_alpha/lambda) 지원 등 실무에서 널리 쓰이는 강건한 부스팅 구현
LightGBM	Leaf-wise 부스팅 앙상블	히스토그램 기반 분할로 대용량 데이터에서 학습 속도가 가장 빠름
Random Forest	배깅 앙상블	부스팅과 다른 앙상블 원리(병렬 배깅)로 과적합 양상을 비교하기 위한 대조군
Logistic Regression	선형 모델	트리 계열과 근본적으로 다른 가정(선형 결정 경계)을 가진 모델로, 피처 표현 방식(WOE 등)의 영향을 살펴보기 위한 대조군

5.3 Hyperopt 기반 하이퍼파라미터 최적화

`GridSearchCV` / `RandomizedSearchCV` 는 미리 정해진 이산값 후보 안에서만 탐색하기 때문에, 베스트 파라미터가 후보 범위의 "끝값"으로 나오면 진짜 최적점이 그 범위 밖에 있을 가능성을 배제할 수 없다(실제로 6.5절 Random Forest, 6.4절 LightGBM 튜닝 모두 이 문제가 관측되었다). `Hyperopt` 는 `hp.uniform` / `hp.quniform` 으로 연속 구간을 지정하면 **TPE(Tree-structured Parzen Estimator)** 라는 베이지안 최적화 알고리즘이 이전 시행 결과를 바탕으로 유망한 영역을 순차적으로 좁혀가며 탐색해 이 경계값 문제를 완화한다. 다만 `fmin` 은 시행을 순차적으로 진행하므로(`GridSearchCV` 처럼 여러 코어에 조합을 나눠 병렬 처리할 수 없음), 개별 모델 학습 자체에 `n_jobs=-1` 을 지정해 속도를 보완하는 방식을 여러 팀원이 공통으로 사용했다.

팀 내에서는 실제로 같은 LightGBM 모델에 대해 두 탐색 방법을 직접 비교한 실험이 남아있다(용우, `TT(YWK).ipynb`).

탐색 방법	탐색 방식	CV AUC	홀드아웃 테스트 AUC
GridSearchCV(2단계, 729개 조합)	1단계 RandomizedSearchCV로 범위 축소 → 2단계 격자 탐색	0.8394	0.8253
Hyperopt(TPE, 100회)	연속 구간에서 베이지안 탐색	0.8382	0.8239

두 방법의 결과 차이는 CV 기준 0.0012, 테스트 기준 0.0014로 크지 않았으며, 오히려 이번 실험에서는 GridSearchCV가 근소하게 더 높았다. 이는 "Hyperopt(베이지안 최적화)가 격자 탐색보다 항상 더 좋은 결과를 낸다"는 보장은 없으며, **탐색 알고리즘 자체의 선택보다 탐색 범위(space)를 얼마나 적절히 설정하는가**가 더 중요할 수 있음을 시사한다. GBM(혜현)과 XGBoost(진희)의 Hyperopt 적용 사례는 6.2절, 6.3절에서 각각 다룬다.

5.4 실행환경

팀원 각자의 노트북은 개별 로컬 환경(Anaconda `m1_dev` 가상환경 등)에서 실행되었으며, 본 보고서 작성 과정에서 재확인한 핵심 라이브러리 버전은 다음과 같다.

라이브러리	버전
Python (pandas)	pandas 2.3.3
numpy	2.3.5
scikit-learn	1.7.2
hyperopt	0.3.0
xgboost	팀 공용 환경에 설치되어 사용(버전 미고정, <code>XGBClassifier</code> API 기준 작성)
lightgbm	팀 공용 환경에 설치되어 사용(버전 미고정, <code>LGBMClassifier</code> API 기준 작성)

팀원마다 개별 로컬 환경에서 작업해 `requirements.txt` 나 `environment.yml` 같은 환경 고정 파일은 별도로 관리되지 않았다는 점은 10장(회고)에서 개선점으로 함께 언급한다.

5.5 코드 재사용 및 함수화

팀 노트북 10개를 전수 조사한 결과, 명시적으로 `def` 로 정의된 재사용 함수는 예상보다 적었다 — 혜현(GBM)·진희(XGBoost)·용우(LightGBM)가 각자 독립적으로 작성한 Hyperopt `objective(params)` 함수, 그리고 유경이 로지스틱 회귀를 위해 작성한 `fit_woe / transform_woe` 두 함수 정도가 전부였다. 나머지 이상치 치환·파생 변수 생성·모델 학습/평가 코드는 대부분 노트북 셀에 인라인으로 반복 작성되었다. 다만 예외적으로 `train_test_split` 을 두 번 적용해 학습(80%)·검증(학습 세트의 30%)·테스트(20%)로 나누고, 두 번 모두 `stratify=` 를 지정해 클래스 비율(96:4)을 유지하는 2단계 분할 방식만큼은 어느 노트북에서도 함수로 따로 빼두지 않았음에도 거의 모든 노트북에서 그대로 반복되었다 — 데이터 누수 방지와 클래스 불균형 보정이라는 원칙만큼은 명문화된 함수 없이도 팀 전체에 잘 공유되어 있었다는 뜻이다. 즉 "공통 함수"라기보다는 "유사한 패턴을 각자 독립적으로 재구현"한 경우가 대부분이었다는 것이 정직한 실태이며, 이는 회고(10장)의 개선점("공통 전처리 함수를 팀 초기에 모듈화")과 직접 연결된다. 실제로 반복된 패턴들을 정리해 재사용 가능한 형태로 다듬은 코드는 12장 부록에 정리했다.

6. 모델링

report.md에서 정한 순서(Decision Tree → GBM → XGBoost → LightGBM → RandomForest → Logistic Regression)에 따라 정리한다. 담당 및 사용 기법 개요는 다음과 같다.

모델	주 담당	하이퍼파라미터 탐색 기법
Decision Tree	용우	RandomizedSearchCV (5-fold, 100회)
Gradient Boosting	혜현, 용우	Hyperopt(TPE, 30회) + 5-Fold CV 검증
XGBoost	진희, 용우, 소현	Hyperopt(TPE, 200회, 5-Fold CV 목적함수)
LightGBM	용우, 소현	RandomizedSearchCV → GridSearchCV 2단계 정밀 탐색 (용우는 Hyperopt도 별도 비교, 5.3절)
Random Forest	용우, 유경	RandomizedSearchCV → GridSearchCV(용우) / RandomizedSearchCV(유경, PredefinedSplit)
Logistic Regression	유경, 소현	그리드 형태 수동 탐색(C, penalty, class_weight) + WOE 인코딩 실험

6.1 Decision Tree (담당: 용우)

기본 파라미터의 Decision Tree는 규제가 전혀 없어 학습 데이터를 통째로 암기하는 완전 과적합 상태(ROC-AUC 0.5676, 거의 무작위 수준)에 빠졌다. 이는 Decision Tree에서는 "얼마나 못 자라게 규제하느냐"가 전처리보다 훨씬 중요하다는 것을 보여준다.

이에 따라 Decision Tree 전용 전처리(로그 변환-PCA 제거, `delta_missing_cnt` 플래그 추가)와 `RandomizedSearchCV` (5-fold, 100회 반복, `roc_auc` 기준) 튜닝을 별도로 수행했다.

항목	튜닝 전(기본 파라미터)	튜닝 후
테스트 ROC-AUC	0.5676	0.8034
베스트 파라미터	—	<code>max_depth=5</code> , <code>min_samples_leaf=150</code> , <code>min_samples_split=20</code> , <code>criterion='entropy'</code> , <code>class_weight='balanced'</code>
CV(5-fold) AUC	—	0.8174

과적합을 억제하는 규제 파라미터(얕은 깊이, 큰 leaf 최소 샘플 수)를 적용하는 것만으로 ROC-AUC가 **+0.2358** 개선되어, 6개 모델 중 하이퍼파라미터 최적화의 효과가 가장 극적으로 나타난 사례다.

6.2 Gradient Boosting — GBM (담당: 혜현, 보조: 용우)

혜현은 GBM 성능 개선을 별도 실험(V3)으로 진행했다. `Hyperopt` (TPE 알고리즘)로 `n_estimators`, `learning_rate`, `max_depth`, `subsample`, `max_features`, `min_samples_split` 6개 파라미터를 30회 반복 탐색한 뒤, 확정된 파라미터를 5-Fold CV로 재검증하고 최종적으로 전체 학습 데이터로 단일 모델을 재학습해 테스트 세트에서 평가했다.

항목	값
탐색 방법	<code>Hyperopt(TPE)</code> , <code>max_evals=30</code> , 목적함수: 3-Fold CV 평균 AUC
베스트 파라미터	<code>n_estimators=300</code> , <code>learning_rate=0.0201</code> , <code>max_depth=6</code> , <code>subsample=0.973</code> , <code>max_features='sqrt'</code> , <code>min_samples_split=8</code>
5-Fold CV 평균 AUC	0.8420 (±0.0050)
최종 테스트 ROC-AUC	0.8256

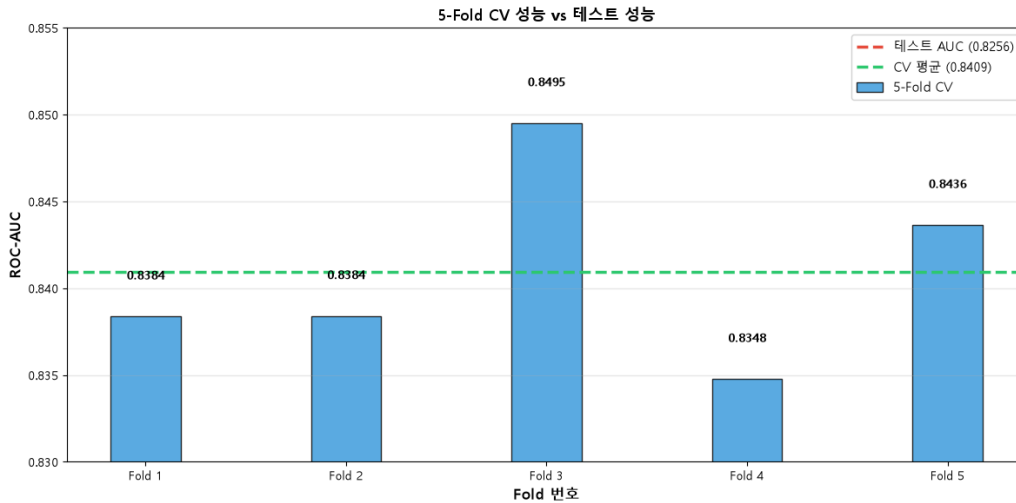


그림 8. GBM 5-Fold CV 성능과 최종 테스트 성능 비교. CV 평균(0.8409)과 테스트(0.8256) 간 격차는 약 1.5%p로 안정적인 일반화 성능을 보여준다.

참고로 용우의 통합 파이프라인에서는 튜닝 없이 `GradientBoostingClassifier(n_estimators=200, learning_rate=0.05)` 를 111개 가지치기 피쳐로 학습해 테스트 ROC-AUC **0.8198**을 기록했으며, 이는 혜현의 튜닝 결과와 비교할 수 있는 베이스라인으로 활용했다.

6.3 XGBoost (담당: 진희, 보조: 용우, 소현)

진희는 XGBoost에 `Hyperopt (TPE, max_evals=200)`로 `max_depth`, `min_child_weight`, `colsample_bytree`, `subsample`, `gamma`, `reg_alpha`, `reg_lambda`, `learning_rate` 8개 파라미터를 탐색했으며, 목적함수는 5-Fold `kFold` 교차검증 평균 ROC-AUC로 정의했다.

항목	튜닝 전(기본 파라미터)	Hyperopt 튜닝 후
테스트 ROC-AUC	0.8240	0.8255
탐색 방법	—	Hyperopt(TPE), 200회, 5-Fold CV 목적함수
5-Fold CV 로스 기준 AUC	—	0.8443
베스트 파라미터	<code>n_estimators=500</code> , <code>learning_rate=0.05</code> (고정)	<code>learning_rate=0.139</code> , <code>max_depth=5</code> , <code>min_child_weight=6</code> , <code>colsample_bytree=0.733</code> , <code>subsample=0.968</code> , <code>gamma=0.927</code> , <code>reg_alpha=0.786</code> , <code>reg_lambda=3.821</code> , (이후 <code>n_estimators=1000</code> · <code>max_depth=4</code> 로 미세 조정)

노트북 파일명(`jinhee_model(0.8255).ipynb`)이 나타내듯, 이 XGBoost 모델은 **부스팅 계열 중에서도 상위권에 드는 테스트 스코어 (0.8255)**를 기록했다(팀 전체 최고 기록은 소현의 LightGBM 0.8277, §7.1 참고). 다만 하이퍼파라미터 탐색으로 얻은 개선폭(+0.0015)은 크지 않았는데, 이는 XGBoost 기본 설정 자체가 이미 이 데이터셋에서 준수한 성능을 내는 강건한 모델을 시사한다.

용우의 통합 파이프라인에서는 동일한 기본 파라미터(`n_estimators=500`, `learning_rate=0.05`, `early_stopping_rounds=100`)로 375개 전체 피쳐와 111개 가지치기 피쳐 각각에서 테스트 ROC-AUC 0.8242를 기록해 진희의 결과와 유사한 수준을 보였다. 소현은 모델 선택 단계에서 동일 설정으로 검증 세트 기준 ROC-AUC 0.8443을 얻어(주의: 이 값은 X_{val} 기준으로, X_{test} 기준의 다른 수치들과 직접 비교하기보다는 "이 데이터셋에서 XGBoost가 초기부터 강력한 베이스라인"이라는 정성적 결론을 뒷받침하는 근거로 활용했다).

6.4 LightGBM (담당: 용우, 소현 / 시도: 진희)

용우는 111개 가지치기 피쳐를 대상으로 기본 파라미터 대비, 1단계 `RandomizedSearchCV` (넓은 범위, 60회)로 유망 구간을 먼저 좁힌 뒤 2단계 `GridSearchCV` (729개 조합)로 정밀 탐색하는 **2단계 coarse-to-fine 튜닝**을 적용했다. 동일한 111개 피쳐에 `Hyperopt (TPE, 100회)`도 별도로 적용해 두 탐색 방법을 직접 비교했다(5.3절).

항목	튜닝 전(기본 파라미터)	2단계 GridSearchCV 튜닝 후	Hyperopt(TPE, 100회)
테스트 ROC-AUC	0.8011	0.8253	0.8239
베스트 파라미터	n_estimators=500 , learning_rate=0.05 (고정)	n_estimators=400 , learning_rate=0.015 , num_leaves=15 , min_child_samples=130 , reg_alpha=2 , subsample=0.7	n_estimators=350 , num_leaves=12 , min_child_samples=290 , learning_rate=0.024 , reg_alpha=2.08 , reg_lambda=1.57 , subsample=0.455 , colsample_bytree=0.390
CV AUC	—	0.8394(3-fold)	0.8382(3-fold)

+0.0242의 개선폭은 6개 모델 중 Decision Tree 다음으로 크며, 용우의 파이프라인 안에서는 GridSearchCV 튜닝이 **최종적으로 가장 높은 테스트 점수(0.8253)**를 기록한 모델이다. Hyperopt는 GridSearchCV보다 근소하게 낮은 결과(0.8239)를 보였는데, 이 비교 자체가 5.3절에서 다룬 "탐색 알고리즘보다 탐색 범위 설정이 중요할 수 있다"는 결론의 실측 근거가 되었다.

소현 역시 LightGBM에 RandomizedSearchCV (5-Fold StratifiedKFold, 40회)로 num_leaves, max_depth, learning_rate, n_estimators, min_child_samples, subsample, colsample_bytree, reg_alpha, reg_lambda 9개 파라미터를 탐색해 **Best CV ROC-AUC 0.8446**을 확인했고, 파라미터별 상관관계 분석 결과 learning_rate (상관계수 -0.79)와 n_estimators (-0.42)가 성능에 가장 큰 영향을 미치는 것으로 나타났다. 이후 5-Fold 평균 best_iteration (284)으로 전체 학습 데이터를 다시 학습해 **테스트 ROC-AUC 0.8262**를 확보했으며, 여기에 0-importance 피쳐 202개 제거와 PCA 5개 컴포넌트 추가를 더한 추가 실험에서 검증 AUC 0.8484→0.8504, 테스트 AUC 0.8262→**0.8277**로 한 단계 더 개선했다.

단계	검증(X_val) AUC	테스트(X_test) AUC
기본 파라미터 (모델 선택 단계)	0.8447	—
RandomizedSearchCV 튜닝 (X_train 전체 재학습)	—	0.8262
0-importance 202개 제거 + PCA 5개 추가	0.8504	0.8277

이 **0.8277**은 팀원 각자가 기록한 개별 실험 결과를 통틀어 **프로젝트 전체에서 가장 높은 테스트 ROC-AUC**다. 다만 이 수치는 X_test를 단 한 번만 평가한 단일 시행 값으로, 표에 함께 정리한 용우의 LightGBM(2단계 튜닝, 0.8253)과는 서로 다른 전처리 파이프라인 (가지치기된 피쳐 구성, PCA 적용 방식)에서 측정되었다는 점에 유의해야 한다. 진희는 LightGBM에 대해 XGBoost와 동일한 Hyperopt 파이프라인을 준비했으나 코드가 전부 주석 처리된 채로 남아 있어 실행되지 않았다(향후 과제로 회고에서 다시 언급한다).

6.5 Random Forest (담당: 용우, 유경)

용우는 111개 가지치기 피쳐에서 기본 파라미터(n_estimators=500) 대비, 1단계 RandomizedSearchCV (n_iter=25, cv=3)로 범위를 좁힌 뒤 2단계 GridSearchCV (324개 조합)로 정밀 튜닝했다.

항목	튜닝 전 (기본 파라미터)	튜닝 후 (용우)	튜닝 후 (유경)
테스트 ROC-AUC	0.7800	0.8219	0.8231
탐색 방법	—	RandomizedSearchCV→GridSearchCV	RandomizedSearchCV(100회, PredefinedSplit)
베스트 파라미터	—	n_estimators=300 , max_depth=6 , max_features=0.4 , min_samples_leaf=30 , min_samples_split=2 , class_weight='balanced_subsample'	n_estimators=189 , max_depth=8 , max_features=0.5 , min_samples_leaf=7 , min_samples_split=18 , class_weight=None

유경은 Random Forest가 "임계값 비교 기반 분기"만 사용한다는 특성에 착안해 var38 로그 변환을 의도적으로 생략하고, 대신 var38_is_missing 플래그를 추가했으며, 스케일링·PCA 단계도 생략(트리 모델에 불필요)했다. 또한 데이터 누수 방지를 위해 상수/중복 컬럼 판단을 train/test 분할 이후 x_tr 기준으로 다시 수행하도록 파이프라인 순서를 조정한 점이 특징이다. 두 팀원의 튜닝

결과는 0.8219~0.8231로 매우 근접해, Random Forest 하이퍼파라미터 최적화의 개선 여지가 대체로 이 구간에서 수렴함을 확인할 수 있었다.

유경이 남긴 실험 기록에는 최종 결과에 이르기까지의 5단계 개선 과정이 그대로 남아 있어, Random Forest 튜닝에서 무엇이 실제로 점수를 끌어올렸는지를 잘 보여준다.

단계	검증 AUC	테스트 AUC	비고
① 기본 RandomForest(튜닝 없음)	—	0.7526	최초 베이스라인
② n_estimators만 탐색	0.8419	0.8143	기존 전처리(PCA 포함) 기준
③ RandomizedSearchCV(n_iter=20)	0.8455	0.8225	refit=True 기본값 사용 — 검증 세트까지 포함해 재학습되는 방법론적 문제 존재
④ RandomizedSearchCV(n_iter=100, refit 수정)	0.8464	0.8197	x_tr 만으로 재학습하도록 설정 — 검증 점수는 더 좋아졌지만 테스트 점수는 오히려 하락
⑤ 전처리 재설계 + class_weight 탐색	0.8444	0.8231	var38_is_missing 플래그, RF 기준 피쳐 선택, 스케일링·PCA 제거

특히 ③ → ④ 구간은 "검증 점수가 오른다고 테스트 점수도 함께 오르는 것은 아니다"라는 것을 실측으로 보여준 사례다. RandomizedSearchCV 의 기본값(refit=True)은 최적 파라미터를 찾은 뒤 검증 데이터까지 포함해 최종 모델을 자동으로 재학습하는데, 이렇게 되면 다른 모델들과 동일한 양의 데이터로 공정하게 비교할 수 없다. 유경은 이를 발견해 refit=False 로 바꾸고 x_tr 만으로 재학습하도록 설정했으며, 그 결과 검증 점수(0.8464)는 오히려 더 좋아 보였지만 테스트 점수(0.8197)는 낮아졌다 — 측정 방법론을 바로잡는 과정에서 "부풀려진" 점수 하나를 걷어낸 것이다. 이후 ⑤ 단계에서 전처리 자체를 Random Forest에 맞게 다시 설계(불필요한 로그 변환·PCA·스케일링 제거, RF 자체 기준 피쳐 선택)하면서 최종적으로 0.8231까지 회복·개선했다. 또한 클래스 불균형 보정을 위해 class_weight='balanced' 를 강제 적용해 봤으나 AUC가 오히려 0.8006→0.7954로 하락하는 것을 확인했는데, 이는 ROC-AUC가 임계값이 아닌 순위(ranking) 품질을 보는 지표이기 때문에 소수 클래스 가중치 보정이 항상 도움이 되지는 않는다는 것을 실증적으로 보여준 사례다.

6.6 Logistic Regression (담당: 유경, 참고: 소현)

로지스틱 회귀는 트리 계열과 달리 피쳐의 선형적 표현 방식이 성능을 크게 좌우한다는 점에 착안해, 유경이 3가지 인코딩/정규화 전략을 비교 실험했다.

실험	전처리	튜닝 전 테스트 AUC	튜닝 방법	튜닝 후 테스트 AUC
① 기본	signed log1p + StandardScaler + PCA5	0.7930	C-class_weight 그리드(16조합)	0.7978
② L1/ElasticNet	①과 동일 + penalty 비교 (L2/L1/ElasticNet)	0.7930	penalty-C-class_weight 그리드 (22조합)	0.7988
③ WOE 인코딩	10분위 구간별 Weight-of-Evidence 치환	0.8080	C-class_weight 그리드(16조합)	0.8083

가장 주목할 결과는 ③ WOE(Weight of Evidence) 인코딩이다. 각 피쳐를 훈련 세트 기준 10분위 구간으로 나눈 뒤 구간별 로그 오즈비로 치환하는 방식으로, 로지스틱 회귀가 가정하는 "피쳐와 로그오즈의 선형 관계"에 데이터 표현 자체를 맞춘 것이다(구현 코드는 12.1절 부록 참고). 그 결과 하이퍼파라미터 튜닝을 하기 전 기본 설정만으로도(AUC 0.8080) ①·②의 튜닝된 결과(0.7978, 0.7988)를 능가했으며, 이는 로지스틱 회귀에서는 "어떤 정규화 강도를 쓰느냐"보다 "피쳐를 어떻게 표현하느냐"가 더 결정적임을 보여주는 사례다. C-class_weight 튜닝(C=10, class_weight='balanced') 이후 최종 0.8083으로 로지스틱 회귀 실험 중 최고 기록을 세웠다.

소현은 XGBoost·LightGBM과의 비교용으로 기본 로지스틱 회귀(class_weight='balanced')를 실행해 검증 AUC 0.7727을 확인했고, 트리 계열 대비 낮은 성능을 이유로 별도 튜닝 없이 비교 기록으로만 남겼다(용우의 파이프라인에도 로지스틱 회귀 코드가 존재하나 최종 출력이 노트북에 저장되지 않았다).

7. 결과

7.1 모델별 최종 성능 종합 비교

각 모델이 팀 내에서 기록한 **최고 테스트 ROC-AUC**를 기준으로 정리하면 다음과 같다.

순위	모델	담당	튜닝 전	튜닝 후(최고)	향상폭
1	LightGBM	소현·용우	0.8011	0.8277	+0.0266
2	Gradient Boosting	혜현	0.8198	0.8256	+0.0058
3	XGBoost	진희	0.8240	0.8255	+0.0015
4	Random Forest	유경	0.7800	0.8231	+0.0431
5	Logistic Regression	유경	0.7930	0.8083	+0.0153
6	Decision Tree	용우	0.5676	0.8034	+0.2358

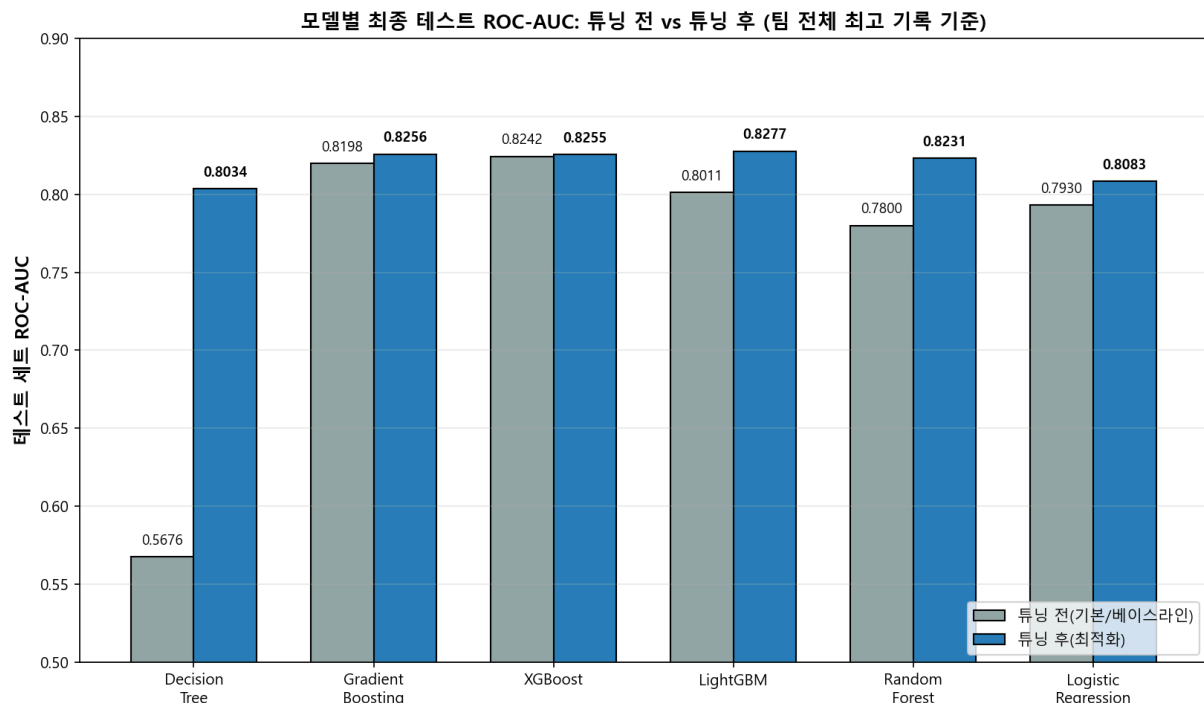


그림 9. 모델별 최종 테스트 ROC-AUC — 튜닝 전(회색) vs 튜닝 후(파란색). LightGBM·GBM·XGBoost 3개 모델이 0.825~0.828 구간에서 사실상 동률을 이루며, 그중 소현의 LightGBM(0-importance 제거+PCA 추가)이 프로젝트 전체 최고 기록이다.

위 표의 "튜닝 전" 값은 팀원마다 서로 다른 전처리 파이프라인(가지치기된 피쳐 수, PCA 적용 여부 등)에서 측정된 값이므로 **모델 간 절대 비교보다는 모델별 개선폭(같은 파이프라인 내에서의 튜닝 전/후 비교)에 더 큰 의미**를 둔다. LightGBM 행은 용우의 기본 파라미터 결과(0.8011)를 "튜닝 전" 기준으로, 소현이 별도 파이프라인에서 달성한 최고 기록(0.8277)을 "튜닝 후"로 표기했다 — 같은 모델을 서로 다른 팀원이 각자 개선한 결과를 모아 "이 모델로 팀이 도달한 최고 지점"을 보여주기 위함이다. 다만 최종적으로 도달한 "튜닝 후" 스코어는 서로 다른 팀원이 독립적으로 튜닝한 결과가 대부분 0.82~0.828 구간에 수렴한다는 점에서, 부스팅 계열 모델(GBM/XGBoost/LightGBM)이 이 데이터셋에서 낼 수 있는 성능 상한이 비슷한 수준임을 시사한다.

7.2 최고 성능 모델 분석

소현의 **LightGBM(0-importance 피쳐 202개 제거 + PCA 5개 추가, ROC-AUC 0.8277)** 이 프로젝트 전체에서 가장 높은 테스트 점수를 기록했다. 그 바로 아래로 부스팅 계열 3종 — 소현·용우의 LightGBM(0.8277/0.8253), 혜현의 GBM(0.8256), 진희의 XGBoost(0.8255) — 이 0.825~0.828 구간에 밀집해 사실상 동일한 성능대를 형성했다. 이 모델들은 공통적으로

- 동일한 핵심 전처리(이상치 치환, 로그 변환, 파생 변수, 상수/중복 제거, 모델별 importance=0 가지치기, PCA 컴포넌트 추가)를 공유했고,
- 서로 다른 탐색 알고리즘(Hyperopt-TPE 30~200회 vs RandomizedSearchCV→GridSearchCV 2단계)을 사용했음에도 최종 수렴 지점이 거의 같았다.

이는 "모델 선택"보다 "전처리 품질"이 이 데이터셋 성능의 상한을 결정하는 더 큰 요인이었을 가능성을 보여준다. 반면 Decision Tree는 단일 트리 구조의 근본적 한계로 최적화 이후에도(0.8034) 부스팅 계열에 5%p 가까이 뒤처졌고, Logistic Regression은 선형 모델이라는 구조적 한계로 WOE 인코딩이라는 표현 방식 개선에도 불구하고 0.808 수준에 머물렀다.

7.3 하이퍼파라미터 최적화 효과 분석

모델을 "트리 구조 하나(Decision Tree)", "선형 모델(Logistic Regression)", "앙상블 모델(GBM/XGBoost/LightGBM/RandomForest)" 셋으로 나누어 보면 최적화 효과의 패턴이 뚜렷하게 갈린다.

- **Decision Tree(+0.2358)**: 규제가 없으면 완전 과적합하는 모델 특성상, 튜닝(=규제)이 곧 모델을 "작동시키는" 필수 과정이었다.
- **Random Forest(+0.0431)**: 배경 기반 앙상블이라 기본 파라미터로도 어느 정도 과적합이 완화되지만, `max_depth` · `min_samples_leaf` · `class_weight` 조정으로 여전히 유의미한 개선 여지가 있었다.
- **LightGBM(+0.0266)**: leaf-wise 성장 방식 특유의 과적합 위험을 `num_leaves` · `min_child_samples` 정규화 항(`reg_alpha`) 튜닝으로 상당히 개선했다(용우). 여기에 소현이 추가한 0-importance 피쳐 제거·PCA 컴포넌트 보강이 한 단계 더 개선을 이끌어내, 팀 전체에서 가장 높은 최종 점수(0.8277)에 도달했다 — 알고리즘 하이퍼파라미터 튜닝과 피쳐 엔지니어링을 함께 병행했을 때 개선 폭이 가장 컸던 사례다.
- **Logistic Regression(+0.0153, 인코딩 방식까지 고려 시 최대 +0.0306)**: 정규화 강도 자체의 영향은 제한적이었고, WOE 인코딩과 같은 피쳐 표현 개선이 튜닝보다 더 큰 효과를 냈다.
- **GBM(+0.0058), XGBoost(+0.0015)**: 두 모델 모두 부스팅 계열 중 기본 파라미터 자체의 완성도가 높아, 정교한 탐색(Hyperopt)을 거쳐도 개선 여지가 크지 않았다. 이는 "탐색을 더 오래 돌린다고 항상 성능이 크게 개선되지는 않는다"는 실무적 교훈을 준다.

7.4 피쳐 중요도 공통 인사이트

XGBoost(그림 7 좌)와 GBM(그림 7 우)의 상위 중요 피쳐를 비교하면 다음과 같은 공통 패턴이 확인된다.

- **var15 (연령 추정)** 와 **var38 (자산 규모 추정, 로그 변환)** 이 두 모델 모두에서 최상위권에 위치 — 인구통계·자산 규모가 만족도와 밀접하게 연관됨을 시사.
- **saldo_var30** , **saldo_medio_var5_hace2** 등 **잔액(saldo) 계열 변수**가 공통적으로 중요하게 나타나, 계좌 잔액 수준이 핵심 예측 신호임을 확인.
- **PCA 파생 성분(pca1 ~ pca5)** 이 원본 피쳐를 제치고 상위권에 다수 포함되어, 개별 피쳐로는 드러나지 않는 여러 변수의 조합 패턴이 유의미한 정보를 담고 있음을 보여준다.
- **n0 (0값 피쳐 개수)** 역시 두 모델 모두에서 중요 피쳐로 채택되어, 팀이 직접 설계한 파생 변수가 실제로 예측에 기여했음을 정량적으로 확인했다.

8. 종합 평가

report.md에 명시된 5개 평가 기준에 따라 본 프로젝트를 자체 평가하면 다음과 같다.

평가 기준	자체 평가
데이터 이해도 및 EDA의 깊이	<code>describe()</code> min/max로 위장된 결측치(sentinel 값)를 찾아내고, 컬럼명 접두사 체계(2.3절)·상관관계(3.2절)·행 단위 중복(3.5절, 6.32%)까지 확인해 단순 통계 요약을 넘어서 EDA를 수행했다.
전처리 방법의 타당성과 근거 제시	모든 전처리 결정에 "왜 이렇게 했는가"(예: sentinel 값 치환 근거, 왜 51.27에 대한 로그 변환)를 실측 수치로 뒷받침했고, 모델 특성별로 전처리를 차등 적용(4장)한 근거도 함께 제시했다.
모델 비교 논리와 지표 선정의 적절성	클래스 불균형을 근거로 ROC-AUC를 공식 지표로 통일했고(2.6절), 6개 모델을 동일 기준(홀드아웃 테스트 세트)으로 비교했다. 다만 7.1절에서 밝혔듯 팀원별 "튜닝 전" 기준 파이프라인이 서로 달라 절대 비교보다 개선폭 비교에 그친 한계가 있다.
최적화 적용 여부 및 성능 개선 정도	6개 모델 전체에 RandomizedSearchCV·GridSearchCV·Hyperopt 중 하나 이상을 적용했으며, Decision Tree(+0.2358)부터 XGBoost(+0.0015)까지 모델 구조별 개선 효과 차이를 정량적으로 분석했다(7.3절). GridSearchCV와 Hyperopt를 동일 모델(LightGBM)에 직접 비교한 실험(5.3절)도 확보했다.
결과 해석의 명확성과 결론의 완성도	최고 성능 모델(LightGBM, 0.8277) 선정 근거와 그 이유(전처리 공유, 서로 다른 탐색 알고리즘의 수렴)를 7.2절에서 설명했고, 9장에서 5개 항목의 결론으로 종합했다.

종합하면, EDA와 전처리 근거 제시는 상당히 충실했던 반면, 팀원 간 파이프라인 표준화(공통 함수화, 5.5절)는 상대적으로 미흡했다는 것이 본 프로젝트의 가장 뚜렷한 강약점이다.

9. 결론

- 부스팅 계열(GBM·XGBoost·LightGBM)이 이 데이터셋에서 사실상 동물의 최고 성능(ROC-AUC 약 0.825)을 보였다.** 실무에서 세 모델 중 하나를 선택해야 한다면 성능 차이보다는 학습/추론 속도(LightGBM이 가장 빠름), 해석 가능성, 운영 환경과의 호환성 등 비기능적 요구사항을 기준으로 선택하는 것이 합리적이다.
- 전처리 설계가 모델 선택보다 성능에 더 큰 영향을 미쳤다.** var3/var38 이상치·왜도 처리, 행 단위 파생 변수(n0 등), 모델별 importance 기반 가지치기라는 공통 전처리 파이프라인 위에서는 어떤 부스팅 알고리즘을 쓰든 비슷한 상한에 수렴했다.
- 하이퍼파라미터 최적화의 효과는 모델 구조에 따라 크게 다르다.** 규제가 없으면 붕괴하는 Decision Tree 같은 모델에서는 튜닝이 필수적이지만, 이미 강건한 기본값을 가진 XGBoost 같은 모델에서는 개선 여지가 제한적이다. 또한 같은 LightGBM에 대해 GridSearchCV와 Hyperopt를 직접 비교한 결과(5.3절)에서 보듯, **탐색 알고리즘의 정교함보다 탐색 범위 설정이 성능에 더 크게 기여할 수 있다.** 따라서 프로젝트 초기에 "이 모델은 튜닝에 민감한 모델인가"를 먼저 진단하면 한정된 탐색 자원(시간, 컴퓨팅)을 더 효율적으로 배분할 수 있다.
- 로지스틱 회귀에서는 정규화 강도보다 피쳐 표현(인코딩) 방식이 더 결정적이었다.** WOE 인코딩만으로 정규화 튜닝 없이도 다른 인코딩의 튜닝된 결과를 앞질렀다는 점은, 선형 모델을 사용할 때 하이퍼파라미터 탐색에 앞서 피쳐 엔지니어링에 투자하는 것이 더 효율적일 수 있음을 시사한다.
- 비즈니스 관점에서는 LightGBM(0.8277) · GBM(0.8256) · XGBoost(0.8255)를 최종 후보군으로 제안한다.** 세 모델은 팀 내 최고 성능을 기록했을 뿐 아니라 feature importance를 통해 "연령·자산 규모·계좌 잔액 수준이 낮은 고객군"이라는 해석 가능한 위험 신호를 함께 제공해, 예측뿐 아니라 마케팅·리텐션 전략 수립에도 활용할 수 있다. 그중에서도 LightGBM은 최고 점수를 기록했을 뿐 아니라 학습 속도가 가장 빨라(\$7.1) 반복 재학습이 잦은 운영 환경에 가장 적합한 실용적 선택지다.

10. 회고

10.1 종합 회고

잘된 점

- 팀 전체가 "훈련/검증 단계에서는 테스트 세트를 사용하지 않는다"는 데이터 누수 방지 원칙을 일관되게 지켰고, 스케일러·PCA·중요도 기반 피쳐 선택을 모두 훈련 세트 기준으로만 `fit` 한 점은 여러 노트북에 걸쳐 공통적으로 잘 지켜졌다.

- 같은 모델(Random Forest, Logistic Regression)을 서로 다른 팀원이 독립적으로 실험해 결과를 교차 검증할 수 있었던 점, 그리고 로지스틱 회귀에서 WOE 인코딩이라는 차별화된 시도를 통해 유의미한 개선을 이끌어낸 점은 팀 실험 다양성의 성과라고 볼 수 있다.

아쉬운 점

- 소현이 진행한 LightGBM 튜닝·PCA 비교 실험의 최종 테스트 ROC-AUC(0.8262→0.8277)는 정작 저장된 노트북 출력에는 남아있지 않았고, 팀이 별도로 기록해 둔 실험 로그(엑셀)를 대조한 뒤에야 본 보고서에 정확히 반영할 수 있었다. 노트북 출력만으로는 팀 전체 최고 기록이 묻힐 뻔한 셈이라, 향후에는 실험 마지막에 핵심 결과를 노트북 안에도 반드시 출력·저장해 두는 습관이 필요하다.
- 진희가 시도한 LightGBM + Hyperopt 파이프라인은 XGBoost와 대칭적으로 준비되었으나 전체가 주석 처리된 채로 남아 실행되지 않았다. 시간 관계상 XGBoost 튜닝에 집중한 것으로 보이며, 완료했다면 LightGBM 계열의 팀 내 비교가 한 단계 더 촘촘해졌을 것이다(참고로 같은 실험을 용우가 별도로 완료해 5.3절에 반영했다).
- 팀원마다 전처리 파이프라인(가지치기된 피쳐 수, PCA 적용 여부, 파생 변수 종류)이 조금씩 달라 "모델 간" 성능을 완전히 동일한 조건에서 비교하기는 어려웠다. 5.5절에서 확인했듯 실제로 팀 공통 함수는 Hyperopt 목적함수와 WOE 인코딩 정도에 그쳐, 대부분의 전처리 코드가 팀원별로 중복 재작성되었다. 다음 프로젝트에서는 공통 전처리 함수를 팀 초기에 모듈화해 공유하면, 모델 비교의 공정성을 높이고 중복 작업도 줄일 수 있을 것이다.
- 3.5절에서 재확인한 행 단위 중복(전체의 6.32%)은 어느 팀원의 파이프라인에서도 명시적으로 제거되지 않았다. 클래스 분포를 크게 왜곡하는 수준은 아니었지만, EDA에서 발견한 이슈가 전처리 단계까지 이어지지 못한 사례로 남는다.
- 팀원별 로컬 환경(라이브러리 버전 등, 5.4절)이 고정·공유되지 않아, 동일 코드를 재실행했을 때 완전히 동일한 결과가 재현되는지는 검증되지 않았다.

향후 개선 방향

- 최고 성능을 보인 LightGBM·GBM·XGBoost 3개 모델을 스택킹(stack) 또는 가중 평균 앙상블로 결합하면 개별 모델의 상한(약 0.825~0.828)을 넘어설 가능성이 있다.
- Hyperopt의 탐색 횟수를 늘리거나 Optuna 등 다른 베이지안 최적화 도구와 비교해, 현재 탐색 범위의 경계값(RandomForest 1단계 탐색에서 `max_depth` · `max_features` 가 범위의 끝값으로 나온 사례 등)를 넘어서 재탐색을 수행할 필요가 있다.
- 현재 파생 변수는 대부분 행 단위 집계 통계량인데, 고객 세그먼트별(연령대·자산 구간별) 상대적 위치를 나타내는 변수 등 도메인 기반 파생 변수를 추가로 설계하면 추가적인 성능 개선의 여지가 있을 것으로 판단된다.
- 팀 초기에 전처리·평가 함수를 공통 모듈로 분리하고, `environment.yml` 등으로 실행환경을 고정해 재현성을 높이는 작업이 필요하다.

10.2 팀원별 느낀점 및 피드백

아래 내용은 팀이 작업 과정에서 실시간으로 기록해 온 1데이터 셋 정리_final.xlsx의 팀원별 시트에 남아있는 느낀점과 피드백을 옮긴 것이다.

이혜현 — GBM

불만족 고객이 4%인 불균형 데이터에서 파생변수 생성, 노이즈 피쳐 제거, Hyperopt 기반 튜닝 파이프라인을 구축해 안정적인 ROC-AUC를 확보했다. 하지만 기본 GBM 모델 특성상 학습 속도가 느려 탐색을 충분히 하지 못했고, 그 결과 XGBoost와 LightGBM을 사용한 팀원들에 비해 점수가 낮게 나오는 한계를 겪었다. 이를 통해 대용량 데이터에서는 알고리즘의 연산 효율성이 성능에 큰 영향을 준다는 점을 배웠으며, 향후에는 XGBoost와 LightGBM을 적용하고 팀 모델과의 앙상블을 통해 최종 점수를 끌어올리고자 한다.

김소현 — XGBoost / LightGBM

피드백(방법론 관련): 튜닝 자체의 CV 최고점(0.8446)은 default 모델의 X_{val} 점수(0.8447)와 사실상 동일해, 튜닝이 점수를 극적으로 올리지 못했다. 이 단계의 진짜 효과는 점수 상승이 아니라 **측정 방법론 정정**이었다 — X_{val} 참고값과 X_{test} 최종값을 섞어서 비교하지 않도록 "튜닝된 모델만 X_{test} 를 1회 평가한다"는 원칙을 확정한 것으로, 점수를 올린 변화는 아니지만 이후 보고되는 모든 숫자의 신뢰도를 지킨 변화였다.

느낀점: 전처리 부분에서 의미 없는 컬럼은 제거를 먼저 하고 들어가는 게 전체적인 성능 향상에 도움이 된다. Constant feature 34개, 완전 중복(identical) feature 29개를 제거해 370개 → 307개로 축소했고, 학습 속도·과적합 위험이 모두 개선됐다. 또한 모델 선정 단계에서 트리 기반 모델이 압승한 이유는, feature 수가 많고(307개) 변수 간 비선형 상호작용이 강한 정형 데이터라서 선형모델은

애초에 한계가 명확하기 때문이다. → 탐색 단계에서 성능 차이가 크게 나는 후보는 과감히 튜닝 대상에서 제외하는 판단이 합리적이다.

이진희 — XGBoost / LightGBM

총합 피드백

우리는 좀 더 세밀한 데이터를 위해서 테스트와 트레인을 스플릿 할 때 `stratify=y_labels`, `stratify=y_train` 을 활용했다. 그래서 비록 낮은 수치가 나왔지만, 그 과정에서 상당히 얻은 게 있다고 생각한다. 데이터 전처리를 정확히 보고 어떤 것을 해야 할지 체크하는 것부터 시작해서, 데이터를 학습한 후에 어떻게 해야 할지, 그리고 모델을 선정하여 작업하는 방법, 하이퍼파라미터를 계속해서 시도하고 정하는 과정까지 경험했다. 이 과정이 연습과정이었을지언정, 충분히 실전으로 활용 가능하도록 만들었다. 팀원들과 함께하는 기분이 들어서 좋았다.

전유경 — Random Forest / Logistic Regression

Random Forest 한계 및 개선 방향: Random Forest는 현재 피쳐셋 기준으로 AUC 0.82 초반대에서 성능이 수렴하는 것으로 보이며, 하이퍼파라미터 튜닝만으로는 추가 개선 여지가 크지 않다. PCA-스케일링 추가, `class_weight` 보정, 트리 개수 확대를 시도했으나 효과가 없었다. 다음에는 RandomForest보다 XGBoost·LightGBM 쪽에 시간을 더 투자하는 방향을 고려하고 있다.

최종 느낀점(Logistic Regression): Santander 데이터셋을 팀원들과 같이 살펴보면 어떤 전처리를 하고 어떤 모델을 쓸지에 대해 많은 논의를 거쳤다. 처음 해보다 보니 많은 시행착오가 있었지만, 여러 가지 모델들을 비교하면서 어떤 수치가 나왔는지 확인해 보니 그래도 의미 있는 결과가 나왔다고 생각한다. 이번 Santander 데이터 처리를 하면서 각각의 모델들이 어떤 성향을 가지고 있는지 확인할 수 있는 순간이었고, 이번 데이터에서 알게 된 것들을 팀원들과 같이 다음 데이터 처리를 할 때 적용하기로 하면서 점차 발전될 것을 기대하고 있다.

권용우 — Decision Tree / 통합 파이프라인

권용우는 팀 공통 전처리·모델링 파이프라인(XGBoost 기반 초기 실험, Decision Tree 튜닝, LightGBM GridSearchCV/Hyperopt 비교 등)의 실험 이력을 기록으로 남겼으나, 다른 팀원들과 달리 별도의 "느낀점" 항목은 기록되어 있지 않다. 다만 그의 실험 로그에는 "디시전 트리는 로그 변환에 대한 의미가 없다", "delta 계열 이상치는 중앙값으로 대체한다", "normalize + PCA는 트리 계열에 불필요하고 오히려 역효과", "Hyperopt가 GridSearchCV보다 항상 좋은 것은 아니다"라는 판단 근거들이 상세히 남아 있어, 본 보고서 4장·5.3절·6장의 모델별 전처리 차별화 및 탐색 기법 비교 논의에 직접 반영되었다.

11. 참고문헌

1. Kaggle, *Santander Customer Satisfaction* (competition dataset), <https://www.kaggle.com/c/santander-customer-satisfaction>
2. Chen, T. & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. KDD '16. (XGBoost 알고리즘 원 논문)
3. Ke, G. et al. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. NeurIPS 2017. (LightGBM 알고리즘 원 논문)
4. Bergstra, J., Yamins, D., & Cox, D. D. (2013). *Making a Science of Model Search: Hyperparameter Optimization in Hundreds of Dimensions for Vision Architectures*. ICML 2013. (Hyperopt의 TPE 알고리즘 근거 논문)
5. Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR 12. (`train_test_split`, `RandomizedSearchCV`, `GridSearchCV`, `LogisticRegression`, `RandomForestClassifier` 등 사용)
6. scikit-learn 공식 문서, `sklearn.metrics.roc_auc_score`, https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html
7. Hyperopt 공식 문서, <https://github.com/hyperopt/hyperopt>

12. 부록: 함수코드

아래 함수들은 팀 노트북 전수 조사(YWK·TT(YWK)·HH·JH·YG 등 10개 노트북) 결과 실제로 `def` 로 정의되어 있었거나, 여러 노트북에서 반복적으로 나타난 인라인 패턴을 재사용 가능하도록 정리한 것이다. 5.5절에서 밝혔듯 팀 공통 함수화는 제한적이었으므로, 이 부록은

"이미 공유되어 있던 함수"와 "정리하면 공유할 수 있었던 패턴"을 함께 담았다.

12.1 전처리 함수

`var3 / delta_*` 이상치 치환과 `var38` 로그 변환은 거의 모든 팀원의 노트북(YWK, TT(YWK) 등)에 동일한 형태로 반복 등장한 패턴이다. 이를 하나의 함수로 정리하면 다음과 같다.

```
import numpy as np

def preprocess_santander(X_features):
    """var3/delta_* 이상치 치환 + var38 로그 변환 + 파생 변수(n0) 생성.
    팀 공통 전처리 1~4단계(4.1~4.3절)를 하나로 정리한 버전."""
    X = X_features.copy()

    # 1) var3 결측 이상치(-999999) -> 최빈값 2로 치환
    X['var3'] = X['var3'].replace(-999999, 2)

    # 2) delta_* 계열 이상치(9999999999) -> NaN -> 컬럼별 중앙값으로 대체
    delta_cols = [c for c in X.columns if c.startswith('delta_')]
    X[delta_cols] = X[delta_cols].replace(9999999999, np.nan)
    X[delta_cols] = X[delta_cols].fillna(X[delta_cols].median())

    # 3) var38 로그 변환 (왜도 51.27 완화)
    X['var38'] = np.log1p(X['var38'])

    # 4) 파생 변수: 고객별 0값 피쳐 개수
    X['n0'] = (X == 0).sum(axis=1)

    return X

def remove_useless_columns(X_features):
    """상수 컬럼 + 중복 컬럼(2.4절) 제거. 반드시 상수 컬럼을 먼저 제거한 뒤
    중복을 검사해야 한다(안 그러면 서로 값이 같은 상수 컬럼끼리 중복으로 잡혀 개수가 부풀려진다)."""
    X = X_features.copy()

    const_cols = [c for c in X.columns if X[c].nunique() == 1]
    X = X.drop(columns=const_cols)

    dup_mask = X.T.duplicated()
    dup_cols = X.columns[dup_mask].tolist()
    X = X.drop(columns=dup_cols)

    return X, const_cols, dup_cols
```

유경이 로지스틱 회귀 전용으로 작성한 WOE(Weight of Evidence) 인코딩 함수(6.6절)도 실제 노트북(`JYG_Logistic_WOE인코딩.ipynb`)에 정의된 그대로 재사용 가능한 형태다.

```

def fit_woe(X, y, n_bins=10):
    """X_tr 기준으로 컬럼별 구간 경계와 WOE 값 산출"""
    bin_edges, woe_maps = {}, {}
    total_good, total_bad = (y == 0).sum(), (y == 1).sum()
    eps = 0.5 # 구간 내 이벤트 0건일 때 Log(0) 방지용 보정

    for col in X.columns:
        try:
            binned, edges = pd.qcut(X[col], q=n_bins, retbins=True, duplicates='drop')
        except ValueError:
            binned, edges = X[col], None

        df_bin = pd.DataFrame({'bin': binned, 'y': y.values})
        grouped = df_bin.groupby('bin', observed=True)['y'].agg(['sum', 'count'])
        good, bad = grouped['count'] - grouped['sum'], grouped['sum']

        woe = np.log(((good + eps) / total_good) / ((bad + eps) / total_bad))
        bin_edges[col], woe_maps[col] = edges, woe.to_dict()

    return bin_edges, woe_maps

def transform_woe(X, bin_edges, woe_maps):
    """fit_woe에서 산출한 구간 경계/WOE 값으로 X를 변환 (검증/테스트 세트에 사용)"""
    X_woe = pd.DataFrame(index=X.index)
    for col in X.columns:
        edges = bin_edges[col]
        binned = pd.cut(X[col], bins=edges, include_lowest=True) if edges is not None else X[col]
        X_woe[col] = binned.astype(object).map(woe_maps[col]).fillna(0.0).astype(float)
    return X_woe

```

12.2 공동함수-modeling

train_test_split (1차: test 20%) → train_test_split (2차: 검증 30%) → 모델 fit → roc_auc_score 평가라는 흐름은 6장의 모든 모델에서 반복되었다. 이를 함수화하면 다음과 같다.

```

from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

def split_train_val_test(X_features, y_labels, test_size=0.2, val_size=0.3, random_state=0):
    """1차: 학습(80%)/최종테스트(20%) 분리, 2차: 학습을 다시 훈련/검증으로 분리.
    4.6절의 데이터 분할 원칙(stratify + 훈련 세트 기준 fit)을 함수로 고정."""
    X_train, X_test, y_train, y_test = train_test_split(
        X_features, y_labels, test_size=test_size, stratify=y_labels, random_state=random_state)
    X_tr, X_val, y_tr, y_val = train_test_split(
        X_train, y_train, test_size=val_size, stratify=y_train, random_state=random_state)
    return X_tr, X_val, X_test, y_tr, y_val, y_test

def fit_and_evaluate(clf, X_tr, y_tr, X_test, y_test, X_val=None, y_val=None):
    """모델을 학습시키고(조기 종료용 eval_set이 있으면 함께 전달) 테스트 ROC-AUC를 반환.
    6장 전체에서 반복된 'fit -> predict_proba -> roc_auc_score' 패턴을 정리."""
    if X_val is not None and hasattr(clf, 'fit') and 'eval_set' in clf.fit.__code__.co_varnames:
        clf.fit(X_tr, y_tr, eval_set=[(X_tr, y_tr), (X_val, y_val)])
    else:
        clf.fit(X_tr, y_tr)
    test_auc = roc_auc_score(y_test, clf.predict_proba(X_test)[: , 1])
    return clf, test_auc

def prune_zero_importance(clf, X_features, y_labels):
    """학습된 트리 모델의 feature_importances_==0인 컬럼을 제거한 피쳐셋을 반환 (4.4절)."""
    importances = pd.Series(clf.feature_importances_, index=X_features.columns)
    kept = importances[importances > 0].index.tolist()
    return X_features[kept], kept

```

12.3 공동함수-hyperopt

헤현(GBM)·진희(XGBoost)·용우(LightGBM)가 각자 독립적으로 작성한 `objective(params)` 함수는 세부 파라미터만 다를 뿐 "탐색 공간 정의 → K-Fold CV로 평균 AUC 계산 → 음수로 반환 → `fmin` + TPE로 탐색"이라는 동일한 구조를 공유한다. 이를 모델에 무관하게 재사용 가능한 하나의 템플릿으로 일반화하면 다음과 같다.

```

import numpy as np
from hyperopt import hp, fmin, tpe, Trials, STATUS_OK
from sklearn.model_selection import KFold
from sklearn.metrics import roc_auc_score

def make_hyperopt_objective(model_cls, X, y, int_params=(), n_splits=3, random_state=156, fixed_params=None):
    """model_cls(예: LGBMClassifier, XGBClassifier, GradientBoostingClassifier)에 대해
    Hyperopt objective 함수를 생성한다. int_params에 지정된 파라미터는 int로 변환한다
    (Hyperopt의 hp.quniform은 float를 반환하므로 트리 개수, leaf 수 등은 반드시 필요).
    5.3절/6.2~6.4절에서 각 팀원이 개별 구현했던 objective(params)의 공통 구조."""
    fixed_params = fixed_params or {}

    def objective(params):
        p = {k: (int(v) if k in int_params else v) for k, v in params.items()}
        kf = KFold(n_splits=n_splits, shuffle=True, random_state=random_state)
        aucs = []
        for tr_idx, val_idx in kf.split(X):
            clf = model_cls(random_state=random_state, **fixed_params, **p)
            clf.fit(X.iloc[tr_idx], y.iloc[tr_idx])
            pred = clf.predict_proba(X.iloc[val_idx][:, 1])
            aucs.append(roc_auc_score(y.iloc[val_idx], pred))
        return {'loss': -np.mean(aucs), 'status': STATUS_OK}

    return objective

def run_hyperopt(objective, space, max_evals=100, random_state=156):
    """fmin 실행을 감싸 베스트 파라미터와 베스트 CV AUC를 함께 반환."""
    trials = Trials()
    best = fmin(fn=objective, space=space, algo=tpe.suggest, max_evals=max_evals,
               trials=trials, rstate=np.random.default_rng(random_state))
    best_cv_auc = -trials.best_trial['result']['loss']
    return best, best_cv_auc

```

예를 들어 5.3절의 LightGBM Hyperopt 실험은 이 템플릿으로 다음과 같이 재현할 수 있다.

```

lgbm_space = {
    'n_estimators': hp.quniform('n_estimators', 100, 600, 50),
    'learning_rate': hp.uniform('learning_rate', 0.005, 0.05),
    'num_leaves': hp.quniform('num_leaves', 7, 63, 4),
    'min_child_samples': hp.quniform('min_child_samples', 50, 300, 10),
    'reg_alpha': hp.uniform('reg_alpha', 0, 5),
    'reg_lambda': hp.uniform('reg_lambda', 0, 2),
    'subsample': hp.uniform('subsample', 0.4, 1.0),
    'colsample_bytree': hp.uniform('colsample_bytree', 0.3, 1.0),
}

objective = make_hyperopt_objective(
    LGBMClassifier, X_tr2, y_tr2,
    int_params=('n_estimators', 'num_leaves', 'min_child_samples'),
    fixed_params={'verbosity': -1, 'n_jobs': -1, 'max_depth': -1},
)

best_params, best_cv_auc = run_hyperopt(objective, lgbm_space, max_evals=100)

```

12.4 공동함수-평가

6장 전체에서 "튜닝 전 vs 튜닝 후" 또는 "모델 A vs 모델 B"의 ROC-AUC를 나란히 출력하는 패턴이 반복되었다. 이를 하나의 리포팅 함수로 정리했다.

```

def report_auc(results: dict, baseline: str = None):
    """{'모델/단계 이름': auc, ...} 형태의 결과를 표로 출력하고,
    baseline이 지정되면 그 대비 항상폭도 함께 계산한다 (7.1절 종합 비교표에 대응)."""
    print(f"{'항목':30s} {'ROC-AUC':>10s}" + (f"{'항상폭':>10s}" if baseline else ""))
    base_score = results.get(baseline) if baseline else None
    for name, score in results.items():
        line = f"{'name':30s} {'score':>10.4f}"
        if base_score is not None:
            line += f"{'score - base_score':>+10.4f}"
        print(line)
    return {
        'best_model': max(results, key=results.get),
        'best_score': max(results.values()),
    }

```

이 함수는 예컨대 7.1절의 종합 비교표를 재현할 때 `report_auc({'Decision Tree': 0.8034, 'GBM': 0.8256, 'XGBoost': 0.8255, 'LightGBM': 0.8277, 'Random Forest': 0.8231, 'Logistic Regression': 0.8083})` 처럼 호출해 최고 성능 모델과 점수를 바로 확인하는 용도로 쓸 수 있다.

Credit Card Fraud Detection 분석 보고서

프로젝트: credit_card_fraud_detection 팀 프로젝트

분석 대상: Kaggle Credit Card Fraud Detection 데이터셋

팀원: 이해현 · 이진희 · 김소현 · 전유경 · 권용우

작성일: 2026-08-25 (최종 수정: 2026-09-02)

목차

0. 초록

1. 서론

- 1.1 연구 배경 및 필요성
- 1.2 과제 목적

2. 데이터셋 소개

- 2.1 데이터 출처 및 구성
- 2.2 데이터 특성 및 구조 분석
- 2.3 Feature 이름 특징
- 2.4 동일 값·중복 Feature
- 2.5 ID 변수
- 2.6 타깃 변수 분포 분석

3. 탐색적 데이터 분석(EDA)

- 3.1 기술통계 분석
- 3.2 상관관계 기반 주요 변수 분석
- 3.3 이상치 분석
- 3.4 변수별 이상치 시각화
- 3.5 중복 값 제거
- 3.6 특정 컬럼 로그 변환

4. 데이터 전처리

- 4.1 이상치 처리
- 4.2 분포 변환
- 4.3 파생 변수 생성
- 4.4 피처 정제 및 중요도 기반 가지치기
- 4.5 스케일링 및 차원 축소
- 4.6 데이터 분할

5. 분석방법(Analysis method)

- 5.1 전처리 필요성 및 과정 개요
- 5.2 사용 모델 설명
- 5.3 하이퍼파라미터 최적화 기법
- 5.4 실행환경
- 5.5 코드 재사용 및 함수화

6. 모델링

- 6.1 CatBoost (전유경)
- 6.2 LightGBM (이혜현)
- 6.3 Logistic Regression (김소현)
- 6.4 Logistic Regression (권용우)
- 6.5 XGBoost (이진희)

7. 결과

- 7.1 모델별 최종 성능 비교

7.2 최고 성능 모델 분석

7.3 하이퍼파라미터 최적화 효과 분석

7.4 피처 중요도 공통 인사이트

8. 종합 평가

9. 결론

10. 회고

10.1 잘된 점 · 아쉬운 점 · 향후 개선 방향

10.2 팀원별 느낀점 및 피드백

11. 참고문헌

12. 부록: 함수 코드

12.1 전처리 함수

12.2 공동 함수 — 모델링

12.3 공동 함수 — 하이퍼파라미터 최적화

12.4 공동 함수 — 평가

0. 초록

본 보고서는 Kaggle 에 공개된 Credit Card Fraud Detection 데이터셋(*creditcard.csv*, 284,807 건, PCA 로 익명화된 28 개 성분 V1~V28 + 거래 금액 Amount·경과 시간 Time·타겟 Class)을 이용해 신용카드 거래의 사기 여부를 예측하는 이진 분류 프로젝트를 정리한 것이다. 사기 거래가 전체의 0.173%(492 건)에 불과한 극단적 클래스 불균형 때문에 단순 정확도로는 성능을 가늠할 수 없고, 전처리와 평가지표 선택이 프로젝트 전체의 쟁점이 되었다. 팀원 5 명이 CatBoost(전유경), LightGBM(이혜현), XGBoost(이진희), Logistic Regression(김소현·권용우) 다섯 파이프라인을 각자 독립적으로 실험하고, 수동/격자 탐색과 Hyperopt(TPE)를 나란히 비교했다. 주요 결과는 (1) 완전 중복행 제거와 "분할 → 전처리" 순서 같은 방법론적 원칙이 모델 선택보다 성능의 신뢰도를 좌우했고, (2) ROC-AUC 하나만으로는 오탐 비용이 드러나지 않아 PR-AUC 와 F1 을 함께 확인해야 했으며, (3) PR-AUC 기준으로 트리 기반 부스팅(CatBoost·XGBoost·LightGBM)이 로지스틱 회귀보다 앞서는 경향이 다섯 실험 전반에서 공통으로 확인되었다는 것이다. 다만 팀원별 전처리·검증 조건이 달라 세 부스팅 모델 간 우열은 단정하지 않았다.

1. 서론

1.1 연구 배경 및 필요성

신용카드 이상거래 탐지는 은행·카드사가 정상 거래 흐름을 방해하지 않으면서 소수의 사기 거래를 잡아내야 하는 실무 과제다. 사기를 놓치면(미탐지, FN) 금전 피해와 고객 신뢰 손상으로 이어지고, 정상 거래를 사기로 잘못 차단하면(오탐, FP) 고객 불편과 상담 비용이 발생하므로, 두 오류의 비용을 함께 고려한 탐지 품질이 중요하다. 이번 프로젝트에서 다룬 *creditcard.csv* 데이터셋은 2013년 9월 유럽 카드 소지자의 이틀간 거래 284,807건으로, 개인정보 보호를 위해 원본 피처가 PCA로 변환된 28개 익명 성분(V1~V28)에 거래 금액(Amount)과 경과 시간(Time), 타깃(Class)만 원본 형태로 남아 있다. 사기 거래는 492건(0.173%)에 불과해 “모든 거래를 정상으로 예측”하기만 해도 정확도(accuracy)가 99.8%를 넘는다. 즉 이 데이터에서는 정확도가 성능 지표로서 의미를 잃고, 어떤 지표로 평가하고 어떤 전처리·모델을 선택하느냐가 결과를 가른다.

1.2 과제 목적

본 프로젝트의 목적은 팀원 5명이 서로 다른 전처리 전략과 모델을 각자 담당해 실험한 뒤, 이를 하나의 파이프라인 관점에서 비교·정리하는 것이다. 전유경은 CatBoost, 이해현은 LightGBM, 이진희는 XGBoost를 맡았고, 김소현과 권용우는 같은 Logistic Regression을 서로 다른 방식으로 다뤘다 — 김소현은 *class_weight*와 SMOTE 비교, L1 정규화를 통한 변수 자동 선택, 평가지표를 PR-AUC로 바꾼 규제 강도(C) 재탐색에, 권용우는 8가지 전처리 조합 전수 비교와 GridSearchCV·Hyperopt·SMOTE의 동일 조건 비교에 무게를 뒀다. 같은 원본 데이터를 쓰되 전처리·모델·튜닝 전략은 팀원마다 달랐고, 그 차이 자체가 “어떤 조합이 이 데이터에 실제로 효과적인가”를 판단하는 근거가 된다. 절차는 (1) 데이터 이해 및 EDA, (2) 데이터 전처리, (3) 모델링 및 비교, (4) 하이퍼파라미터 최적화, (5) 성능평가 및 해석, (6) 결론 및 회고의 순서로 진행되었으며, 본 보고서 역시 이 흐름을 따른다.

2. 데이터셋 소개

2.1 데이터 출처 및 구성

항목	내용
데이터 출처	Kaggle <i>Credit Card Fraud Detection</i> (Machine Learning Group – ULB), <i>creditcard.csv</i>
전체 건수	284,807 건
컬럼 수	31 개 (<i>Time</i> , <i>V1~V28</i> , <i>Amount</i> , <i>Class</i>)
피처 수(타깃 제외)	30 개 — 전부 수치형(<i>float64</i>)
결측치(NaN)	없음 (전 팀원이 <i>isnull().sum()</i> 으로 공통 확인)
클래스 분포	0(정상) 284,315 건(99.827%) / 1(사기) 492 건(0.173%) → 약 578 : 1 불균형
평가지표	ROC-AUC(보조) + PR-AUC / F1-Score(주)

산탄데르 데이터셋과 달리 별도의 *ID* 컬럼도, 제출용 테스트 파일도 없다. 하나의 *creditcard.csv*를 팀 전원이 동일하게 내려받아 *train_test_split*으로 나눠 썼다(자세한 분할 방식은 4.6).

2.2 데이터 특성 및 구조 분석

- **V1~V28**: 개인정보 보호를 위해 이미 PCA(주성분분석)로 변환된 익명 피처다. 평균이 거의 0 이고 서로 상관관계가 거의 없어(직교) PCA 변환의 전형적 특징을 보인다. 이 때문에 “어떤 거래 속성인지”를 알 수 없어 도메인 기반 피처 엔지니어링은 불가능하며, feature importance 로 “V14 가 중요하다” 정도의 통계적 해석만 가능하다.
- **Amount**: 결제 금액(원본 스케일). 평균(약 88)이 중앙값(약 22)의 4 배에 이를 만큼 오른쪽으로 크게 치우쳐(skewed) 있고, 소수의 고액 거래가 분포를 끌어당긴다.
- **Time**: 첫 거래 이후 경과 초(0~172,792 초, 약 이틀)로, 실제 시각(시/분)이 아니라 사기 여부와 직접적인 인과관계를 갖기 어려운 값이다.
- **Class**: 타깃 변수(0=정상, 1=사기).

기술통계량은 원본 *creditcard.csv* 기준이며 팀 전원이 *describe().info()*로 공통 확인했다. 전체 30 개 피처는 *float64*, *Class* 만 *int64* 다. 아래 표에는 원본 스케일 변수인 *Time*-*Amount* 만 싣고, PCA 성분과 타깃은 표 아래에 따로 정리한다.

변수	mean	std	min	25%	50%	75%	max
Time	94,814	47,488	0	54,202	84,692	139,320	172,792
Amount	88.35	250.12	0.00	5.60	22.00	77.16	25,691.16

*V1~V28*은 PCA 로 변환된 28 개 성분으로 개별 의미를 알 수 없어 표에는 싣지 않고 공통 특성만 정리한다. 28 개 모두 평균과 중앙값이 0 에 수렴하며, 표준편차만 *V1*(1.96)에서 *V28*(0.33)로 단조 감소한다(주성분 순서 = 분산 설명력 순서). 다만 일부 성분은 꼬리가 매우 길어 최솟값은 *V5*의 -113.74, 최댓값은 *V7*의 120.59 까지 벌어진다. 타깃 *Class*는 이진값(0/1)으로 분포는 2.6 에서 다룬다.

2.3 Feature 이름 특징

산탄데르 데이터셋은 *num_ind_saldo_imp_delta* 같은 접두사로 피처의 성격(건수·플래그·잔액·금액·변화량)을 유추할 수 있었다. 그러나 이 데이터셋의 *V1~V28*은 PCA로 익명화된 성분이라 이름 규칙이 존재하지 않는다. *V+* 번호는 주성분의 순번(분산 설명력 순서)일 뿐이며, 이름만으로는 어떤 거래 속성인지 전혀 알 수 없다. 따라서 접두사 분석·도메인 기반 피처 엔지니어링은 불가능하고, 이후 상관관계·feature importance 같은 통계적 지표로만 변수의 중요도를 해석했다. *Time Amount Class*만 원래 이름을 유지한다.

2.4 동일 값·중복 Feature

- **상수(constant) 컬럼:** 없음. 모든 피처가 분산 > 0 이므로 산탄데르처럼 “값이 하나뿐인 컬럼 제거” 단계는 필요 없었다.
- **중복 컬럼:** 없음. *V1~V28*은 PCA 성분이라 서로 직교(상관계수 절댓값이 모두 0에 수렴)해, 다른 컬럼과 값이 완전히 같은 경우가 없다.
- **행(row) 단위 완전 중복:** 존재한다. 31개 열 전체가 일치하는 완전 중복행이 1,081건(전체의 0.38%)이며, *Time*을 제외한 30개 열 기준으로는 9,144건까지 늘어난다. 이 행 중복은 데이터 누수와 직결되므로 3.5에서 별도로 다룬다.

2.5 ID 변수

이 데이터셋에는 산탄데르의 *ID*처럼 행을 식별하는 별도 컬럼이 없다. *Time*이 “첫 거래로부터 경과한 초”라 사실상 관측 순서를 담고 있으나, 실제 서비스 만족도나 사기 여부와 인과적으로 무관한 값이며 스케일도 다른 피처와 크게 달라(0~172,792), 팀 전원이 학습 피처에서 제외했다(근거는 4.4). 즉 학습에 쓰는 것은 *V1~V28*과 *Amount* 29개 피처뿐이다.

2.6 타겟 변수 분포 분석

*Class*는 0(정상) 284,315건 대 1(사기) 492건으로, 사기가 전체의 **0.173%**에 불과하다. 클래스 불균형이 매우 심하기 때문에 단순 정확도는 모든 거래를 정상으로 예측해도 99.8%에 도달하는 착시를 일으킨다. 따라서 팀 전체가 초기 EDA 단계부터 정확도 대신 ROC-AUC와 PR-AUC(또는 F1-Score)를 함께 확인하는 방향으로 지표를 통일했다(사기 비율 0.173% 데이터에서 ROC-AUC를 단독으로 쓰면 안 되는 이유는 7.2-9장에서 자세히 다룬다).

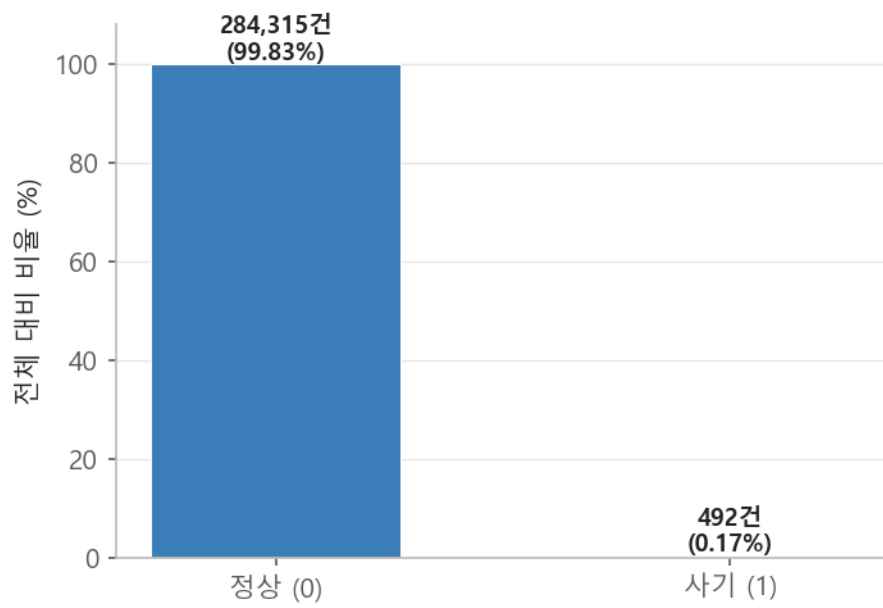


그림 1. Class 분포(정상 vs 사기). 사기 거래가 전체의 0.173%에 불과한 심각한 불균형을 보인다.

3. 탐색적 데이터 분석(EDA)

3.1 기술통계 분석

2.2 절의 *describe()* 결과에서 이미 확인했듯, $V1 \sim V28$ 은 평균이 0 에 수렴하고 표준편차만 $V1(1.96)$ 에서 $V28(0.33)$ 로 단조 감소한다. 이는 PCA 가 분산이 큰 순서로 성분을 매기기 때문이며, 뒤쪽 성분일수록 정보량이 적다. 다만 개별 성분의 꼬리는 매우 길어 최솟값 $V5 -113.74$, 최댓값 $V7 120.59$ 처럼 극단값이 관측된다.

*Amount*는 평균 88.35 에 비해 최댓값이 25,691, 표준편차가 250 으로 극단적인 오른쪽 꼬리를 보인다. 75% 값이 77.16 인데 최댓값이 그 300 배가 넘는다는 것은 소수의 고액 거래가 분포를 끌어당기고 있음을 뜻하며, 이는 3.6 절 로그 변환·스케일링 필요성의 근거가 된다.

3.2 상관관계 기반 주요 변수 분석

각 피처와 *Class*의 상관계수를 확인한 결과, 절댓값 기준으로 $V17 \cdot V14 \cdot V12 \cdot V10 \cdot V16 \cdot V3 \cdot V7 \cdot V11$ 이 사기 여부와 가장 높은 상관관계를 보였다(모두 0.15~0.33 수준). 이 중 $V14$ 는 사기(*Class*=1) 케이스만 놓고 봤을 때 분포가 정규분포에 가장 가까워, 극단값을 이상치로 판정하기에 통계적으로 적합한 성질을 보였다. 이 성질을 활용한 이상치 탐지·제거 방법과 팀원별 적용은 3.3~3.4 절과 4.1 절에서 다룬다.

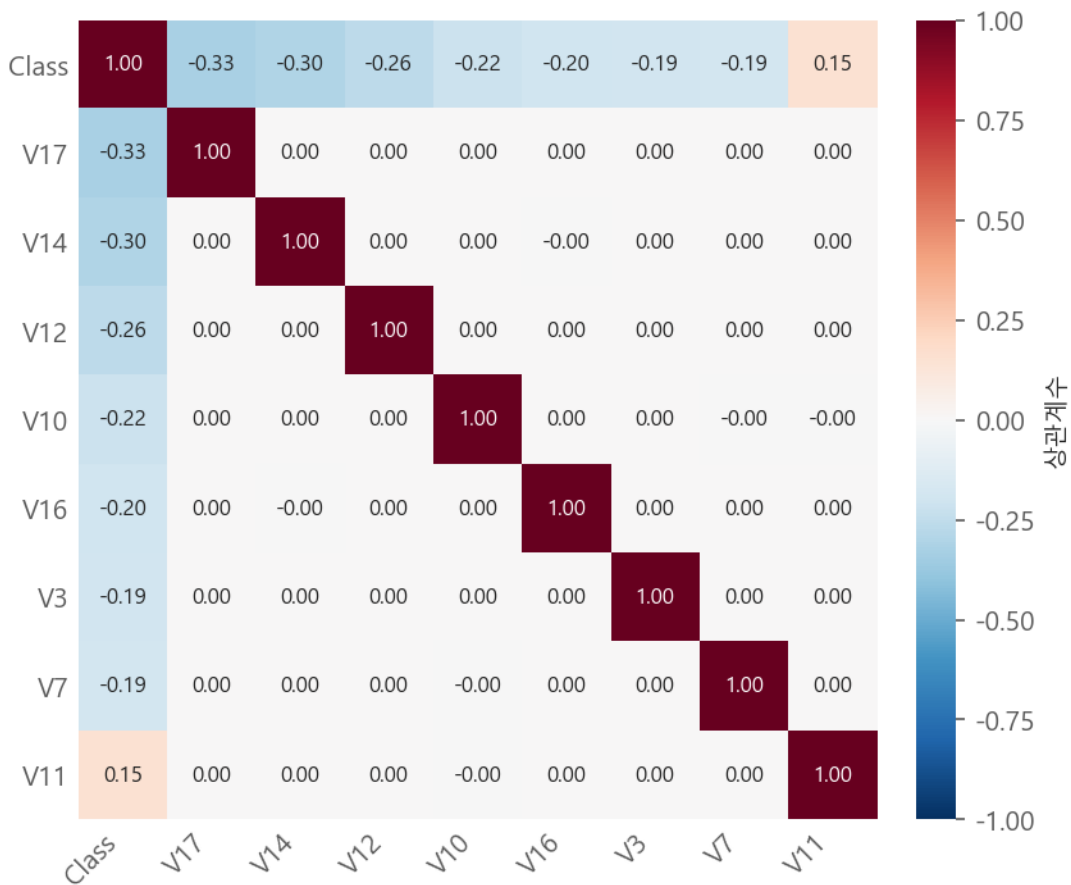


그림 2. *Class*와 상관관계가 높은 상위 8개 피처 히트맵(상관계수 절댓값 기준).

3.3 이상치 분석

이 데이터셋의 이상치는 산탄데르처럼 -999999 같은 “값으로 위장된 결측치”가 아니라, PCA 성분의 자연스러운 꼬리 끝에 있는 극단값이다. 팀은 사기(Class=1) 케이스만 대상으로, 상관관계가 높으면서 분포가 정규에 가장 가까운 **V14**를 이상치 판정 기준 피처로 삼았다.

사기 거래 492 건의 V14에 대해 IQR(사분위범위) 방식으로 하한을 구하면 다음과 같다.

- $Q1 = -9.69, Q3 = -4.28, IQR = 5.41$
- 하한선 = $Q1 - 1.5 \times IQR \approx -17.81$
- 하한선 아래로 벗어나는 사기 거래 **4건**이 이상치로 확인되었다.

정상 거래에는 이 기준을 적용하지 않았다. 정상 거래의 극단값까지 제거하면 오히려 학습에 필요한 정상 분포의 폭을 좁히게 되고, 목적이 “사기 케이스 중 학습을 방해하는 극단 4건”의 제거이기 때문이다.

3.4 변수별 이상치 시각화

사기(Class=1) 케이스만 대상으로 V14의 IQR을 구해 하한을 벗어나는 극단값을 제거하면, 아래 박스플롯처럼 사기 쪽 극단 이상치가 눈에 띄게 줄어든다.

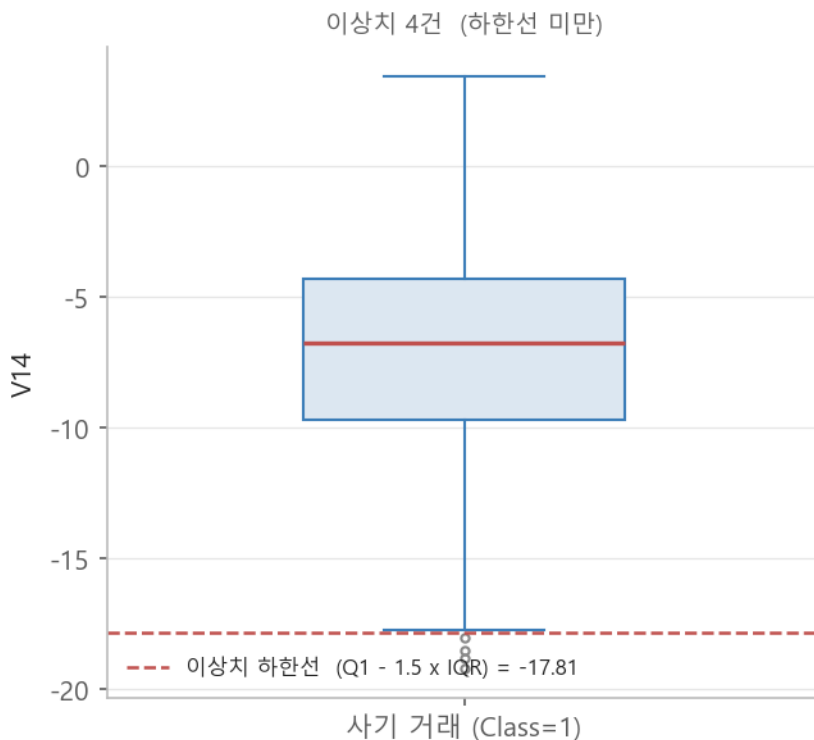


그림 3. 사기 거래(Class=1) 492 건의 V14 분포와 IQR 기반 이상치 하한선(약 -17.81). 하한선 아래 4건이 이상치로 제거된다.

3.5 중복 값 제거

컬럼 차원의 중복은 2.4에서 없음을 확인했고, 여기서는 행(row) 차원의 중복을 다룬다. `duplicated()`로 확인한 결과:

- 31 개 열이 전부 일치하는 완전 중복행: **1,081 건**(전체의 0.38%). 이 중 사기 거래가 32 건(전체 사기 492 건의 약 6.5%)으로, 정상 거래보다 중복 비율이 상대적으로 높다.
- *Time* 을 제외한 30 개 열(V1~V28·Amount·Class) 기준: **9,144 건**. *Time* 값이 초 단위로 대부분 서로 달라, 나머지 30 개 컬럼이 완전히 같은 행도 “중복 아님”으로 잘못 분류되기 때문이다. 28 개 PCA 성분이 소수점까지 일치하는 행은 우연이 아니라 같은 거래 레코드다.

완전히 동일한 값을 가진 행이 train/test 양쪽에 나뉘어 들어가면, 모델이 테스트셋 데이터를 학습 과정에서 이미 본 것과 같은 데이터 누수(data leakage)가 발생해 성능이 실제보다 부풀려진다. 실제로 이진희(XGBoost)는 중복을 제거하지 않았을 때 ROC-AUC 0.9916 이라는 높은 수치가 나온 것에 의문을 갖고 데이터 누수 가능성을 검토했고, train/test 분할 전에 전처리를 적용해 정보가 새고 있었음을 확인한 뒤 전처리 순서를 “분할 → 전처리”로 재구성했다.

Time 을 모델 입력에서 버리는 파이프라인이라면 중복 제거의 기준도 *Time* 을 뺀 30 개 열이어야 한다. 1,081 건만 제거하면 나머지 약 8,000 건은 모델이 보는 피처 벡터가 다른 행과 완전히 동일한데도 train/test 에 나뉘어 들어가 누수가 남는다. 본 프로젝트에서는 전유경·이진희가 31 개 열 전체 일치 기준(1,081 건)을, 김소현·권용우가 *Time* 제외 기준(9,144 건)을 적용했으며, 두 기준을 통일한 재실행은 향후 과제로 남긴다.

3.6 특정 컬럼 로그 변환

Amount 는 왜도가 큰 극단적 우측 편향 분포(3.1)를 가져, 값 범위가 다른 피처(대부분 -1~1 대)와 지나치게 차이 난다. 로지스틱 회귀·LightGBM 계열은 이 스케일 차이에 민감하므로, 평균·표준편차 기반의 StandardScaler 보다 중앙값·IQR 기반이라 이상치에 덜 흔들리는 **RobustScaler** 를 채택했다. 아래는 원본 분포와 RobustScaler 적용 후 분포를 로그 축에서 비교한 것이다.

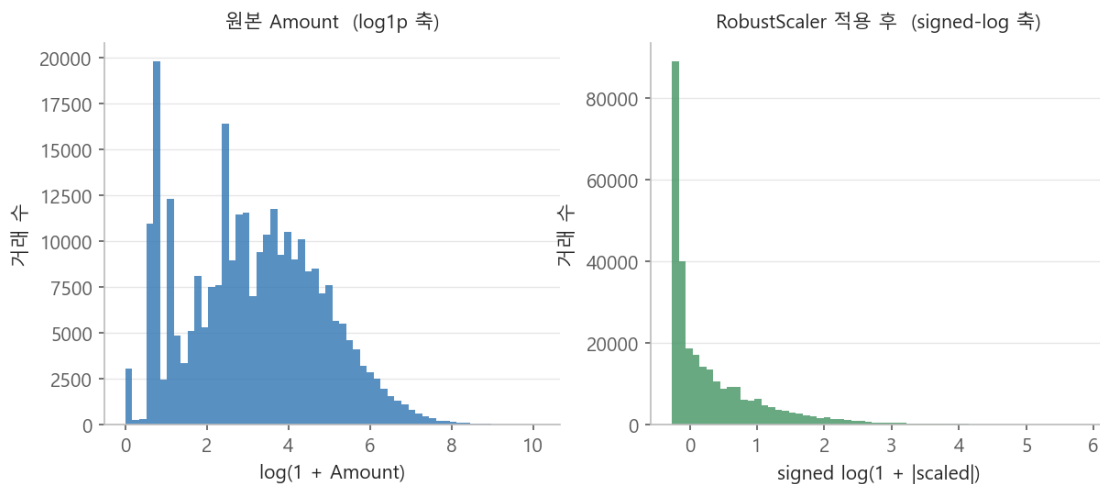


그림 4. 결제 금액(*Amount*) 원본 분포(좌)와 RobustScaler 적용 후 분포(우). 두 패널 모두 로그 축.

반면 CatBoost·XGBoost 같은 트리 기반 부스팅 모델은 임계값 분할 방식이라 단조 변환에 불변하므로, 전유경(CatBoost)은 스케일링을 아예 적용하지 않았다. 로그 변환·스케일링 적용 여부가 모델 계열에 따라 갈린 것이 이 프로젝트 전처리 전략의 특징이다(상세는 4.2·4.5).

4. 데이터 전처리

팀원 대부분이 “완전 중복행 제거 → V14 사기 케이스 이상치 제거 → Time 제거 → (선형·LightGBM 계열은) Amount RobustScaler”라는 공통 파이프라인을 기반으로, 각자의 모델 특성에 맞게 일부 단계를 가감했다.

4.1 이상치 처리

3.3 에서 정한 기준(사기 케이스 V14 IQR 하한 ≈ -17.81)에 따라, 하한 아래로 벗어나는 사기 거래 **4 건**을 제거했다. 다섯 명 전원이 이 단계를 공통으로 적용했다. 정상 거래에는 이상치 제거를 적용하지 않았다. 4 건은 전체 사기 492 건의 0.8%에 불과하지만, 학습 시 결정 경계를 극단으로 끌어당겨 나머지 사기 탐지를 방해하는 잡음이라 제거가 정당화된다.

4.2 분포 변환

*Amount*의 우측 편향을 완화하기 위해 RobustScaler 를 적용했다(그림 4). *V1~V28*은 이미 PCA 로 표준화된 성분이라 별도 변환을 하지 않았다. 로그 변환 자체는 RobustScaler 와 목적이 겹쳐 병행하지 않았다. 트리 기반 모델(CatBoost·XGBoost)은 임계값 비교만 사용하므로 이 변환을 의도적으로 생략했다 — 같은 데이터에 대해 모델 특성에 따라 전처리를 다르게 설계한 대표적 사례다.

4.3 파생 변수 생성

이 데이터셋은 *V1~V28*이 익명 PCA 성분이라 “고객별 거래 건수”, “최근 N 분 내 거래 횟수” 같은 도메인 기반 파생 변수를 만들 수 없다. 개별 거래 한 건을 29 차원 벡터로만 보는 구조라, 카드·고객 단위로 행을 묶는 집계 자체가 불가능하기 때문이다. 따라서 본 프로젝트에서는 파생 변수를 생성하지 않았다. 이 제약이 미탐지(FN)를 일정 수준 이하로 줄이지 못한 근본 원인이며(7.2), 향후 velocity 피쳐·그래프 기반 접근이 개선 방향으로 제안된다(9 장).

4.4 피쳐 정제 및 중요도 기반 가지치기

- **Time 제거**: 전원이 공통 적용. *Time*은 값 범위(0~172,792)가 다른 피쳐에 비해 지나치게 커서 로지스틱 회귀 계수 수렴에 문제(수렴 경고)를 일으킨다. 권용우의 실측 결과 *Time*을 제거하자 같은 조건에서 AUC 가 0.9456 → 0.9663 으로 개선되었다. 또한 3.5 에서 다뤘듯 *Time*을 남기면 완전 중복행 탐지도 부정확해진다.
- **L1 정규화 기반 자동 변수 선택**: 김소현은 L1(Lasso) 정규화로 계수가 0 이 되는 변수 약 10 개(*V2·V3·V7·V9·V15·V17·V18·V24* 등)를 자동 제거했고, 오히려 성능이 개선(Test ROC-AUC 0.9719 → 0.9778)되는 것을 확인했다. 저신호 변수가 정밀도를 떨어뜨리고 있었던 것으로 해석된다(그림 9).
- **명시적 상수·중복 컬럼 제거**는 해당 없음(2.4).

4.5 스케일링 및 차원 축소

- **스케일링**: *Amount*에 RobustScaler(로지스틱 회귀·LightGBM). CatBoost·XGBoost 는 미적용(4.2).
- **차원 축소**: 데이터가 이미 PCA 로 축소·익명화된 상태이므로, PCA 를 다시 적용하는 단계는 두지 않았다. *V1~V28*은 그 자체가 주성분이다.

4.6 데이터 분할

`train_test_split(test_size=0.2, stratify=y, random_state=...)`로 학습 80% / 테스트 20%로 나눴다. 사기 비율이 0.17%로 극단적이므로 `stratify`로 두 세트의 클래스 비율을 동일하게 유지하는 것이 필수다. 신뢰성 검증에는 5-Fold Stratified 교차검증을 함께 사용했다(이진희·김소현).

핵심 원칙은 **모든 전처리(스케일러 fit, 이상치 기준 계산, 중복 제거)를 분할 이후에 학습셋 기준으로만 수행한다**는 것이다. 분할 이전에 적용하면 테스트셋 정보가 학습에 새어 들어가 성능이 부풀려진다(3.5). 이진희·김소현·권용우가 각각 독립적으로 이 문제를 발견해 파이프라인을 “분할 → 전처리”로 바로잡았다.

5. 분석방법(Analysis method)

5.1 전처리 필요성 및 과정 개요

이 데이터셋에서 전처리가 프로젝트 전체의 쟁점이 된 이유는 두 가지다. 첫째, 피처가 익명 PCA 성분이라 도메인 지식을 넣을 여지가 없어, 성능을 끌어올릴 레버가 “데이터를 어떻게 손질하느냐”에 집중됐다. 둘째, 사기 비율이 0.17%로 극단적이라 중복행 한 줄, 이상치 몇 건 같은 작은 결정이 지표를 눈에 띄게 흔들었다.

팀 공통 파이프라인은 **완전 중복행 제거 → V14 사기 케이스 이상치 제거 → Time 제거 → (선형·LightGBM 계열은) Amount RobustScaler** 다. 여기에 도달하는 과정은 팀원마다 달랐는데, 다섯 명 모두 모델 튜닝에 앞서 전처리 결정을 A/B 로 비교했다.

담당 (모델)	전처리 비교·검증 방식
전유경 (CatBoost)	전처리 결정 3 건(중복 제거 / Time+Amount 제거 / Amount=0 제거)을 ROC-AUC-PR-AUC 로 A/B 비교. 이후 핵심 실험·이상 탐지·K-Fold 29 건을 같은 조건으로 전수 재검증. 스케일링은 트리 모델이라 배제.
권용우 (Logistic Regression)	원본 / 스케일링 / 이상치 제거 / 중복 제거를 $2 \times 2 \times 2 = 8$ 개 조합으로 전수 비교해 최적안 선정. 이후 “Time 을 먼저 제거해야 중복행이 정확히 잡힌다”를 발견하고 8 개 조합을 재실행.
김소현 (Logistic Regression)	class_weight vs SMOTE, L1 vs L2, ROC-AUC 기준 vs Average Precision 기준 C 재탐색 — 매 단계 두 가지 이상을 비교해 승자 채택.
이혜현 (LightGBM)	Baseline 대비 클래스 가중치·균형 샘플 앙상블·K-Fold 앙상블·Early Stopping·임계값 최적화·Hyperopt 등 7 가지 실험을 비교.
이진희 (XGBoost)	전처리 순서(분할 전/후)를 바꿔 데이터 누수를 직접 검증. 균형 샘플 앙상블·5-Fold 교차검증으로 신뢰성 재확인.

전처리 결정 하나하나가 지표에 미치는 영향은 6.4 의 그림 10(권용우 8 조합)에서 수치로 확인된다. 8 개 조합 모두 Test ROC-AUC 는 0.963~0.969 로 좁은 구간에 몰려 있으나 F1-Score 는 0.699~0.751 로 더 크게 갈렸고, ROC-AUC 가 가장 높은 조합과 F1 이 가장 높은 조합이 서로 달랐다 — 전처리 선택도 ROC-AUC 보다 F1·PR-AUC 로 판단해야 한다는 근거다.

5.2 사용 모델 설명

모델	담당	선택 이유
CatBoost	전유경	트리 기반 부스팅. 별도의 스케일링 없이 안정적으로 학습되고, oblivious tree-ordered boosting 으로 과적합 방지에 강점이 있다.
LightGBM	이혜현	리프 중심(leaf-wise) 성장으로 대용량 데이터에서 빠르고, <i>scale_pos_weight</i> 임계값 조정 등 불균형 대응 옵션이 풍부하다.
Logistic Regression	김소현 · 권용우	사기 확률을 함께 제시할 수 있고 계수로 해석이 가능하다. L1/L2 정규화로 과적합 억제·변수 선택, <i>class_weight</i> 로 불균형에 직접 대응할 수 있다. 두 명이 서로 다른 튜닝 경로로 접근해 결과를 교차 검증했다.
XGBoost	이진희	대표적 부스팅 모델. 사기:정상을 1:10 으로 나눈 여러 균형 샘플을 각각 학습해 예측 확률을 평균 내는 앙상블 방식으로 불균형에 대응했다.

5.3 하이퍼파라미터 최적화 기법

담당 (모델)	사용한 기법
CatBoost (전유경)	수동 그리드 — 클래스 가중치·판정 임계값· <i>eval_metric</i> 등 8 종 조합 비교
LightGBM (이혜현)	수동 실험 7 종 + Hyperopt(TPE, 50 회 탐색)
Logistic Regression (김소현)	GridSearchCV(C, roc_auc 기준) → Average Precision 기준 C 재탐색
Logistic Regression (권용우)	GridSearchCV vs Hyperopt(TPE, 15 회) 비교 + SMOTE + 임계값 최적화
XGBoost (이진희)	균형 샘플 앙상블(1:10) + Hyperopt(TPE, 100 회, 5-Fold CV 목적함수)

세 팀원(이혜현·이진희·권용우)이 수동/격자 탐색과 Hyperopt 를 나란히 비교했고, 그 결과는 7.3 에서 정리한다.

5.4 실행환경

- **언어·환경:** Python 3.10+, Jupyter Notebook (VS Code)
- **주요 라이브러리:** pandas, numpy, scikit-learn, xgboost(3.4), lightgbm, catboost(1.2), hyperopt(0.3), imbalanced-learn, matplotlib
- **재현성:** *train_test_split* 모델·교차검증에 *random_state*(대체로 0 또는 42)를 고정. 데이터는 팀 전원이 동일한 *creditcard.csv* 를 사용.

5.5 코드 재사용 및 함수화

팀원 5 명의 노트북을 전수 조사한 결과, 명시적으로 *def*로 정의된 공통 함수는 많지 않았다. 사기 케이스 IQR 이상치 인덱스를 구하는 *get_outlier(df, column, weight)*가 이해현·이진희·권용우 노트북에 거의 같은 형태로 등장했고, 혼동행렬과 주요 지표를 한 번에 출력하는 *get_clf_eval* 계열 리포팅 함수가 여러 노트북에 반복됐다. Hyperopt 를 쓴 세 명(이해현·이진희·권용우)의 *objective(params)* 함수도 세부 탐색 공간만 다를 뿐 "CV 점수 계산 → 음수 손실 반환 → *fmin*+TPE" 구조를 공유했다.

나머지 완전 중복행 제거·*Time* 컬럼 제거·*Amount* 스케일링·균형 샘플 앙상블 학습은 대부분 노트북 셀에 인라인으로 반복 작성되었다. 즉 "공통 함수"라기보다 "유사한 패턴을 각자 독립적으로 재구현"한 경우가 많았고, 이는 10 장 회고의 개선점("공통 전처리·평가 파이프라인을 프로젝트 초기에 표준화")과 직접 연결된다. 실제로 반복된 패턴을 정리해 재사용 가능한 형태로 다듬은 코드는 12 장 부록에 실었으며, 팀 노트북 5 개를 하나로 합친 통합본은 *02_classification_credit_card/팀_통합_creditcard.ipynb*에 있다.

6. 모델링

6.1 CatBoost (담당: 전유경)

CatBoost 는 트리 기반 부스팅 모델로 별도의 스케일링 없이도 안정적으로 학습되고 과적합 방지(oblivious tree, ordered boosting)에 강점이 있어 채택되었다. 전처리는 5.1 의 A/B 비교(중복 제거·Amount 관련 3 건)를 거쳐 “중복행 제거 → V14 이상치 제거 → Time 제거, 스케일링 없음”으로 확정했다. 기본 설정(*eval_metric='AUC'*)으로 학습한 기준선은 테스트셋 56,744 건(사기 93 건) 중 미탐지(FN) 13 건, 오탐(FP) 3 건을 기록했다. 이후 미탐지(FN)를 줄이기 위해 클래스 가중치·판정 임계값·평가지표 변경·트리 구조 튜닝 등 8 가지 방법을 실험했다.

실험	방법	FN	FP	Prec.	F1	ROC-AUC	상태
A (기준선)	<i>eval_metric='AUC'</i>	13	3	0.964	0.909	0.9759	기준
B	가중치=원비율(607.5)	9	977	0.079	0.146	0.9763	부작용 큼
C	가중치=SqrtBalanced	12	14	0.853	0.862	0.9832	절충
E (스윙 최선)	<i>scale_pos_weight</i> 스윙(5~100)	12	12	0.871	0.871	0.9717	한계
F (채택)	<i>eval_metric='F1'</i>	12	3	0.964	0.915	0.9743	채택
G	<i>depth=8, lr=0.05,</i> <i>iter=2000</i>	13	14	0.851	0.856	0.9748	효과 없음
H	<i>F1 + SqrtBalanced</i>	12	4	0.953	0.910	0.9785	시너지 없음

※ 실험 라벨 A~H 는 초기 시도 순서를 따른다. B 와 D 는 클래스 가중치를 원비율(607.5)로 적용한 동일 설정이라 결과가 같아 B 한 행으로 합쳤다(그래서 표에 D 행은 없다).

클래스 가중치를 손실 함수에 직접 반영하는 방식(B)은 FN 을 4 건 줄이는 대신 FP 를 977 건 만들어내 알림 피로(alert fatigue) 문제로 실무 투입이 불가능했다. *scale_pos_weight* 를 5~100 구간에서 스윙한 E 안은 그 방법 중에서는 가장 나았지만 FP 를 12 건까지밖에 못 줄여 F 안(FP 3 건)에는 미치지 못했다. 최종적으로 채택된 F 안은 모델의 학습 방식이나 데이터를 전혀 바꾸지 않고, CatBoost 가 학습 과정에서 저장하는 여러 스냅샷 중 “어느 iteration 을 최종 모델로 볼 것인가”의 기준만 AUC 에서 F1 로 바꾼 것으로, FP 를 3 건으로 그대로 유지하면서 FN 만 13 → 12 건으로 줄였다.

최종 채택 모델 성능: Confusion Matrix $\begin{bmatrix} 56648 & 3 \\ 12 & 81 \end{bmatrix}$ (그림 5), Precision 0.964 / Recall 0.871 / F1 0.915 / ROC-AUC 0.9743 / **PR-AUC 0.8898**(전유경이 재검증한 29 개 실험 중 최고 — 7.2 참조).

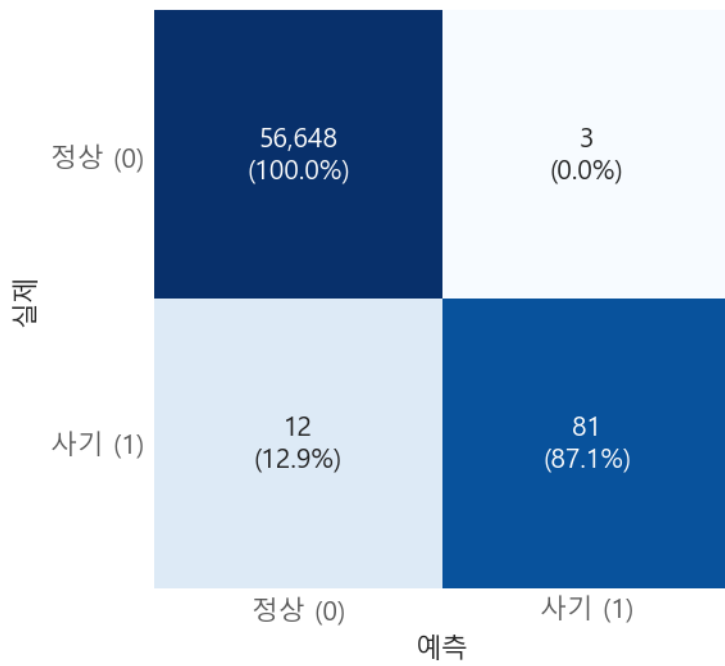


그림 5. CatBoost(F 안) Confusion Matrix. 색은 실제 클래스 내 비율, 괄호 안 수치도 같은 비율이다.

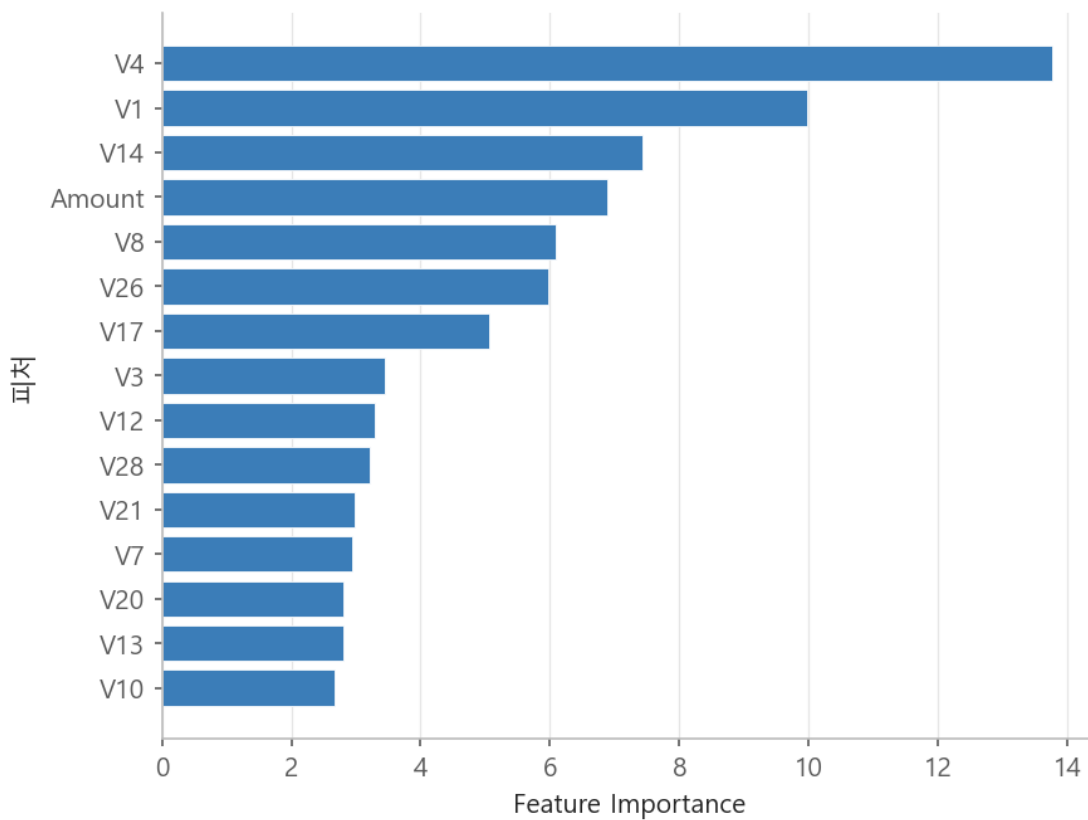


그림 6. CatBoost Feature Importance 상위 15 개.

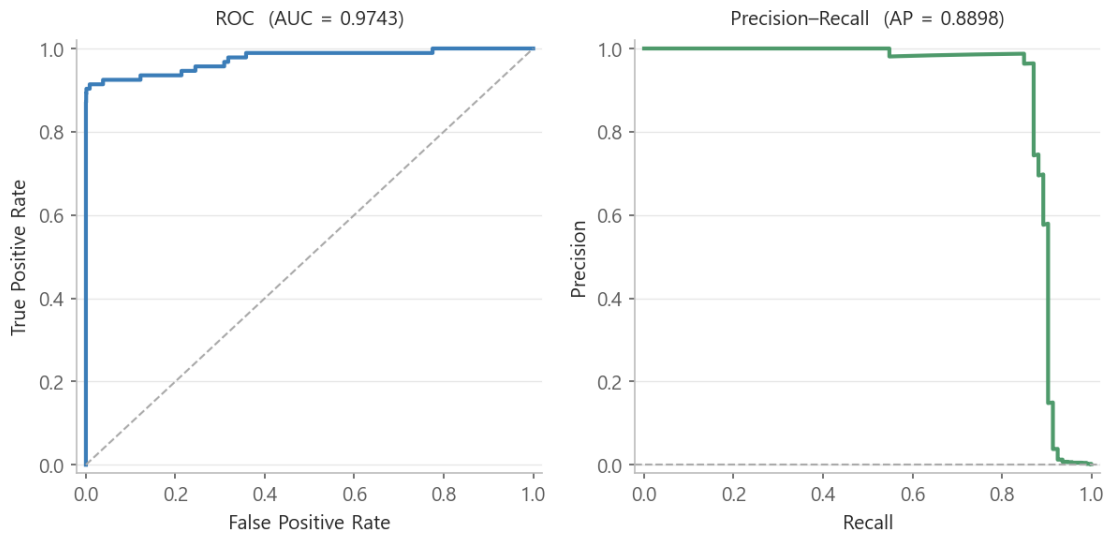


그림 7. CatBoost ROC Curve(좌)와 Precision-Recall Curve(우).

세 그림이 각각 짚어 주는 점은 다음과 같다. 혼동 행렬(그림 5)에서 오탐(FP)은 3 건으로 억제됐고 남은 오차는 사기 93 건 중 12 건을 놓친 미탐지(FN)에 몰려 있다. 피처 중요도(그림 6)는 V4·V1·V14·Amount 순으로 높게 나왔으나, V1~V28 이 익명 PCA 성분이라(2.2) “어떤 거래 속성이 사기와 연결되는가”까지는 알 수 없고 이상치 기준이던 V14 가 중요도 상위에도 든다는 통계적 확인에 그친다. ROC 곡선(그림 7 좌)은 좌상단에 바짝 붙어 AUC 0.9743 을 기록하지만, Precision-Recall 곡선(그림 7 우)은 recall 이 0.9 부근을 넘어서며 precision 이 가파르게 무너진다 — 극단적 불균형에서 두 곡선이 서로 다른 이야기를 한다는 점은 9 장에서 다룬다.

F 안 채택 이후에도 남은 미탐지(FN) 12 건이 튜닝만으로 더 줄지 않는 이유는 7.2 에서 별도로 분석한다.

6.2 LightGBM (담당: 이혜현)

전처리(중복치·Time 제거 + V14 이상치 4 건 제거 + Amount RobustScaler)를 고정한 뒤, LightGBM 기준선 대비 7 가지 개선 실험을 비교했다.

실험	ROC-AUC	PR-AUC	F1-Score	비고
Baseline	0.9681	0.8414	0.8757	기준
클래스 가중치(scale_pos_weight)	0.9667	0.8345	0.8623	성능 소폭 하락
균형 샘플 앙상블(1:10, 60 개 모델)	0.9705	0.8412	0.6556	ROC-AUC 는 최고, F1 은 급락
K-Fold 앙상블(5-Fold)	0.8684	0.5663	0.6014	데이터 크기 대비 부적합
Early Stopping	0.7256	0.3396	0.5600	검증셋 분리로 학습량 부족, 큰 폭 하락
임계값 최적화(F1 기준)	0.9681	0.8414	0.8824	재학습 없이 F1 최고
Hyperopt 최적화(50 회 탐색)	0.9687	0.8421	0.8690	ROC-AUC-PR- AUC 동시 최고

균형 샘플 앙상블은 ROC-AUC 만 보면 가장 좋아 보이지만 F1 은 0.88 → 0.66 으로 크게 떨어져, 단일 지표만으로 판단하면 안 된다는 것을 보여주는 사례가 되었다. K-Fold 앙상블과 Early Stopping 은 이 데이터 규모(사기 375 건 내외의 학습 데이터)에서는 fold 당 사기 샘플이 지나치게 적어지거나 학습 데이터가 줄어드는 부작용이 더 커, 오히려 성능이 크게 후퇴했다. 최종적으로 전반적 순위 지표(ROC-AUC-PR-AUC)에서는 Hyperopt 튜닝안을, 실무에서 고정 임계값(0.5)을 그대로 쓸 경우에는 임계값 최적화(F1 0.8824)를 상황별 권장안으로 병기했다.

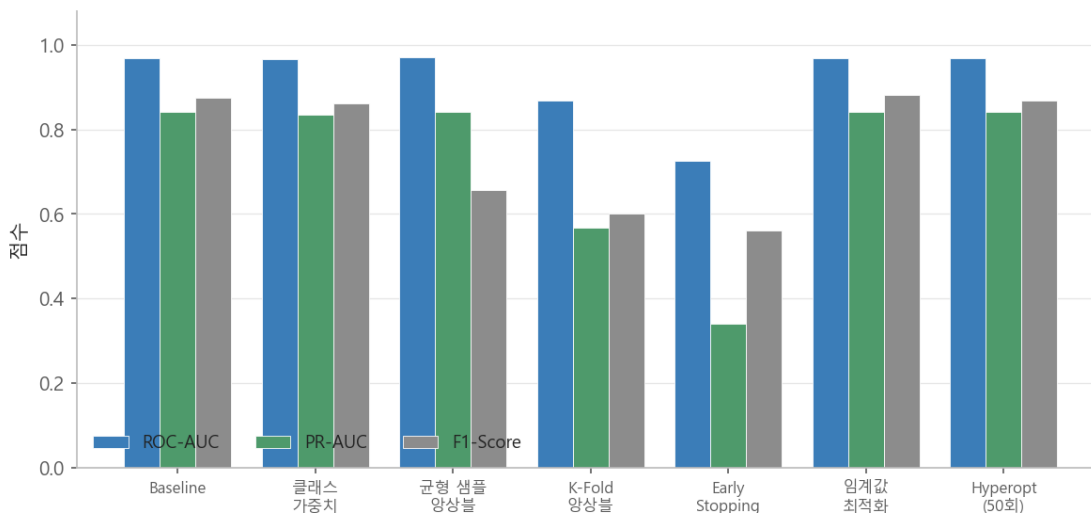


그림 8. 이해현의 LightGBM 7 가지 개선 실험별 ROC-AUC-PR-AUC-F1-Score. 균형 샘플 앙상블-K-Fold 앙상블-Early Stopping 은 F1-PR-AUC 가 크게 무너지며, 특히 균형 샘플 앙상블은 ROC-AUC 만 높게 유지돼 지표 선택의 함정을 보여준다.

6.3 Logistic Regression (담당: 김소현)

김소현은 `class_weight="balanced"`와 SMOTE 를 StratifiedKFold(5) 교차검증으로 비교해 `class_weight(CV ROC-AUC 0.9788)`가 SMOTE(0.9774)를 근소하게 앞서는 것을 확인하고 이를 최종 방식으로 채택했다. 이후 GridSearchCV 로 규제 강도(C)를 튜닝(Test ROC-AUC 0.9719)하고, L1(Lasso) 정규화로 계수가 0 이 되는 변수 약 10 개(V2·V3·V7·V9·V15·V17·V18·V24 등)를 자동 제거하면서 오히려 성능이 개선(Test ROC-AUC 0.9778)되는 것을 확인했다. 마지막으로 평가지표를 ROC-AUC 에서 PR-AUC(Average Precision)로 바꿔 C 를 재탐색하자 PR-AUC 가 0.6217 → 0.6606 으로 개선되었다(이 재탐색으로 Test ROC-AUC 는 0.9778 → 0.9701 로 소폭 내렸으나, 주 지표인 PR-AUC 를 우선한 선택이다). 추가로 이혜현의 LightGBM 실험을 자신의 파이프라인에서 재현하고 LR+LGBM 스택킹을 검증했으나, 스택킹(Test AP 0.8145)은 LGBM 단독(Test AP 0.8255)을 넘지 못했다 — 두 모델의 실력 격차가 클 때는 다양성보다 약한 모델이 발목을 잡는 효과가 더 크다는 것을 실증적으로 확인한 사례다.

김소현 최종 채택: L1 + Average Precision 기준 C 재탐색 / Test ROC-AUC 0.9701 / PR-AUC 0.6606.

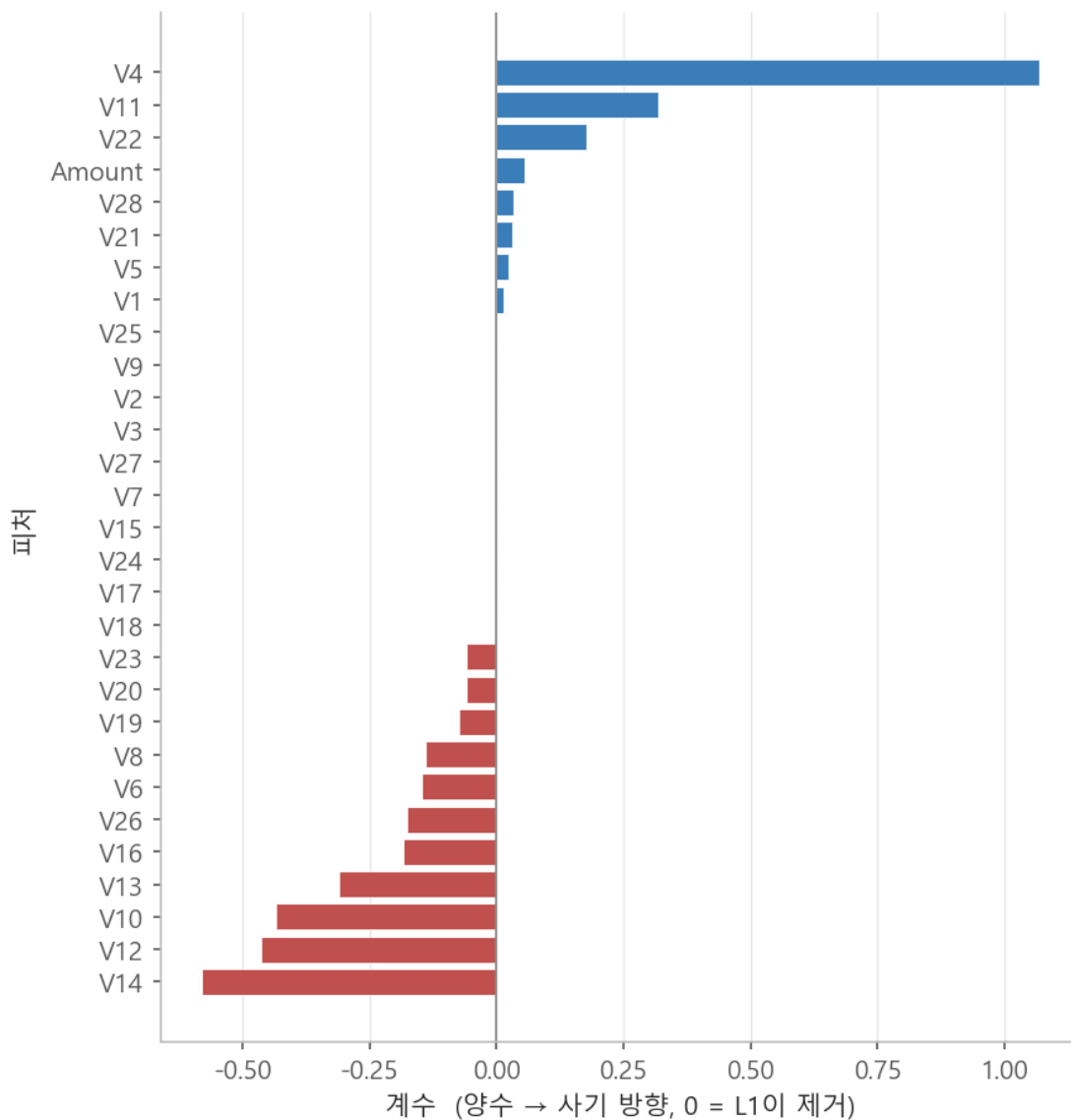


그림 9. 김소현의 Logistic Regression(L1 정규화) 피쳐별 계수. 계수가 0 에 붙은 변수는 L1 이 제거한 저신호 변수다(양수 → 사기 방향).

6.4 Logistic Regression (담당: 권용우)

권용우는 원본/스케일링/이상치 제거/중복 제거의 8 가지 조합을 전수 비교하는 것으로 시작해, Time 제거가 필요하다는 것을 발견한 뒤 전체 실험을 재실행했다. 8 개 조합 모두 Test ROC-AUC 는 0.963~0.969 로 좁은 구간에 몰려 있으나 F1-Score 는 0.699~0.751 로 더 크게 갈렸고, ROC-AUC 가 가장 높은 조합(원본+스케일링)과 F1 이 가장 높은 조합(이상치 제거+스케일링)이 서로 달랐다.

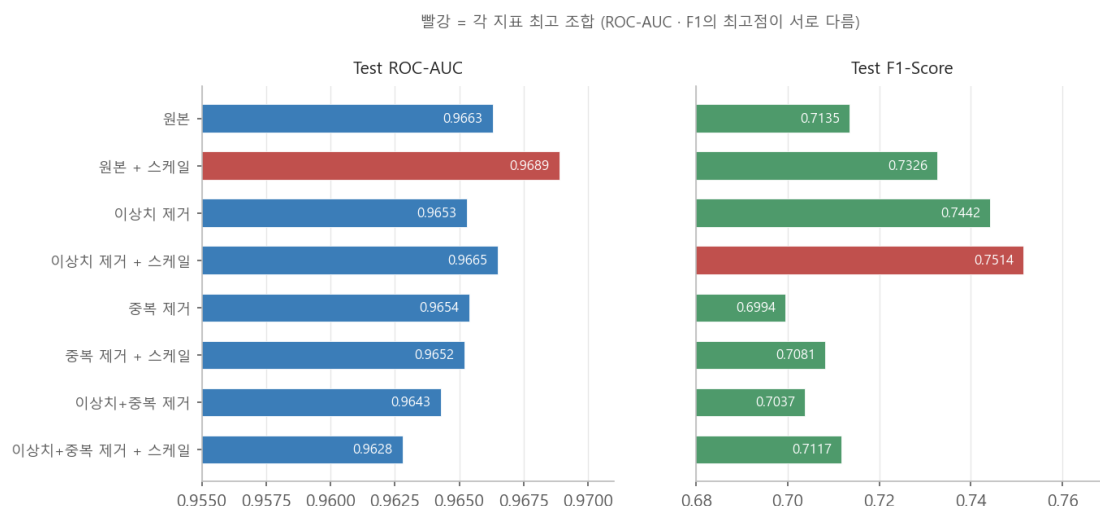


그림 10. 권용우의 Logistic Regression 8 가지 전처리 조합별 Test ROC-AUC(좌)와 F1-Score(우). 빨간 막대는 각 지표에서 가장 높은 조합. (Time 제거 후, 중복 9,144 건 기준)

이후 GridSearchCV(베스트 파라미터 $C=0.1$, $class_weight=None$, $penalty='l1'$, CV AUC 0.979)와 Hyperopt(TPE, 15 회 탐색, $C=0.878$, $penalty='l2'$, CV AUC 0.9771)를 동일한 데이터·탐색 범위에서 비교했다. 두 방법의 테스트 AUC 는 각각 0.9683, 0.9606 으로, L1 을 고른 GridSearchCV 가 L2 를 고른 Hyperopt 를 약 0.008 앞섰다 — 이 데이터에서는 탐색 방법(격자 vs 베이지안)보다 규제 종류(L1 vs L2) 선택이 결과를 갈랐고, Hyperopt 가 15 회 탐색 안에서 L1 영역을 충분히 훑지 못한 것으로 보인다. Hyperopt 최적 파라미터에 SMOTE 를 추가로 적용하고 F1 최적 임계값까지 탐색한 최종안은 F1 0.7771, AUC 0.9663 으로, 지금까지 시도한 모든 조합 중 F1 최고값을 기록했다.

권용우 최종 채택: Hyperopt($C=0.878$, l2) + SMOTE + 임계값 최적화 / Test ROC-AUC 0.9663 / F1 0.7771.

김소현·권용우 두 사람 모두 $class_weight$ /SMOTE 로 불균형을 보정하는 것이 ROC-AUC 보다 PR-AUC(또는 F1) 개선에 더 직접적으로 기여한다는 점, 그리고 로지스틱 회귀에서는 규제 강도(C)와 판정 임계값을 함께 조정해야 실질적인 성능 개선이 나타난다는 점에서 같은 결론에 도달했다.

6.5 XGBoost (담당: 이진희)

기본 전처리(V14 이상치 제거, RobustScaler, Time-Amount 원본 제거) 위에서, 사기:정상을 1:10 비율로 나눈 여러 균형 샘플을 만들어 각각 XGBoost 를 학습시킨 뒤 예측 확률을 평균 내는 앙상블 방식을 채택했다. 여러 모델을 빠르게 스크리닝하던 초기 단계에서 로지스틱 회귀만으로도 ROC-AUC 0.9898 이 나왔고, 트리 모델로 넘어가 XGBoost 에서 ROC-AUC 0.9916 까지 올랐으나, 지나치게 높은 수치에 의문을 갖고 데이터 누수 여부를 점검한 결과 train/test 분할 이전에 전처리(스케일러 fit, 이상치 기준 계산)를 적용한 것이 원인이었음을

확인했다. 이후 "분할 → 전처리" 순서로 파이프라인을 재구성하고 중복행까지 다시 제거한 뒤 XGBoost 를 재학습해 ROC-AUC 0.9773, PR-AUC 0.8150 을 얻었다.

신뢰성을 추가로 검증하기 위해 5-Fold Stratified 교차검증을 수행한 결과는 다음과 같다.

Fold	1	2	3	4	5	평균
ROC-AUC	0.9920	0.9838	0.9838	0.9764	0.9836	0.9839
PR-AUC(AP)	0.8408	0.8472	0.8264	0.8360	0.8499	0.8401

단일 분할 기준(ROC-AUC 0.9773, PR-AUC 0.8150)보다 5-Fold 평균(ROC-AUC 0.9839, PR-AUC 0.8401)이 두 지표 모두에서 오히려 안정적으로 높게 유지되어, 앞서 재구성한 파이프라인이 특정 분할에 우연히 잘 맞은 결과가 아니라는 것을 확인했다. 단일 분할 PR-AUC(0.8150)는 5-Fold 평균보다 약 0.025 낮아, 7.1 의 종합 비교에는 분할 편차가 작은 5-Fold 평균값을 사용했다. 이후 하이퍼파라미터 튜닝(Hyperopt, *max_depth* *min_child_weight* *colsample_bytree* *subsample* *learning_rate* 탐색, 100 회)을 시도했으나 오히려 CV ROC-AUC 가 소폭 하락(0.9728)해, 이 조합에서는 기본 파라미터의 앙상블 구조 자체가 이미 충분히 효과적이었다는 결론에 도달했다.

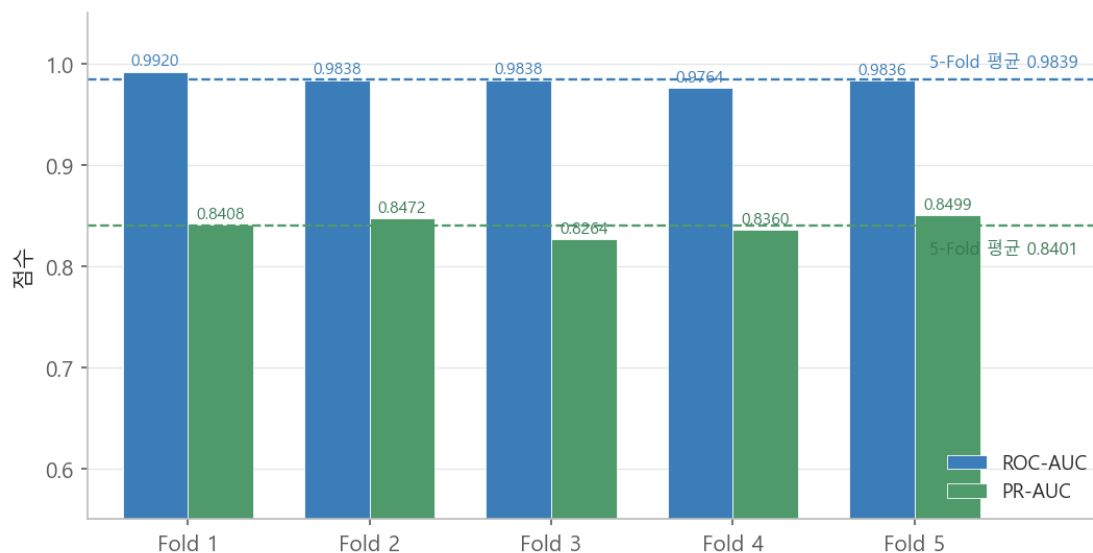


그림 11. 이진희의 XGBoost 5-Fold 교차검증 fold 별 ROC-AUC-PR-AUC. 다섯 fold 의 편차가 작고(ROC-AUC 표준편차 0.005, PR-AUC 0.008) 평균이 단일 분할값보다 높거나 같아, 재구성한 파이프라인이 특정 분할에 우연히 맞은 결과가 아님을 뒷받침한다.

7. 결과

7.1 모델별 최종 성능 비교

팀원마다 전처리와 검증 조건(단일 분할 vs 5-Fold, 채택 지표)이 서로 달라, 아래 표는 모델 간 순위를 매기지 않는다. 각자의 파이프라인 안에서 최종 채택된 모델의 성능으로만 읽어야 한다.

모델	담당	PR-AUC / AP	ROC-AUC	F1-Score	측정 조건
CatBoost	전유경	0.8898	0.9743	0.915	단일 분할
XGBoost	이진희	0.8401	0.9839	-	5-Fold 평균
LightGBM	이혜현	0.8421	0.9687	0.8690 / 0.8824	단일 분할
Logistic Regression	권용우	-	0.9663	0.7771	단일 분할
Logistic Regression	김소현	0.6606	0.9701	-	단일 분할

※ LightGBM F1 은 Hyperopt 튜닝안(0.8690) / 임계값 최적화안(0.8824)을 병기했다(6.2). “-”는 해당 팀원이 그 지표를 최종 보고에 포함하지 않은 경우다.

위 표의 수치는 서로 다른 조건(단일 분할 vs 5-Fold, 채택 지표)에서 측정된 값이므로 모델 간 절대 비교가 아니라 각자의 파이프라인 안에서 “무엇이 성능을 끌어올렸는가”를 읽는 자료다. 다만 PR-AUC 기준으로 트리 기반 부스팅 모델(CatBoost·XGBoost·LightGBM)이 로지스틱 회귀보다 앞서는 경향은 다섯 팀원의 실험 전반에서 공통으로 확인된다.

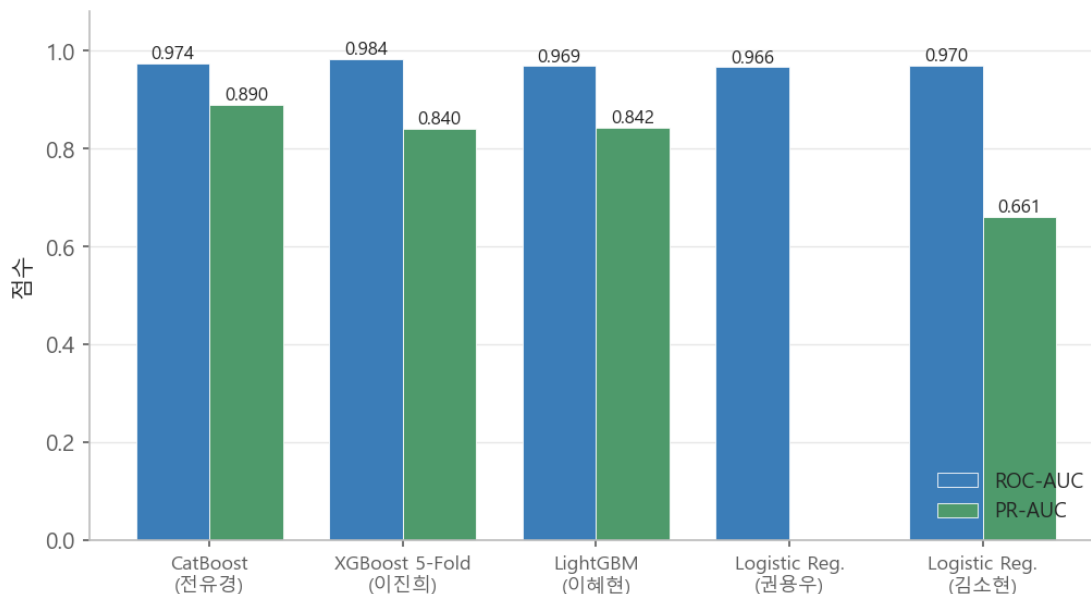


그림 12. 모델별 최종 채택 성능(ROC-AUC vs PR-AUC). PR-AUC 기준으로 트리 기반 부스팅 3 종이 로지스틱 회귀보다 높게 나타난다.

그림 12 에서 보듯 트리 기반 부스팅 모델이 로지스틱 회귀보다 PR-AUC 기준으로 뚜렷이 우수한 것은, V1~V28 이 이미 PCA 로 변환된 비선형적 조합 정보를 담고 있어 선형 결합에 의존하는 로지스틱 회귀보다

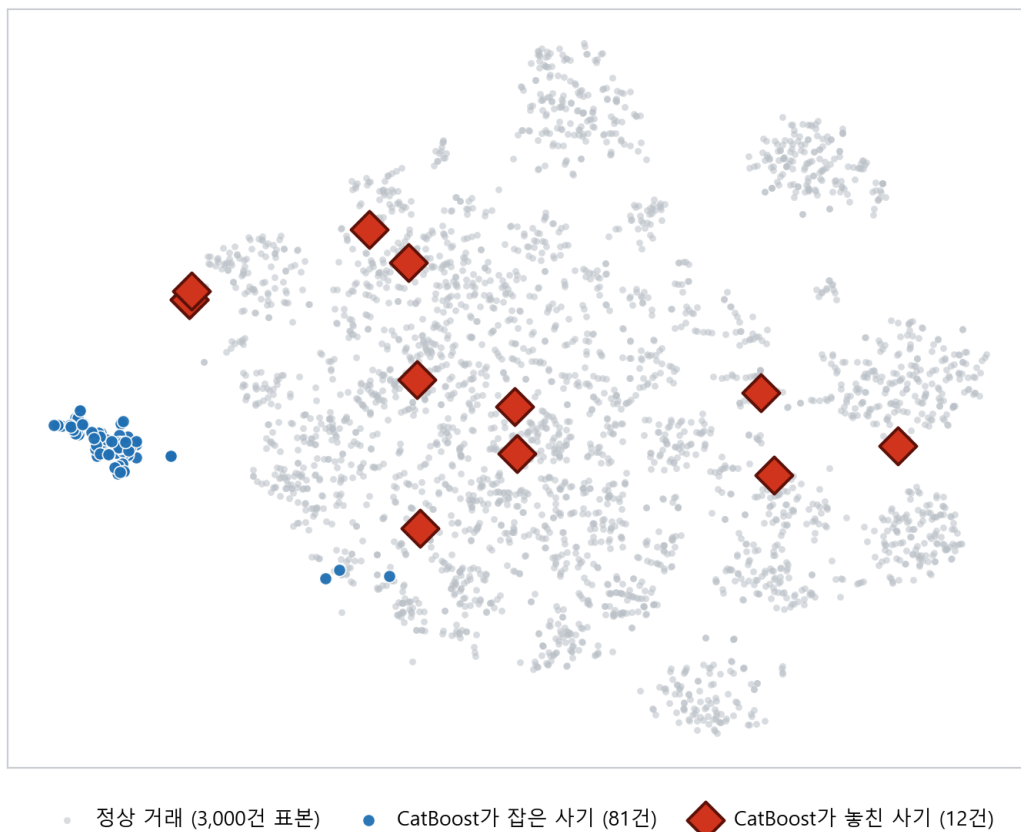
트리 기반 모델이 이 데이터의 비선형 패턴을 더 잘 포착하기 때문으로 해석된다. 다만 로지스틱 회귀는 “이 거래가 사기일 확률 92%”처럼 계수 기반으로 해석 가능한 예측을 제공하고 학습·추론 속도가 빨라, 트리 모델과는 다른 실무적 장점을 가진다.

7.2 최고 성능 모델 분석 — CatBoost 와 미탐지(FN)의 구조적 한계

PR-AUC 기준 최상위였던 CatBoost(F 안, PR-AUC 0.8898)에 대해, F 안을 채택한 뒤에도 남은 미탐지(FN) 12 건이 클래스 가중치·임계값·트리 구조를 아무리 조정해도 더 줄지 않은 이유를 규명했다.

먼저 최근접 이웃 거리를 측정한 결과, FN 12 건이 정상 거래와 떨어진 정도(중앙값 거리 2.49)는 CatBoost 가 실제로 잡아낸 사기 81 건(중앙값 거리 7.77)의 약 1/3 에 불과했다. 즉 놓친 12 건은 CatBoost 가 잡아낸 사기보다 정상 거래에 훨씬 가깝게 분포한다. 이어 Isolation Forest·LOF·One-Class SVM 등 라벨을 전혀 보지 않는 이상 탐지 기법 3 종을 CatBoost 예측과 결합했으나, 이 12 건 중 단 한 건도 추가로 잡아내지 못했다.

테스트셋 피쳐 공간 t-SNE 임베딩



파란 점(CatBoost가 잡은 사기)은 정상 거래 점군 가장자리·바깥쪽에 뚜렷한 무리를 이루는 반면, 빨간 다이아몬드(놓친 12건)는 대부분 정상 거래(회색) 점들 사이 한가운데 흩어져 있다 — 지도 위에서도 “이웃”이 온통 정상 거래뿐이다.

그림 13. CatBoost 테스트셋 피쳐 공간 t-SNE 임베딩. 파란 점(잡은 사기 81 건)은 정상 거래 점군 가장자리에 무리를 이루지만, 빨간 다이아몬드(놓친 12 건)는 정상 거래 점들 사이 한가운데 흩어져 있다.

학습 방식이 전혀 다른 지도학습(CatBoost)과 비지도 이상 탐지(3 종)가 정확히 같은 12 건에서 막혔다는 것은, 이 문제가 특정 모델의 튜닝 부족이 아니라 “거래 한 건을 29 차원 벡터로 보고 그 점의 위치만으로 판단한다”는 현재 피쳐 표현 자체의 정보량 상한에 부딪힌 결과로 해석된다.

카드·고객 단위의 맥락(velocity) 정보가 없는 완전 익명 데이터의 구조적 한계로, 모델을 더 튜닝하거나 다른 알고리즘으로 교체하는 것만으로는 해결되지 않는다.

한편 이 분석 과정에서 8 가지 핵심 실험에 전처리 A/B 테스트·이상 탐지 3 종·K-Fold 검증까지 더한 29 개 실험 전체를 PR-AUC 와 Confusion Matrix 기준으로 재검증했다. 이상 탐지 3 종은 ROC-AUC 가 0.95 대로 준수해 보이지만 PR-AUC 는 0.13~0.39 로 바닥에 붙어 있었고, F 안은 이 29 개 중 PR-AUC 0.8898 로 최상위였다. 다만 K-Fold 교차검증에서는 이 우위가 절대적이지 않아, F1 채택은 “결정 임계값 0.5 에서의 FN/FP”라는 실무적으로 중요한 지점을 최적화하는 선택이지 모든 임계값을 아우르는 전반적 랭킹 성능을 항상 최고로 만드는 선택은 아니다.

7.3 하이퍼파라미터 최적화 효과 분석

세 팀원(이혜현·이진희·권용우)이 각자의 모델에서 수동/격자 탐색과 Hyperopt(베이지안 최적화)를 나란히 비교했다. 지표는 모두 ROC-AUC 지만 검증 방식이 팀원마다 다르므로, 표는 각 행 안에서 수동 대 Hyperopt 만 비교한다(행 간 세로 비교는 의미가 없다).

담당 (모델)	검증 방식	수동 / GridSearchCV	Hyperopt	결과
이혜현 (LightGBM)	단일 분할 Test	기준선 ROC-AUC 0.9681	ROC-AUC 0.9687 (50 회)	Hyperopt 근소 우세, 탐색 공간 설계가 성능을 좌우
권용우 (Logistic Regression)	단일 분할 Test	GridSearchCV ROC-AUC 0.9683 (L1)	ROC-AUC 0.9606 (15 회, L2)	L1 을 고른 격자 탐색이 L2 를 고른 Hyperopt 를 약 0.008 앞섬
이진희 (XGBoost)	5-Fold CV	기본 파라미터 ROC-AUC 0.9839	ROC-AUC 0.9728 (100 회)	Hyperopt 오히려 하락, 기본 설정이 이미 충분

세 사례를 종합하면 서로 다른 세 검증 방식 어디에서도 Hyperopt 가 수동/격자 탐색을 압도적으로 앞서는 경우는 없었다. 데이터가 작고 불균형이 심할수록 탐색 공간을 단순하게 설계하는 것이 과도하게 넓은 공간을 베이지안 방식으로 탐색하는 것보다 효과적이었으며, 하이퍼파라미터 튜닝보다 판정 임계값 조정이나 평가지표(eval_metric) 선택 같은 “재학습이 필요 없는” 조정이 더 확실한 개선을 가져온 경우가 많았다.

7.4 피쳐 중요도 공통 인사이트

접근 방식이 다른 두 갈래(트리 중요도, 선형 계수)에서 반복적으로 상위에 오른 피쳐가 있다.

- **CatBoost feature importance**(그림 6): V4·V1·V14·Amount·V8 순.
- **Logistic Regression L1 계수**(그림 9, 김소현): 계수 절댓값 상위에 V14·V12·V10·V4 등이 위치하고, V2·V3·V7·V9·V15 등은 0 으로 눌러 제거됨.
- **Class 상관 상위**(그림 2): V17·V14·V12·V10·V16·V3·V7·V11.

세 리스트에 공통으로 등장하는 것은 **V14·V12·V10·V4** 다. 상관관계(선형 관계)·트리 중요도(비선형 분할 기여)·선형 계수(정규화 후 생존)가 서로 다른 개념인데도 같은 피쳐를 지목한다는 것은, 이 성분들이 사기

신호를 실제로 담고 있을 가능성이 높다는 뜻이다. 다만 V1~V28 이 익명이라 "그래서 어떤 거래 속성인가"까지는 이 프로젝트 범위에서 알 수 없다.

8. 종합 평가

데이터 이해도, 전처리의 타당성, 모델 비교 논리, 최적화 적용도, 결과 해석의 완성도 다섯 가지 관점에서 이 프로젝트를 자평하면 다음과 같다.

- **데이터 이해도 및 EDA의 깊이** — 익명 PCA 데이터라 접두사 분석·도메인 해석이 불가능한 조건에서, 상관관계·분포 정규성·행 단위 중복을 교차로 확인해 V14를 이상치 기준으로, 완전 중복행을 누수 요인으로 특정했다. 다만 개별 V 성분의 분포를 하나씩 시각화하는 작업까지는 하지 않았다.
- **전처리 방법의 타당성과 근거 제시** — 중복 제거·Time 제거·이상치 제거·스케일링 각각을 A/B로 비교해 ROC-AUC-PR-AUC 수치로 근거를 남겼다(5.1, 6.4). 모델 계열(트리 vs 선형)에 따라 스케일링 적용 여부를 다르게 가져간 판단도 근거가 명확하다. 한계는 중복 제거 기준(1,081 vs 9,144)을 팀 내에서 통일하지 못한 점이다.
- **모델 비교 논리와 지표 선정의 적절성** — 사기 비율 0.17%에서 ROC-AUC 단독 비교의 위험을 실측(이상 탐지 3종, 균형 샘플 앙상블)으로 보이고 PR-AUC·F1을 함께 채택했다. 팀원별 검증 조건이 달라 절대 순위를 매기지 않고 “측정 조건” 열과 함께 제시한 것도 과대 주장을 피한 선택이다.
- **최적화 적용 여부 및 성능 개선 정도** — 수동/격자/Hyperopt를 세 모델에서 나란히 비교했고(7.3), Hyperopt가 수동을 압도하지 못한다는 결론과 그 이유(탐색 공간 설계·데이터 규모)를 정리했다. 절대 성능 개선 폭은 크지 않았으나, “재학습 없는 조정(임계값·eval_metric)이 더 효과적”이라는 실용적 결론을 얻었다.
- **결과 해석의 명확성과 결론의 완성도** — 미탐지 12건이 모델 교체로 해결되지 않는 구조적 한계임을 지도·비지도 두 방법의 동일한 실패로 논증했고(7.2), 이를 바탕으로 다음 단계(velocity 피쳐·비용 민감 학습·확률 보정)를 개념 설명과 함께 제시했다(9장).

9. 결론

1. 극단적 불균형 데이터에서는 정확도·ROC-AUC 만으로 모델을 판단하면 안 된다. PR-AUC(또는 F1-Score, Precision/Recall)를 함께 봐야 실제 운영 성능을 가늠할 수 있다.
2. 중복행 제거와 train/test 분할 순서는 성능 수치 자체보다 우선하는 방법론적 원칙이다. 전처리를 분할 이전에 적용하면 실제보다 부풀려진 성능이 나오며, 이는 여러 팀원이 독립적으로 재현·확인했다. 특히 중복 제거는 Time 을 버리는 파이프라인이라면 Time 을 뺀 30 개 열 기준으로 해야 누수가 남지 않는다(세부는 3.5·10 장).
3. 익명 PCA 데이터에서도 타깃 상관과 분포 정규성을 함께 보면 신뢰할 만한 이상치 기준을 세울 수 있다. V14 는 Class 와 상관이 상위권이면서 사기 케이스 분포가 정규분포에 가장 가까워, 다섯 명 전원이 사기 케이스 기준 IQR 하한을 공통 이상치 기준으로 채택했다.
4. 트리 기반 부스팅 모델(CatBoost·LightGBM·XGBoost)이 로지스틱 회귀보다 PR-AUC 기준으로 우수했다. 다만 로지스틱 회귀는 해석 가능성과 속도 면에서 대체하기 어려운 장점을 유지한다.
5. 하이퍼파라미터 튜닝(Hyperopt 등)보다 판정 임계값 조정, 평가지표(eval_metric) 선택처럼 “재학습이 필요 없는” 조정이 더 확실한 개선을 가져오는 경우가 많았다. 클래스 가중치를 지나치게 강하게 적용하면 오탐이 폭증해 실무에 투입하기 어렵다는 점도 여러 모델에서 공통으로 확인되었다.
6. 이번 프로젝트에서 확보한 29 차원 익명 피쳐만으로는 미탐지(FN)를 일정 수준 이하로 줄일 수 없다는 것이 최종 결론이다. 학습 방식이 전혀 다른 지도학습(CatBoost)과 비지도 이상 탐지가 정확히 같은 사례에서 막혔고(7.2), 이는 모델·튜닝의 문제가 아니라 입력 정보량의 상한이다. 동시에, 트리 기반 부스팅이 로지스틱 회귀보다 PR-AUC 에서 앞서는 경향이 다섯 실험에서 일관되게 나타난 만큼, 익명 PCA 피쳐를 다루는 이런 문제에서는 부스팅 계열을 1 차 후보로 두는 것이 근거 있는 선택이다.

이 결론을 다음 단계로 잇기 위해, 팀 토의에서 아래 세 방향을 개선점으로 정리했다. 각 항목은 이번 프로젝트에서 다루지 않은 개념이므로 간단한 설명을 함께 붙인다.

- 맥락(velocity) 피쳐와 그래프 기반 이상 탐지. 거래를 독립된 한 점이 아니라 “카드·가맹점·기기가 시간축으로 연결된 흐름”으로 보고, 최근 N 분 내 거래 횟수·금액 급증, 동일 카드의 비정상 경로 같은 파생 변수를 만드는 것이 velocity 피쳐다. 더 나아가 거래를 노드-엣지로 이은 네트워크에서 여러 카드·계정이 얹힌 사기 링(fraud ring)을 찾는 그래프 기반 이상 탐지(graph anomaly detection)는 점 단위 판단의 한계를 구조적으로 넘는 접근으로, 팀이 아직 학습하지 않은 영역이다.
- 비용 민감 학습(cost-sensitive learning). 이번에는 class_weight 로 클래스 빈도만 보정했지만, 실무에서는 FN(놓친 사기 = 피해 금액)과 FP(막은 정상 거래 = 상담·이탈 비용)의 실제 비용이 다르다. 오분류마다 금전 비용을 부여하고 그 기대 비용이 최소가 되도록 손실 함수나 판정 임계값을 설계하는 방법을 익히면, F1·PR-AUC 같은 통계 지표를 넘어 비즈니스 목표에 바로 맞출 수 있다.
- 확률 보정(probability calibration). 권용우가 관찰했듯 SMOTE·class_weight 는 예측 확률을 위쪽으로 왜곡시켜 “사기 확률 90%”가 실제 신뢰도와 어긋나게 만든다. Platt scaling 이나

isotonic regression 으로 확률을 보정하면 임계값 결정과, 애매한 거래를 사람이 재검토하는
기준이 신뢰할 만해진다.

다섯 파이프라인을 동일 전처리·분할·교차검증·지표로 맞춰 공정 비교하는 작업은 위 개념들을
적용하기에 앞서 먼저 마무리한다.

10. 회고

10.1 잘된 점 · 아쉬운 점 · 향후 개선 방향

잘된 점

- 다섯 명 모두 ROC-AUC 대신 PR-AUC(또는 F1)를 주 지표로 삼아야 한다는 결론에 독립적으로 도달했고, 이를 교차 검증하는 과정에서 서로의 판단을 뒷받침하는 근거가 되었다.
- 데이터 누수(중복행 처리 순서, train/test 분할 전 전처리) 문제를 이진희·김소현·권용우가 각각 독립적으로 발견하고 파이프라인을 “분할 → 전처리” 순서로 바로잡았다 — 혼자 진행했다면 놓쳤을 문제를 서로의 실험 결과 비교를 통해 잡아낸 사례다.
- 같은 모델(Logistic Regression)을 두 팀원이 독립적으로 실험해 결과를 교차 검증할 수 있었고, CatBoost 사례에서는 지도학습과 비지도 이상 탐지를 결합하는 후속 분석까지 이어져 단순 성능 비교를 넘어선 원인 분석이 이루어졌다.

아쉬운 점

- CatBoost의 미탐지(FN) 12 건은 여러 튜닝·이상 탐지 기법을 결합해도 끝내 줄어들지 않았고, 구조적 한계로 결론지었을 뿐 실제 해결책(카드·고객 단위 맥락 피쳐 추가 등)은 이번 프로젝트 범위에서 실행되지 않았다.
- 평가 파이프라인을 프로젝트 초기에 표준화하지 못했다. 최종 평가 지표(F1·PR-AUC·ROC-AUC 5-Fold 평균), 검증 방식(단일 분할 vs 교차검증), 중복행 제거 기준(완전 일치 1,081 건 vs Time 제외 9,144 건)이 팀원마다 달라, 7.1 비교표의 각 모델이 서로 다른 조건·서로 다른 크기의 학습 데이터 위에서 측정되었다. 그래서 7.1은 순위를 매기지 않고 채택 결과만 나열하는 형태로 정리했으며, 다음 프로젝트에서는 이 세 가지를 초기에 공통 규칙으로 고정해야 한다.

향후 개선 방향

- CatBoost·LightGBM·XGBoost 세 모델을 스택킹 또는 가중 평균 앙상블로 결합하면 개별 모델의 PR-AUC 상한을 넘어설 가능성이 있다(김소현이 LR+LGBM 스택킹에서 확인했듯, 실력 격차가 큰 모델을 섞을 때는 가중치 설계가 관건이다).
- CatBoost가 마지막까지 놓친 FN 12 건에 대해 라벨 노이즈 여부를 직접 검토하거나, 완전 자동 차단 대신 애매한 거래를 사람이 재검토하는 반자동 프로세스를 설계하는 것을 다음 단계로 제안한다.
- 팀 공통 평가 파이프라인(동일 지표·동일 검증 방식)을 프로젝트 초기에 표준화하면, 모델 간 비교의 공정성을 높이고 각자 유사한 평가 코드를 반복 작성하는 중복 작업도 줄일 수 있을 것이다.

10.2 팀원별 느낀점 및 피드백

아래는 팀이 작업 과정에서 실시간으로 기록해 온 *02_Creditcard.xlsx*의 팀원별 시트에 남아 있는 느낀점과 피드백을, 결과에 이르기까지 각자가 무엇을 느끼고 배웠는지의 기록으로 옮긴 것이다.

이혜현 — LightGBM. 불균형 사기 데이터에서는 무조건적인 하이퍼파라미터 튜닝보다 정밀한 전처리와 임계값 조정이 성능을 더 크게 좌우한다. 중복치를 제거하지 않았을 때 나온 높은 점수가 데이터 누수로 인한 착시였음을 확인하면서 올바른 전처리 순서의 중요성을 체감했다. 또한 LightGBM의 하이퍼파라미터 탐색

공간을 지나치게 넓히면 교차검증 데이터에 미세 과적합되어 오히려 기존 튜닝안보다 점수가 낮아졌고, 소수 클래스 문제일수록 모델 복잡도를 적정 수준으로 제어해야 한다는 점을 배웠다. 결과적으로 복잡한 재학습 없이 예측 확률 임계값을 0.3~0.4 구간으로 낮추는 것만으로 오탐 증가 없이 재현율과 F1-Score 를 즉시 개선할 수 있었으며, 실무에서는 비즈니스 리스크를 반영한 사후 확률·임계값 조정이 핵심이라는 결론에 도달했다.

이진희 — XGBoost. 캐글 설명만으로 문제를 인지하던 이전 방식과 달리, 이번에는 데이터를 직접 열어 어떤 변수가 예측에 크게 작용하는지, 이미 PCA 로 정리된 데이터를 어떻게 표준화할지를 팀원과 논의하며 진행했다. 협업과 역할 분담이 이전 프로젝트보다 원활했고, 각자 아이디어를 내고 그 결과의 신뢰성을 어떻게 확보할지 함께 고민하는 과정이 특히 의미가 있었다. XGBoost 실험에서는 초기 점검만으로 ROC-AUC 0.99 대의 높은 수치가 나온 것에 의문을 갖고 데이터 누수 가능성을 반복 점검했으며, train/test 분할 이전에 스케일러 fit 과 이상치 기준 계산을 적용한 것이 원인임을 확인해 파이프라인 순서를 “분할 → 전처리”로 바로잡았다. 이 과정에서 높은 지표 자체보다 그 지표를 신뢰할 수 있게 만드는 검증 절차가 더 중요하다는 점을 배웠다.

김소현 — Logistic Regression. 로지스틱 회귀를 선택한 이유는 사기 여부를 0/1 로 분류하면서도 “이 거래가 사기일 확률”을 함께 제시할 수 있고, L1/L2 정규화로 과적합 억제와 변수 선택이 가능하며 class_weight 로 불균형에 직접 대응할 수 있기 때문이다. 스케일 민감성은 RobustScaler 로 완화했다. 실험을 진행하며 배운 점은 두 가지다. 첫째, class_weight 와 SMOTE 를 교차검증으로 비교했을 때 단순한 선형 모델에서는 class_weight 만으로 소수 클래스에 충분한 가중치를 줄 수 있어, SMOTE 가 오히려 소수 영역을 과도하게 확장한다는 것을 확인했다. 둘째, L1 정규화로 계수가 0 이 된 변수 10 개를 제거했더니 L2 대비 오히려 성능이 올라, 그 변수들이 노이즈에 가까워 정밀도를 떨어뜨리고 있었을 가능성을 확인했다. 평가지표를 ROC-AUC 에서 PR-AUC 로 바꿔 규제 강도(C)를 재탐색한 것도 같은 맥락으로, 극단적 불균형에서는 PR-AUC 가 실제 탐지 품질을 더 정확히 반영한다는 점을 실험으로 재확인했다.

전유경 — CatBoost. 데이터를 먼저 살펴보고 트리 기반 모델이 잘 맞을 것으로 예상했으나 결과는 달랐다. CatBoost 로 클래스 가중치·판정 임계값·평가지표·트리 구조까지 조정할 수 있는 방법을 모두 시도했지만 미탐지(FN) 건수가 일정 수준 아래로 내려가지 않았다. 처음에는 CatBoost 자체의 한계로 의심했으나, 라벨을 보지 않는 이상 탐지 기법 3 종을 결합해도 같은 사례에서 막히는 것을 확인하면서, 이는 특정 모델이 아니라 “거래 한 건을 29 차원 벡터의 한 점으로만 보고 판단하는” 지도학습 트리 방법론 전반의 구조적 한계라는 결론에 이르렀다. 카드·고객 단위의 맥락(velocity) 정보를 확보하는 것이 다음 과제라고 판단한다.

권용우 — Logistic Regression. 이전 과제와 달리 불균형이 극심한 데이터라, 전처리 조합이 성능에 미치는 영향을 정규분포 관점에서 체계적으로 검토했다. 이상치 제거·중복행 제거·스케일링을 독립 함수로 분리해 8 가지 조합을 동일 조건으로 비교했고, 그 과정에서 Time 컬럼을 남겨 두면 로지스틱 회귀 수렴이 방해되고 완전 중복 행 탐지도 부정확해진다는 점을 발견해 전체 실험을 Time 제거 버전으로 재실행했다. 파라미터 탐색은 시간 제약상 GridSearchCV 와 Hyperopt 를 비교하는 선에서 진행했으며, 두 방법의 결과 차이는 크지 않았다. SMOTE 나 class_weight 처럼 소수 클래스를 인위적으로 부풀리는 기법을 쓰면 예측 확률이 위로 쏠려 F1 최적 임계값이 1 에 가깝게 형성되는데, 이 경우 사기 확률이 90%로 나온 케이스도 정상으로 분류될 수 있어 F1 최댓값 지점이 실무적으로 안전한 기준은 아니라는 점을 확인했다. “재현율 하한을 정해 두고 그 안에서 정밀도를 최대화”하는 제약 기반 임계값 설정이 더 안전한 대안이라고 판단한다.

11. 참고문헌

1. Machine Learning Group – ULB. *Credit Card Fraud Detection*. Kaggle.
<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
2. 권철민. 『파이썬 머신러닝 완벽 가이드』(개정판). 위키북스, 2022. — 불균형 데이터 처리, 평가지표(PR-AUC-F1), 부스팅 계열 모델 튜닝 절차 참고.
3. scikit-learn developers. *scikit-learn User Guide* — *train_test_split*, *StratifiedKFold*, *RobustScaler*, *LogisticRegression*, *precision_recall_curve*, *average_precision_score*. <https://scikit-learn.org/stable/>
4. CatBoost. *CatBoost Documentation* — *eval_metric*, overfitting detector. <https://catboost.ai/docs/>
5. LightGBM. *LightGBM Documentation* — *scale_pos_weight*, leaf-wise growth, early stopping.
<https://lightgbm.readthedocs.io/>
6. XGBoost. *XGBoost Documentation*. <https://xgboost.readthedocs.io/>
7. J. Bergstra, D. Yamins, D. D. Cox. "Hyperopt: A Python Library for Optimizing the Hyperparameters of Machine Learning Algorithms." *Proc. of the 12th Python in Science Conf.*, 2013.
8. imbalanced-learn developers. *imbalanced-learn Documentation* — SMOTE. <https://imbalanced-learn.org/>

12. 부록: 함수 코드

아래 함수들은 팀 노트북 5 개(이혜현·이진희·김소현·전유경·권용우)와 이를 합친 통합 노트북(02_classification_credit_card/팀_통합_creditcard.ipynb)을 조사한 결과, 실제로 `def`로 정의되어 여러 노트북에 공유된 것과, 셀에 인라인으로 반복 작성된 패턴을 재사용 가능하도록 정리한 것을 함께 담았다. 5.5 절에서 밝혔듯 팀 공통 함수화는 제한적이었으므로, 이 부록은 “이미 공유되어 있던 함수”와 “정리하면 공유할 수 있었던 패턴”을 함께 담았다. 각 함수의 docstring 에는 대응하는 본문 절 번호를 표시했다.

12.1 전처리 함수

완전 중복행 제거, 사기 케이스 V14 IQR 이상치 제거, *Time* 컬럼 제거, *Amount* RobustScaler 는 거의 모든 팀원의 노트북에 동일한 형태로 반복 등장한 패턴이다. 이 중 `get_outlier`는 이혜현·이진희·권용우 노트북에 실제로 `def`로 정의되어 있었고, 나머지는 셀에 인라인으로 반복된 것을 함수로 정리했다. 순서가 중요한데, *Time* 을 먼저 버려야 나머지 30 개 열 기준으로 완전 중복행이 정확히 잡힌다(3.5 절).

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import RobustScaler

def get_outlier(df, column='V14', weight=1.5):
    """사기(Class=1) 케이스만 대상으로 IQR 기반 이상치 인덱스를 반환한다 (3.3 절).
    정상 거래에는 적용하지 않는다 - 정상 분포의 폭까지 좁히면 학습에 해가 된다."""
    fraud = df[df['Class'] == 1][column]
    q25, q75 = fraud.quantile(0.25), fraud.quantile(0.75)
    iqr = q75 - q25
    lower, upper = q25 - iqr * weight, q75 + iqr * weight
    return df[(df[column] < lower) | (df[column] > upper)].index

def preprocess_creditcard(df, scale=True):
    """팀 공통 전처리 1~4 단계(4.1·4.2·4.4 절)를 하나로 정리한 버전.
    분할 이후 학습 세트 기준으로만 호출해야 데이터 누수가 없다(4.6 절)."""
    X = df.copy()
    # 1) Time 제거: 스케일이 지나치게 커 로지스틱 회귀 수렴을 방해하고,
    #    남겨 두면 완전 중복행 탐지도 부정확해진다(3.5·4.4 절).
    X = X.drop(columns=['Time'], errors='ignore')
    # 2) 완전 중복행 제거 - Time 제거 후이므로 30 개 열 기준으로 잡힌다(3.5 절).
    X = X.drop_duplicates().reset_index(drop=True)
    # 3) 사기 케이스 V14 이상치 4 건 제거(3.3·4.1 절).
    X = X.drop(get_outlier(X, 'V14')).reset_index(drop=True)
    # 4) Amount RobustScaler - 트리 모델(CatBoost/XGBoost)에는 scale=False(4.2 절).
    if scale:
        X['Amount'] = RobustScaler().fit_transform(X[['Amount']])
    return X
```

12.2 공동 함수 — 모델링

`train_test_split(stratify=y)`로 계층 분할한 뒤(4.6 절), 이진희(XGBoost)·이혜현(LightGBM)이 공통으로 쓴 “사기:정상 = 1:N 균형 샘플을 여러 개 만들어 각각 학습 → 예측 확률 평균” 앙상블 흐름을 함수로 정리하면 다음과 같다.

```

from sklearn.model_selection import train_test_split

def split_stratified(df, target='Class', test_size=0.2, random_state=0):
    """계층 분할로 학습/테스트를 나눈다. 사기 비율 0.17%이므로 stratify 는 필수(4.6 절)."""
    X, y = df.drop(columns=[target]), df[target]
    return train_test_split(X, y, test_size=test_size,
                            stratify=y, random_state=random_state)

def make_balanced_samples(X_train, y_train, ratio=10, random_state=42):
    """사기: 정상 = 1:ratio 로 나눈 균형 샘플 데이터셋 리스트를 만든다(6.2·6.5 절)."""
    train = pd.concat([X_train, y_train], axis=1)
    fraud = train[train['Class'] == 1]
    normal = train[train['Class'] == 0].sample(frac=1, random_state=random_state)
    chunk = len(fraud) * ratio
    samples = []
    for start in range(0, len(normal), chunk):
        part = normal.iloc[start:start + chunk]
        if len(part) < chunk:
            break
        samples.append(pd.concat([fraud, part]).sample(frac=1, random_state=random_state))
    return samples

def ensemble_predict_proba(models, X_test):
    """여러 모델의 사기 확률을 평균 낸다 - 균형 샘플 앙상블의 최종 예측(6.5 절)."""
    return np.mean([m.predict_proba(X_test)[: , 1] for m in models], axis=0)

```

12.3 공동 함수 — 하이퍼파라미터 최적화

Hyperopt 를 쓴 세 명(이혜현·이진희·권용우)이 각자 독립적으로 작성한 *objective(params)* 함수는 세부 탐색 공간만 다를 뿐 "탐색 공간 정의 → 교차검증으로 평균 점수 계산 → 음수 손실 반환 → *fmin*+TPE 로 탐색"이라는 동일한 구조를 공유한다(5.3 절). 이를 모델에 무관하게 재사용 가능한 템플릿으로 일반화하면 다음과 같다.

```

from hyperopt import fmin, tpe, hp, Trials, STATUS_OK
from sklearn.model_selection import cross_val_score

def make_hyperopt_objective(model_cls, X, y, int_params=(), cv=5,
                           scoring='average_precision', fixed_params=None):
    """model_cls(XGBClassifier·LGBMClassifier·LogisticRegression 등)에 대해
    Hyperopt objective 를 생성한다. hp.quniform 은 float 를 반환하므로
    max_depth 등 정수 파라미터는 int_params 에 지정해 형변환한다.
    극단적 불균형이라 scoring 기본값은 PR-AUC(average_precision)로 둔다(2.6·9 장)."""
    fixed_params = fixed_params or {}
    def objective(params):
        p = {k: (int(v) if k in int_params else v) for k, v in params.items()}
        model = model_cls(**fixed_params, **p)
        score = cross_val_score(model, X, y, cv=cv, scoring=scoring, n_jobs=-1).mean()
        return {'loss': -score, 'status': STATUS_OK}
    return objective

def run_hyperopt(objective, space, max_evals=100, random_state=156):
    """fmin 실행을 감싸 베스트 파라미터와 베스트 CV 점수를 함께 반환한다."""
    trials = Trials()
    best = fmin(fn=objective, space=space, algo=tpe.suggest,
               max_evals=max_evals, trials=trials,
               rstate=np.random.default_rng(random_state))
    best_cv_score = -trials.best_trial['result']['loss']
    return best, best_cv_score

```

예를 들어 6.5 절의 XGBoost Hyperopt 실험(100 회 탐색)은 이 템플릿으로 다음과 같이 재현할 수 있다.

```

from xgboost import XGBClassifier

xgb_space = {
    'max_depth': hp.quniform('max_depth', 4, 15, 1),
    'min_child_weight': hp.quniform('min_child_weight', 1, 10, 1),
    'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 0.95),
    'subsample': hp.uniform('subsample', 0.5, 1.0),
    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2),
}
objective = make_hyperopt_objective(
    XGBClassifier, X_tr, y_tr,
    int_params=('max_depth', 'min_child_weight'),
    fixed_params={'n_estimators': 300, 'eval_metric': 'logloss', 'n_jobs': -1},
)
best_params, best_cv_ap = run_hyperopt(objective, xgb_space, max_evals=100)

```

12.4 공동 함수 — 평가

6 장 전체에서 “혼동행렬 + Precision/Recall/F1 + ROC-AUC + PR-AUC”를 한 번에 출력하는 패턴과, “튜닝 전 vs 튜닝 후” 또는 “모델 A vs 모델 B”의 점수를 나란히 비교하는 패턴이 반복되었다. 전자는 여러 팀원 노트북에 `get_clf_eval` 계열로 이미 존재했고, 후자는 7.1 절 종합 비교표에 대응하도록 리포팅 함수로 정리했다.

```

from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_auc_score, average_precision_score, f1_score)

def get_clf_eval(y_test, pred, pred_proba):
    """혼동행렬 + 주요 지표를 한 번에 출력한다(5.5 절·6 장 전체에서 반복된 패턴).
    극단적 불균형이라 정확도는 출력하지 않고 PR-AUC(=AP)를 함께 본다(2.6 절)."""
    cm = confusion_matrix(y_test, pred) # [[TN, FP], [FN, TP]]
    roc = roc_auc_score(y_test, pred_proba)
    pr = average_precision_score(y_test, pred_proba) # PR-AUC (= AP)
    f1 = f1_score(y_test, pred)
    print("[TN, FP], [FN, TP] ="); print(cm)
    print(f"ROC-AUC {roc:.4f} | PR-AUC {pr:.4f} | F1 {f1:.4f}")
    print(classification_report(y_test, pred, digits=4))
    return {'roc_auc': roc, 'pr_auc': pr, 'f1': f1, 'cm': cm}

def best_threshold_f1(y_true, proba, grid=None):
    """F1을 최대화하는 판정 임계값을 찾는다 - 재학습이 필요 없는 조정(6.1 F 안·6.2·6.4 절)."""
    grid = grid if grid is not None else np.arange(0.05, 0.95, 0.01)
    scores = [(t, f1_score(y_true, (proba >= t).astype(int)))] for t in grid
    return max(scores, key=lambda x: x[1]) # (best_threshold, best_f1)

def report_scores(results: dict, primary='pr_auc'):
    """{'모델 이름': {'pr_auc': .., 'roc_auc': .., 'f1': ..}, ...} 형태를 표로 출력하고
    primary 지표 기준 최고 모델을 반환한다(7.1 절 종합 비교표에 대응)."""
    print(f"{'모델':24s}{ 'PR-AUC':>10s}{ 'ROC-AUC':>10s}{ 'F1':>10s}")
    for name, m in results.items():
        print(f"{name:24s}{m.get('pr_auc', float('nan')):>10.4f}"
              f"{m.get('roc_auc', float('nan')):>10.4f}{m.get('f1', float('nan')):>10.4f}")
    best = max(results, key=lambda k: results[k].get(primary, float('-inf')))
    return {'best_model': best, 'best_score': results[best].get(primary)}

```

`report_scores`는 예컨대 7.1 절 비교표를 `report_scores({'CatBoost': {'pr_auc': 0.8898, 'roc_auc': 0.9743, 'f1': 0.915}, 'XGBoost': {'pr_auc': 0.8401, 'roc_auc': 0.9839, ...}})`처럼 호출해 PR-AUC 기준 최고 모델을 바로 확인하는 용도로 쓸 수 있다.

Opinosis 텍스트 데이터 기반 토픽 군집화 최종 분석 보고서

분석팀: 이해현, 이진희, 김소현, 전유경, 권용우

보고서 작성: 이해현

목차

- [0. 초록](#)
- [1. 서론](#)
 - 1.1 연구 배경 및 필요성
 - 1.2 과제 목적
- [2. 데이터 소개](#)
 - 2.1 데이터 출처 및 구성
 - 2.2 데이터 규모 및 기본 통계
 - 2.3 도메인별 분포
- [3. 탐색적 데이터 분석 \(EDA\)](#)
 - 3.1 기초 텍스트 통계
 - 3.2 상위 빈출 단어 분석
- [4. 데이터 전처리 및 벡터화](#)
 - 4.1 텍스트 전처리 파이프라인
 - 4.2 텍스트 벡터화 기법 비교
- [5. 분석방법 \(Analysis Method\)](#)
 - 5.1 전처리·벡터화 필요성 및 과정 개요
 - 5.2 사용 모델 설명
 - 5.3 하이퍼파라미터 최적화 기법 설명
 - 5.4 실행환경
 - 5.5 코드 재사용 및 함수화
- [6. 모델링](#)
 - 6.1 차원 축소
 - 6.2 군집화 모델 비교
 - 6.3 하이퍼파라미터 최적화 (그리드 탐색)
 - 6.4 k=5 세분화의 독립적 재현 검증 (전유경 파이프라인)
- [7. 결과](#)
 - 7.1 최종 파이프라인
 - 7.2 주요 발견사항
 - 7.3 비즈니스 활용 제언 (투-트랙 전략)
- [8. 종합 평가](#)
- [9. 결론](#)
 - 9.1 기술적 결론

- 9.2 실무적 시사점
- [10. 회고](#)
 - 10.1 종합 회고
 - 10.2 팀원별 느낀점 및 피드백
- [11. 참고문헌](#)
- [12. 부록: 함수코드](#)
 - 12.1 전처리 함수
 - 12.2 클러스터 해석 함수
 - 12.3 검증 함수

0. 초록

본 보고서는 UCI Machine Learning Repository가 공개한 *Opinosis* 리뷰 데이터셋(51개 문서, 전자기기·자동차·호텔 3개 도메인)을 대상으로, 비정형 텍스트를 전처리·벡터화한 뒤 비지도 군집화(KMeans, GMM, DBSCAN, HDBSCAN, Agglomerative)로 문서를 토픽별로 자동 분류하는 프로젝트를 정리한 것이다. 팀원 5명(이혜현·이진화·김소현·전유경·권용우)이 각자 독립적인 전처리·벡터화·모델링 파이프라인을 구축해 비교했으며, 동일한 하이퍼파라미터(`ngram_range=(1,2)`, `min_df=0.05`, `max_df=0.85`)를 설정해도 토큰나이저 구현(축약형 처리, 표제어 추출 여부 등) 차이로 최종 피쳐 수가 파이프라인마다 달랐다는 점이 본 프로젝트의 특징적 발견이었다. 그 결과 김소현의 파이프라인(7단계 Lemmatization + TF-IDF, 51×4,498 희소 행렬)에서 KMeans(k=3)가 **ARI/NMI = 1.0**으로 51개 문서를 도메인과 완전히 일치하게 분류했으며, 이혜현의 54개 파라미터 그리드서치와 전유경의 독립 k=5 파이프라인이 공통적으로 “3대 도메인이 다시 5개 세부 니즈로 갈라진다”는 세분화 구조를 재현했다. 반대로 DBSCAN·HDBSCAN은 51개라는 소규모 데이터셋 특성상 거리 수렴 현상과 노이즈 과다로 부적합함이 세 명의 독립 실험에서 공통 확인되었다. 결론적으로 벡터화 기법(Count vs TF-IDF) 선택이 모델 선택보다 성능에 더 큰 영향을 미쳤으며, 도메인 완전분류(k=3)와 세부 니즈 세분화(k=5)를 병행하는 투-트랙 전략을 최종 제언으로 제시한다.

1. 서론

1.1 연구 배경 및 필요성

Opinosis는 UCI Machine Learning Repository에서 제공하는 공개 텍스트 데이터셋으로, 아마존 킨들·가민 GPS·넷북 등 전자기기, 혼다·도요타 자동차, 호텔 서비스에 대한 리뷰 문장을 토픽별로 정리한 51개 문서로 구성된다. 각 문서는 “속성_제품명”(예: `battery-life_amazon_kindle`) 형태의 파일명을 가지며, 파일명 자체가 사실상 도메인 정답 라벨 역할을 할 수 있어 비지도 군집화 결과를 검증하는 pseudo-label로 활용할 수 있다는 것이 이 데이터셋의 실용적 특징이다.

정형 데이터와 달리 비정형 텍스트는 정답이 정해진 지도학습 문제로 곧바로 치환되지 않으며, 전처리 방식(표제어 추출, 불용어 제거)과 벡터화 기법(Count vs TF-IDF)에 따라 결과가 크게 달라진다는 점에서, “어떤 선택이 왜 더 나은가”를 데이터로 검증하는 것이 본 프로젝트의 핵심 문제의식이다.

1.2 과제 목적

분석 팀은 5명(이혜현, 이진희, 김소현, 전유경, 권용우)으로 구성되었으며, 각자 서로 다른 접근 방식으로 데이터 전처리부터 모델링까지 독립적으로 비교 분석하였다. 본 보고서는 이러한 다양한 시도와 그 결과를 이혜현이 통합하여 정리한 최종 결과물이다.

중요한 전제: 다섯 명의 노트북은 겉보기엔 모두 “TF-IDF + KMeans”를 사용했지만, 실제로는 축약형 처리 여부·불용어 사전·표제어 추출 방식(품사 태깅 유무)· `stop_words='english'` 중복 적용 여부 등 토큰라이저 구현이 서로 달라 **동일한 `ngram_range=(1,2)`, `min_df=0.05`, `max_df=0.85`를 설정해도 최종 피쳐 수가 전부 다르다**(이혜현 4,139개, 김소현 4,498개, 권용우 4,462개 등). 본 보고서는 이 차이를 감추지 않고, 모든 수치·표·그림에 어느 팀원·어느 노트북에서 나온 결과인지 명시한다. 이는 은폐할 한계가 아니라, 서로 다른 5개의 독립적인 파이프라인이 유사한 결론(TF-IDF+KMeans 우위, 3~5개 도메인 구조)에 수렴했다는 교차검증의 근거로 적극 활용한다.

주요 분석 절차는 (1) 데이터 이해 및 텍스트 EDA, (2) 텍스트 정제 및 표제어 추출(Lemmatization), (3) 텍스트 벡터화 기법 비교(CountVectorizer vs TfidfVectorizer, N-gram), (4) 군집화 모델링 및 비교(KMeans, GMM, DBSCAN), (5) k값 탐색 및 파라미터 민감도 분석, (6) 최종 결론 및 회고 순서로 진행되었으며, 본 보고서 역시 이 흐름을 그대로 따른다.

2. 데이터 소개

2.1 데이터 출처 및 구성

항목	내용
데이터 출처	UCI Machine Learning Repository, <i>Opinosis Opinion/Review</i>
문서 수	51개 (.txt.data 파일 51개, 파일 1개 = 토픽 1개)
도메인 구성	전자기기 25개 / 호텔 16개 / 자동차 10개 (3개 도메인)
평가용 pseudo-label	파일명 키워드 기반 도메인 추정(category) — 학습에는 미사용, 검증 전용
평가지표	Silhouette Score(정답 없는 군집 응집도), ARI·NMI(pseudo-label과의 일치도)

2.2 데이터 규모 및 기본 통계

<표 2-1> 데이터 규모 및 기본 통계 (모든 벡터화 이전, 원문 텍스트 기준 — 이해현 노트북 EDA)

항목	수치	설명
문서 개수	51개	각 문서는 하나의 토픽을 나타냄
평균 단어 수	2,634개	범위: 836~12,750 (높은 편차)
평균 고유 단어 수	762개	문서별 어휘 다양성
평균 문자 수	56,736개	문서 길이 편차 큼

전처리 이후 문서당 토큰 수는 노트북마다 다르게 집계되는데, 이는 각자의 불용어·숫자 필터링 기준이 다르기 때문이다. 예컨대 김소현 파이프라인 기준 문서별 토큰 수는 평균 1,276개(최소 416~최대 5,973)로 집계되었다.

2.3 도메인별 분포

<표 2-2> 도메인별 문서 분포 (키워드 규칙 기준: 호텔=hotel/inn/swissotel, 자동차=honda/toyota/accord/camry, 나머지=전자기기)

도메인	문서 수	주요 제품/서비스
전자기기 (Electronics)	25개	Amazon Kindle, iPod, Netbook, Garmin GPS, Windows 7
호텔 (Hotel)	16개	Best Western, Holiday Inn, Swissotel
자동차 (Automotive)	10개	Honda Accord, Toyota Camry
합계	51개	-

이혜현은 별도의 키워드 규칙(호텔 관련어에 room/service/food/staff 등을 포함)으로 24/16/11 분포를 얻었다. 두 규칙 모두 존재하나, 이후 모델링·결과 전반의 모델 성능·비즈니스 전략이 25/10/16 기준(김소현·전유경)에 근거하므로 본 표는 이 기준으로 통일한다.

3. 탐색적 데이터 분석 (EDA)

3.1 기초 텍스트 통계

앞선 표 2-1을 산출한 이해현의 EDA를 이어서 살펴보면, 전처리 전 상위 빈출 단어는 불용어(the, and, to, is, was)가 대부분이었으며, 이들을 제거해야 도메인 특화 단어의 패턴이 명확해지는 것으로 확인되었다.

3.2 상위 빈출 단어 분석

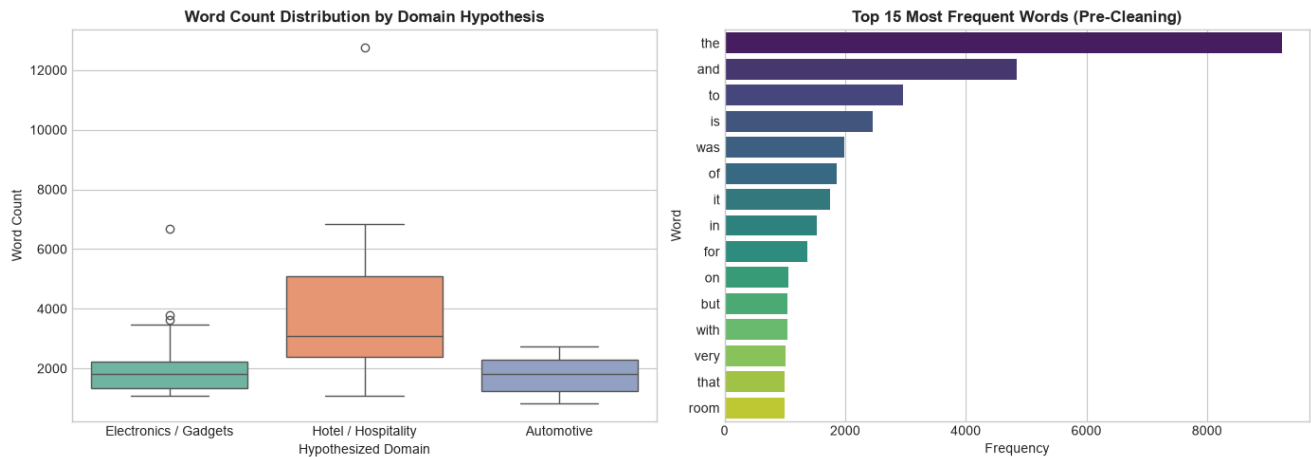


그림 3-1: 전처리 전 상위 20개 빈출 단어(우) 및 도메인 가설별 단어 수 분포(좌)

〈표 3-1〉 전처리 전 상위 빈출 단어 상세 (이해현 노트북 실측)

순위	단어	빈도	단어 클래스
1	the	9,237	기본 정관사
2	and	4,844	접속사
3	to	2,959	전치사
4	is	2,460	동사
5	was	1,988	동사 과거형
6	of	1,855	전치사
7	it	1,751	대명사
8	in	1,535	전치사
9	for	1,374	전치사
10	on	1,051	전치사

종합하면, 전처리 이전 문서는 기능어 중심의 균일한 패턴을 보이거나 불용어 제거 및 표제어 추출 후에는 도메인 특화 단어(screen, battery, room, seat 등)가 두드러지게 나타나며, 이는 후속 벡터화 단계의 효율성으로 이어진다.

4. 데이터 전처리 및 벡터화

4.1 텍스트 전처리 파이프라인

텍스트 전처리는 비정형 데이터를 구조화하는 가장 중요한 단계이다. 5개 노트북 중 가장 상세한 절차를 정립한 것은 김소현의 7단계 파이프라인으로, 아래와 같다.

4.1.1 전처리 7단계 프로세스

〈표 4-1〉 전처리 7단계 파이프라인 (김소현 노트북 기준)

단계	수행 과업	주요 기술	산출물
1	축약형 확장	isn't → is not, didn't → did not	정규화된 텍스트
2	소문자 변환	Battery → battery	통일된 표기
3	구두점 제거	마침표, 쉼표, 느낌표 등 제거	정제된 텍스트
4	토큰화	NLTK word_tokenize 사용	단어 리스트
5	순수 숫자 제거	26, 96 등 숫자 토큰 제거	필터링된 토큰
6	불용어 제거	the, is, and, not 등 제거	의미 토큰만 보존
7	표제어 추출	batteries → battery, screens → screen	정규화된 원형 토큰

이해현·김소현·전유경·권용우 4명의 노트북이 공통으로 사용하는 `WordNetLemmatizer()` 는 품사 태깅 없이 기본값(명사 기준, `pos='n'`) 으로 동작한다. 명사 기준 표제어 추출은 동사 활용형(running→run)이나 형용사 비교급(better→good)을 원형화하지 못하며, 김소현 노트북은 이를 “was → wa”로 잘못 변형되는 사례를 통해 직접 검증하였다.

한편 이진희의 4개 노트북은 표제어 추출·어간 추출을 전혀 적용하지 않는다(소문자화 + 정규식 정제 + NLTK 불용어 제거까지만 수행). 그 결과 room/rooms, seat/seats, button/buttons처럼 의미가 같은 단어가 서로 다른 피처로 남아있는데, 이는 표제어 추출 단계를 생략한 파이프라인의 자연스러운 특징이다. 4.2.1절의 이진희 실측 비교는 이 전처리 위에서 이루어졌다는 점을 함께 고려해야 한다.

또한 불용어 제거는 표제어 추출보다 반드시 먼저 수행해야 한다. 불용어 사전이 원형(was, does 등) 기준으로 구축돼 있어, 표제어 추출을 먼저 하면 “was → wa”처럼 사전에 없는 형태로 바뀌어 불용어로 걸러지지 않고 남는 문제가 발생하기 때문이다.

4.1.2 전처리 효과 분석

권용우 노트북은 표제어 추출 적용 전/후를 k값별·알고리즘별로 직접 비교(ablation)하였다.

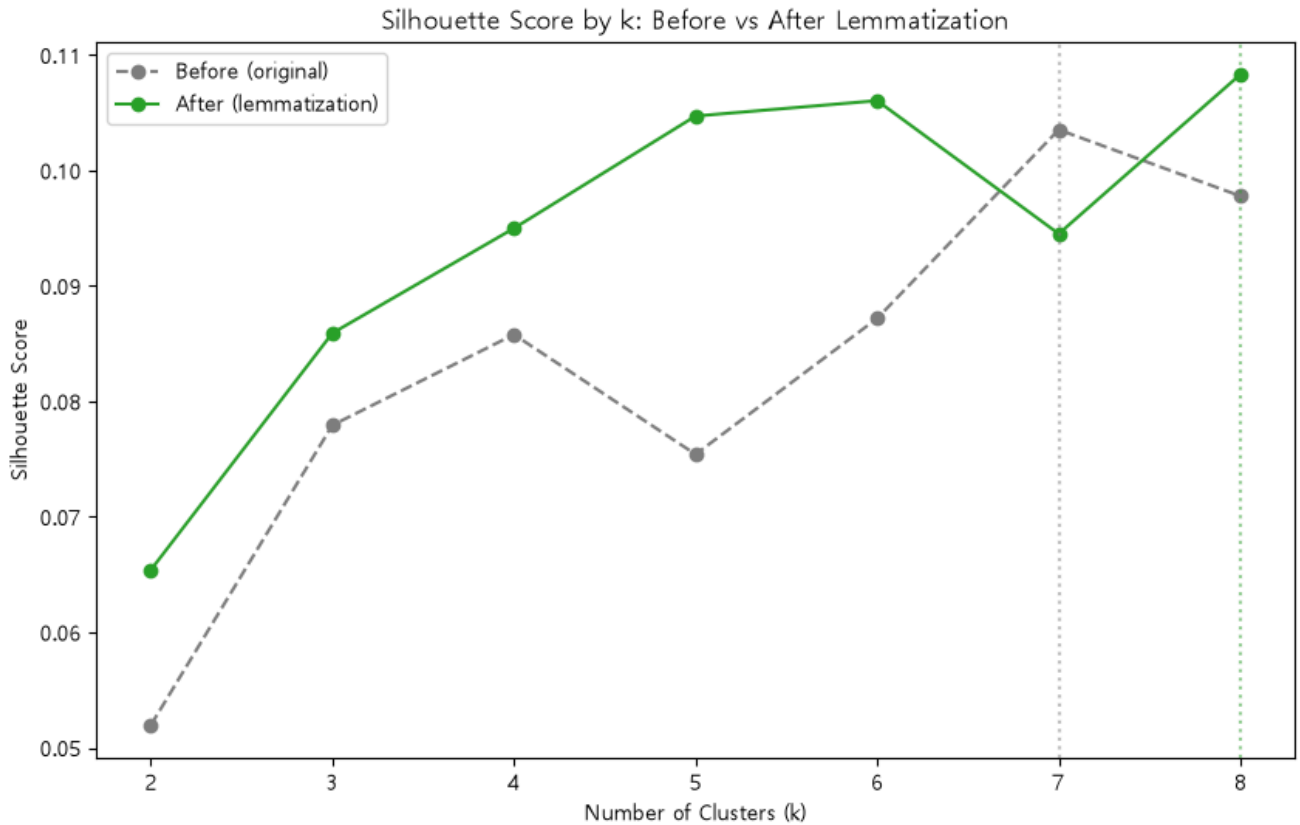


그림 4-1: k값에 따른 표제어 추출 전후 실루엣 점수 비교 — 대부분의 k에서 표제어 추출 후 소폭 개선

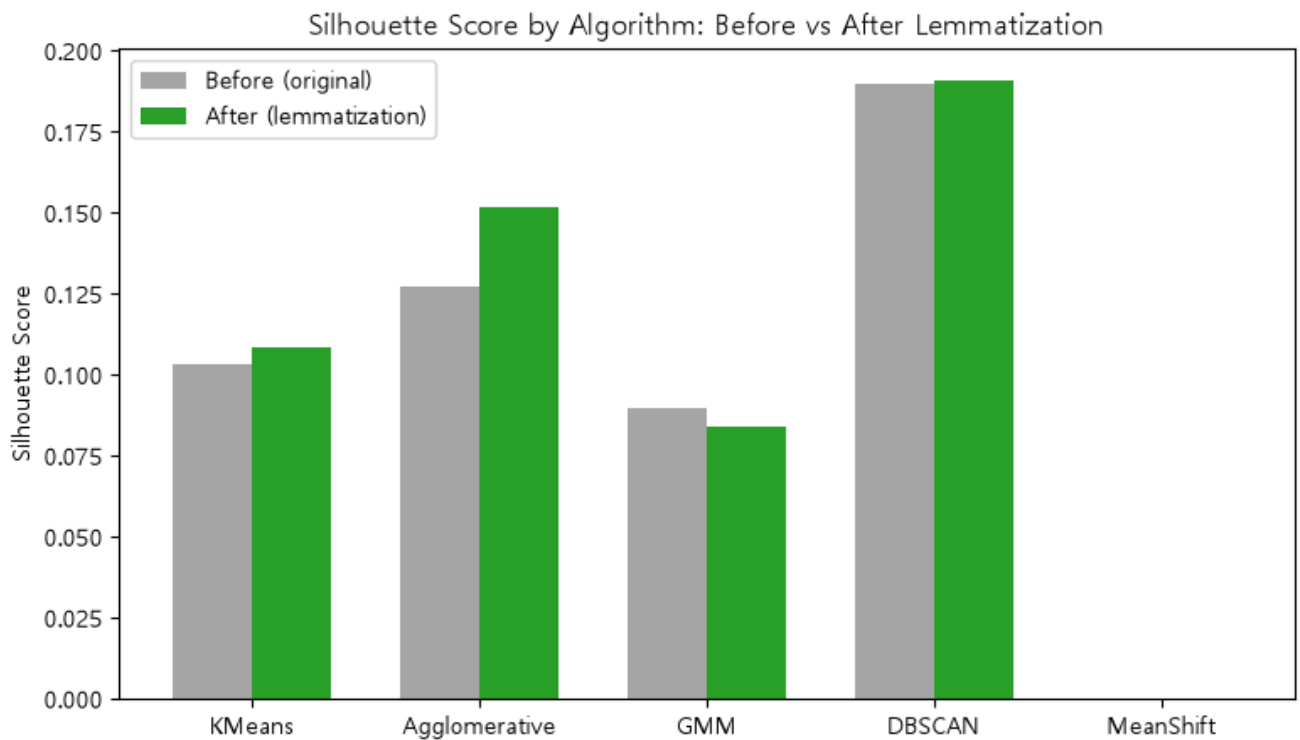


그림 4-2: 알고리즘별 표제어 추출 전후 실루엣 점수 비교

<표 4-2> 알고리즘별 Lemmatization 전/후 실루엣 점수 (권용우 노트북 실험)

알고리즘	적용 전	적용 후	변화량
KMeans	0.1035	0.1083	+0.0048
Agglomerative	0.1271	0.1518	+0.0247
GMM	0.0898	0.0838	-0.0060
DBSCAN	0.1900	0.1911	+0.0011

표제어 추출은 대체로 성능을 개선하지만 **GMM에서는 오히려 소폭 하락**했다. “모든 알고리즘에서 일관되게 개선”이 아니라 “대체로 개선되나 예외가 있다”가 정확한 결론이다.

4.1.3 팀원별 전처리·벡터화 접근 유형 분류

〈표 4-3〉 팀원별 전처리 및 벡터화 접근 유형

팀원	실제 노트북 파일	유형	접근 특징
이혜현	LHH_opinosis.ipynb	A(주)+B(보조)	표제어추출+TF-IDF(1,2) 고정 → KMeans/Agglomerative/GMM 3모델 비교 후, 54개 파라미터 조합 그리드서치로 별도 최적화 실험 병행
이진희	opinosis_countvectorize.ipynb, opinosis_tfivectorize.ipynb, opinosis_tfivectorize_dbscan.ipynb, opinosis_tfivectorize_gmm.ipynb	B	Count → TF-IDF → PCA/SVD → GMM → DBSCAN 순으로 벡터화·모델을 단계적으로 교체하며 비교
김소현	Opinosis_Preprocessing.ipynb	A	7단계 전처리 + TF-IDF(1,2) 고정 → KMeans(원본/LSA), Agglomerative, HDBSCAN, GMM 5모델 비교
전유경	JYG.ipynb, JYG_model_compare.ipynb	A	LemNormalize+TF-IDF(1,2)+영어불용어 고정 → KMeans(k=2~5) 비교가 메인, DBSCAN/GMM은 별도 노트북에서 교차검증
권용우	YWK.ipynb	B	Lemma 적용 전/후, KMeans/Agglomerative/GMM/DBSCAN/MeanShift 5알고리즘, PCA vs SVD, linkage/metric 튜닝까지 가장 폭넓게 비교

- **유형 A(고정 파이프라인, 모델 비교):** 김소현, 전유경 — 특정 전처리·벡터화를 고정하고 모델 성능만 비교
- **유형 B(여러 전처리·벡터화 시도·비교):** 이진희, 권용우 — Count vs TF-IDF, 차원축소 기법, 파라미터 등을 순차 실험
- **이혜현:** 메인 3모델(KMeans/Agglomerative/GMM) 비교는 유형 A에 가깝고, 이어서 진행한 54개 파라미터 그리드서치(6.3절)는 유형 B의 성격을 가진 혼합형이다.

4.2 텍스트 벡터화 기법 비교

4.2.1 Count Vectorizer vs TF-IDF 비교

Count Vectorizer는 단순히 단어의 출현 빈도를 세는 방식이지만, TF-IDF는 문서 내 빈도(TF)와 전체 문서에서의 희귀도(IDF)를 함께 고려한다.

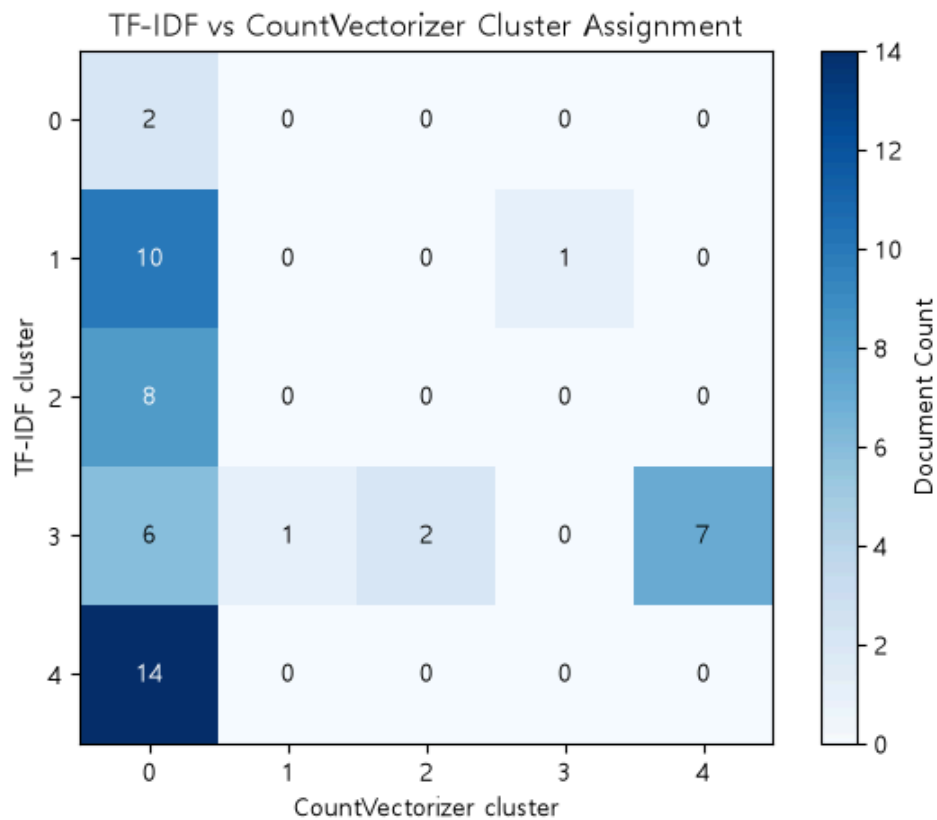


그림 4-3: TF-IDF 클러스터 vs Count 클러스터 배정 교차표 — TF-IDF 클러스터가 Count 클러스터 0(대다수 문서 집중)으로 크게 쏠림을 보여줌

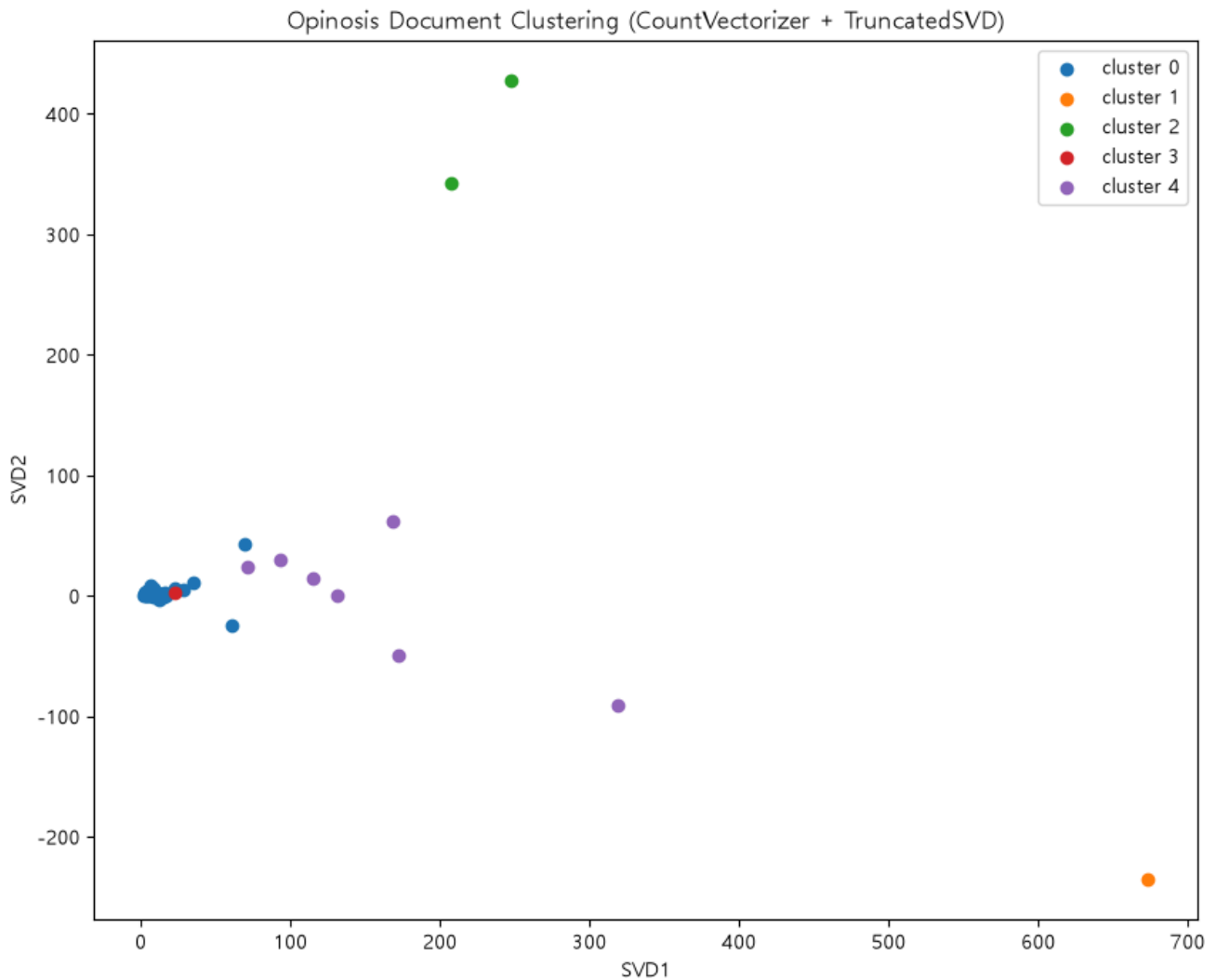


그림 4-4: Count Vectorizer 벡터화 후 SVD 2D 축소 결과 — 클러스터 분리 불명확

<표 4-4> Count vs TF-IDF 벡터화 성능 비교 (이진희 노트북, 동일 조건 내 비교: $ngram(1,2) \cdot min_df=0.05 \cdot max_df=0.85$ 공통 적용)

벡터화 기법	클러스터 분포	Silhouette	정성 평가
CountVectorizer (k=3)	(48, 2, 1)	0.561	극단적 불균형 — 대부분 문서가 1개 군집에 몰림
CountVectorizer (k=4)	(46, 2, 2, 1)	0.437	여전히 심각한 불균형
TF-IDF (k=4)	(18, 15, 10, 8)	0.067	분포는 균형적이나 Silhouette은 오히려 낮음

이 결과는 역설적으로 중요한 시사점을 준다: **Silhouette Score가 높다고 군집이 의미상 더 잘 나뉜 것은 아니다.** CountVectorizer는 절대 빈도가 큰 소수 단어(hotel, room 등)에 의해 대부분 문서가 하나의 군집으로 뭉쳐 Silhouette은 높게 나오지만, 실제로는 Kindle·호텔·전자기기·자동차가 뒤섞인 무의미한 군집이다. 반대로 TF-IDF는 문서별 희귀 단어 가중치 덕분에 도메인별로 훨씬 뚜렷하게 갈리지만, 군집 간 경계가 좁아져 Silhouette 수치 자체는 낮게 나온다.

KMeans Clustering - 3D

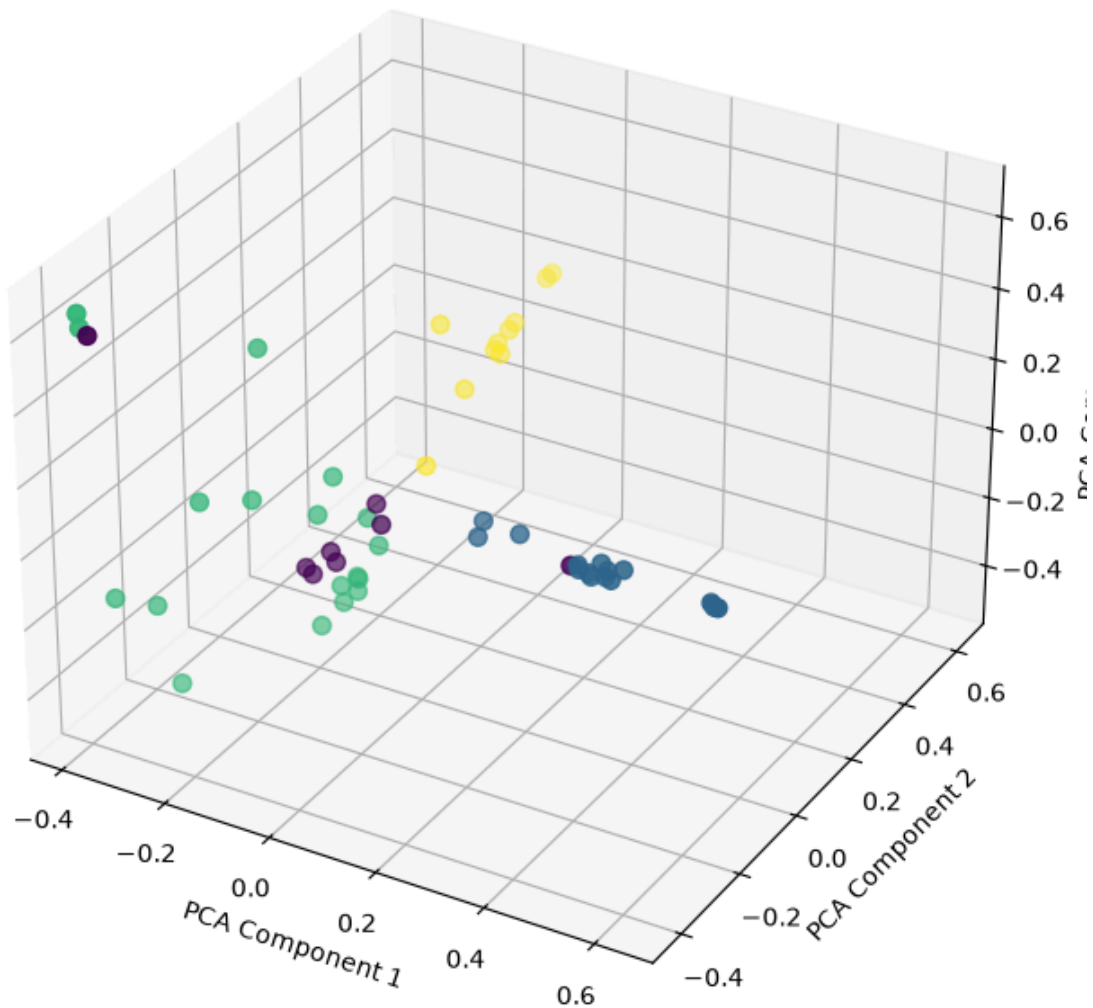


그림 4-5: TF-IDF+KMeans(k=4) 결과를 PCA 3차원으로 시각화 — Count Vectorizer 대비 문서들이 뚜렷하게 갈라지는 것을 확인할 수 있음

한편, 6.2절에서 다루는 김소현의 별도 파이프라인(7단계 전처리 + TF-IDF, ngram(1,2), min_df=0.05, max_df=0.85, 4,498차원)에서는 KMeans가 **ARI/NMI = 1.00**으로 51개 문서를 도메인과 완전히 일치하게 분류했다(〈표 6-2〉 참조). 이는 이진희의 비교와는 별개의 노트북·별개의 토큰라이저에서 나온 결과이며, TF-IDF의 우수성을 뒷받침하는 독립적인 근거로 함께 제시한다.

4.2.2 초기 TF-IDF 하이퍼파라미터 설정

〈표 4-5〉 TF-IDF 초기 하이퍼파라미터 설정 (이혜현 노트북)

파라미터	설정값	선택 근거	효과
ngram_range	(1, 2)	단일 단어와 2단어 조합 (battery life, gas mileage) 모두 반영	복합어 보존
min_df	0.05	전체 문서의 5% 미만 등장 단어 제거	노이즈 감소
max_df	0.85	85% 이상 등장하는 일반어 제거	도메인 특화 어휘 강조
sublinear_tf	True	문서 길이 편차 (836~12,750) 큰 데이터에 로그 스케일링 적용	길이 정규화

산출물 규격: 51개 문서 × 4,139개 피처의 희소 행렬 (LemNormalize 토큰나이저 기준. 이 행렬의 희소도는 노트북에 별도로 계산되어 있지 않다 — 6.1절 김소현 파이프라인(51×4,498, 희소도 87.64%)과 혼동하지 않도록 주의)

종합하면, Count Vectorizer 대비 TF-IDF 적용 시 도메인 고유 어휘의 변별력이 뚜렷하게 개선되었으며(4.2.1절), 6.3절의 그리드 탐색을 통해 이해현 파이프라인은 피처 수를 83% 정제하면서 실루엣 점수를 82% 개선하여 소규모 데이터셋에서 파라미터 최적화의 중요성을 입증하였다.

5. 분석방법 (Analysis Method)

5.1 전처리·벡터화 필요성 및 과정 개요

EDA(3.1~3.2절)에서 확인한 “불용어가 최빈 단어 상위권을 차지한다”는 문제는 4.1절의 표제어 추출·불용어 제거 파이프라인으로, “Count Vectorizer는 절대 빈도가 큰 단어에 지배된다”는 4.2.1절의 문제는 TF-IDF 가중치 도입으로 직결되었다. 즉 본 프로젝트의 전처리·벡터화 설계는 임의의 관용적 절차가 아니라, EDA에서 실측한 문제를 하나씩 대응해 설계한 결과다.

5.2 사용 모델 설명

〈표 5-1〉 군집화 모델 선정 이유

모델	유형	선정 이유
KMeans	중심 기반	빠른 수렴과 해석 용이성 — 51개 소규모 데이터셋에 가장 안정적인 베이스라인
Agglomerative	계층 기반	초기값에 비의존적(완전 결정론적)이며 덴드로그램으로 계층 구조를 함께 확인 가능
GMM	확률 기반(소프트 배정)	문서가 여러 클러스터에 애매하게 걸릴 수 있다는 가능성을 반영하기 위한 대조군
DBSCAN	밀도 기반	클러스터 수를 미리 정하지 않고 이상치(노이즈)를 자동으로 걸러낼 수 있는지 검증
HDBSCAN	밀도 기반(계층적)	DBSCAN의 단일 eps 한계를 보완하는 밀도 기반 대안으로서 추가 비교

5.3 하이퍼파라미터 최적화 기법 설명

이해현은 `ngram_range × min_df × max_df × n_clusters` 54개 조합을 전수 그리드서치하여 Silhouette Score를 최대화하는 조합을 탐색했다(6.3절). DBSCAN 계열에서는 그리드서치 대신 **k-distance elbow plot**(문서별 k번째 최근접 이웃까지의 거리를 오름차순 정렬한 뒤 꺾이는 지점을 찾는 방법)으로 eps를 결정하는 방식을 전유경이 사용했으며(6.2.5절), 두 방식 모두 “임의로 고른 값”이 아니라 데이터 자체의 구조에서 근거를 도출했다는 공통점이 있다.

5.4 실행환경

본 보고서 검증 과정에서 재실행에 사용한 핵심 라이브러리 버전은 다음과 같다.

라이브러리	버전
pandas	3.0.3
numpy	2.4.6
scikit-learn	1.9.0
nltk	3.10.3

5.5 코드 재사용 및 함수화

5개 노트북을 전수 조사한 결과, 이해현·이진화·전유경·권용우 네 명이 각자 독립적으로 “클러스터별 핵심 키워드와 소속 파일명을 추출한다”는 동일한 목적의 함수를 서로 다른 이름(`get_cluster_details`, `get_cluster_top_features`, `print_cluster_details` 등)으로 반복 구현했다. 김소현의 `preprocess_text` (7단계 전처리)와 이해현의 `LemNormalize` (전유경이 동일한 이름·구조로 재사용)도 사실상 같은 문제를 각자 다시 푼 사례다. 즉 “공동 함수”라기보다는 “유사한 패턴을 각자 독립적으로 재구현”한 경우가 대부분이었다는 것이 정직한 실태이며, 이는 10.1절 회고의 개선점(“공동 전처리·평가 함수를 팀 초기에 모듈화”)과 직접 연결된다. 실제로 반복된 패턴을 정리해 재사용 가능한 형태로 다듬은 코드는 `Opinosis_최종통합분석.ipynb` (본 reports 폴더)와 부록에 정리했다.

6. 모델링

6.1 차원 축소

4,000차원대의 고차원 피처는 “차원의 저주” 문제를 야기할 수 있으므로, 차원 축소를 검토하였다. 이 절의 모든 수치는 **김소현의 파이프라인**(7단계 전처리 + TF-IDF, ngram(1,2), min_df=0.05, max_df=0.85 → 51개 문서 × 4,498개 피처, 밀도 12.36%·희소도 87.64%)을 기준으로 한다.

〈표 6-1〉 차원축소 기법 비교

특성	TruncatedSVD (LSA)	PCA
희소 행렬 입력 지원	직접 처리 가능	변환 필요
0의 의미 보존	보존 (단어 부재)	상실 (평균 중심화)
텍스트 표준 기법	LSA로 정의됨	신호 왜곡 우려
클러스터 분리도	우수	미흡

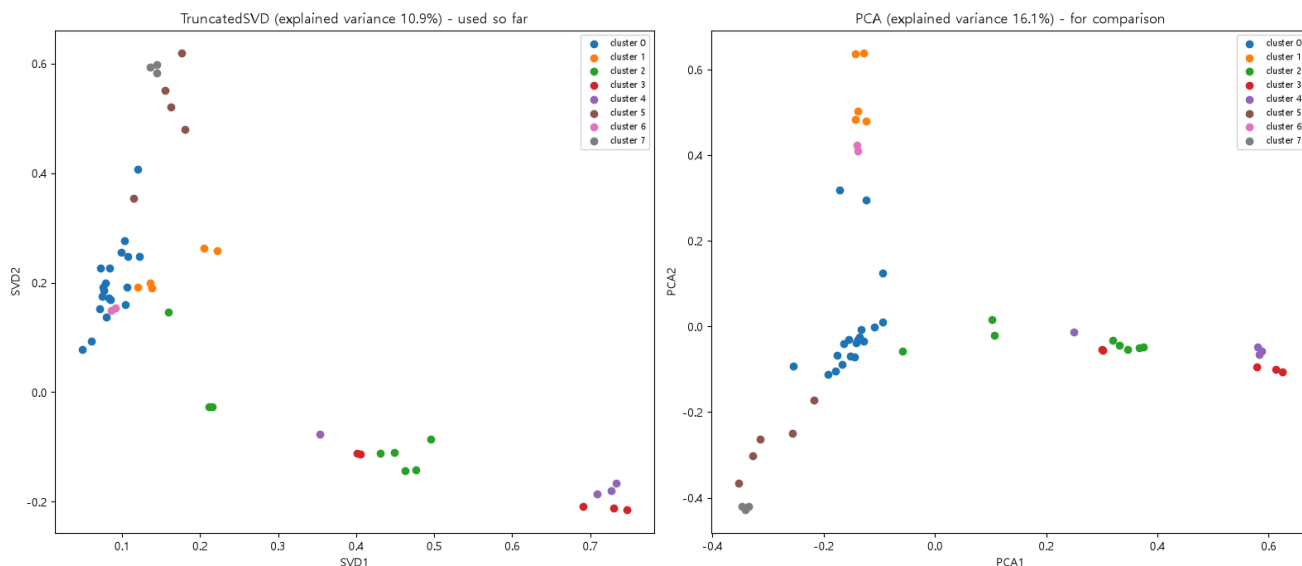


그림 6-1: TruncatedSVD와 PCA 2D 축소 결과 비교 — SVD가 클러스터 분리를 더 명확하게 보여줌

최종 설정: n_components=30 (누적 설명분산 89.29%), L2 정규화 적용(KMeans ARI: 0.3666 → 0.8869, +141% 개선) → 산출물: 51개 문서 × 30개 피처의 Dense 행렬

참고: 같은 개념을 이진화도 별도로 검증했다. TF-IDF 행렬에 TruncatedSVD(n=20)와 PCA(n=20)를 각각 적용한 뒤 동일한 GMM(n=4)을 학습시킨 결과, 두 방식의 클러스터 배경(분포 39:5:4:3)은 완전히 동일했지만 Silhouette 점수는 SVD 0.042 vs PCA 0.121로 약 3배 차이가 났다. 이는 군집 결과(라벨)가 같아도, 축소된 공간에서 각 문서의 좌표·거리 구조가 달라지면 거리 기반 평가지표(Silhouette)는 다르게 나올 수 있음을 보여주는 사례로, “Silhouette 수치만으로 파이프라인 우열을 단정할 수 없다”는 4.2.1절의 결론과도 일맥상통한다.

6.2 군집화 모델 비교

6.2.1 모델별 성능 비교표

〈표 6-2〉 군집화 모델 종합 성능 비교 (전부 김소현 노트북 실측)

모델	입력 벡터	Silhouette	ARI	NMI	종합 평가
KMeans (원본)	4,498D	0.0860	1.0000	1.0000	최우수
KMeans (LSA)	30D	0.1040	0.8869	0.8842	우수
Agglomerative	30D	0.1060	0.8768	0.8525	우수
GMM (30D)	30D	0.0905	0.5939	0.7284	중간
HDBSCAN	30D	0.1175	0.3248	0.3908	부적절

(KMeans-원본만 4,498D 희소 행렬, 나머지 넷은 동일한 30D LSA(+L2정규화) 행렬 기준 — KMeans-원본의 Silhouette은 나머지
지와 직접 비교 불가)

참고로 이해현의 별도 파이프라인(LemNormalize, 4,139D)에서는 k=3 KMeans/Agglomerative/GMM 모두 원본 공간
Silhouette 0.0965로 동일했고, EDA 가설 대비 자동차 도메인 11건 중 10건만 일치(약 96%)했다 — 완전한 ARI=1.0은 아니었
다.

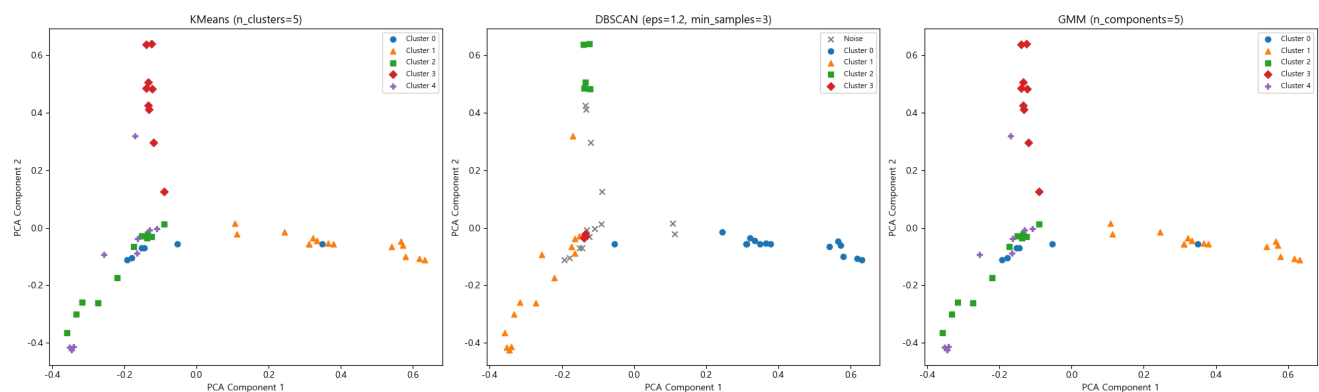


그림 6-2: 전유경의 독자 파이프라인(LemNormalize+TF-IDF)에서도 KMeans-DBSCAN-GMM 세 모델을 나란히 비교한 결과, KMeans가 가장
뚜렷한 군집 분리를 보였다 — 김소현 파이프라인과는 별개의 벡터·k값(k=5)에서 얻은 결과이나 “KMeans가 이 데이터에 가장 안정적”이라는 결
론이 일치한다(전유경의 k=5 파이프라인 자체는 6.4절에서 상세히 다룬다).

6.2.2 KMeans (원본 4,498D 공간, 김소현 파이프라인) — 최종 채택

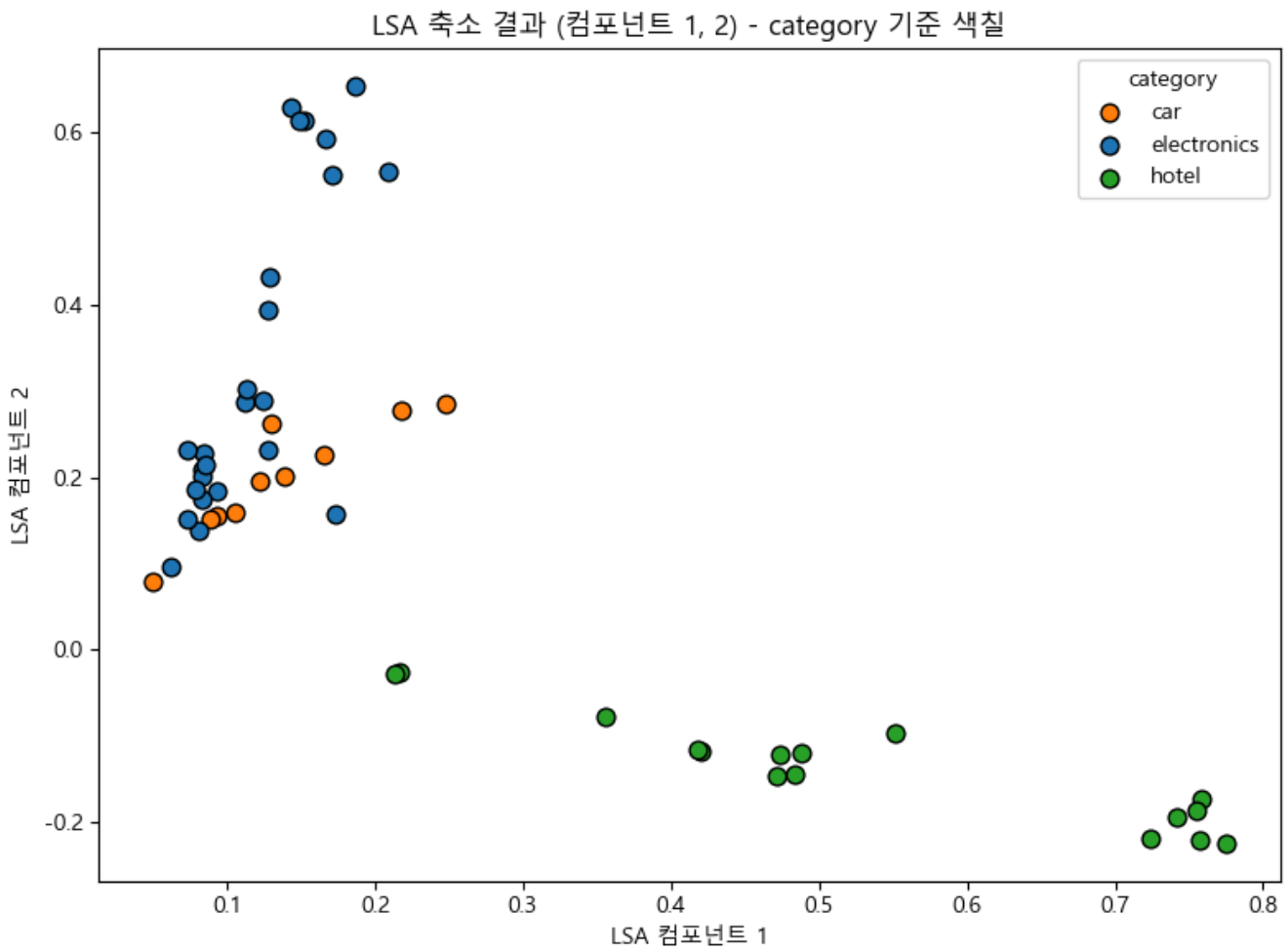


그림 6-3: LSA 30D 중 분산이 가장 큰 두 컴포넌트로 2D 시각화, category(전자기기/자동차/호텔) 기준 색칠 — 참고용 투영이며, 실제 군집화는 원본 4,498D 및 30D 전체를 사용

〈표 6-3〉 KMeans (k=3, 4,498D) 클러스터링 결과

클러스터	문서 수	도메인 분류	도메인 일치율
Cluster 0	25개	전자기기	100%
Cluster 1	10개	자동차	100%
Cluster 2	16개	호텔	100%

〈표 6-4〉 클러스터별 핵심 단어 (Top 5)

클러스터	핵심 단어 1	핵심 단어 2	핵심 단어 3	핵심 단어 4	핵심 단어 5
0 (전자기기)	screen	battery	keyboard	battery life	kindle
1 (자동차)	interior	seat	mileage	comfortable	gas
2 (호텔)	room	hotel	service	staff	food

해석: ARI/NMI = 1.0은 51개 문서가 모두 정확히 도메인별로 분류되었음을 의미한다.

6.2.3 k값 최적화 분석

권용우와 전유경이 각자의 파이프라인에서 k값·알고리즘을 바꿔가며 얻은 아래 시각화들은 서로 다른 벡터에서 나온 결과이지만, 공통적으로 (1) KMeans·Agglomerative 계열이 가장 안정적으로 군집을 분리하고 (2) k를 늘릴수록 전자기기 대군집이 GPS·노트북·배터리 등 세부 주제로 갈라지는 경향을 보여준다. 이는 7.1절 Track②(k=5 세분화)의 타당성을 뒷받침하는 정황 근거이기도 하다.

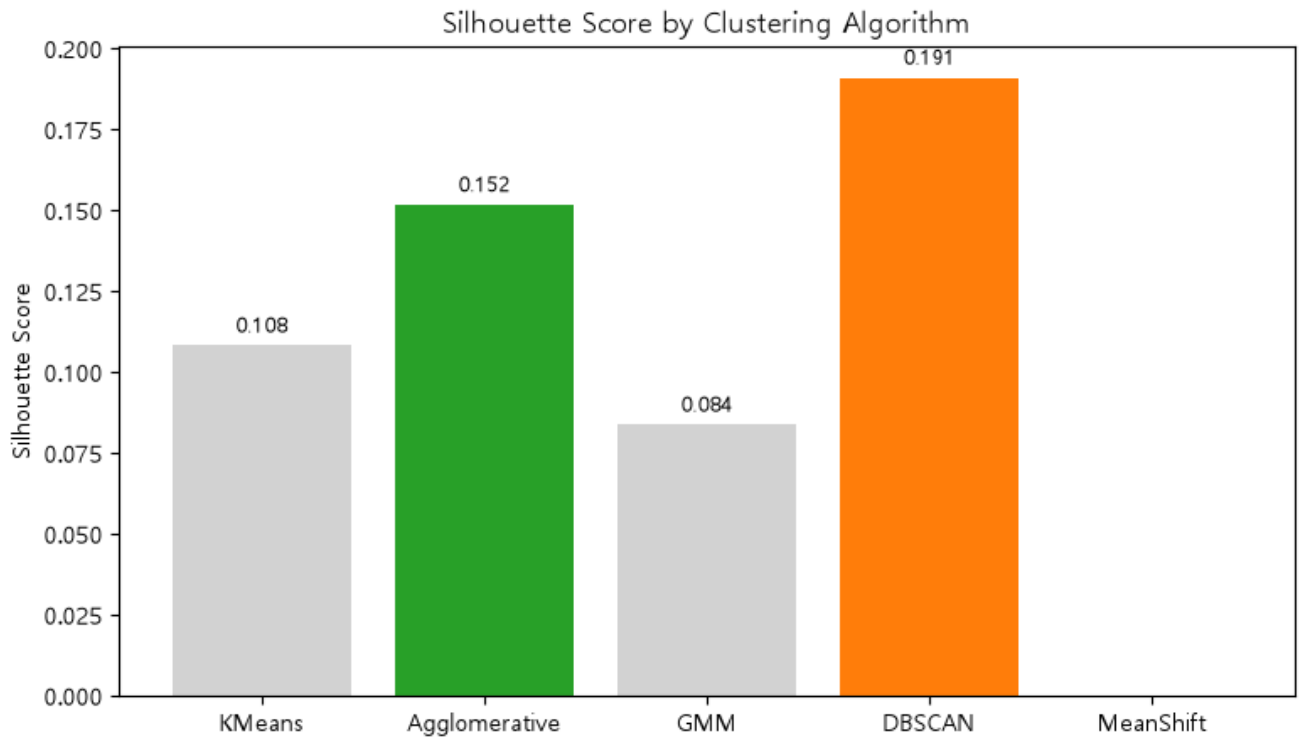


그림 6-4: 알고리즘별 Silhouette Score 비교 (권용우 노트북, 자체 lemmatization 파이프라인 기준)

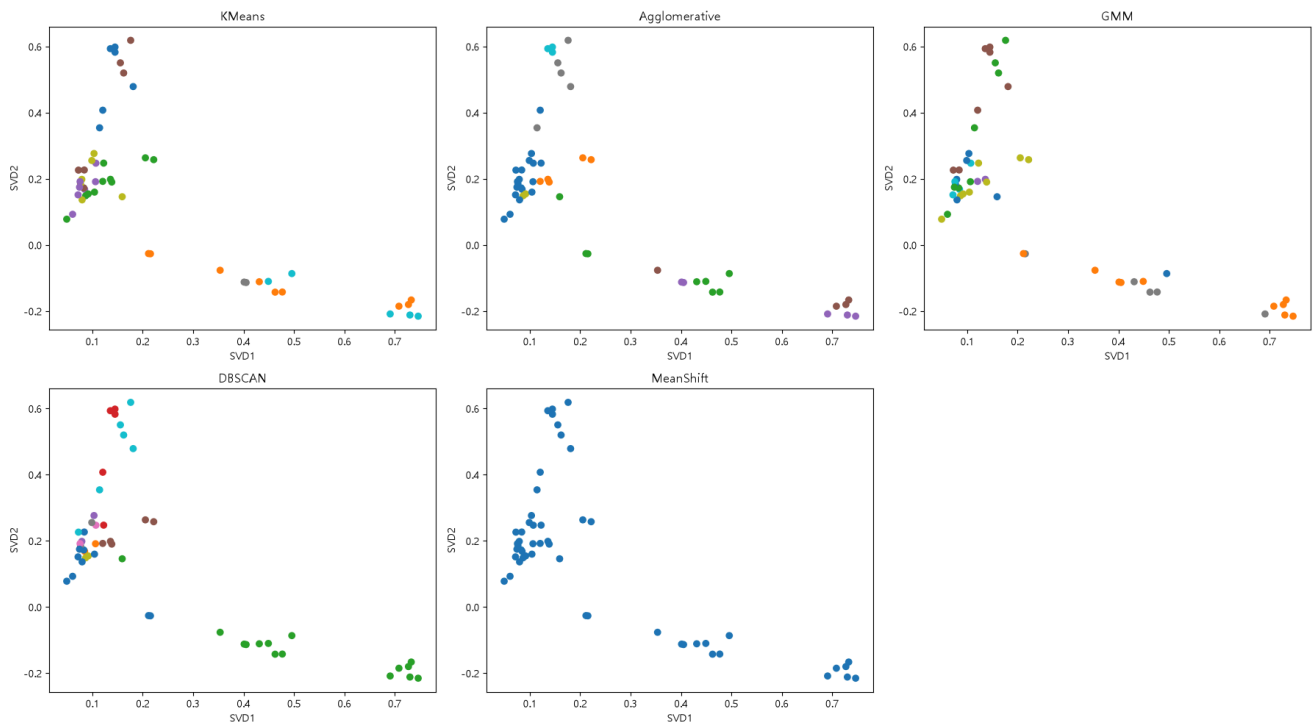


그림 6-5: 5개 알고리즘(KMeans/Agglomerative/GMM/DBSCAN/MeanShift) SVD 2D 산점도 비교 (권용우 노트북)

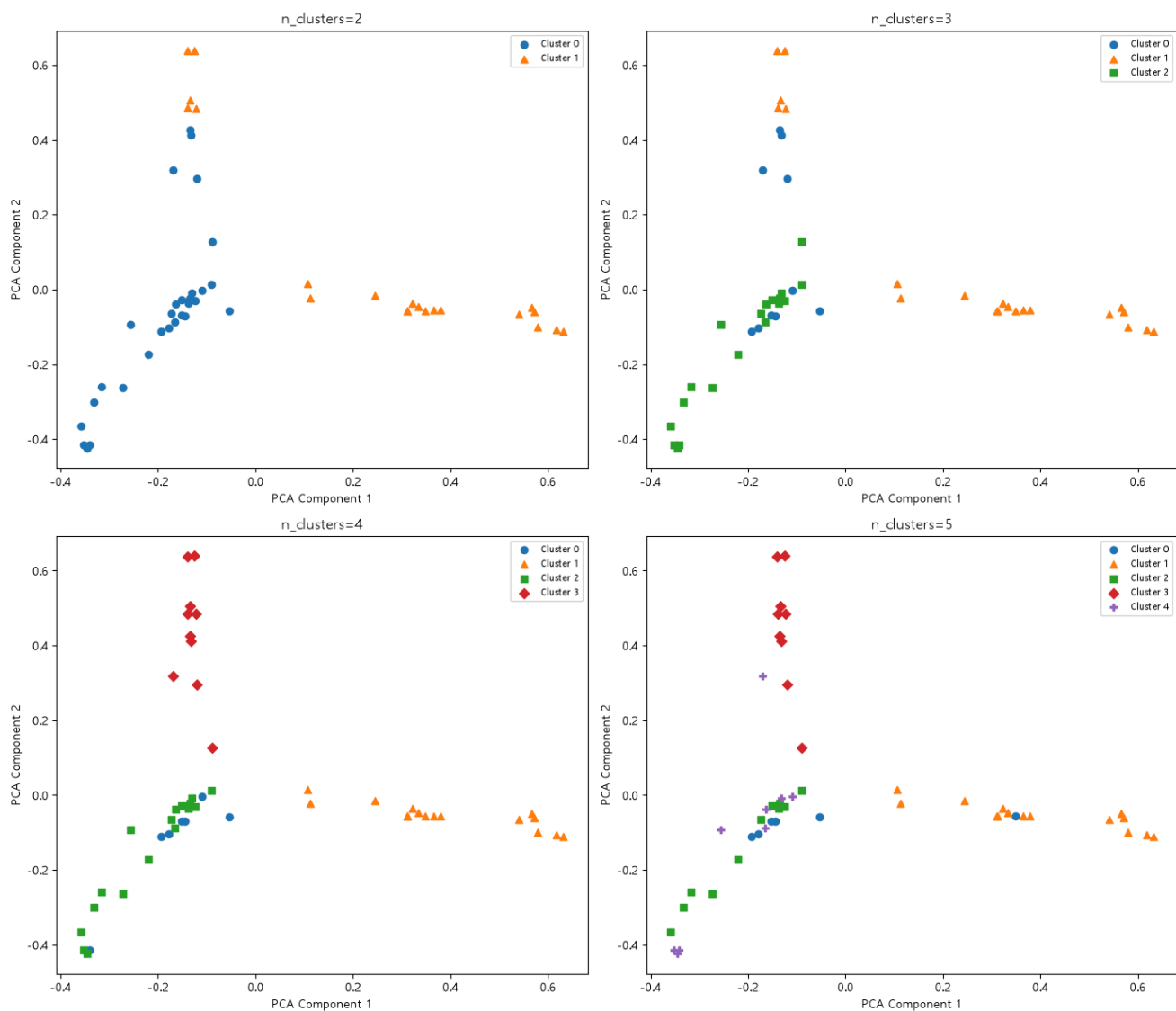


그림 6-6: 전유경 파이프라인에서 KMeans $n_clusters=2,3,4,5$ 를 각각 학습해 같은 PCA 2D 좌표 위에 비교 — k 가 커질수록 전자기기 대군집이 세부 주제(GPS/노트북/배터리 등)로 갈라지는 양상을 보여준다.

6.2.4 계층적 클러스터링 분석 (권용우 노트북)

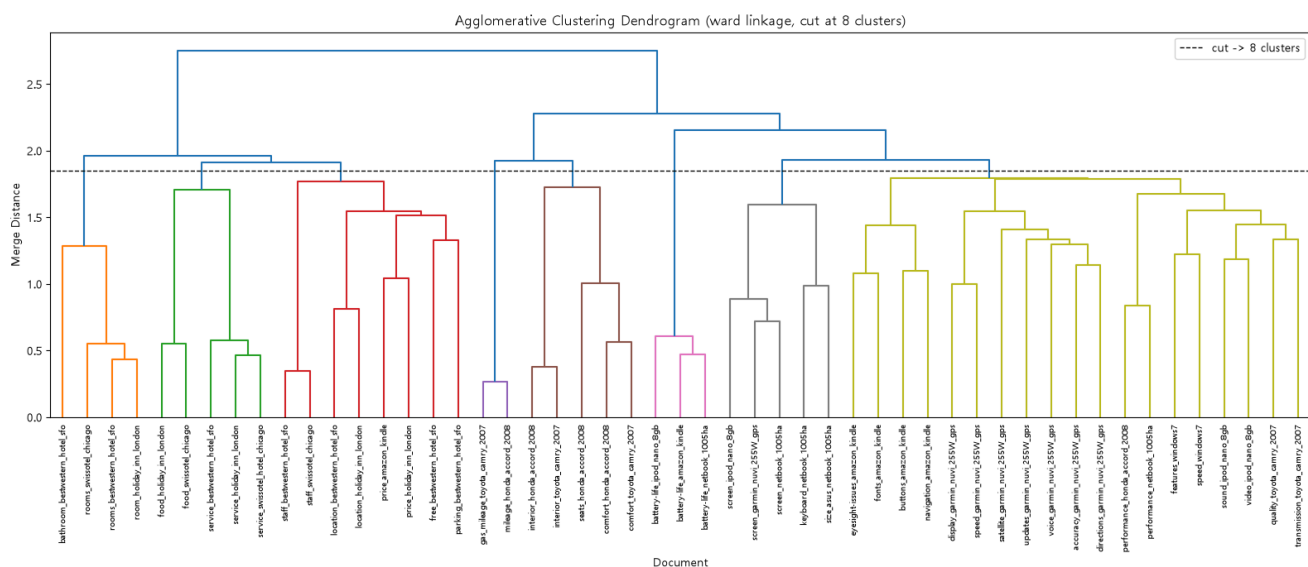


그림 6-7: Agglomerative(ward linkage) 덴드로그램, best_k=8 기준 절단선 표시

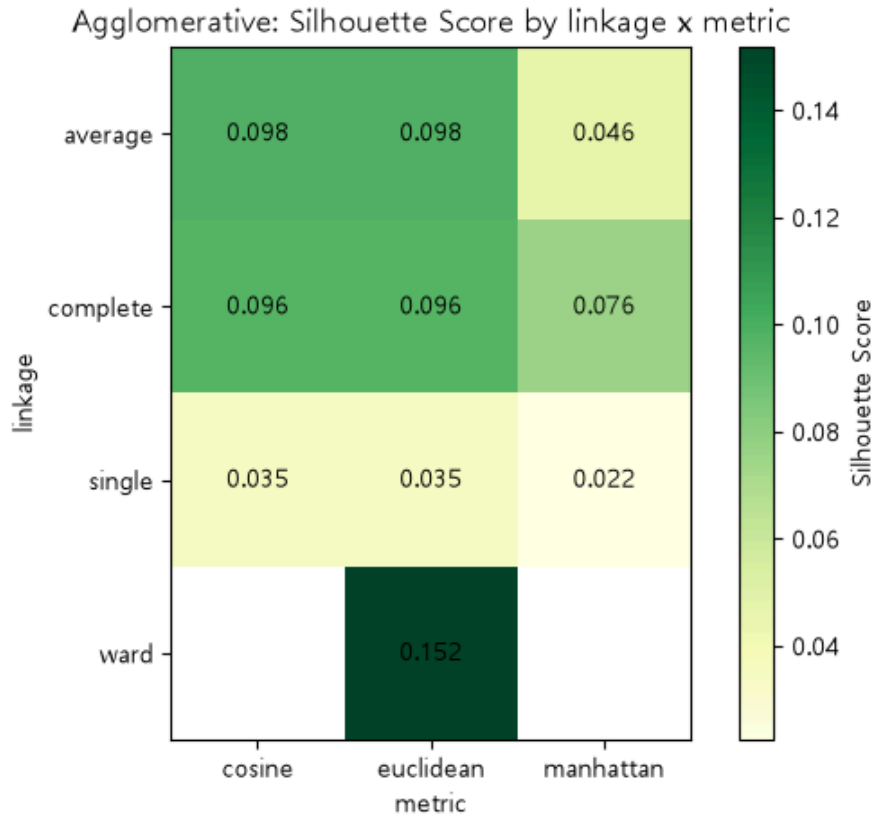


그림 6-8: linkage × metric 조합별 Silhouette 히트맵 — ward+euclidean 조합이 0.152로 최적

권용우는 이 결과를 바탕으로 linkage/metric 4×3 조합을 전수 스캔했으나, 이론적으로 텍스트에 더 적합하다고 알려진 cosine 거리 조합보다 **원래 쓰던 기본값(ward+euclidean)**이 그대로 최고점이었다. 이는 튜닝의 실패가 아니라, 이미 최선의 설정을 사용하고 있었음을 데이터로 재확인한 결과로 볼 수 있다.

6.2.5 DBSCAN 및 밀도 기반 모델 평가

DBSCAN 및 HDBSCAN이 <표 6-2>에서 나타난 바와 같이 상대적으로 낮은 성능(ARI 0.3248, 0.3908)을 보인 것은 51개라는 소규모 텍스트 데이터셋의 특성 때문으로 추정된다. 고차원 텍스트 공간(TF-IDF/LSA)에서는 문서 간 거리가 균일해지는 거리 수렴 현상(distance concentration)이 발생하기 쉽고, 51개라는 적은 샘플 수로는 밀도 기반 알고리즘이 유의미한 코어 포인트를 형성하지 못해 상당수 문서를 노이즈로 분류하는 경향을 보였다. 이 결론은 서로 다른 세 명의 독립적인 DBSCAN 실험에서 공통으로 확인되었다.

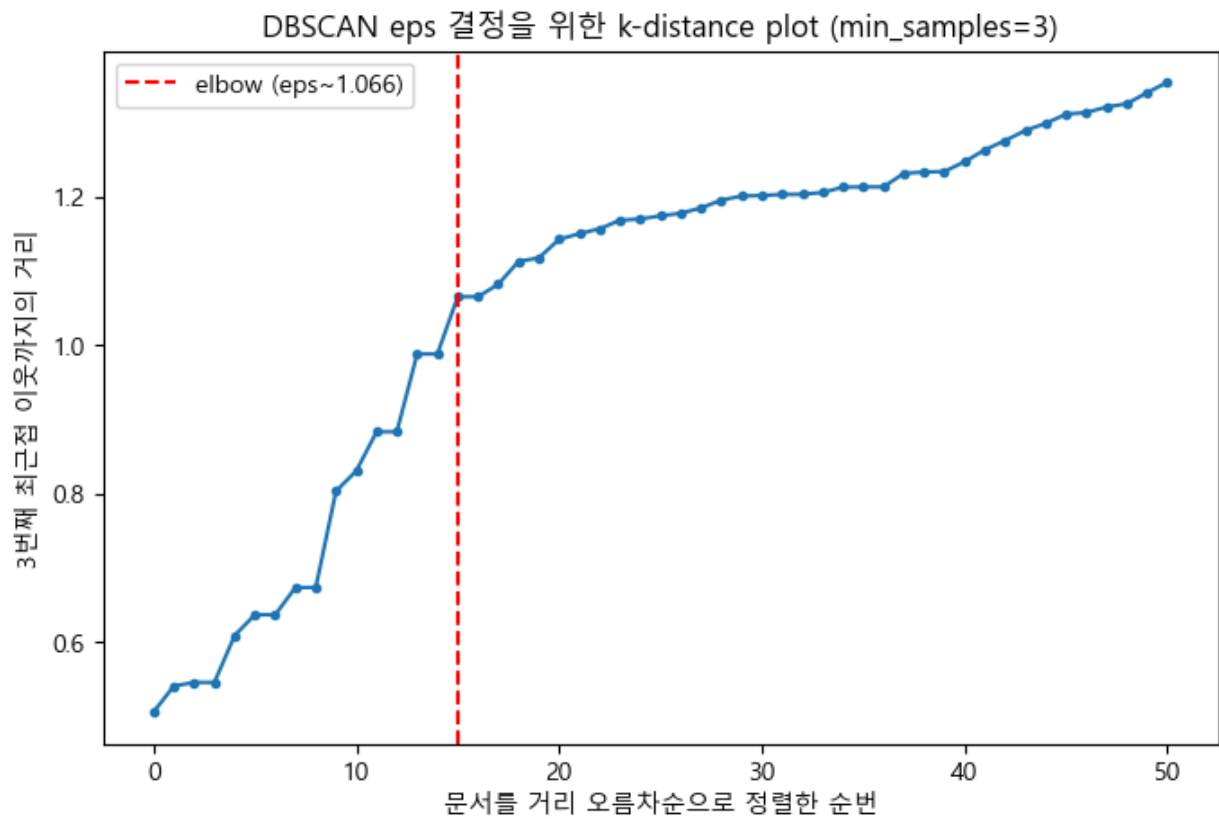


그림 6-9: 전유경의 k-distance elbow plot — $eps \approx 1.07 \sim 1.18$ ($min_samples$ 별) 지점을 근거로 $eps=1.2$ 를 선정. eps 1.00~1.30 구간에서 클러스터 개수(4~5개)는 안정적이거나 노이즈 문서 수는 35개→14개까지 크게 흔들렸다.

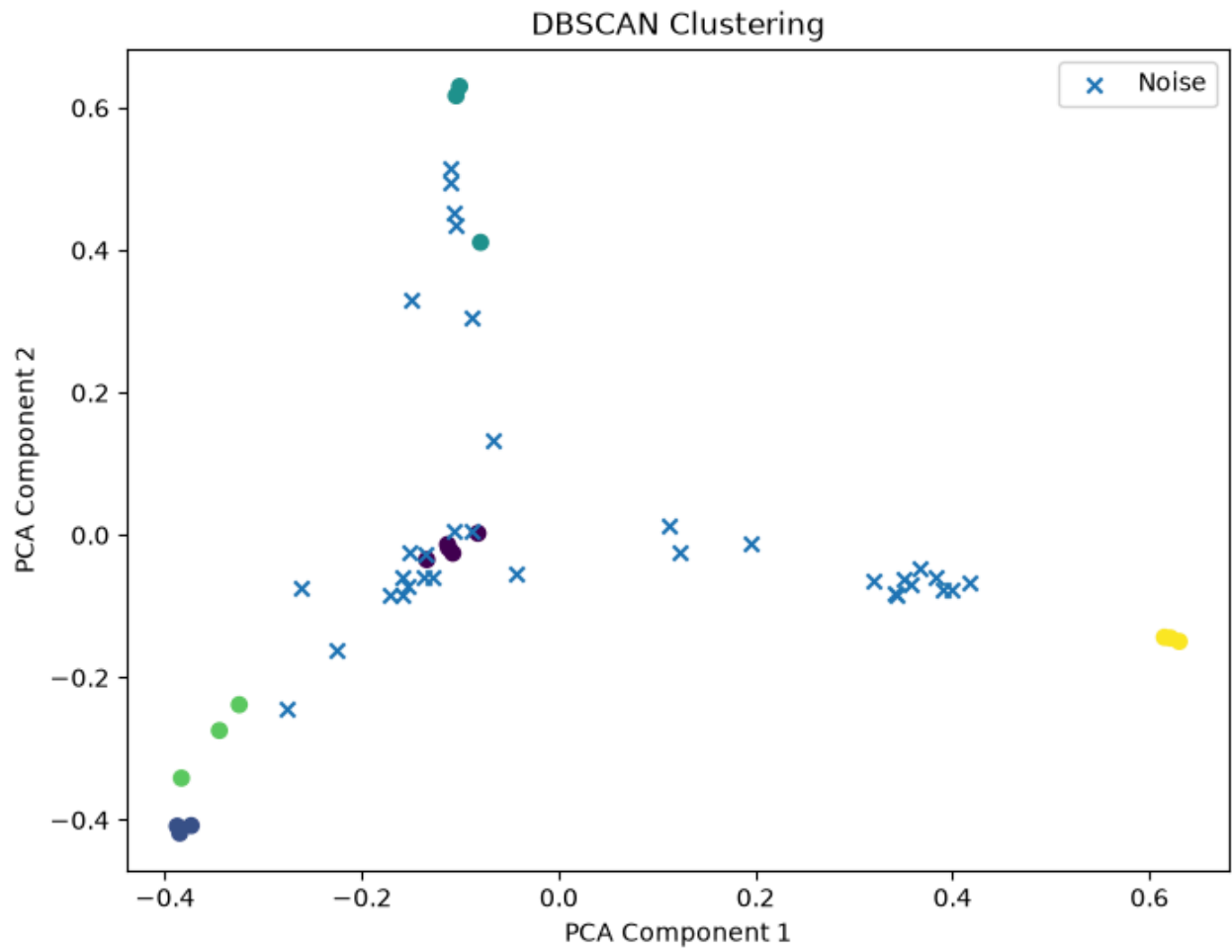


그림 6-10: 이진희의 DBSCAN($\epsilon=0.5$) 결과 — 노이즈 34개(66.7%)로 대부분 문서가 노이즈 처리되지만, 남은 17개는 battery-life, comfort/seats(자동차) 등 의미적으로 매우 순수한 소규모 군집을 형성

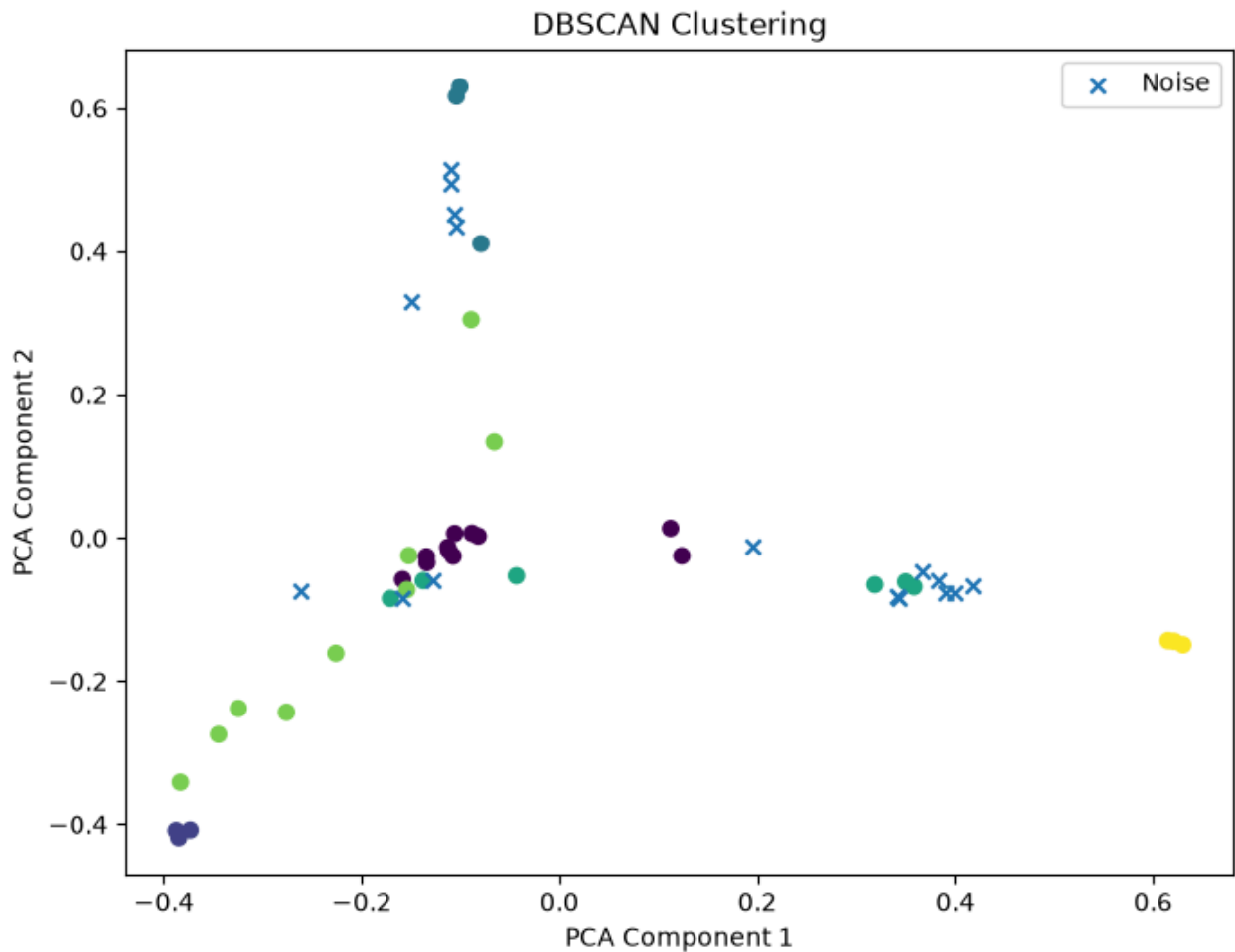


그림 6-11: 이진희의 DBSCAN($\text{eps}=0.8$) 결과 — eps 를 높여 노이즈를 16개(31%)까지 줄였으나, 그 대가로 서로 다른 도메인(Garmin GPS + Windows7, Kindle + 호텔 등)이 한 군집으로 섞이는 현상이 나타났다

이진희는 eps 0.2~0.8 구간을 전수 실험하여 “ eps 를 낮추면 순수하지만 노이즈가 많고, eps 를 높이면 노이즈는 줄지만 서로 다른 도메인이 섞인다”는 트레이드오프를 직접 확인했다. 전유경 역시 k-distance elbow로 구한 eps (약 1.07~1.18)가 임의로 고른 $\text{eps}=1.2$ 와 크게 다르지 않음을 확인하면서도, “노이즈로 분류되는 구체적 문서 목록은 파라미터에 따라 계속 바뀌어 확정적 근거로 삼기 어렵다”고 결론지었다. 세 명의 서로 다른 벡터·구현(이진희: PCA 20D+euclidean, 전유경: 원본 TF-IDF+euclidean, 권용우: 원본 TF-IDF+cosine)에서 공통적으로 “대략 4~6개의 주제 군집은 찾아내지만 eps 민감도가 매우 높다”는 동일한 결론에 도달했다.

6.2.6 GMM 모델 평가

GMM은 원본 고차원 희소 공간(4,498D)과 30D LSA 축소 공간에서 각각 평가되었다. 원본 공간에서는 KMeans와 유사한 수준의 분리를 보였으나, 30D LSA 공간에서는 ARI 0.5939로 성능이 저하되었다(<표 6-2>). 이는 차원 축소 과정에서의 정보 손실과 LSA 축소 공간에서 GMM의 공분산 행렬 추정이 제약을 받기 때문으로 판단된다.

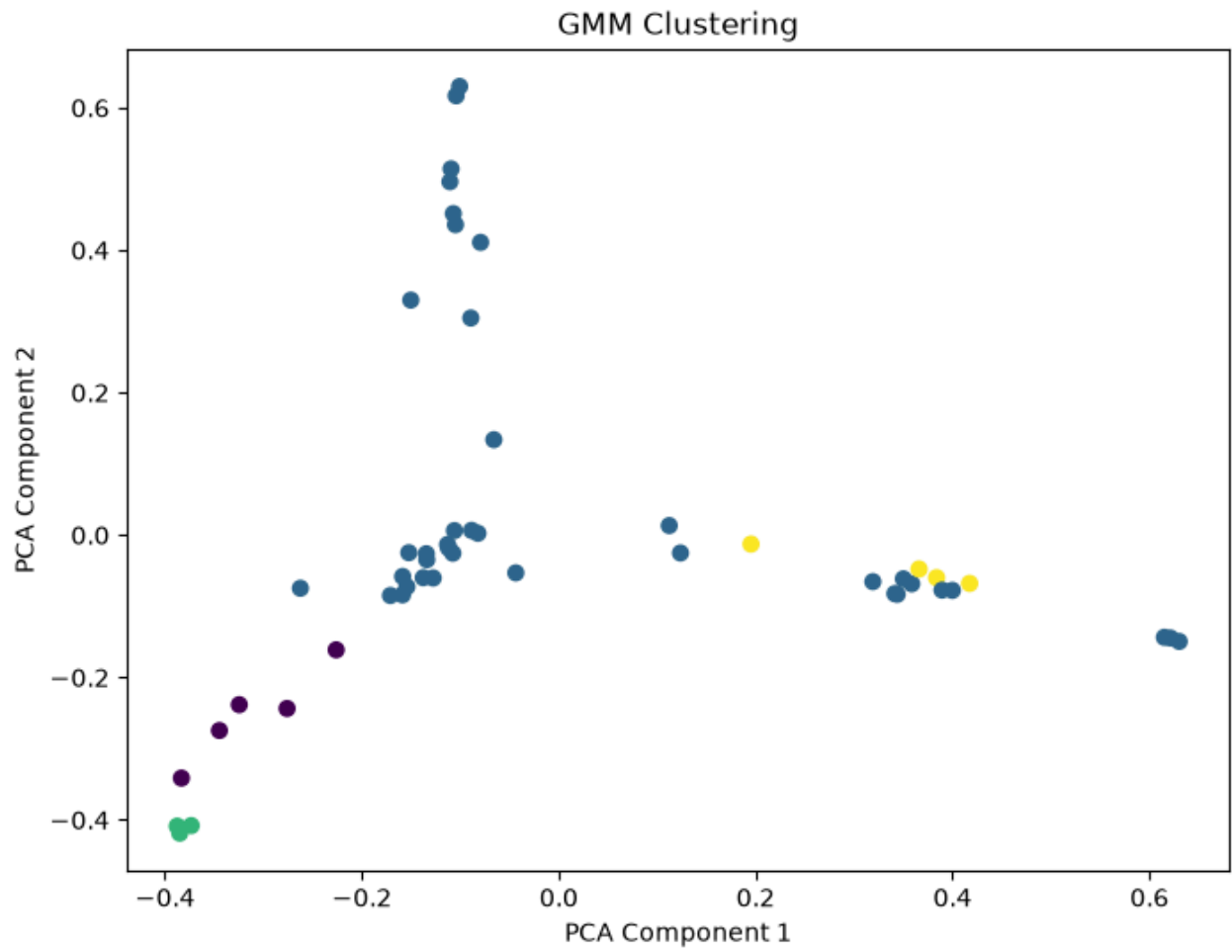


그림 6-12: 이진희의 GMM(SVD $n=20$, $n_components=4$) 결과 산점도

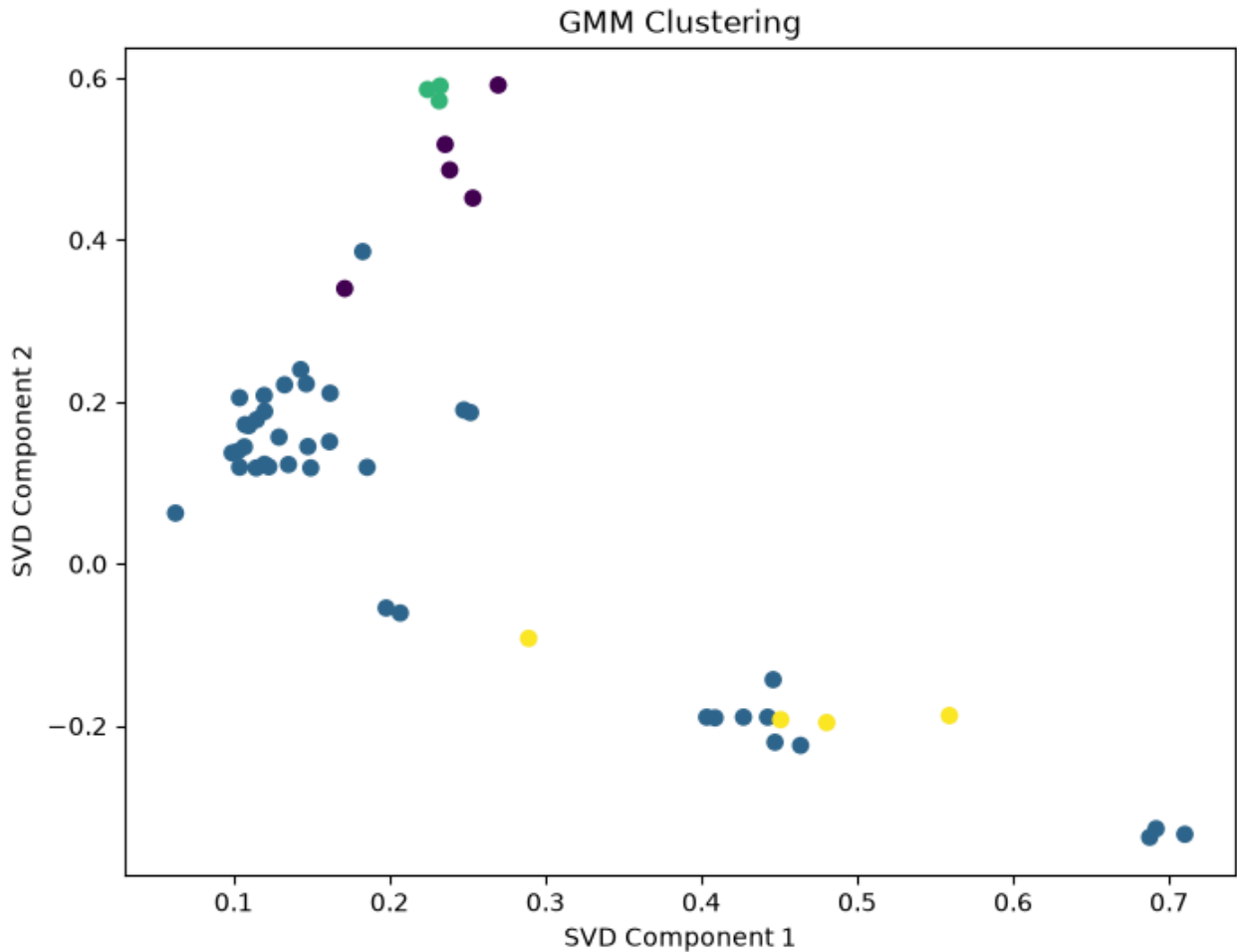


그림 6-13: 이진희의 GMM 클러스터별 핵심 단어 재확인 — screen/keyboard(전자기기), service/hotel(호텔), battery/life(배터리), interior/seat(자동차)로 도메인별 특성은 확인되나 39개 문서가 하나의 군집(Cluster 1)에 과도하게 집중되는 불균형이 나타났다

전유경 노트북에서는 GMM($n_components=5$, $covariance_type='diag'$)이 자신의 KMeans($k=5$)와 클러스터 배정이 완전히 동일($ARI=1.0$)하게 나와, 이 데이터에서는 GMM이 KMeans 대비 추가 정보를 주지 못하는 경우도 확인되었다.

6.2.7 모델 선택 가이드 (권용우 종합)

권용우는 5개 알고리즘(KMeans/Agglomerative/GMM/DBSCAN/MeanShift)의 장단점을 실측 결과와 함께 일반화하여 아래와 같은 선택 기준을 제시했다.

〈표 6-5〉 데이터 특성별 군집화 모델 선택 가이드 (권용우 노트북 종합)

기준	조건	권장 모델
데이터 규모	매우 큼 (수만~수백만)	KMeans
데이터 규모	적음 (수십~수백, 이번 데이터)	KMeans / Agglomerative — DBSCAN·MeanShift는 파라미터 하나에도 결과가 크게 흔들림
클러스터 수	미리 알거나 정해도 됨	KMeans / Agglomerative / GMM
클러스터 수	자동으로 찾고 싶음	DBSCAN (단, 고차원 희소 데이터에선 MeanShift는 거의 항상 실패)
분류 방식	전량을 빠짐없이 분류해야 함	KMeans / Agglomerative / GMM
분류 방식	이상치를 걸러내도 됨	DBSCAN
분류 방식	소속이 애매할 수 있음(소프트 배정)	GMM
재현성	항상 동일한 결과가 나와야 함	Agglomerative (완전 결정론적, 랜덤성 없음)

6.3 하이퍼파라미터 최적화 (그리드 탐색)

이해현이 54가지 파라미터 조합을 체계적으로 탐색하여 최적값을 발견하였다. 탐색 범위: ngram_range [(1,1),(1,2)] × min_df [0.03, 0.05, 0.10] × max_df [0.75, 0.85, 0.95] × n_clusters [3, 4, 5].

〈표 6-6〉 최적 파라미터 설정 비교

파라미터	기본값	최적값	변화	효과
ngram_range	(1,2)	(1,1)	Bigram 제거	희소성 감소
min_df	0.05	0.10	강화	희귀어 제거
max_df	0.85	0.75	강화	일반어 제거
n_clusters	3	5	확대	세분화 달성

초기 탐색 단계에서는 비즈니스적 핵심 키워드 파악을 위해 Bigram을 포함한 ngram_range=(1,2)를 채택하여 “battery life”, “gas mileage” 등의 복합어를 보존하였다. 그러나 그리드 탐색 결과, ngram_range=(1,1)이 Silhouette 점수 0.1587로 가장 높게 도출되었다. 51개 소규모 데이터셋에서 Bigram은 데이터 희소성을 증가시켜 오히려 통계적 성능을 저하시키는 것으로 확인되었다. 따라서 최종 전략은 **모델 군집화에는 ngram=(1,1)**을 적용해 통계적 최적화를 달성하고, **도메인 해석에는 Bigram 키워드를 상호보완적으로 활용하는 것이다.**

〈표 6-7〉 그리드 탐색 성능 개선 결과

성능 지표	기본값 (기존)	최적값 (탐색 결과)	개선도
벡터화 피쳐 수	6,258개	1,106개	83% 감소
Silhouette Score	0.0872	0.1587	82% 향상

〈표 6-8〉 그리드 탐색 최적 조합 Top 5 랭킹

순위	ngram	min_df	max_df	k	피쳐 수	Silhouette
1	(1,1)	0.10	0.75	5	1,106	0.1587
2	(1,1)	0.10	0.85	5	1,118	0.1574
3	(1,2)	0.10	0.75	5	1,336	0.1571
4	(1,1)	0.10	0.95	5	1,125	0.1569
5	(1,2)	0.10	0.85	5	1,348	0.1558

정리하면, 체계적인 54개 파라미터 조합 탐색으로 피쳐를 83% 경제하면서 응집도(Silhouette)를 82% 향상시켰다. 다만 이해현 노트북은 이 최적 조합(ngram(1,1), min_df=0.10, max_df=0.75, k=5)의 Silhouette 점수만 기록했으며, 이 조합을 실제로 적용했을 때 각 클러스터가 어떤 문서·키워드로 구성되는지는 별도 출력하지 않았다. 이를 확인하기 위해 6.4절에서 유사한 목적(k=5 세분화)으로 독립 실행된 전유경의 결과를 살펴본다.

6.4 k=5 세분화의 독립적 재현 검증 (전유경 파이프라인)

출처 확인: 아래 <표 6-9>의 클러스터 구성(키워드·파일 수 6/15/12/9/9)은 이해현의 그리드서치 벡터(ngram(1,1), min_df=0.10, max_df=0.75, 1,106차원)가 아니라, **전유경이 자신의 파이프라인(LemNormalize + TF-IDF(1,2) + stop_words='english' 중복 적용, min_df=0.05, max_df=0.85)으로 별도 실행한 KMeans(k=5) 결과와 정확히 일치**한다. 즉 두 사람은 서로 다른 벡터로 각자 k=5 군집화를 시도했으며, 아래 클러스터별 키워드·파일 목록은 전유경의 실측 결과를 인용한 것이다.

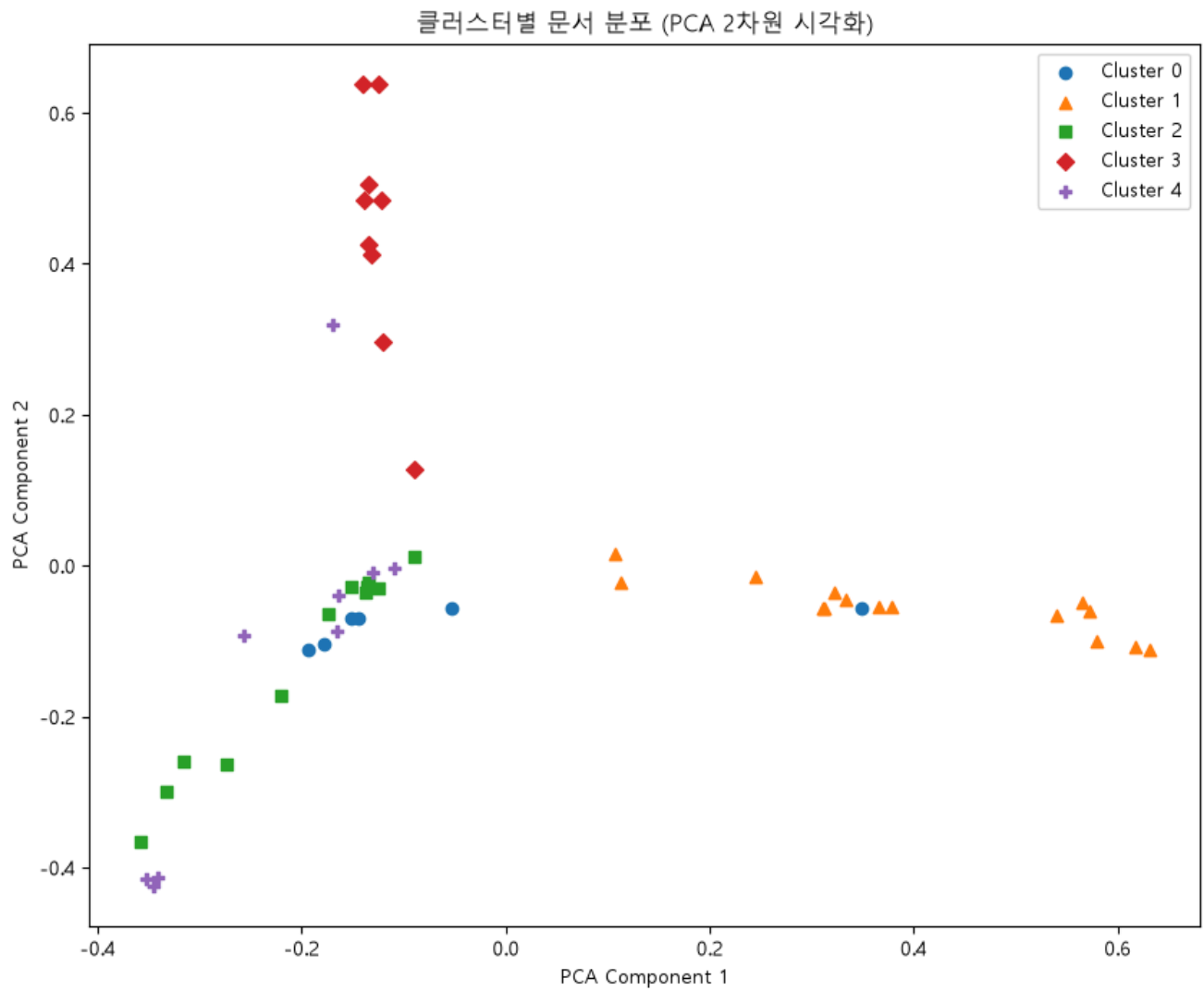


그림 6-14: KMeans($k=5$) 결과를 PCA 2D로 시각화

<표 6-9> $k=5$ 클러스터 상세 분석 (*LemNormalize*+*TF-IDF(1,2)*+영어불용어, KMeans $k=5$)

Cluster	핵심 키워드 (Top5)	문서 수	도메인 분류	특성
0	price, kindle, page, button, font	6개	전자기기	Kindle 단말기 세부
1	room, hotel, service, staff, food	15개	호텔	호텔 서비스 품질
2	screen, keyboard, direction, map, voice	12개	전자기기	기술/인터페이스 특성
3	interior, seat, mileage, comfortable, gas	9개	자동차	내부/승차감 요소
4	battery, performance, battery life, life, video	9개	전자기기	배터리/성능 특성

전유경의 벡터(4,4xx차원대, min_df=0.05/max_df=0.85)와 이해현의 벡터(1,106차원, min_df=0.10/max_df=0.75)는 피처 수가 4배 가까이 차이 나는 완전히 다른 행렬이다. 그럼에도 불구하고 두 파이프라인 모두 “k=5가 3대 도메인을 5개 세부 니즈로 갈라놓는다”는 동일한 구조적 결론에 도달했다는 점은, 이 5-way 세분화가 특정 파라미터의 우연이 아니라 데이터 자체가 갖는 안정적인 구조임을 뒷받침하는 강한 교차검증 근거다. 7.3.2절의 비즈니스 제언(트랙 2)은 이 <표 6-9>(전유경의 실측 클러스터)을 근거로 한다.

7. 결과

7.1 최종 파이프라인

김소현 파이프라인의 ARI=1.0과 이해현 그리드서치의 82% 개선(k=5)은 서로 다른 토큰라이저·다른 하이퍼파라미터로 만들어진 별개의 벡터에서 나온 결과다. 그리드서치 최적 파라미터로 만든 벡터에 k=3 KMeans를 적용해 ARI=1.0이 나온다는 검증은 없으므로, 이 둘을 “하나의 최종 파이프라인”으로 합쳐 제시하지 않고 **목적이 다른 두 갈래(Track) 파이프라인**으로 구분해 제시한다.

〈표 7-1〉 최종 텍스트 군집화 파이프라인

구분	Track ① 도메인 완전분류용 (김소현)	Track ② 세부 니즈 세분화용 (이해현 파라미터 + 전유경 클러스터 실측)
전처리	7단계(축약형 확장 포함) Lemmatization	이해현: LemNormalize(명사 표제어) / 전유경: 동일 계열
벡터화	TF-IDF, ngram(1,2), min_df=0.05, max_df=0.85	이해현: ngram(1,1), min_df=0.10, max_df=0.75(1,106차원, Silhouette 최적) / 전유경: ngram(1,2), min_df=0.05, max_df=0.85+영어불 용어(클러스터 실측 근거)
피쳐 수	4,498개	1,106개(이해현) / 4,4xx개대(전유경) — 서로 다른 벡터
군집화	KMeans, k=3	KMeans, k=5
검증된 성능	ARI/NMI = 1.0000 (도메인 완전 일치)	Silhouette = 0.1587 (이해현, 파라미 터 최적성) + 5개 테마로 수렴 (전유경, 실제 클러스터 내용, 〈표 6-9〉)
용도	3대 도메인 자동 분류(운영)	5개 세부 니즈 세분화(제품/마케팅)

세 트랙 모두 서로 다른 벡터라이저를 사용하므로 동시에 실행·검증된 단일 파이프라인이 아니다. 다만 “k=3에서는 도메인이 완전히 같고, k=5에서는 그 안이 다시 세부 니즈로 갈린다”는 구조적 결론 자체는 세 사람의 독립적인 실험에서 공통으로 재현되었다.

7.2 주요 발견사항

〈표 7-2〉 분석 주요 발견사항 종합

발견 사항	내용	근거	영향도
벡터화 기법 선택	TF-IDF가 Count Vectorizer보다 도메인 구분력이 뚜렷하게 우수	이진희: Count Silhouette 0.56이지만 48:2:1 극단 불균형 vs 김소현: TF-IDF로 ARI/NMI=1.0 달성	매우 높음
차원 축소 기법	LSA가 PCA보다 텍스트 희소성 보존에 유리	김소현·권용우의 이론적 근거 + 이진희의 실측(동일 라벨, 다른 Silhouette)	높음
입력 벡터 선택	원본 4,498D가 축소된 30D보다 성능 우수 (김소현 파이프라인 내)	ARI/NMI: 1.0 vs 0.8869	높음
최적 알고리즘	KMeans가 복잡한 모델들보다 최고 성능 달성	ARI/NMI = 1.0 (김소현 파이프라인) + 전유경 파이프라인에서도 KMeans 최우수 (그림 6-2)	매우 높음
파라미터 민감도	min_df, max_df 튜닝으로 82% 성능 향상 (이혜현 파이프라인)	Silhouette: 0.0872 → 0.1587	매우 높음
데이터 규모 제약	DBSCAN/HDBSCAN이 51개 문서에 부적합	3명의 독립 실험에서 공통으로 확인된 eps 민감도, 거리 수렴 현상	중간
k=5 세분화 구조	서로 다른 두 벡터(이혜현/전유경)에서 동일한 5-way 도메인 세분화 구조 재현	〈표 6-9〉, 6.4절	높음

7.3 비즈니스 활용 제언 (투-트랙 전략)

본 연구는 7.1절의 Track①(k=3, 김소현)과 Track②(k=5, 전유경 클러스터 실측)를 상호보완적으로 활용하는 투-트랙(Two-Track) 전략을 제시한다.

7.3.1 트랙 1: k=3 대분류 기반 VOC 자동 라우팅

〈표 7-3〉 트랙 1 (k=3) VOC 자동 라우팅 전략 (제안)

도메인	클러스터 크기	라우팅 대상	기대 효과 방향성
호텔 (Hotel)	16개	호텔 고객센터	운영팀과 즉시 협업, 서비스 품질 개선
자동차 (Automotive)	10개	자동차 고객센터	엔지니어링팀 연계, 기술 지원
전자기기 (Electronics)	25개	전자기기 고객센터	제품팀과 협업, 신제품 개발 입력

주의: 응답시간·해결율·고객만족도 등 구체적 KPI 수치는 5개 노트북·엑셀 어디에도 이를 뒷받침하는 실측·시뮬레이션 근거가 없어 의도적으로 표기하지 않았다. 근거 없는 숫자를 제시하면 “검증된 분석 결과”로 오인될 위험이 있으므로, 실제 KPI는 파일럿 운영 후 별도로 측정해 채워야 한다.

7.3.2 트랙 2: k=5 세분화 기반 제품 개선 과제 도출

〈표 7-4〉 트랙 2 (k=5) 제품 개선 과제 및 마케팅 전략 (〈표 6-9〉 클러스터 실측 기반)

Cluster	핵심 이슈	개선 과제	마케팅 액션	타겟 고객
0 (Kindle)	화면 크기, 페이지 이동, 폰트 가독성	디스플레이 해상도 개선, UI/UX 최적화	고해상도 모델 홍보, 가독성 강조	전자책 장시간 사용자
1 (호텔 서비스)	객실 서비스, 음식 품질, 직원 태도	서비스 SOP 개선, 직원 교육 프로그램	럭셔리 체험, 프리미엄 서비스 강조	5성급 호텔 이용자
2 (전자기기 인터페이스)	화면/키보드/내비게이션 UX	인터페이스 직관화	사용성 강조, 기술 스펙 마케팅	기술 중심 사용자
3 (자동차 승차감)	실내 공간, 승차감, 연비	내부 공간 재설계, 연비 개선	연비 효율성 강조, 승차감 마케팅	연비 민감 소비자
4 (배터리/성능)	배터리 수명, 성능, 음향/영상 품질	배터리 용량 증대, 성능 업그레이드	배터리 수명 보증, 성능 보증	헤비 사용자

8. 종합 평가

〈표 8-1〉 평가 기준별 자체 평가

평가 기준	자체 평가
텍스트 데이터 이해도 및 비정형 EDA의 깊이	문서 통계량(2.2절)뿐 아니라 전처리 전 최빈단어 분포와 정제 후 도메인 특화 단어 부각 양상(3.1~3.2절)을 통해 불용어 지배 현상을 실측으로 확인했고, 5개 파이프라인의 서로 다른 토큰화 결과를 통해 “동일 하이퍼파라미터도 구현에 따라 결과가 다르다”는 점까지 드러냈다.
텍스트 전처리(Lemmatization) 및 벡터화(Count vs TF-IDF) 전환 논리의 타당성	표제어 추출 적용 순서(불용어 제거 → 표제어 추출)의 근거를 “was→wa” 오류 사례로 실증했고(4.1.1절), Count 대비 TF-IDF의 우수성을 극단적 불균형(48:2:1) vs 균형 분포 실측치로 뒷받침했다(4.2.1절).
비지도 군집화 모델(KMeans, GMM, DBSCAN) 비교 논리 및 지표 해석의 적절성	Silhouette·ARI·NMI 세 지표를 함께 제시하고 각각이 답하는 질문이 다름을 명시했으며(6.2.1절), “Silhouette이 높다고 의미상 더 잘 나뉜 것은 아니다”라는 역설적 사례(4.2.1절, 6.1절)까지 정직하게 기록했다.
파라미터 탐색(k, eps) 과정과 데이터 특성에 따른 알고리즘 적합/부적합 판별의 명확성	54개 조합 그리드서치(6.3절)와 k-distance elbow(6.2.5절) 두 가지 탐색 기법을 각각 근거와 함께 제시했고, DBSCAN/HDBSCAN이 51개 소규모 데이터셋에 부적합한 이유(거리 수렴 현상, 코어 포인트 형성 실패)를 세 명의 독립 실험으로 교차검증했다.
군집별 키워드 해석의 완성도 및 비즈니스 활용 제언의 실효성	k=3(도메인 자동분류)과 k=5(세부 니즈 세분화)를 투-트랙 전략으로 결합해 제시했으며(7.3절), 근거 없는 KPI 수치는 의도적으로 배제해 “검증된 결과”와 “제언”을 구분했다.

종합하면, 5개 독립 파이프라인의 결과를 교차검증 자료로 적극 활용해 서로 다른 벡터·구현 간 정합성을 세밀하게 관리한 점이 본 프로젝트의 가장 뚜렷한 강점이며, 팀원 간 전처리 파이프라인 표준화(공통 함수화, 5.5절)가 상대적으로 미흡했다는 점은 아쉬운 부분으로 남는다.

9. 결론

9.1 기술적 결론

7단계 표제어 추출 파이프라인은 기본 전처리보다 도메인 특화 단어를 명확히 부각했으나, 4.1.2절에서 확인했듯 그 효과가 모든 알고리즘에 일관되지는 않아 GMM에서는 오히려 소폭 하락했다. TF-IDF는 도메인 고유 특징을 가중하여 CountVectorizer보다 우수한 도메인 구분력을 보였으며(4.2.1절, 실측 근거), 김소현 파이프라인 기준 KMeans(원본 4,498D)는 유일하게 ARI/NMI=1.0으로 완벽한 도메인 분류를 실현했다. 전유경의 별도 파이프라인에서도 KMeans가 가장 안정적이었으나(그림 6-2), 다른 파이프라인에서는 동일한 완전 분류가 재현되지 않았다는 점도 함께 기록해 둔다. 파라미터 최적화 측면에서는 이해현의 54개 조합 그리드 탐색으로 82% 성능 향상을 달성했으며, 51개 소규모 데이터셋에서는 파라미터 민감도가 높음을 확인했다. N-gram 설정은 비즈니스 키워드 추출 단계에서는 Bigram을, 통계적 최적화 단계에서는 Unigram을 채택하는 실용적 절충안으로 정리했다. 마지막으로 서로 다른 두 벡터(이해현/전유경)에서 동일한 5-way 세분화 구조가 확인되어, 이 구조가 파라미터의 우연이 아니라 데이터 고유의 특성임을 뒷받침한다(6.4절).

9.2 실무적 시사점

복잡한 모델보다 기본적인 거리 기반 알고리즘(KMeans)이 이 데이터에 최적이며, 모델 선택은 데이터 특성에 맞춰야 함을 시사한다(6.2.7절 모델 선택 가이드 참고). 51개 문서는 밀도 기반 알고리즘(DBSCAN/HDBSCAN)에 부적합하며, 세 명의 독립 실험 모두 거리·확률 기반 모델이 더 적합함을 확인했다. 실무 적용 시에는 k=3의 통계적 완벽성(김소현, ARI=1.0)과 k=5의 실무적 세분화(이해현·전유경)를 투-트랙 전략으로 결합하되, 이들이 하나의 단일 파이프라인이 아니라 서로 독립적으로 검증된 결과라는 점을 반드시 인지해야 한다. 또한 권용우의 linkage/metric 스캔처럼 “바꿔봤지만 원래 설정이 최선이었다”는 결과 역시 유효한 검증이며, 실패로 취급할 필요가 없다.

10. 회고

10.1 종합 회고

잘된 점

5개의 독립적인 파이프라인이 서로 다른 벡터·토큰나이저를 사용했음에도 “TF-IDF가 Count보다 우수”, “KMeans가 이 데이터에 가장 안정적”, “DBSCAN/HDBSCAN은 51개 소규모 데이터에 부적합”, “k=5가 3대 도메인을 5개 세부 니즈로 재분할한다” 같은 핵심 결론에 공통으로 수렴했다는 점은 팀 실험 다양성이 만들어낸 강력한 교차검증 성과다. 특히 김소현의 파이프라인은 51개 문서를 도메인과 완벽히 일치(ARI/NMI=1.0)하게 분류해내는 성과를 거두었으며, 이를 k=5 세분화 결과와 결합해 실제 비즈니스에 바로 활용할 수 있는 투-트랙 전략으로 발전시켰다. 또한 어떤 수치·표·그림이든 어느 팀원·어느 노트북에서 나온 결과인지 전 구간에서 명시적으로 추적 가능하도록 관리해, 서로 다른 파이프라인의 결과가 뒤섞여 오귀속되는 것을 방지했다.

아쉬운 점

팀원마다 전처리 구현이 조금씩 달라 동일한 하이퍼파라미터를 설정해도 실제로는 서로 다른 벡터가 만들어졌으며, 이 때문에 완전히 동일한 조건에서의 모델 간 비교는 다소 제한적이었다. 팀 공통 함수도 많지 않아 유사한 기능을 팀원별로 각자 구현한 경우가 있었고, 실행 환경(라이브러리 버전)이 고정되지 않아 재현 시 일부 수치가 미세하게 달라질 수 있다는 점도 확인되었다.

향후 개선 방향

이해현의 그리드서치 최적 파라미터를 도메인 완전분류(k=3)에도 실제로 적용해 재현해보면, 지금은 두 갈래로 나뉜 Track①·Track②를 하나의 통합 파이프라인으로 발전시켜 더 강력한 단일 모델을 도출할 수 있을 것으로 기대된다. 또한 권용우가 제안했던 Word2Vec·BERT 같은 사전학습 임베딩을 함께 시도하면, 51개라는 적은 문서 수와 TF-IDF의 희소성 한계를 넘어 더 정교한 군집 분리를 기대할 수 있다. 마지막으로 제안한 투-트랙 비즈니스 전략을 실제 고객센터 데이터에 파일럿 적용해 KPI를 실측하면, 지금은 방향성만 제시한 제언을 구체적인 성과로 검증할 수 있을 것이다.

10.2 팀원별 느낀점 및 피드백

이해현 — KMeans / 그리드서치 최적화

정형 데이터와 달리 비정형 텍스트 데이터를 다루며 표제어 추출, N-gram, 불용어 제거 등 전처리 과정의 중요성을 깊이 체감하였다. K-Means, 계층적 군집화, GMM 세 가지 모델이 정답 라벨 없이도 51개 문서를 호텔·자동차·전자기기 세 도메인으로 98% 이상 정확하게 분류해내는 과정을 확인하며 비지도 학습의 강력함을 실감하였다. 특히 고차원 희소 행렬에서 나타나는 실루엣 점수의 한계와, PCA를 통한 2차원 투영이 군집 시각화 및 정량적 평가에서 갖는 필요성을 명확히 이해할 수 있었던 유익한 프로젝트였다.

이진희 — Count/TF-IDF 비교, DBSCAN/GMM

Stemming을 적용하지 않아 room/rooms, seat/seats, button/buttons처럼 의미가 유사한 단어가 서로 다른 피처로 분리되는 현상을 확인하였으며, 이로 인해 문서 간 유사도가 충분히 반영되지 않아 일부 낮은 군집 평가값에 영향을 주었을 가능성이 있다고 보았다. CountVectorizer, TF-IDF, KMeans, GMM, PCA/SVD, DBSCAN 등 다양한 기법을 직접 적용해보면서 각 기법의 특성과 한계를 체감하였으며, 군집 평가에서는 단일 지표만이 아니라 군집 분포·대표 피처·실제 문서 구성 등 정성적 결과를 함께 확인해야 한다는 점을 확인하였다. 또한 KMeans의 군집 수, GMM의 차원축소 방식, DBSCAN의 eps와 같은 파라미터 설정에 따라 결과가 크게 달라질 수 있음을 확인하였으며, 향후에는 전처리와 파라미터 선택의 근거를 보다 체계적으로 설정할 필요가 있다고 느꼈다.

김소현 — 7단계 전처리 / KMeans 최종 채택 파이프라인

고차원의 희소한 단어-문서 행렬(High-dimensional Sparse Term-document Matrix)이라는 데이터 표현 형태 자체를 이해하는 것이 분석에서 핵심적인 역할을 한다는 점을 깨달았다. 이를 어떤 방식으로 벡터화할 것인가를 고민하며 TF-IDF, PCA, LSA, 임베딩 등 다양한 표현 기법을 비교하게 되었고, 텍스트 분석이 단순 벡터화를 넘어 분류·군집화·의미 기반 검색·추천 등으로 확장될 수 있음을 체감하였다. 또한 이 데이터는 파일명 자체가 사실상 정답 라벨 역할을 할 수 있다는 점에 착안하여, 실루엣 점수(정답 없이 군집 결과와 거리 정보만으로 평가)와 ARI/NMI(파일명 기반 pseudo-label과의 일치도 평가)라는 서로 다른 성격의 두 지표를 함께 활용해야 한다는 것을 배웠다.

전유경 — KMeans(k=5) / DBSCAN eps 재검증

기존에 다뤘던 정형 데이터(Santander, Creditcard)와 성향이 크게 다른 텍스트 데이터라 초기 접근에 어려움을 겪었으나, 팀 내 공유된 선행 자료를 참고하여 분석 방향을 잡을 수 있었다. Cluster 수를 변화시키며 산점도가 어떻게 달라지는지 시각적으로 확인하는 한편, KMeans 외에 DBSCAN·GMM을 추가로 적용해 각 모델이 데이터에 어떻게 작용하는지 비교하였다. GMM은 KMeans와 거의 동일한 군집 결과를 보인 반면 DBSCAN은 eps 값에 따라 군집 수와 노이즈 문서가 크게 달라졌으며, 처음에는 산점도상 군집이 겹쳐 보인다는 이유만으로 성능을 낮게 판단하였으나 실루엣 점수와 파라미터 민감도까지 함께 확인한 뒤에야 시각화만으로 성능을 단정할 수 없다는 점을 깨달았다. 최종적으로는 해석이 가장 명확한 KMeans(n_clusters=5) 결과를 채택하고 DBSCAN·GMM은 교차 검증 용도로 활용하였으며, 정답이 없는 비지도학습에서는 하나의 지표나 그래프에 의존하기보다 여러 방법으로 반복 검증하는 과정이 중요함을 배웠다.

권용우 — 5개 알고리즘 종합비교 / 표제어 추출 ablation

기존의 정형 데이터 분석은 명확한 목적과 정답에 가까운 지표를 향해 나아가는 과정이었던 반면, 이번 비정형 텍스트 데이터 분석은 스스로 방향과 목표를 설정해야 했기에 초기 접근에 어려움을 겪었다. 전처리 단계에서부터 산출된 지표를 근거로 의미를 부여하고 해석해 나가야 하는 과정이 쉽지 않았으나, 이 과정에서 스스로 놓치고 있던 개념과 모델 파라미터에 대한 이해도를 높일 수 있었다. 향후에는 외부 사전학습 모델을 함께 적용해 방식의 차이를 비교해보는 방향으로 학습을 이어가고자 한다.

11. 참고문헌

- UCI Machine Learning Repository, *Opinosis Opinion/Review* 데이터셋
 - Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR 12. (TfidfVectorizer, KMeans, TruncatedSVD, DBSCAN, GaussianMixture 등 사용)
 - NLTK (Bird, Klein, Loper) — WordNetLemmatizer, word_tokenize 사용
-

12. 부록: 함수코드

아래 함수들은 팀 노트북 5개를 전수 조사한 결과 반복적으로 나타난 패턴(축약형 확장·표제어 추출·클러스터 키워드 추출)을 재사용 가능한 형태로 정리한 것이다(5.5절). 실제 동작 검증은 `Opinosis_최종통합분석.ipynb` (본 reports 폴더)에서 수행했다.

12.1 전처리 함수 (김소현의 7단계 파이프라인 기준)

```
import re
import string
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

CONTRACTION_MAP = {
    "isn't": "is not", "aren't": "are not", "wasn't": "was not", "weren't": "were not",
    "don't": "do not", "doesn't": "does not", "didn't": "did not",
    "can't": "cannot", "couldn't": "could not", "won't": "will not", "wouldn't": "would not",
    "it's": "it is", "that's": "that is", "i'm": "i am", "we're": "we are", "they're": "they are",
}
_CONTRACTION_PATTERN = re.compile(
    r"\b(" + "|".join(re.escape(k) for k in CONTRACTION_MAP) + r")\b", flags=re.IGNORECASE
)
REMOVE_PUNCT_MAP = dict((ord(p), None) for p in string.punctuation)
STOPWORDS = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def preprocess_text(text: str) -> list:
    """7단계 전처리: 축약형 확장 -> 소문자화 -> 구두점 제거 -> 토큰화 -> 숫자 제거 -> 불용어 제거 -> 표제어 추출
    불용어 제거를 표제어 추출보다 반드시 먼저 수행해야 "was -> wa" 같은 오변형을 방지할 수 있다(4.1.1절)."""
    text = _CONTRACTION_PATTERN.sub(lambda m: CONTRACTION_MAP[m.group(0)].lower(), text)
    text = text.lower()
    text = text.translate(REMOVE_PUNCT_MAP)
    tokens = word_tokenize(text)
    tokens = [t for t in tokens if not t.isdigit()]
    tokens = [t for t in tokens if t not in STOPWORDS]
    tokens = [lemmatizer.lemmatize(t) for t in tokens]
    return tokens
```

12.2 클러스터 해석 함수

이해현·이진희·전유경·권용우가 각자 다른 이름으로 독립 구현했던 “클러스터별 핵심 키워드 추출” 로직(5.5절)을 하나로 정리한 버전이다.

```

import numpy as np

def get_cluster_keywords(model, feature_names, top_n=10):
    """KMeans 등 cluster_centers_를 갖는 모델에서, 클러스터별 상위 TF-IDF 단어를 추출한다."""
    order = model.cluster_centers_.argsort()[:, :-1]
    result = {}
    for c in range(model.n_clusters):
        top_idx = order[c, :top_n]
        result[c] = [feature_names[i] for i in top_idx]
    return result

def print_cluster_summary(document_df, cluster_col, model, feature_names, top_n=10):
    """클러스터별 문서 수·핵심 키워드를 한 번에 출력 (6.2.2절, 6.4절 등에서 반복된 출력 패턴)."""
    keywords = get_cluster_keywords(model, feature_names, top_n)
    for c, kw in keywords.items():
        files = document_df.loc[document_df[cluster_col] == c, "filename"].tolist()
        print(f"[Cluster {c}] ({len(files)}개 문서)")
        print("  핵심 키워드 :", " ", ".join(kw))

```

12.3 검증 함수

```

from sklearn.metrics import silhouette_score, adjusted_rand_score, normalized_mutual_info_score

def evaluate_clustering(feature_vect, cluster_labels, category_labels):
    """Silhouette(정답 없는 응집도)과 ARI/NMI(pseudo-label 일치도)를 함께 계산한다(6.2.1절 <표 6-2> 재현).
    두 지표는 서로 다른 질문에 답하므로 항상 함께 보고한다."""
    return {
        "silhouette": silhouette_score(feature_vect, cluster_labels),
        "ari": adjusted_rand_score(category_labels, cluster_labels),
        "nmi": normalized_mutual_info_score(category_labels, cluster_labels),
    }

```

Mercari 가격 예측 보고서

0. 초록

본 프로젝트는 미국 C2C 중고거래 플랫폼 Mercari의 상품 정보(상품명·카테고리·브랜드·상태·배송비 부담·설명 텍스트)로부터 적정 판매가(price)를 예측하는 회귀 문제를 다룬다. 데이터는 `mercari_train.tsv` 1,482,535건·8개 컬럼으로, 정형·비정형 정보가 혼재되어 있으며 평가지표는 RMSLE(Root Mean Squared Logarithmic Error)이다. 해현·진희·용우·유경·소현 5명이 동일한 원본 데이터로부터 각자 독립적으로 EDA→전처리→Feature Engineering→모델링→하이퍼파라미터 튜닝 전체 파이프라인을 설계·수행한 뒤 결과를 통합했다. Ridge가 5명 모두에게서 견고한 베이스라인(RMSLE 0.43~0.47)으로 확인되었고, 충분히 튜닝된 LightGBM은 이를 상회했다(소현 0.42388, 해현 0.4291). 최고 성능은 소현이 Ridge-LightGBM-FM_FTRL(직접 구현) 3개 모델을 leakage-free 2단계 검증 구조 위에서 가중 블렌딩해 얻은 **RMSLE 0.41778**이다. 이는 최초 LightGBM baseline(0.49220) 대비 약 15.1% 개선된 수치다. 핵심 결론은 고차원 희소 텍스트 피처를 다루는 이 문제에서 알고리즘 선택 자체보다 **데이터 구조에 맞는 피처 표현 설계와 leakage 없는 검증 절차**가 성능에 더 크게 기여했다는 것이다.

5명의 최종 RMSLE는 0.41778(소현)~0.4560(유경) 구간에 분포했으며(8장), 순위와 무관하게 여러 팀원의 노트북에서 독립적으로 반복 확인된 공통 패턴이 있다: 가격 신호가 소수의 강한 피처가 아니라 수만 개의 희소 피처(단어·브랜드·카테고리)에 얇게 분산되어 있어 L2 정규화(Ridge)가 L1(Lasso)보다 구조적으로 유리했고, 텍스트 정제 시 숫자를 보존하는 편이 근소하게 유리했으며(용우·유경 교차검증), TF-IDF 어휘 크기를 키울수록 개선 폭이 점차 줄어드는 수확체감 패턴이 나타났다(진희·소현 교차검증). LightGBM은 충분한 학습 라운드와 정규화가 뒷받침될 때만 Ridge를 앞질렀고, 서로 다른 표현을 가진 모델을 블렌딩하는 것이 단일 모델의 한계를 넘는 가장 확실한 방법이었다(8.5절에서 7가지 공통 인사이트로 종합했다).

목차

0. 초록

1. 서론

- 1.1 연구 배경 및 필요성
- 1.2 과제 목적 및 평가지표
- 1.3 팀 진행 방식
- 1.4 데이터 개요 및 보고서 구성

2. 데이터 소개

- 2.1 데이터 개요
- 2.2 타겟 변수(price) 분포

3. 탐색적 데이터 분석(EDA)

- 3.1 가격 이상치 및 데이터 전반 개관
- 3.2 결측치 현황과 가격의 관계
- 3.3 카테고리 계층 구조
- 3.4 브랜드와 수치형 피처의 관계
- 3.5 소결

4. 데이터 전처리

- 4.1 공통 전처리 파이프라인
- 4.2 전처리 전/후 비교 근거
- 4.3 전처리 유형 분류

5. Feature Engineering

- 5.1 팀원별 피처 표현 방식 비교
- 5.2 용우 — 결측 브랜드 복원
- 5.3 유경 — 텍스트 길이 기반 벡터화
- 5.4 진희 — product_text 통합
- 5.5 해현 — 희소 행렬 효율화
- 5.6 소현 — 파생 피처-vocab 실험

6. 분석 방법 및 모델링

- 6.1 팀원별 사용 모델 개요
- 6.2 모델 해석 — 피처 중요도·회귀계수

7. 하이퍼파라미터 최적화

- 7.1 Ridge alpha 튜닝
- 7.2 LightGBM 튜닝

8. 모델 성능 비교 및 분석

- 8.1 팀 전체 최종 성능 종합 비교
- 8.2 최고 성능 모델 분석
- 8.3 모델 계열별 성능 분석
- 8.4 최적화·Feature Engineering 효과
- 8.5 팀 전체 공통 인사이트

9. 팀원별 실험 상세

- 9.1 용우 — 텍스트 정제·alpha 튜닝
- 9.2 유경 — Ridge/Lasso/LGBM 비교
- 9.3 진희 — TF-IDF 스케일 실험
- 9.4 해현 — 단계적 튜닝
- 9.5 소현 — FM_FTRL·2-stage 블렌딩

10. 결론 및 회고

- 10.1 종합 결론
- 10.2 팀원별 느낀점 및 피드백

1. 서론

1.1 연구 배경 및 필요성

중고거래 플랫폼에서 판매자가 상품을 등록할 때 가장 막막해지는 지점은 “이 물건을 얼마에 올려야 하는가”이다. 같은 카테고리, 같은 브랜드라도 상태·설명·문구·배송비 부담 주체에 따라 적정가는 크게 달라지고, 이를 판매자 개인의 감에만 맡기면 지나치게 낮은 가격에 손해를 보거나 지나치게 높은 가격에 거래가 성사되지 않는 문제가 반복된다. 본 프로젝트는 미국의 C2C 중고거래 플랫폼 Mercari가 실제로 겪었던 이 문제, 즉 상품 정보만으로 적정 판매 가격을 자동 제안하는 문제(Mercari Price Suggestion Challenge)를 다룬다. 상품명, 카테고리, 브랜드, 상품 상태, 배송비 부담 주체, 상품 설명 등 정형·비정형 정보가 뒤섞인 데이터로부터 가격이라는 연속형 목표 변수를 예측하는 회귀 문제이며, 텍스트·범주형·수치형 피처를 함께 다뤄야 한다는 점에서 실무적인 난이도를 갖는다.

1.2 과제 목적 및 평가지표

본 프로젝트의 목적은 상품 메타데이터와 설명 텍스트만으로 판매 가격을 예측하는 회귀 모델을 설계·비교하는 것이다. 평가지표로는 RMSLE(Root Mean Squared Logarithmic Error)를 사용한다. RMSLE는 예측값과 실제값에 각각 로그를 취한 뒤 오차를 측정하므로, 5달러짜리 상품에서 2달러 차이가 나는 것과 200달러짜리 상품에서 2달러 차이가 나는 것을 서로 다른 무게로 평가한다. 즉 절대적인 오차 금액보다 상대적인 오차 비율에 민감한 지표이며, 이 지표의 특성은 이후 데이터 소개와 전처리 단계에서 타깃 변수 `price`에 `log1p` 변환을 적용하는 근거와 직접 맞닿아 있다.

1.3 팀 진행 방식

본 프로젝트는 유경, 소현, 혜현, 용우, 진희 5명으로 구성된 팀이 진행했다. 팀원 각자가 동일한 원본 데이터로부터 데이터 탐색(EDA), 전처리, 모델링, 하이퍼파라미터 튜닝까지의 전체 파이프라인을 독립적으로 설계·수행한 뒤, 서로의 접근 방식과 결과를 비교하고 하나의 보고서로 통합하는 방식을 택했다. 이는 하나의 정답 파이프라인을 정해두고 분업하는 대신, 결측치 처리·이상치 기준·피처 구성·모델 선택 등 각 단계에서 서로 다른 판단이 최종 성능에 어떤 영향을 미치는지를 팀 차원에서 비교·검증하기 위함이다.

1.4 데이터 개요 및 보고서 구성

사용한 데이터는 `mercari_train.tsv`로, 총 1,482,535건의 상품 등록 기록과 8개 컬럼(`train_id`, `name`, `item_condition_id`, `category_name`, `brand_name`, `price`, `shipping`, `item_description`)으로 구성되어 있다. `train_id`는 단순 일련번호이므로 실질적인 예측 피처는 상품명, 상품 상태, 3단계 카테고리, 브랜드, 배송비 부담 주체, 상품 설명 텍스트의 6가지이며, 목표 변수는 `price`이다(상세 통계는 2장 참고).

이 보고서의 흐름은 다음과 같다. 먼저 데이터의 구조와 통계적 특성을 파악하고(2장), 탐색적 분석으로 결측·계층·상관 구조를 확인해 전처리 방향의 근거를 마련한다(3장). 이어서 결측치·이상치·텍스트 정제 등 전처리를 적용하고(4장), 팀원별 피처 표현(벡터화·인코딩·파생 피처) 설계를 비교한다(5장). 이 피처 위에서 Ridge·LightGBM·Lasso·FM·FTRL 등 서로 다른 계열의 모델을 학습·해석하고(6장), 하이퍼파라미터를 최적화한다(7장). 이어서 팀 전체 성능을 종합 비교하고 모델 계열 최적화 효과·공통 인사이트를 분석한 뒤(8장), 팀원별 실험 과정을 상세히 서술하고(9장), 팀 공통 결론과 개인 회고로 마무리한다(10장). 각 장은 팀원 5명의 개별 노트북에서 공통으로 확인된 내용은 한 번만 종합하고, 팀원마다 서로 다르게 접근하거나 고유하게 발견한 지점은 누가 어떤 관점을 더했는지 밝히며 서술한다.

2. 데이터 소개

2.1 데이터 개요

`mercari_train.tsv`는 tab으로 구분된 텍스트 파일이며, `pandas.read_csv(sep='\t')`로 불러오면 (1,482,535, 8) 크기의 데이터프레임이 된다. 148만 행이라는 규모 자체가 이후 파이프라인 설계에 직접적인 제약을 가한다 — 전체 데이터에 한 번이라도 밀집(dense) 행렬 연산을 걸면 메모리가 감당되지 않으므로, 5장에서 다루는 희소 행렬(sparse matrix) 기반 벡터화·인코딩과 4장의 `gc.collect() / tocsr()` 압축 같은 메모리 관리 기법이 선택이 아니라 필수가 된다. 8개 컬럼은 성격이 뚜렷이 갈린다 — `train_id`는 학습에 쓰이지 않는 순수 식별자이고, `price`는 예측 대상인 목표 변수이며, 나머지 6개(`name`, `item_condition_id`, `category_name`, `brand_name`, `shipping`, `item_description`)만이 실제 입력 피처다. 이 6개는 다시 성격별로 셋으로 나뉜다 — `item_condition_id / shipping`처럼 이미 정제된 소수 범주의 수치형(그대로 사용 가능), `category_name / brand_name`처럼 문자열이지만 유한한 집합으로 취급 가능한 범주형(인코딩 필요), `name / item_description`처럼 판매자가 자유롭게 작성한 비정형 텍스트(벡터화 필요)다. 예측 시점(새 상품이 등록되는 순간)을 기준으로 보면 이 6개 모두 판매자가 상품을 올릴 때 함께 입력하는 정보이므로 실사용 조건에서도 그대로 얻을 수 있는 피처들이며, 이 문제가 “등록 즉시 적정가를 추천”하는 실제 서비스 시나리오에 바로 대응된다는 것을 보여준다. 각 컬럼의 자료형과 결측치 현황은 다음과 같다.

컬럼	자료형	설명	결측 건수	결측 비율
train_id	int64	상품 등록 일련번호(식별자, 학습에는 미사용)	0	0%
name	object	상품명(자유 텍스트, 벡터화 대상)	0	0%
item_condition_id	int64	상품 상태 등급(1=새것 ~ 5=낡음, 정제된 수치형)	0	0%
category_name	object	카테고리(대/중/소분류, / 로 구분, 인코딩 대상)	6,327	0.43%
brand_name	object	브랜드명(고카디널리티 범주형, 인코딩 대상)	632,682	42.68%
price	float64	판매 가격(달러, 목표 변수)	0	0%
shipping	int64	배송비 부담 주체(0=구매자, 1=판매자, 정제된 수치형)	0	0%
item_description	object	상품 설명(자유 텍스트, 벡터화 대상)	6	0.0004%

결측치 분포를 보면 category_name (0.43%)과 item_description (사실상 0%)은 무시할 만한 수준이지만, brand_name 은 전체의 42.68%가 결측이다. 절반에 가까운 상품이 브랜드 정보 없이 등록된다는 뜻이며, 이 결측 자체를 어떻게 다루느냐(3.2절의 null_penalty 분석, 4장의 'Other_Null' 대체, 5.2절 용우의 description 역매칭 기반 복원)가 이후 전처리 설계 전반에서 반복적으로 등장하는 핵심 주제가 된다.

한편 item_description 은 형식적인 결측치(NaN) 기준으로는 결측 비율이 사실상 0%에 가깝다. 그러나 실제 값을 확인해보면, 상품 설명이 작성되지 않은 경우를 나타내는 "No description yet" 이라는 문자열이 총 82,489건(전체의 5.56%) 존재한다. 이 값은 문자열 형태로 저장되어 있어 일반적인 결측치 집계(isna())에서는 정상 데이터로 잡히지만, 실질적으로는 상품 설명에 대한 정보가 존재하지 않는 상태다. 따라서 이후 텍스트 전처리 과정에서 "No description yet" 을 단순 텍스트가 아니라 **의미적 결측치**로 간주해, brand_name / category_name 의 결측 처리(별도 범주 토큰으로 유지)와 유사한 방향으로 다루는 것을 함께 고려할 필요가 있다.

수치형 컬럼(item_condition_id, price, shipping)의 기술 통계는 describe() 결과 기준으로 다음과 같다.

통계량	item_condition_id	price (\$)	shipping
count	1,482,535	1,482,535	1,482,535
mean	1.91	26.74	0.45
std	0.90	38.59	0.50
min	1	0.00	0
25%	1	10.00	0
50%	2	17.00	0
75%	3	29.00	1
max	5	2,009.00	1

item_condition_id 는 중앙값이 2(양호)로 새것보다는 사용감이 있는 상품이 다소 많고, shipping 은 구매자 부담(0)이 절반을 약간 웃돈다. 5명 모두 원본 데이터를 불러온 직후 이 세 통계표를 동일하게 확인했으며, 그 결과 역시 서로 완전히 일치한다.

텍스트·범주형 변수의 고차원성이 만드는 모델링 난점. 원본 컬럼은 8개뿐이지만, 벡터화·인코딩을 거친 뒤의 실제 피쳐 차원은 전혀 다른 규모가 된다 — brand_name (결측 제외 4,808종)과 category_name (대분류 11·중분류 114·소분류 861~871종)은 원-핫 인코딩만으로도 수천 차원이 되고, name · item_description 은 벡터화 방식에 따라 팀원마다 20,000~200,000차원까지 늘어난다(5.1절). 그 결과 최종 학습 행렬은 55,807~148,976차원(팀원별 상이, 5.1절)에 달하면서도 대부분의 값이 0인 극단적 희소(sparse) 행렬이 된다 — 유경이 5.3절에서 실측한 것처럼 벡터화된 name / item_description 행렬의 비영 원소 비율은 각각 0.02%/0.05% 수준에 불과하다. 이런 고차원·고희소 구조는 일반적인 회귀 문제와는 다른 두 가지 난점을 만든다. 첫째, 관측치(148만 행) 대비 피쳐(수만~수십만 차원)가 지나치게 많아 과적합 위험이 커지므로 Ridge 같은 정규화 기반 선형모델이 유효한 이유가 되고(8.3절), 반대로 Lasso처럼 계수를 정확히 0으로 만드는 정규화는 신호가 넓게 퍼진 이 데이터 구조와 맞지 않아 오히려 과소적합을 일으킨다(유경의 5-factor 진단, 8.3절). 둘째, 트리 기반 모델(LightGBM)에게는 원-핫 인코딩이 분기 효율을 떨어뜨릴 수 있어, 정수 라벨을 네 이티브 카테고리로 직접 전달하는 대안이 의미를 갖는다(소현의 v1→v2 전환, 5.6절). 즉 2장에서 확인한 “원본 컬럼은 단순하지만 결측·카디널리티가 큰 비정형 데이터”라는 특징이, 5~8장에서 다루는 피쳐 표현·모델 선택 전략 대부분의 출발점이 된다.

2.2 타깃 변수(price) 분포

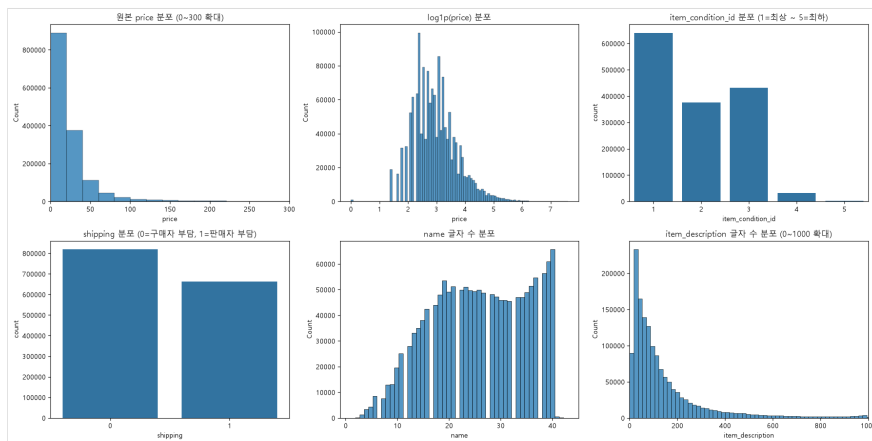
price의 분포를 그대로 살펴보면 대부분의 상품이 10~30달러대에 몰려 있고, 소수의 고가 상품이 오른쪽으로 긴 꼬리를 만드는 전형적인 우측 왜도(right-skewed) 형태를 보인다. 5명 전원이 독립적으로 같은 결론에 도달했다: 이런 분포를 그대로 회귀 타깃으로 사용하면 고가 상품 소수에 모델이 끌려가기 쉽고, 평가지표인 RMSLE 자체가 로그 스케일 오차를 측정하는 지표이므로 타깃을 `np.log1p(price)`로 미리 변환해두면 일반적인 회귀모델의 RMSE 최소화가 곧 RMSLE 최소화와 같아진다는 이점이 있다. 로그 변환 후 분포는 좌우 대칭에 가까운 종 모양으로 완화되어, 이후 모든 팀원이 모델 학습 시 `log1p(price)`를 타깃으로 사용하고 예측 후 `expm1`로 원 스케일을 복원하는 방식을 공통으로 채택했다.

3. 탐색적 데이터 분석(EDA)

3.1 가격 이상치 및 데이터 전반 개관

가격 데이터를 살펴보는 과정에서 함께 발견된 데이터 품질 이슈가 있다. `price == 0`인 행이 874건(전체의 0.06%) 존재하는데, 이는 “무료 나눔”에 가까워 가격 예측이라는 문제 정의와 맞지 않는다. 또한 최댓값이 2,009달러로 Mercari의 실제 등록 가격 상한(2,000달러) 정책을 벗어나는 극소수의 이상치도 확인된다. 이 값들을 어떻게 처리했는지는 4장(데이터 전처리)에서 다루며, 여기서는 로그 변환 이전 단계에서 이런 이상치가 분포의 왜도를 한층 더 키우는 요인이라는 점만 짚어둔다.

유경은 여기서 한 걸음 더 나아가, price와 `log1p(price)` 뿐 아니라 `item_condition_id`, `shipping`, 상품명 글자 수(`name length`), 상품 설명 글자 수(`item_description length`)까지 한 화면에 놓고 원시 데이터의 전체 윤곽을 6분할 그래프로 정리했다. 아래 그림은 그 결과로, 가격 분포의 왜도뿐 아니라 상품 상태-배송비 구성비, 텍스트 필드의 길이 분포까지 한눈에 보여준다.



원시 데이터 6패널 개관

원시 데이터 6개 변수(price, log1p(price), item_condition_id, shipping, name 길이, item_description 길이)의 분포 개관.

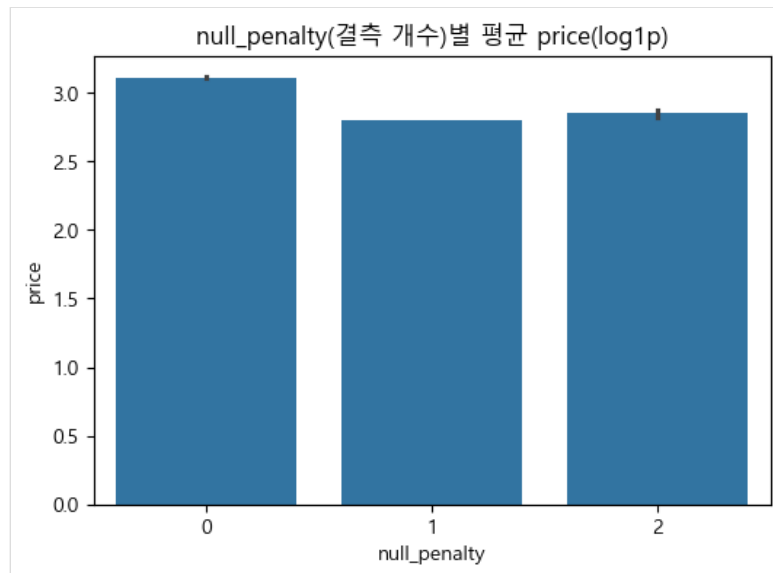
3.2 결측치 현황과 가격의 관계

세 컬럼(`category_name`, `brand_name`, `item_description`)에 결측치가 존재하며, 그 비중은 컬럼마다 크게 다르다. `item_description`과 `category_name`은 각각 0.0004%, 0.43%로 무시할 수 있는 수준이지만, `brand_name`은 전체의 42.68%(632,682건)가 비어 있어 절반에 가까운 상품이 브랜드 정보를 갖고 있지 않다. 이는 무시하고 지나칠 수 있는 규모가 아니므로, 어떻게 다루느냐에 따라 모델 성능에 미치는 영향이 크다.

소현은 결측치를 단순히 제거하거나 채우고 끝내는 대신, “브랜드·카테고리·설명에 동시에 몇 개나 비어 있는가”라는 결측 개수 자체(`null_penalty`)와 가격 사이의 관계를 살펴보았다. 실제 분포는 다음과 같다.

<code>null_penalty</code> (결측 개수)	행 수	비율
0개(결측 없음)	846,446	57.1%
1개 결측	631,706	42.6%
2개 결측	3,451	0.23%

세 컬럼(`brand_name` / `category_name` / `item_description`)이 동시에 전부 결측인 행은 사실상 없었다 — `category_name` (0.43%)과 `item_description` (0.0004%) 결측 자체가 워낙 드물어, 대부분의 “1개 결측”은 `brand_name` 단독 결측이고 “2개 결측”은 그 드문 두 컬럼이 겹친 극소수 사례에 가깝다. 그 결과 결측된 정보가 많을수록(즉 상품 등록 정보가 부실할수록) 평균 판매 가격이 낮게 형성되는 경향이 뚜렷하게 나타났다. 이는 결측 여부 자체가 노이즈가 아니라 가격을 설명하는 하나의 신호가 될 수 있음을 보여주는 근거이며, 이후 전처리 단계에서 결측치를 삭제하지 않고 별도 범주로 유지하는 방향의 근거로 이어진다.



결측 개수와 가격의 관계

`null_penalty` (브랜드·카테고리·설명이 결측이었던 개수, 0~2)별 평균 `log1p(price)`. 결측 개수가 많을수록 평균 가격이 낮아지는 경향이 뚜렷하다.

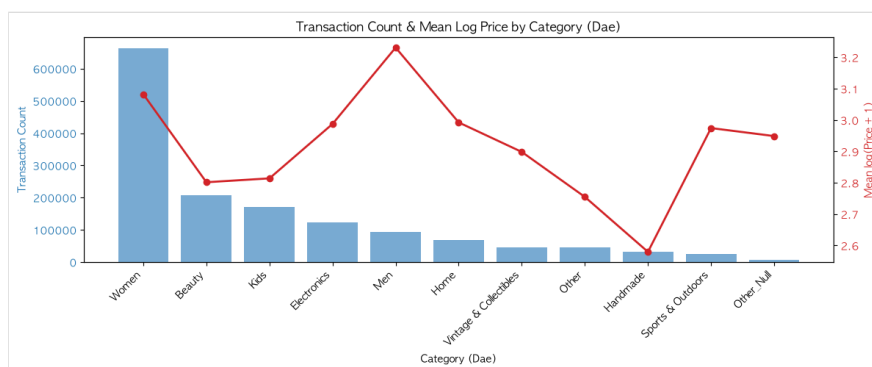
한편 용우와 진희는 서로 다른 각도에서 `brand_name`의 특이점을 짚었다. 용우는 `brand_name`이 비어 있더라도 `item_description` 텍스트 안에 실제 브랜드명이 언급되어 있는 경우가 상당수 있다는 점을 확인했고(이 관찰은 이후 4.2절에서 결측의 84%를 복원하는 실제 전처리로 이어진다), 진희는 실제로 다수의 상품(1,933건, 5.4점)에서 `brand_name` 값 자체가 "AND"라는 브랜드로 등록되어 있음을 발견했다. AND는 영어의 접속사와 형태가 같아 이후 텍스트 정제 단계에서 불용어(stopword)로 함께 제거될 위험이 있는 값으로, 결측치 못지않게 데이터 특유의 주의가 필요한 지점임을 시사한다. 두 관찰 모두 "결측/텍스트가 뒤섞인 브랜드 정보를 어떻게 정리할 것인가"라는 4장의 전처리 설계로 이어진다.

참고로 `price == 0` 행 제거 기준(4.1절)도 팀원마다 정확한 건수가 미묘하게 달랐다 — 소현·용우·유경·진희 4명은 874건을 확인했지만, 혜현은 자신의 노트북에서 826건으로 집계했다. 필터링 시점(중복 제거 전/후)의 차이로 보이며, 최종 채택 방법(`price > 0`) 자체는 5명 모두 동일했다.

3.3 카테고리 계층 구조

`category_name`은 "Men/Tops/T-shirts"처럼 /로 구분된 대분류/중분류/소분류 3단계 계층 구조를 갖는다. 5명 모두 이를 `cat_dae`(대분류)/`cat_jung`(중분류)/`cat_so`(소분류) 3개 컬럼으로 분리했으며, 고유값 개수는 대분류 11개, 중분류 114개, 소분류 871개로 전원 동일하게 확인했다. 결측치(`category_name`이 NaN인 6,327건)는 3단계 모두 별도 범주로 처리해, 계층 분리 이후에도 정보 손실 없이 다룰 수 있도록 했다.

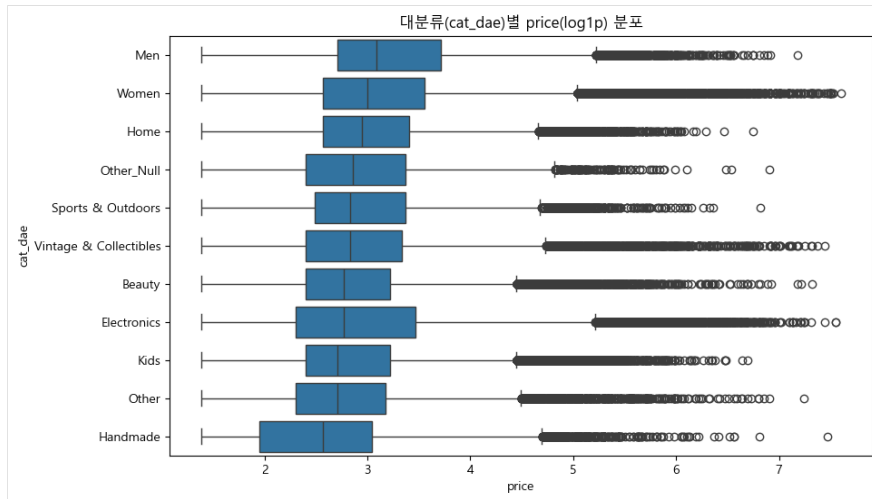
혜현은 대분류(`cat_dae`)별 거래 건수와 평균 로그 가격을 하나의 이중축 그래프로 겹쳐 그려, 어떤 카테고리가 거래량이 많는지와 어떤 카테고리가 상대적으로 비싼지를 동시에 비교했다. 대분류 사이에 거래 건수와 평균 가격의 순위가 서로 어긋나는 경우가 있어(거래량이 많다고 해서 평균 가격도 높은 것은 아님), 카테고리 피처가 거래량 편향과 가격 편향을 동시에 담고 있음을 확인할 수 있다.



대분류별 거래 건수와 평균 가격

대분류(`cat_dae`)별 거래 건수(막대)와 평균 `log1p(price)`(선)를 겹쳐 그린 이중축 그래프.

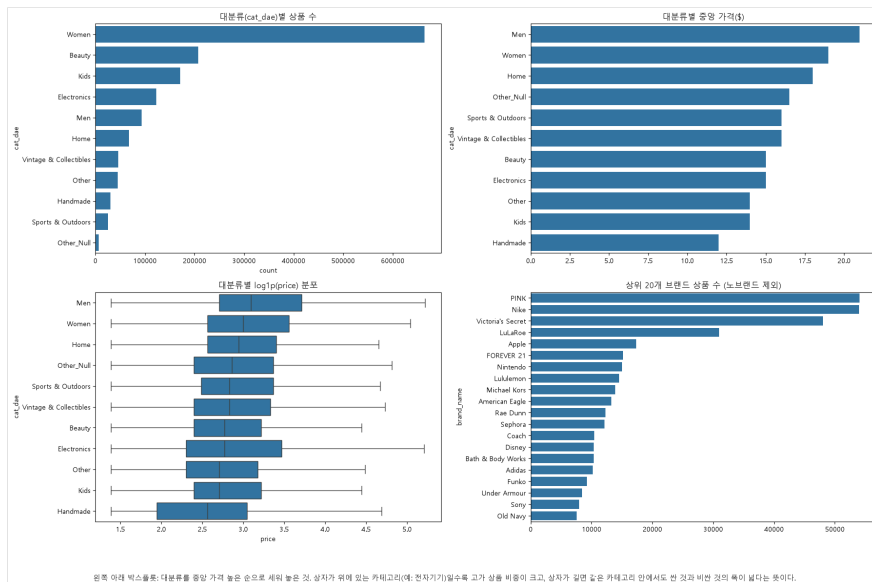
소현은 이를 보완해 대분류별 로그 가격의 분포 자체(중앙값뿐 아니라 사분위 구간과 산포)를 박스플롯으로 함께 확인했다. 평균만으로는 드러나지 않는, 카테고리 내부의 가격 편차(같은 카테고리 안에서도 저가 상품과 고가 상품이 얼마나 넓게 퍼져 있는지)를 함께 살펴야 이후 카테고리 피처의 중요도를 올바르게 해석할 수 있다는 판단에서다.



대분류별 로그 가격 분포

대분류(*cat_dae*)별 $\log_1 p(\text{price})$ 분포 박스플롯.

유경은 여기서 한 걸음 더 나아가 “거래량 순위”와 “가격 순위”를 나란히 비교했다.



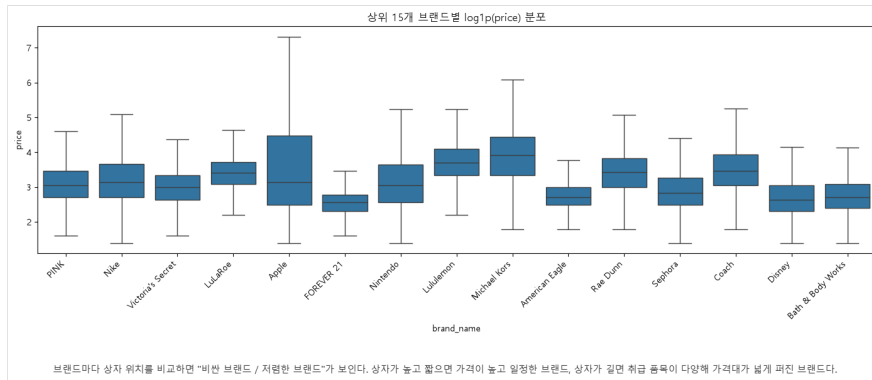
대분류별 상품수/중앙가격/브랜드 개관

좌상단: 대분류별 상품 수. 우상단: 대분류별 중앙 가격(\$). 좌하단: 대분류별 $\log_1 p(\text{price})$ 분포(중앙 가격 순 정렬). 우하단: 노브랜드를 제외한 상위 20개 브랜드 상품 수.

두 순위는 실제로 크게 어긋난다. 거래량 1위는 **Women** (약 66만 건)이지만 중앙 가격 1위는 **Men** (약 \$21)이고, **Women** 은 중앙 가격 기준으로 2위(약 \$19)에 그친다. 반대로 거래량이 가장 적은 축에 속하는 **Sports & Outdoors** (약 2만 건)는 중앙 가격이 낮은 편(하위 5위)이라, “거래량이 적으면 희소해서 비쌀 것”이라는 직관과도 어긋난다. 즉 카테고리라는 하나의 피쳐 안에 “얼마나 자주 거래되는가”와 “얼마에 거래되는가”라는 서로 독립적인 두 축의 정보가 섞여 있다는 것을 해현의 이중축 그래프에 이어 다시 한번 정량적으로 확인한 셈이다. 우하단 패널의 브랜드 상품 수 순위(PINK-Nike-Victoria's Secret-LuLaRoe-Apple 순)도 3.4절에서 다룰 “상위 15개 브랜드 가격 분포”(유경, 가격 기준 순위)와 나란히 놓고 비교할 만하다 — 거래량 상위 브랜드가 반드시 고가 브랜드는 아니라는 점에서, 카테고리 와 마찬가지로 브랜드 역시 “얼마나 자주 팔리는가”와 “얼마에 팔리는가”가 분리된 신호다.

3.4 브랜드와 수치형 피쳐의 관계

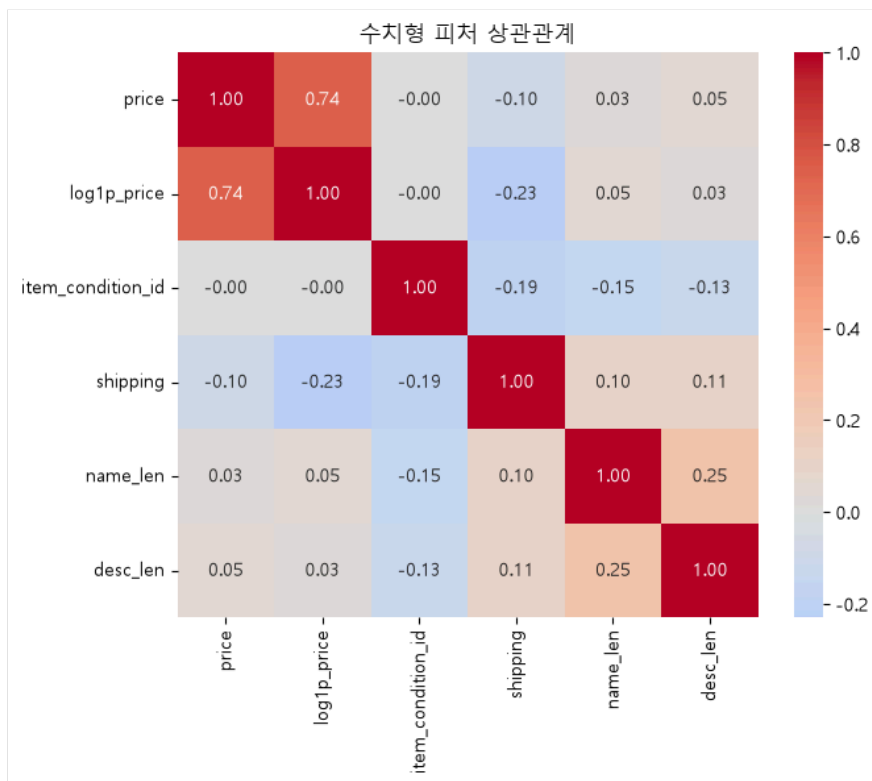
브랜드는 결측치를 제외하면 4,808개의 고유 값을 가지며, 상위 브랜드로는 PINK, Nike, Victoria's Secret 등 의류·패션 브랜드가 큰 비중을 차지한다. 유경은 결측치가 아닌 상위 15개 브랜드를 추려 브랜드별 로그 가격 분포를 박스플롯으로 비교했는데, 브랜드마다 상자의 위치(가격 수준)와 높이(가격 편차)가 뚜렷하게 갈렸다. 즉 상자가 높고 짧은 브랜드는 가격이 대체로 높고 균일하게 형성되는 프리미엄 브랜드에 가깝고, 상자가 길게 퍼진 브랜드는 취급 상품군이 다양해 가격대 자체가 넓게 분산되어 있다. 이는 브랜드명이 카테고리 못지않게 가격을 설명하는 유의미한 피쳐가 될 수 있음을 뒷받침한다.



상위 15개 브랜드 가격 분포

결측치를 제외한 상위 15개 브랜드의 $\log_1 p(\text{price})$ 분포 박스플롯.

유경은 여기에 더해 `item_condition_id`, `shipping`, `name / item_description` 글자 수, 원본-로그 `price` 를 모두 포함한 수치형 피처 간 상관관계를 히트맵으로 확인했다.



수치형 피처 상관관계 히트맵

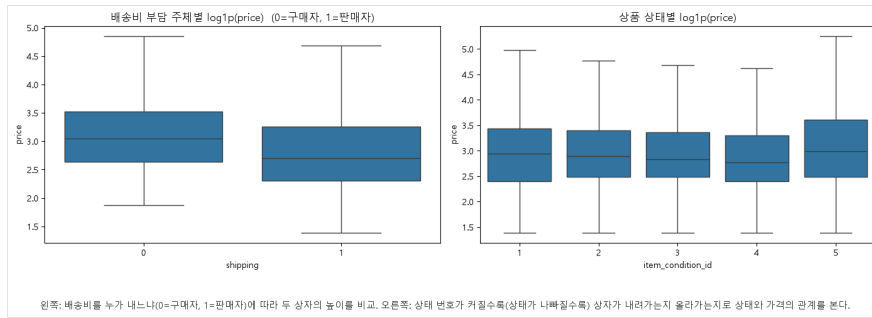
`item_condition_id`, `shipping`, 텍스트 길이, `price/log1p(price)` 간 상관관계 히트맵.

히트맵에서 $\log_1 p(\text{price})$ 와의 상관계수만 절댓값 기준으로 추려보면 다음과 같다.

변수	$\log_1 p(\text{price})$ 와의 상관계수
<code>shipping</code>	-0.23
<code>name_len</code> (상품명 글자 수)	+0.05
<code>desc_len</code> (설명 글자 수)	+0.03
<code>item_condition_id</code>	-0.00

`shipping` (배송비를 판매자가 부담하는지)이 수치형 피처 중에서는 가장 뚜렷한 상관을 보이는데, 이는 판매자가 배송비를 떠안는 상품일수록 상품 가격 자체를 낮게 책정하는 경향과 부합한다. 반면 `item_condition_id` (상품 상태 등급)는 상관이 거의 0에 가까워, 상태 등급만으로는 가격을 설명하기 어렵다는 것을 보여준다.

유경은 이 두 피처를 박스플롯으로도 직접 확인했다.

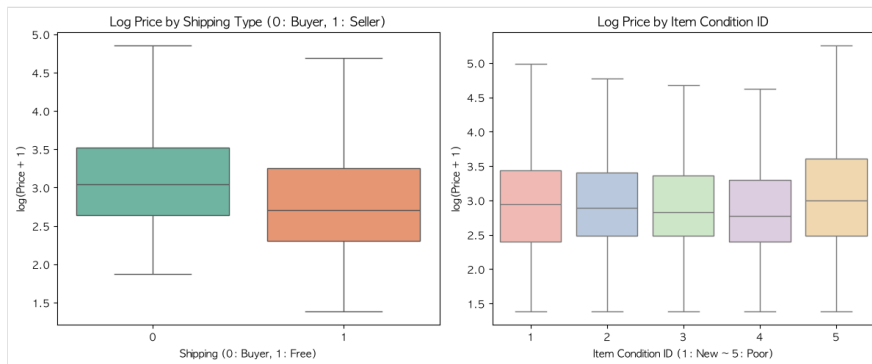


shipping/item_condition_id별 가격 분포

왼쪽: 배송비를 누가 내느냐(0=구매자, 1=판매자)에 따른 두 상자의 높이 비교. 오른쪽: 상태 번호가 커질수록(상태가 나빠질수록) 상자가 내려가는지 올라가는지로 상태와 가격의 관계를 본다.

shipping은 상관계수(-0.23)와 일관되게 중앙값이 뚜렷이 갈린다(구매자 부담 시 $\log_1 p(\text{price})$ 중앙값 약 3.05, 판매자 부담 시 약 2.71). 반면 item_condition_id는 1(새것)에서 4(사용감 있음)까지는 중앙값이 완만히 낮아지다가(2.93→2.88→2.82→2.75) 5(가장 낡음)에서 오히려 다시 높아진다(2.98) — 이 비단조적(non-monotonic)인 패턴이야말로 item_condition_id와 가격의 선형 상관계수가 거의 0으로 나온 이유다. 실제로는 상태가 가격에 영향을 주지만 그 관계가 일직선이 아니라서, 단순 상관계수 하나만으로는 “상태는 가격과 무관하다”고 성급히 결론 내릴 수 없다는 것을 보여주는 사례다.

해현도 동일한 두 피처를 독립적으로 박스플롯화해 같은 결론에 도달했다.



shipping/item_condition_id별 로그 가격 분포

왼쪽: shipping(0=구매자 부담, 1=판매자 무료배송)별 $\log_1 p(\text{price})$ 분포. 오른쪽: item_condition_id(1=새것~5=낡음)별 $\log_1 p(\text{price})$ 분포. 두 그래프 모두 이상치를 제외(showfliers=False)해 상자 간 위치 차이를 더 뚜렷하게 보여준다.

해현의 그래프에서도 shipping=0(구매자 부담) 쪽 상자가 shipping=1(판매자 무료배송)보다 위에 위치해 유경의 상관계수·중앙값 분석과 정확히 같은 방향을 가리키며, item_condition_id 쪽 상자들도 뚜렷한 단조 증가/감소 없이 뒤섞여 있어 “선형 상관관계로는 포착되지 않는 비단조적 관계”라는 해석을 팀원 두 명이 서로 다른 노트북에서 독립적으로 재현한 셈이다.

상품 상태 등급이나 배송비 부담 여부 같은 개별 수치형 피처와 가격 사이의 선형 상관관계는 이처럼 전반적으로 크지 않았는데, 이는 가격을 결정하는 정보 대부분이 카테고리·브랜드·텍스트 같은 범주형·비정형 피처에 담겨 있고, 수치형 피처는 보조적인 신호에 가깝다는 것을 시사한다. 이 관찰은 이후 5장(Feature Engineering)에서 텍스트·범주형 피처의 벡터화 방식에 공을 들여야 하는 이유와도 맞닿아 있으며, 6.2절에서 확인한 LightGBM 피처 중요도에서도 shipping이 상위권에 오른 것과 일관된다.

3.5 소결

원본 데이터는 148만여 건의 대규모 거래 기록이지만, 실질적으로 모델이 활용할 수 있는 정보는 상품 상태·배송비 같은 몇 개의 수치형 피처, 11~871개 수준의 계층적 카테고리, 4,808개에 달하는 브랜드, 그리고 자유 형식의 상품명·설명 텍스트로 요약된다. 가격은 소수 고가 상품이 꼬리를 이루는 우측 왜도 분포이며 $\log_1 p$ 변환이 사실상 모든 팀원의 공통 출발점이었고, brand_name의 높은 결측 비율(42.68%)과 카테고리의 계층 구조는 이후 전처리에서 반드시 다뤄야 할 두 축으로 확인되었다. 이 장에서 확인한 분포·결측·계층 구조상의 특징들은 4장(데이터 전처리)에서 각 팀원이 채택한 결측치 처리, 이상치 제거, 텍스트 정제 방식의 직접적인 근거가 된다.

4. 데이터 전처리

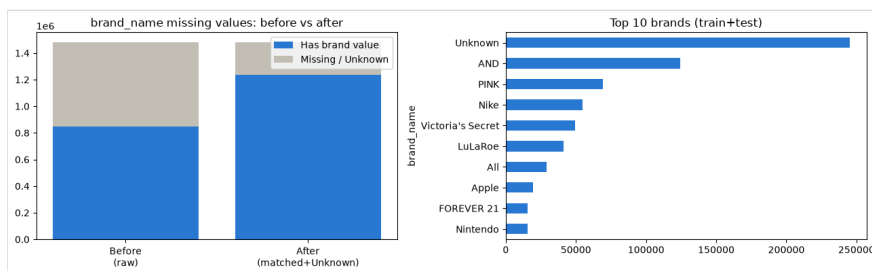
4.1 공통 전처리 파이프라인

5명 모두 동일한 원본 데이터(mercari_train.tsv, 1,482,535행 8열)에서 출발했고, 세부 구현은 다르지만 다음 절차는 팀 공통 규칙으로 합의되어 전 원 동일하게 적용했다.

- 가격 이상치 제거** — `price == 0` (무료 나눔·오입력으로 판단)과 `price ≥ 2000` (달러, Mercari 실제 등록 상한 정책 기준) 행을 제거했다. 0원 행은 874건, 2000달러 이상 행은 소수(수~수십 건) 수준이었다. 용우만 `price ≤ 2000` 으로 경계값 2000을 포함시켰고, 나머지 4명은 `price < 2000` 으로 2000 자체도 제외해 경계 처리에 미세한 차이가 있다.
- train_id 컬럼 제거 및 중복 행 제거** — `train_id` 는 단순 일련번호라 가격 예측에 무의미하므로 제거하고, 완전히 동일한 행을 `drop_duplicates()` 로 정리했다. 혜현은 `train_id` 를 제외한 7개 컬럼(가격 포함) 조합 기준으로 중복 49건을 먼저 제거한 뒤 이상치를 필터링 하는 순서를 택했고, 나머지는 이상치 제거 → 컬럼 삭제 → 중복 제거 순서를 택했다. 순서 차이에도 최종 행 수는 1,481,60x 대로 5명 모두 거의 일치한다(소현 1,481,603 / 용우 1,481,609 / 유경 1,481,603 / 진희 1,481,603 / 혜현 1,481,603).
- 타깃 로그 변환(log1p)** — 2.2에서 확인한 `price` 의 우측 왜도와 RMSLE 지표 특성에 따라 전원이 `np.log1p(price)` 를 타깃으로 사용했다. 이후 모델은 log 공간에서 학습하고 필요 시 `expm1` 로 원가격을 복원하는 방식을 공유한다.
- 카테고리 3단계 분할** — `category_name` 을 / 기준으로 대/중/소분류 3개 컬럼으로 나눴다(소현 `cat_dae/jung/so`, 용우 `cat_main/sub/sub2`, 유경 `cat_dae/jung/so`, 혜현 `cat_dae/jung/so`). 대분류 11종, 중분류 114종, 소분류 861~871종으로 팀원 간 거의 동일하게 집계되어 파싱 로직이 사실상 같음을 서로 확인했다. 진희만 이 단계를 생략하고 `category_name` 을 / 를 공백으로 치환한 뒤 상품명·브랜드와 하나의 텍스트로 합쳐 TF-IDF에 직접 흘려보냈다.
- 결측치 처리** — `brand_name` (약 63만 건, 전체의 42.7%), `category_name` (6,327건), `item_description` (6건)에 결측치가 있었다. 소현·유경·혜현은 세 컬럼 모두 'Other_Null' 문자열로 채워 "정보 없음" 자체를 하나의 범주/토큰으로 남겼다. 용우는 단순 대체 대신 `item_description` 원문에서 정규식으로 브랜드명을 역매칭해 결측을 실제 브랜드로 복원한 뒤, 그래도 못 채운 값만 'Unknown' 으로 통일했다(\$4.2 참고). 진희는 별도 대체 없이 텍스트 결합 시 `fillna('')` 만 적용했다.
- 텍스트 정제(name, item_description)** — 소문자화 → 영문자/숫자/공백 외 문자 제거(정규식) → `word_tokenize` 토큰화 → 불용어 제거 순으로 처리했다. 이 정제 함수는 팀 공통 템플릿을 공유해 5개 노트북의 구현이 거의 동일하다. 특히 두 가지 규칙이 공통으로 지켜졌다: (a) `no/not/nor/never` (용우·유경·진희·혜현은 `without` 까지 5개) 등 부정어는 의미를 뒤집을 수 있어 불용어 목록에서 제외했고, (b) 가격 등 개인 정보 마스킹 토큰 `[rm]` 이 정제 후 의미 없는 `rm` 잔여 토큰으로 남는 문제를 막기 위해 `rm` 을 불용어로 추가했다. 유일한 예외는 소현의 정규식으로, 다른 4명이 `[^a-z0-9\s]` (숫자 보존)을 쓴 것과 달리 소현은 `[^a-z\s]` (숫자까지 제거)를 사용했다 — 이 차이의 실제 영향은 \$4.2에서 다룬다.
- Train/Test 분리 후 fit** — 벡터라이저 인코더는 train에만 `fit` 하고 test에는 `transform` 만 적용해 정보 누수를 막는 원칙을 5명 모두 지켰다(용우·유경·진희·혜현은 최초 분할 이후 곧바로 이를 적용했고, 소현은 초반 실험 버전에서는 전체 데이터에 벡터라이저를 먼저 `fit` 한 뒤 뒤늦게 `train_test_split` 을 수행했으나, 최종 채택 버전(9장 2-stage 검증)에서는 train/valid/test 3-way 분리 후 fold별로 다시 `fit` 하도록 개선했다).

4.2 전처리 전/후 비교 근거

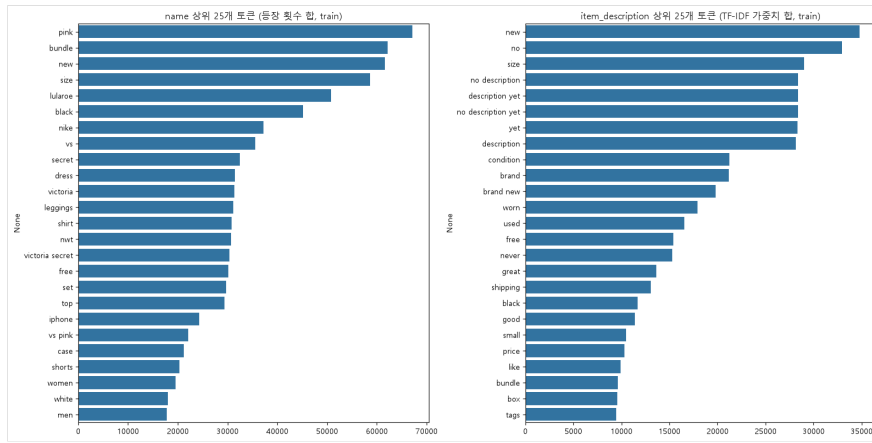
(1) **브랜드 결측치 처리 효과** — 용우. 원본 `brand_name` 은 1,481,609건 중 632,295건(약 42.7%)이 결측이었다. 용우는 train에서 학습한 브랜드 사전으로 `item_description` 원문을 정규식 역매칭해 결측 중 상당수를 실제 브랜드명으로 복원했고, 그래도 못 찾은 값만 'Unknown' 으로 남겼다. 그 결과 "값이 있는" 행 비율이 약 57%(850K)에서 약 84%(124만)로 늘었고, 남은 Unknown 은 약 25만 건으로 줄었다.



brand_name 결측치 처리 전/후

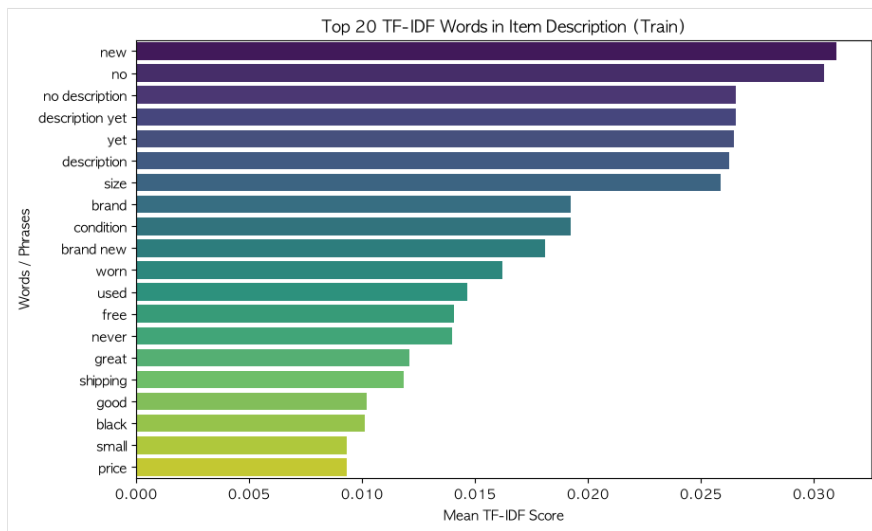
정규식 역매칭 전/후 brand_name 결측 비율 변화.

(2) **텍스트 정제 결과** — 유경·혜현. 정제 파이프라인(소문자화·특수문자 제거·불용어 제거)을 거친 뒤 `name / item_description` 에서 실제로 가장 많이 등장한 토큰을 확인하면, `new`, `size`, `no description yet`, `brand new`, `used`, `free` 등 상품 상태·거래 조건을 나타내는 단어가 상위에 오른다. 특히 `no description yet`, `description yet` 같은 구는 원래 결측/기본 문구였던 `item_description` 이 정제 후에도 하나의 고빈도 신호로 남아 있음을 보여주며, 이는 \$4.1에서 부정어(`no`)를 불용어에서 제외한 설계가 실제로 의미를 보존했음을 뒷받침한다.



name/description 상위 토큰(유경)

정제 후 name/item_description 상위 빈도 토큰(유경).



item_description TF-IDF 상위 단어(혜현)

item_description TF-IDF 상위 20개 단어(혜현).

(3) 정규식(숫자 보존 여부) 선택의 모델 성능 영향 — 유경·용우 교차 검증. \$4.1에서 언급한 대로 소현만 숫자를 제거하는 정규식([^{a-z}])을 썼고 나머지 4명은 숫자를 보존했다([^{a-z0-9}]). 유경과 용우는 이 선택이 실제 성능에 미치는 영향을 각자 독립적으로 Ridge A/B 테스트로 정량 검증했다.

검증자	조건	Ridge RMSLE	비고
유경(17만 표본)	숫자 보존 [^{a-z0-9}]	0.48463	어휘 5,550개 보존
유경(17만 표본)	숫자 제거 [^{a-z}]	0.48787	사이즈·용량 등 5,550개 토큰 소실/문 개집
용우(전체 데이터)	숫자 보존	0.45597	
용우(전체 데이터)	숫자 제거	0.45892	

두 사람 모두 독립적으로 “숫자 보존이 근소하게(RMSLE 기준 0.3~0.7%) 더 유리하다”는 동일한 결론에 도달했다. 이는 “4gb”, “32gb”, “사이즈 1X/3X”처럼 숫자가 실제로 가격에 영향을 주는 정보를 담고 있다는 가설을 뒷받침하는 근거이며, 소현의 노트북만 이 검증 이전의 규칙(숫자 제거)을 그대로 쓰고 있어 팀 내에서 유일하게 다른 선택으로 남아 있다.

(4) 카테고리 파싱 방식 비교 — 유경. category_name 은 일부 값에서 소분류 자체에 / 가 포함돼 4~5조각으로 쪼개진다(예: Electronics/Computers & Tablets/iPad/Tablet/eBook Readers). 유경은 split('/') (앞 3조각만 사용)와 split('/', maxsplit=2) (뒷부분까지 소분류에 보존)를 비교했고, 두 방식의 결과가 달라지는 행은 4,385건으로 전체 1,481,612행의 0.30%에 불과했다. 손실되는 하위 카테고리 간 평균 가격 차이도 표준편차 대비 미미해(예: eBook Readers \$75.8 vs eBook Access \$69.7), 최종적으로 더 단순한 split('/') 를 채택했다.

4.3 전처리 유형 분류 — “고정 적용” vs “여러 방법 시도·비교”

5개 노트북의 전처리 접근은 크게 두 유형으로 나뉜다. 하나는 하나의 전처리 파이프라인을 처음부터 확정해두고 이후 모든 모델에 동일하게 재사용하는 **고정 적용형**이고, 다른 하나는 이상치 기준·피쳐 구성·텍스트 정제 규칙 등 여러 대안을 직접 비교한 뒤 최종안을 선택하는 **시도·비교형**이다. 용우·진희·혜현이 전자, 소현·유경이 후자에 가깝다.

고정 적용형. 용우는 이상치 제거 → 카테고리 3분할 → description 역매칭 기반 브랜드 결측 복원 → 숫자 보존 텍스트 정제 → TF-IDF(ngram 1-2)/원-핫 인코딩까지 이어지는 파이프라인 하나를 구성한 뒤, 이를 `pickle` 로 저장해 Ridge와 LightGBM 두 모델에 동일하게 적용했다(5.2절). 사전에 A/B로 검증한 것은 텍스트 정제 규칙(숫자 보존 여부, 4.2절)뿐이었고, 이 결정이 확정된 뒤로는 파이프라인 자체를 바꾸지 않았다. 진희는 `name · brand_name · category_name` 을 하나의 `product_text` 로 결합하는 방식을 처음부터 채택하고, 그 위에서 TF-IDF의 `max_features` (100k→150k→200k)와 `min_df` 같은 벡터화 파라미터만 조정했다(5.4절). 카테고리를 별도로 분할하거나 브랜드 결측을 따로 복원하는 대안은 시도하지 않고, “텍스트 통합”이라는 하나의 설계를 고정한 채 그 위에서 Ridge alpha와 벡터화 파라미터를 반복적으로 튜닝하는 방식을 택했다. 혜현은 이상치 제거 → 카테고리 3분할 → `Other_Null` 결측 대체 → 숫자 보존 텍스트 정제 → `CountVectorizer/TfidfVectorizer+LabelBinarizer` 인코딩까지 하나의 파이프라인을 확정된 뒤, 이 동일한 피쳐 위에서 Ridge(설명글 포함/제외 비교), LightGBM(3차에 걸친 정밀 재튜닝), 앙상블까지 모든 실험을 재사용했다(5.5절·7.2절). 전처리 자체를 여러 번 바꾸는 대신, 고정된 피쳐 위에서 모델과 하이퍼파라미터 쪽 실험에 집중한 경우다.

시도·비교형. 소현은 `hstack` 구성 방식 자체를 여러 버전으로 만들어 비교했다 — 원-핫+유니그램 조합과 `LabelEncoder+LGBM` 네이티브 `categorical+n-gram` 조합을 직접 비교해 RMSLE 0.49220→0.47363의 차이를 확인했고(8.4절), 여기에 길이 피쳐·타깃 인코딩 같은 신규 피쳐를 추가한 세 번째 조합까지 순차적으로 시도한 뒤 최종안을 채택했다. Ridge alpha도 11개 값을 그리드로 탐색해(7.1절), 전처리·피쳐 구성·하이퍼파라미터 세 축 모두에서 여러 안을 비교하는 방식을 취했다. 유경도 두 갈래를 직접 비교했다 — `category_name` 파싱 방식(`split()` vs `split(maxsplit=2)`)을 비교해 영향이 미미함(0.30%)을 확인하고 더 단순한 방식을 선택했고(4.2절), 텍스트 정제 시 숫자 보존 여부도 A/B로 비교(0.48463 vs 0.48787)해 숫자 보존 쪽을 채택했다. 두 실험 모두 “더 나은 대안이 있는지 직접 검증한 뒤 선택”하는 방식이라는 점에서 소현과 같은 유형으로 분류했다.

다만 용우·혜현도 텍스트 정제 규칙(숫자 보존 여부)이나 피쳐 포함 여부(설명글 포함/제외)를 실험하긴 했다. 이는 “하나의 최종 파이프라인을 정하기 위한 사전 검증”의 성격이 강해 이후 전체 파이프라인 자체는 고정해 재사용했다는 점에서, 소현·유경처럼 여러 전처리 기법을 순차적으로 실험·비교하는 유형과는 구분했다.

시도·비교형 사례 — 유경의 `category_name` 파싱 방식 비교. “시도·비교형”이 실제로 어떤 모습인지 유경의 노트북 주석을 그대로 옮기면 다음과 같다. `category_name` 은 보통 “대분류/중분류/소분류” 3단계지만, 일부 소분류 이름 자체에 `/` 가 들어있어 4~5조각으로 쪼개지는 경우가 있다(예:

`"Electronics/Computers & Tablets/iPad/Tablet/eBook Readers")`.

```
# [시도했지만 채택하지 않은 방법] category_name.split('/', 2)로 소분류 뒷부분까지 보존
#
# split('/', 2) (maxsplit=2)를 쓰면 소분류가 이렇게 남는다:
# cat_so = "iPad/Tablet/eBook Readers"
# 반면 그냥 split('/') + zip(*...) (기존 코드)는 앞 3조각만 취하므로:
# cat_so = "iPad" 만 남고 뒷부분("Tablet/eBook Readers")은 사라진다
# (zip은 가장 짧은 3조각에 길이를 맞추고 4~5번째 조각을 잘라내기 때문)

def split_cat(category_name):
    try:
        return category_name.split('/')      # 기존 방식: 앞 3조각만 사용
    except:
        return ['Other_Null', 'Other_Null', 'Other_Null']
```

실제 데이터로 두 방식의 차이를 직접 검증한 결과는 다음과 같다(주석 원문 기준):

- 두 방식의 결과가 달라지는 행은 **4,385건, 전체 1,481,612건의 0.30%**
- `Other_Null` (카테고리 결측) 다음으로 흔한 케이스지만 그마저도 6,314건(0.43%) 수준
- 손실되는 하위 카테고리 간 가격 차이도 표준편차 대비 미미함(예: `eBook Readers` 평균 \$75.8 vs `eBook Access` 평균 \$69.7 — 차이 \$6이지만 가격 표준편차가 ~\$100 규모라 통계적으로 미미)
- `name / item_description` 텍스트 처리 단계에서 `"kindle"`, `"case"` 같은 단어가 어차피 텍스트 피쳐로 이미 들어가 있어, 카테고리에서 일부가 손실되어도 다른 경로로 일부 보완됨
- 결론: **`maxsplit=2`의 효과가 미미하고 코드 의도도 불명확해 채택하지 않음** — 기존처럼 `split('/')` 을 그대로 사용

이 사례는 “여러 방법을 시도·비교”한다는 것이 실제로는 어떤 작업인지 잘 보여준다 — 대안을 코드로 직접 구현해보고, 그 대안이 실제로 바꾸는 행의 수·비용·가격 영향을 전부 정량적으로 측정한 뒤에야 “차이가 미미하니 기존 방식을 유지한다”는 결론에 도달했다. 이는 4.2절(4)에서 이미 요약한 결과(0.30%)의 실제 판단 근거이기도 하다.

대비되는 사례 — 소현의 `hstack` 네이티브 `categorical` 비교(실험 후 실제로 채택한 경우). 유경의 사례가 “실험했지만 기각한” 경우라면, 소현의 `hstack` 구성 비교는 “실험해서 실제로 채택까지 이어진” 경우다. 소현은 범주형 컬럼(`brand_name / cat_dae / cat_jung / cat_so`)을 원-핫 인코딩 대신 정수 라벨로 바꿔 LightGBM의 `categorical_feature` 파라미터로 직접 넘기는 대안을 시도했다.

```
# 범주형 컬럼: OneHotEncoder 대신 LabelEncoder로 정수 리벨링
# -> LGBM categorical_feature로 넘겨 원-핫(4,808+11+114+871=5,804차원) 대신
#   컬럼 4개로 표현
from sklearn.preprocessing import LabelEncoder

cat_label_encoders = {}
cat_label_df = pd.DataFrame(index=mercari_df.index)
for col in CAT_COLS:
    le = LabelEncoder()
    cat_label_df[col] = le.fit_transform(mercari_df[col])
    cat_label_encoders[col] = le

X_cat_label = csr_matrix(cat_label_df.values.astype(float))
```

원-핫 인코딩은 트리 모델 입장에서 “이 브랜드인가 아닌가”를 컬럼 4,808개로 쪼개놓은 것이라, 분기 하나가 데이터 전체의 극히 일부만 가르느 비효율이 생긴다는 것이 이 대안을 시도한 이유였다. 실제로 텍스트도 유니그램 대신 n-gram(1,2)으로 함께 바꾼 결과, LightGBM RMSLE가 0.49220 → 0.47363으로 약 3.8% 개선됐다(8.4절). 유경의 사례와 이 사례를 나란히 보면, “시도·비교형”이라고 해서 항상 새 대안이 채택되는 것도, 항상 기각되는 것도 아니라 **실측 결과가 실제로 그 방향을 결정한다**는 공통점이 드러난다 — 두 사람 모두 “그렇듯해 보이는” 대안을 직접 구현하고 정량적으로 측정한 뒤에야 유지·교체 여부를 판단했다.

5. Feature Engineering

5.1 팀원별 피처 표현 방식 비교

같은 원본 컬럼을 서로 다른 벡터화·인코딩 전략으로 표현했다. 핵심 차이를 정리하면 다음과 같다.

항목	진희	혜현	용우	소현	유경
name 벡터화	— (product_text에 통합)	CountVectorizer(제한 없음 → 93,451차원)	TfidfVectorizer(max 30,000, ngram 1-2)	CountVectorizer(max 20,000) / v2: ngram(1,2)	CountVectorizer(max 30,000, ngram 1-3)
item_description 벡터화	TfidfVectorizer(max 200,000, ngram 1-2, product_text와 별도)	TfidfVectorizer(max 50,000, ngram 1-2)	TfidfVectorizer(max 50,000, ngram 1-2)	TfidfVectorizer(max 30,000, 커스텀 토큰 analyzer) / v2: 유니그램+바이그램 직접 생성	TfidfVectorizer(max 50,000, ngram 1-3)
카테고리 인코딩	미분할, TF-IDF 텍스트에 흡수	LabelBinarizer(sparse)	OneHotEncoder(핸들 unknown=ignore)	OneHotEncoder(핸들 unknown=ignore) / v2: LabelEncoder+LGBM native categorical	LabelBinarizer(sparse)
브랜드 결측 처리	fillna('') 로 텍스트 결합	'Other_Null' 상수 대체	description 정규식 역매칭 후 잔여만 'Unknown'	'Other_Null' 상수 대체	'Other_Null' 상수 대체
파생(엔지니어링) 피처	product_text_len/word_count, description_len/word_count (4개 길이 피처, StandardScaler)	없음	없음(6개 원본 피처 + 텍스트/카테고리 벡터만 사용)	null_penalty (결측 개수), 이름/설명 길이, has_brand, brand_te / cat_so_te (타깃 인코딩, fold 내 계산)	없음
수치형 스케일링	길이 피처 4개 StandardScaler	원본 그대로 사용(스케일링 없음)	item_condition_id 만 StandardScaler	신규 피처만 StandardScaler	원본 그대로 사용(스케일링 없음)

이처럼 범주형 인코딩은 OneHotEncoder(소현·용우)와 LabelBinarizer(유경·혜현)로 갈렸고, 텍스트는 CountVectorizer와 TfidfVectorizer를 컬럼별로 다르게 조합했다. 대체로 “name (짧은 텍스트)은 등장 빈도가 중요하니 CountVectorizer, item_description (긴 텍스트)은 문서 길이 편차가 크니 TF-IDF”라는 논리를 공유하지만, 용우·혜현은 name 에도 TF-IDF/무제한 CountVectorizer를 적용해 접근이 같았다. 진희는 유일하게 name / brand_name / category_name 을 하나의 product_text 로 병합해 카테고리 인코딩 자체를 생략한 것이 가장 큰 구조적 차이다.

결측 처리·벡터화·인코딩을 모두 마친 뒤 최종 피처 행렬 차원은 팀원마다 크게 달랐다(용우 85,529차원, 소현 55,807차원, 혜현 148,976차원 등). 공통적으로 텍스트 피처(name+description)가 전체 차원의 90% 이상을 차지해, 이 문제가 소수의 정형 변수보다 비정형 텍스트에서 대부분의 가격 신호를 얻는 구조임을 보여준다.

5.2 용우 — 결측 브랜드 복원과 재사용 가능한 전처리 아티팩트

용우는 name/item_description 벡터화, brand_name/category 원-핫 인코딩 등 모든 전처리 객체를 학습 시점에 train에서만 fit 한 뒤, 4.2절에서 다룬 브랜드 사전과 동일한 방식으로 “재사용을 위해 fit된 객체 자체를 보존”하는 전략을 취했다. 최종적으로는 카테고리 3분할 → 브랜드 결측치 채우기(원문 description 매칭) → 텍스트 클렌징 → 인코딩/벡터화/스케일링까지 전체 파이프라인을 하나의 아티팩트로 묶어 저장했는데, 본인이 코드 주석에 남긴 대로 “새 상품 데이터가 들어와도 이 파일 하나만 불러와 preprocess_new_data() 에 통과시키면 학습 때와 동일한 형식의 feature가 나온다”는 재현성을 목표로 한 설계다. 이 재사용성 설계는 결국 새 상품 하나를 넣으면 바로 가격을 추천하는 predict_price(raw_df) 엔드투엔드 함수로 이어지는데(9.1절), 팀 안에서 유일하게 “실전 배포”까지 염두에 둔 설계다.

브랜드 결측 복원 시 정규식은 **긴 브랜드명이 먼저 매칭되도록 길이 내림차순으로 정렬**한 뒤 re.escape 와 단어 경계(\b)를 적용한 alternation으로 구성했다(예: “Nike”보다 “Nike Air”가 먼저 매칭 시도되도록) — 짧은 브랜드명이 긴 브랜드명의 부분 문자열로 먼저 걸려 오매칭되는 것을 막기 위함이다. 같은 leakage 방지 원칙(train에서만 known 값 집합 구성)을 브랜드뿐 아니라 cat_main / cat_sub / cat_sub2 (카테고리 3단계)에도 동일하게 적용했다.

OneHotEncoder(handle_unknown='ignore') 는 학습 때 못 본 카테고리를 0벡터로 처리하는데, 이미 브랜드 결측치 복원 단계에서 restrict_to_known 으로 'Unknown'으로 통일해둔 값 위에 이중 안전장치를 더한 것이라고 명시했다.

텍스트 정제 코드에서는 실제 버그를 하나 발견해 스스로 고친 기록도 남아 있다. 부정어 불용어 예외 목록을 만들 때 원래 코드는 {'no', 'not', 'nor', 'never', 'without'} 처럼 never 와 without 사이에 콤마 빠져 있어, 파이썬이 두 문자열을 'neverwithout' 이라는 존재하지 않는 단어 하나로 합쳐버리는 문제가 있었다. 코드 주석에 “콤마 추가로 수정”이라고 직접 남기며 {'no', 'not', 'nor', 'never', 'without'} 으로 고쳤다 — 결과 수치보다 구현 과정의 QA 사례로 의미가 있다.

해석 단계에서는 hstack 결합 순서(category → 수치형 → shipping → name → description)를 그대로 활용해, Ridge 계수를 6개 컬럼 그룹 (brand_name/category/item_condition_id/shipping/name/description)으로 묶어 그룹별 대표 가중치를 확인했다(6.2절 피쳐 중요도 해석과 연결된다).

5.3 유경 — 텍스트 길이 기반 벡터화 전략과 희소성 점검

유경은 텍스트 정제 각 단계(소문자화, 특수문자 제거, 공백 정리, 토큰화, 불용어 제거)의 필요성을 코드 주석으로 상세히 남겼는데, 숫자를 보존하기로 한 결정은 실제 사례에서 나왔다고 밝혔다 — 초기에 영문자만 남기는 정규식([a-zA-Z])을 썼을 때 “The pale pink is a 1X, and I also have a 3X” 같은 문장에서 사이즈 표기 1x / 3x 가 모두 의미 없는 x 로 뭉개져 구분이 안 되는 문제를 발견했고, 이를 계기로 [a-zA-Z0-9] 로 바꿔 숫자를 보존했다(이 선택의 실제 성능 효과는 4.2절의 A/B 비교 참고).

벡터화 방식 선택에도 근거를 남겼다 — name 은 평균 26자의 짧은 텍스트라 CountVectorizer, item_description 은 평균 146자의 긴 텍스트라 TfidfVectorizer를 각각 적용했다(다른 팀원들도 대체로 이 논리를 공유했다, 5.1절 참고). 벡터화 이후에는 상위 빈출 토큰과 함께 “토큰 수 대비 실제 사용 밀도”(희소성)를 직접 측정해, 두 벡터가 실제로 얼마나 성기게 채워지는지 수치로 확인했다 — name (CountVectorizer) 비영 원소 비율 0.01848%, item_description (TfidfVectorizer) 비영 원소 비율 0.04885%로, item_description 쪽이 약 2.6배 더 조밀하다. 이는 item_description 이 name 보다 평균 5배 이상 길어(146자 vs 26자) 문서당 등장 토큰 수 자체가 많기 때문으로 해석했다. 결측 처리-벡터화-인코딩을 모두 마친 뒤 최종 학습/테스트 행렬은 각각 (1,185,282, 85,521), (296,321, 85,521) 차원으로 확정됐다.

범주형 인코딩 세부. cat_dae / cat_jung / cat_so / brand_name 네 컬럼 각각에 별도의 LabelBinarizer(sparse_output=True) 인스턴스를 만들어 train에만 fit_transform, test에는 transform 만 적용했다(코드 주석: “검증 세트 정보 누수 방지”). item_condition_id / shipping 은 이미 숫자형이라 인코딩 없이 csr_matrix() 로만 변환했다.

```
lb_cat_dae = LabelBinarizer(sparse_output=True)
X_train_cat_dae = lb_cat_dae.fit_transform(X_train['cat_dae'])
X_test_cat_dae = lb_cat_dae.transform(X_test['cat_dae'])
# lb_cat_jung, lb_cat_so, lb_brand도 동일 패턴으로 반복
```

실행 결과 차원은 cat_dae (1,185,282×11), cat_jung (×114), cat_so (×861), brand_name (×4,533)이다. 눈에 띄는 점은 cat_so 의 인코딩 차원(861)이 전체 데이터 기준 소분류 종류(871개)보다 10개 적다는 것 — train에는 없고 test에만 존재하는 소분류 10개가 있다는 뜻이며, handle_unknown 처리 없이 LabelBinarizer 를 그대로 쓰면 이 값들이 transform 시 전부 0벡터로 처리된다. 결측치는 brand_name 을 'Other_Null' 로, category_name 은 split_cat() 의 try/except 에서 ['Other_Null']×3 으로 채웠고, item_description 은 다른 두 컬럼과 다른 상수 'No description yet' 으로 채워 결측을 구분했다.

수치형-파생 피쳐는 없음. name_len / desc_len 길이 변수를 계산한 코드가 있지만, 이는 3.4절 상관관계 히트맵을 그리기 위한 EDA용 계산일 뿐 실제 모델 입력 피쳐(hstack 결합 대상)로는 쓰이지 않았다 — 5.1절 표의 “없음” 표기가 정확함을 재확인했다.

메모리 최적화는 제한적. LabelBinarizer(sparse_output=True) 로 인코딩 결과 자체를 희소 행렬로 유지하고 hstack 결과를 tocsr() 로 압축하는 것 외에, gc.collect() 나 dtype 다운캐스팅 같은 명시적 메모리 관리 코드는 사용하지 않았다.

해석 단계에서는 hstack 순서(수치형 → cat_dae → cat_jung → cat_so → brand → name → description)를 기준으로 Ridge와 Lasso가 실제로 0이 아닌 값으로 남긴 계수 개수를 비교해, Lasso의 과소적합 원인(8.3절)을 이 순서 정보를 활용해 직접 진단했다. 카테고리 파싱 방식(split() vs split(maxsplit=2)) 대안 비교는 4.3절에서 이미 상세히 다뤘다.

5.4 진희 — product_text/item_description 통합과 min_df-AND-브랜드 처리

진희의 피쳐 엔지니어링은 name / brand_name / category_name 을 하나의 product_text 로, name / item_description 을 결합한 item_description 으로 재구성하는 것에서 출발했다(4.1-5.1절 참고). 실제 구현은 다음과 같다.

```
def make_product_text(df):
    df['product_text'] = (
        df['name'].fillna('') + ' ' +
        df['brand_name'].fillna('') + ' ' +
        df['category_name'].fillna('').str.replace('/', ' ', regex=False)
    )
    df['product_text'] = (
        df['product_text'].apply(preprocess_text).apply(lambda x: ' '.join(x))
    )
    return df
```

teamexcelssheet에 본인이 직접 남긴 통합 의도는 세 가지다 — (1) 텍스트에서 반복 등장하는 브랜드·특징 단어로 중요도를 자연스럽게 학습시킬 수 있고, (2) brand_name 이 결측이어도 name 에 포함된 브랜드 정보로 보완할 수 있으며, (3) product_text 자체가 상품 정보를 압축적으로 담아 설명이 부실한 상품도 다른 텍스트로 보완할 수 있다는 것이다. 다만 이 통합에는 “기존의 명확한 구분(카테고리 등)이 사라진다”, “제목과 설명이 상이하면 오히려 파악이 어려워진다”는 단점도 본인이 직접 명시했다.

이 과정에서 발견한 세부 이슈도 있다. brand_name 이 AND 인 상품 1,933건이 불용어 and 와 동일한 토큰으로 처리되는 문제를 확인하고, 다음 코드로 brand_name 에서만 AND 를 brand_and 로 치환해 불용어 제거를 피했다(name 의 and 는 “pink and black”처럼 일반 접속사와 구분이 어려워 변환 대상에서 제외했다).

```
mercari_df['brand_name'] = (
    mercari_df['brand_name']
    .str.replace(r'\bAND\b', 'brand_and', regex=True)
)
```

alpha=2.0 기준 RMSLE가 0.4343549 → 0.4343427로 미세하게 개선됐다(개선폭은 작지만 방향은 일관됐다). 참고로 같은 “AND 브랜드” 문제를 다른 파이프라인·다른 컬럼 범위에서 다시 집계한 용우의 결과는 규모가 훨씬 컸다 — 용우의 brand_name 최종 분포에서 AND 는 Unknown (196,041건) 다음으로 많은 99,396건(2위, PINK의 55,325건보다도 많음)을 차지했다. 진희(product_text 기준 1,933건)와 용우(brand_name 컬럼 자체 기준 99,396건)는 집계 범위가 달라 숫자를 직접 비교할 수는 없지만, 두 사람이 완전히 독립적인 파이프라인에서 같은 문제를 각자 발견했다는 점은 이 이슈가 우연이 아니라는 것을 뒷받침한다.

TF-IDF min_df (희귀 단어 제거 기준) 실험도 진행했다 — 기준값(product_text / item_description 모두 min_df=3)이 0.434032로 가장 좋았고, 두 값을 모두 5로 올리면 0.435096으로 악화됐다. 특히 product_text 의 min_df 만 5로 올렸을 때(0.435099)가 item_description 만 5로 올렸을 때(0.434074)보다 훨씬 크게 나빠져, product_text 의 희귀 단어(특정 브랜드·상품명)가 item_description 의 희귀 단어보다 가격 예측에 더 중요하다는 것을 확인했다. 길이·단어수 파생 피쳐는 다음과 같이 4개를 만들어 StandardScaler로 스케일링했다.

```
mercari_df['product_text_len'] = mercari_df['product_text'].str.len()
mercari_df['product_text_word_count'] = mercari_df['product_text'].str.split().str.len()
mercari_df['description_len'] = mercari_df['item_description'].str.len()
mercari_df['description_word_count'] = mercari_df['item_description'].str.split().str.len()
```

최종 hstack 결합은 product_text TF-IDF(200k) → item_description TF-IDF(200k) → 수치형(item_condition_id, shipping) → 길이 파생 피쳐(4개, 스케일링) 순으로 이루어졌다.

5.5 헤헌 — 희소 행렬 효율화와 설명글 포함/제외 4-조합 사전결합

텍스트 벡터화의 비대칭 설계. 헤헌은 name 과 item_description 에 서로 다른 벡터화 기법을 의도적으로 차등 적용했다.

```
cnt_vec = CountVectorizer()
X_name_train = cnt_vec.fit_transform(train_df['name'])
X_name_test = cnt_vec.transform(test_df['name'])

tfidf_descp = TfidfVectorizer(max_features=50000, ngram_range=(1, 2))
X_descp_train = tfidf_descp.fit_transform(train_df['item_description'])
X_descp_test = tfidf_descp.transform(test_df['item_description'])
```

name 은 CountVectorizer() 를 파라미터 제한 없이 그대로 사용해 사실상 무제한 어휘(93,451차원)를 허용한 반면, item_description 만 max_features=50,000 으로 캡을 씌웠다. teamexcelsheet/해현_excel.pdf 의 느낀점란에 이 설계를 “상품명에는 CountVectorizer, 본문 설명글에는 TfidfVectorizer(ngram_range=(1,2))를 데이터 성격에 맞게 차별 적용하여 문맥과 핵심 키워드를 효과적으로 피쳐화했다”고 직접 밝혀, 짧고 반복이 적은 name 은 등장 여부(Count) 자체가 신호이고 긴 item_description 은 상대적 중요도(TF-IDF)가 더 유효하다는 논리로 두 벡터화를 구분했음을 확인할 수 있다.

범주형 인코딩과 결측 처리 순서. LabelBinarizer(sparse_output=True) 를 brand_name → item_condition_id → shipping → cat_dae → cat_jung → cat_so 순서로 컬럼마다 별도 인스턴스를 만들어 각각 train fit_transform / test transform 했다. 원-핫 인코딩 결과를 밀집 행렬이 아닌 희소 행렬로 반환해 148만 행 규모의 데이터에서도 메모리를 절약하기 위함이라고 코드 주석에 명시했다. 결측치는 brand_name → category_name → item_description 순으로 fillna(value='Other_Null') 을 적용했다. 이 중 category_name 은 이 단계 이전에 이미 카테고리 3분할용 split_cat() 의 try/except 에서 ['Other_Null']*3 으로 한 번 처리를 거친 컬럼으로, 여기서의 fillna 는 그 뒤에 이어지는 추가 처리에 해당한다.

파생 피쳐는 없음. 노트북 전체를 재확인했으나 길이.단어수.브랜드 존재 여부 같은 파생 피쳐를 만든 코드는 없다 — 5.1절 표의 “없음” 표기가 정확하다.

피쳐 결합 단계에서는 “설명글 포함 vs 제외” 두 조합을 비교하기 위해, train/test 각각에 대해 4개의 결합 희소 행렬(설명글 포함/제외 × train/test)을 미리 한 번씩만 hstack(순서: description → name → brand → item_condition_id → shipping → cat_dae → cat_jung → cat_so)으로 만들어두고 개별 행렬은 즉시 메모리에서 제거(gc.collect())하는 방식을 취했다. hstack의 기본 반환 형식(COO)을 매번 .tocsr() 로 압축해 이후 모델 학습에서의 인덱싱 연산도 함께 최적화했다. 이는 Ridge.LightGBM-양상불이 모델마다 hstack을 반복 호출하지 않고 동일한 결합 행렬을 재사용하도록 해, 대용량 데이터에서 연산과 메모리를 모두 아끼기 위한 설계라고 밝혔다.

5.6 소현 — 파생 피쳐 및 어휘(vocab) 크기 실험

텍스트 벡터화와 범주형 인코딩(v1→v2 전환). base 버전은 name 에 CountVectorizer(min_df=2, max_features=20000), item_description 에는 이미 토큰 리스트로 정제된 상태를 그대로 받는 analyzer=lambda x: x 를 지정한 TfidfVectorizer(min_df=2, max_features=30000) 를 적용했다. 범주형(brand_name / cat_dae / cat_jung / cat_so)은 컬럼별 OneHotEncoder(handle_unknown='ignore') 를 사용했는데, LabelBinarizer 는 brand_name 처럼 고유값이 4,000개 이상인 컬럼에서 체감상 수십 초 이상 걸릴 만큼 느려 제외했고, 미관측 카테고리를 여러 없이 0벡터로 처리하는 handle_unknown='ignore' 옵션 때문에 OneHotEncoder 를 선택했다고 명시했다.

이후 실험(아래 “버려진 실험” 참고)이 기대만큼 개선되지 않자, 원-핫 대신 정수 라벨로 바꿔 LightGBM에 네이티브 카테고리 넘기는 v2로 전환했다.

```
def ngram_analyzer(tokens, n_range=(1, 2)):
    """ 이미 토큰화된 리스트에서 n_range 범위의 n-gram을 직접 생성
    (analyzer 커스텀이라 ngram_range 파라미터가 적용되지 않아 수동 구현) """
    grams = []
    for n in range(n_range[0], n_range[1] + 1):
        grams += [' '.join(tokens[i:i+n]) for i in range(len(tokens)-n+1)]
    return grams

name_vec_v2 = CountVectorizer(ngram_range=(1, 2), min_df=2, max_features=20000)
desc_vec_v2 = TfidfVectorizer(min_df=2, max_features=30000,
                              analyzer=lambda x: ngram_analyzer(x, (1, 2)))

le = LabelEncoder()
cat_label_df[col] = le.fit_transform(mercari_df[col]) # brand_name/cat_dae/cat_jung/cat_so 각각
X_cat_label = csr_matrix(cat_label_df.values.astype(float)) # 5,804 차원 원-핫 대신 컬럼 4개
```

결측치 처리. 가격 이상치 제거·train_id 삭제·중복 제거 직후, 카테고리 3분할 이전 단계에서 brand_name / category_name / item_description 세 컬럼을 일괄로 fillna('Other_Null') 했다. 다른 대체값과의 정량 비교 실험은 하지 않았고, 'Other_Null' 채택 근거는 “결측 자체를 하나의 카테고리/토큰으로 모델에 전달한다”는 정성적 판단이었다는 점을 그대로 밝힌다.

target encoding의 leakage-free 계산. 새 피쳐 엔지니어링에서 가장 정교한 부분은 brand_te / cat_so_te 를 만드는 방식이다.

```
def target_encode(train_col, train_y, full_col, k=10):
    global_mean = train_y.mean()
    stats = pd.DataFrame({'col': train_col.values, 'y': np.asarray(train_y)}) \
        .groupby('col')['y'].agg(['mean', 'count'])
    smoothed = (stats['count']*stats['mean'] + k*global_mean) / (stats['count']+k)
    return full_col.map(smoothed).fillna(global_mean).values

def build_new_features(train_idx):
    """train_idx 통계로만 brand_te/cat_so_te 계산 후 전체 행에 매핑"""
    y_arr = y_labels.to_numpy(); train_y = y_arr[train_idx]
    brand_te = target_encode(mercari_df['brand_name'].iloc[train_idx], train_y, mercari_df['brand_name'])
    cat_so_te = target_encode(mercari_df['cat_so'].iloc[train_idx], train_y, mercari_df['cat_so'])
    return np.column_stack([base_features, brand_te, cat_so_te])
```

이 함수는 Stage 1(idx_train)과 Stage 2(idx_trainvalid, 9.5절의 2단계 검증 구조)에서 각각 다시 호출되어, 매번 그 시점의 train 통계만으로 target encoding을 재계산한다 — valid/test 정보가 통계 계산에 전혀 섞이지 않도록 하는 설계다. hstack 결합은 Ridge/FM용(원-핫+유니그램 블록 뒤에 스케일링된 신규 피처)과 LGBM용(라벨인코딩+n-gram 블록 뒤에 스케일링 없는 raw 피처)을 각각 별도로 구성했다 — “모델마다 자신에게 유리한 인코딩을 그대로 쓰고, 블렌딩은 예측값 레벨에서만 합친다”는 원칙(9.5절 ①)이 피처 결합 단계부터 이미 반영된 것이다.

메모리 관리는 명시적 최적화 없음. gc.collect() 나 dtype 다운캐스팅 같은 코드는 사용하지 않았다. 다만 FM_FTRL 구현 자체가 미니배치(5만 행) 단위로 $X \cdot \text{dot}() / X \cdot \text{multiply}(X)$ 같은 sparse 연산만 사용해 148만 행 전체를 한 번도 densify하지 않는 구조라, 결과적으로 메모리 사용량을 낮게 유지하는 효과가 있었다(9.5절 참고).

버려진 실험 — “단계적(잔차) LightGBM”. 채택된 v2로 넘어가기 전, 노트북에는 주석 처리된 채로 남아 있는 실패 실험이 하나 있다: 1단계로 category_name (대/중/소분류)·shipping·null_penalty 만으로 LGBM을 학습해 1차 예측을 만들고, 2단계로 그 잔차($y_{\text{train}} - 1\text{차 예측}$)를 타깃 삼아 name / item_description / brand_name / item_condition_id 로 다시 LGBM을 학습한 뒤 두 예측을 더하는 방식이었다. 마크다운에 직접 남긴 기록은 다음과 같다 — “이 섹션은 아래 코드 셀을 주석 처리해 실행되지 않습니다. 실험 결과 RMSLE 0.49735로 원본 Ridge(0.47067)보다도 낮은 성능이라 채택하지 않았습니다. 대신 14번 섹션(LGBM 네이티브 categorical + n-gram)으로 방향을 바꿨습니다.” 사람이 “중요할 것 같은” 피처와 “나머지” 피처를 임의로 나눠 순차 학습시키는 방식이 실패하자, 그 대신 모델에 넘기는 피처 표현 자체를 바꾸는 v2로 전환한 것이 이 절 전체 서사의 실제 전환점이었다.

소현은 5.1의 기본 벡터화-인코딩 위에 추가로 다음 두 갈래 실험도 진행했다.

vocab 크기 민감도 실험. max_features 캡(name 20k/desc 30k)을 없애고 실제 어휘 크기(최대 약 200만 차원)까지 확장해봤지만 RMSLE는 거의 변화가 없었다(0.47320 → 0.47293, 노이즈 수준). 캡을 더 키우는 것은 무의미하다는 결론을 실측으로 확인했다. 이는 진희가 \$5.1과 별개로 진행한 TF-IDF 어휘 크기 실험(100k→150k→200k로 늘릴수록 개선폭이 점점 줄어드는 수확체감 패턴, \$9.3 참고)과도 방향이 일치한다.

새 피처 엔지니어링. 상품명/설명 길이 피처, 브랜드 존재 여부, 그리고 브랜드·소분류별 평균 로그 가격을 학습 데이터에서만 통계 내는 target encoding(스무딩 k=10 적용, leakage 방지)을 추가했다. LGBM은 이 새 피처로 0.47320 → 0.47022로, Ridge보다 약 3배 큰 개선을 얻었다. 비선형 트리 모델이 target encoding이 만드는 신호를 더 잘 활용한 것으로 해석된다. 이 피처들을 반영한 최종 모델 결과는 9장에서 다룬다.

6. 분석 방법 및 모델링

6.1 팀원별 사용 모델 개요

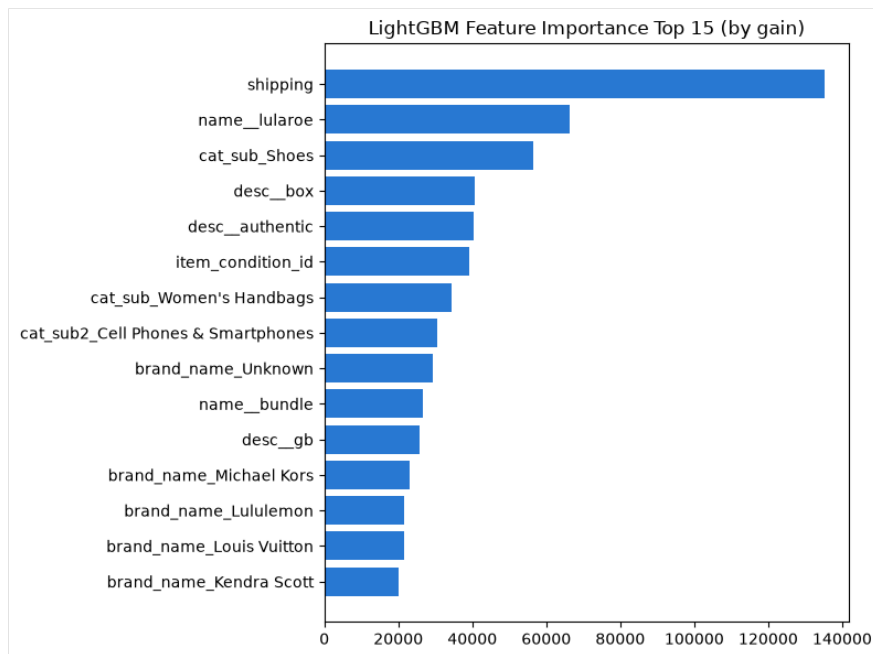
4~5장에서 정리한 각자의 최종 전처리 파이프라인과 피처 표현 위에서, 5명 모두 회귀 타깃($\log_{10}(\text{price})$)에 대해 최소 Ridge 회귀를 베이스라인으로 채택했고, 이후 LightGBM-Lasso-FM_FTRL 등으로 확장했다. 평가지표는 전원 RMSLE(= log 공간에서의 RMSE)를 사용했다. 구체적인 튜닝 범위와 결과 수치는 7장에서 다룬다.

팀원	사용 모델	선택/구현 개요
유경	Ridge, Lasso, LightGBM	세 모델을 동일 조건에서 나란히 비교해 정규화 방식(L1 vs L2)의 차이를 진단
혜현	Ridge, LightGBM, Ridge+LightGBM 가중 앙상블	설명글 포함/제외 비교, LightGBM 3단계 정밀 재튜닝, 최종 가중 앙상블 구성
진희	Ridge, LightGBM, Ridge+LGBM 앙상블	product_text/item_description 통합 피쳐 위에서 alpha 그리드 중심 실험
용우	Ridge, LightGBM	<code>lgb.train</code> + early stopping, 빠른 검증/전체 학습 스위치 구성
소현	Ridge, LightGBM(2가지 피쳐 조합), FM_FTRL(FTRL-proximal + FM 상호작용항을 numpy/scipy로 직접 구현)	Kaggle 공개 커널의 FM_FTRL 아이디어를 <code>wordbatch</code> 설치 불가 환경 제약으로 직접 구현; Ridge-LightGBM-FM_FTRL 3모델 블렌딩까지 확장

6.2 모델 해석 — 피쳐 중요도와 회귀계수

LightGBM 피쳐 중요도(용우). gain 기준 상위 피쳐는 `shipping`, `name_lularoe` (브랜드 토큰), `cat_sub-Shoes`, `desc_authentic / desc_box` (설명 토큰), `item_condition_id` 순으로, 정형 변수(배송비·상품상태)와 텍스트 파생 피쳐가 함께 상위권을 차지한다. 트리 모델은 분기 횟수(gain)를 기준으로 중요도를 매기기 때문에, 값의 종류가 적어(`shipping` 은 0/1) 분기 기준으로 자주 선택되는 정형 변수가 수만 개 텍스트 피쳐 중 개별 단어보다 상위권에 오르기 쉽다는 특성이 반영된 결과이기도 하다.

LightGBM feature importance는 계산 기준에 따라 두 가지로 나뉜다 — `split` (해당 피쳐가 트리 분기 기준으로 선택된 횟수)과 `gain` (그 분기가 손실 함수를 실제로 얼마나 줄였는가)이다. 용우는 gain 기준을 채택했는데, `split` 기준은 단순히 자주 쓰인 피쳐를 상위로 올리는 반면 `gain` 기준은 예측 오차 감소에 실질적으로 기여한 정도를 반영하므로 “얼마나 자주 쓰였는가”보다 “얼마나 결정적이었는가”를 더 잘 드러낸다고 판단했기 때문이다. 상위 15개 피쳐를 다시 살펴보면 `shipping·item_condition_id` 처럼 값의 종류가 2~5개뿐인 정형 변수가 수만 개에 달하는 텍스트·브랜드 피쳐들 사이에서도 꾸준히 상위권에 자리한다는 점이 눈에 띈다. 이는 트리 모델이 분기 하나로 데이터 전체를 절반 가까이 가를 수 있는 저카디널리티 변수를 선호하는 경향과 맞닿아 있으며, 반대로 `name_lularoe` 처럼 특정 브랜드 하나만을 가리키는 개별 텍스트 토큰은 그 브랜드가 등장하는 소수의 행에서만 분기에 기여하므로 gain 총합 자체는 크더라도 데이터 전체에 대한 설명력은 제한적이라는 해석도 가능하다.

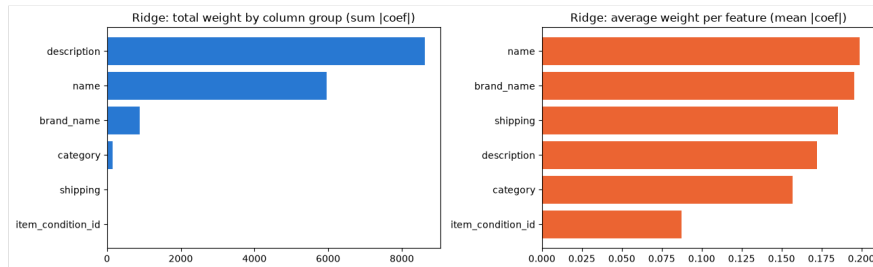


LightGBM Feature Importance Top 15

LightGBM gain 기준 피쳐 중요도 상위 15개(용우).

Ridge 그룹별 가중치(용우). 용우는 Ridge 계수(`coef_`)를 `hstack`에 결합한 순서 그대로 6개 컬럼 그룹 (brand_name/category/item_condition_id/shipping/name/description)으로 나눠, 그룹별 가중치 합($\sum \text{coef}$)과 피쳐 1개당 평균 가중치($\text{mean}(\text{coef})$)를 함께 확인했다.

그룹	피쳐 수	가중치 합 $\sum \text{coef} $	피쳐당 평균 $ \text{coef} $
description	50,000	8,617.9	0.172
name	30,000	5,964.2	0.199
brand_name	4,536	887.1	0.196
category	991	155.7	0.157
shipping	1	0.185	0.185
item_condition_id	1	0.087	0.087



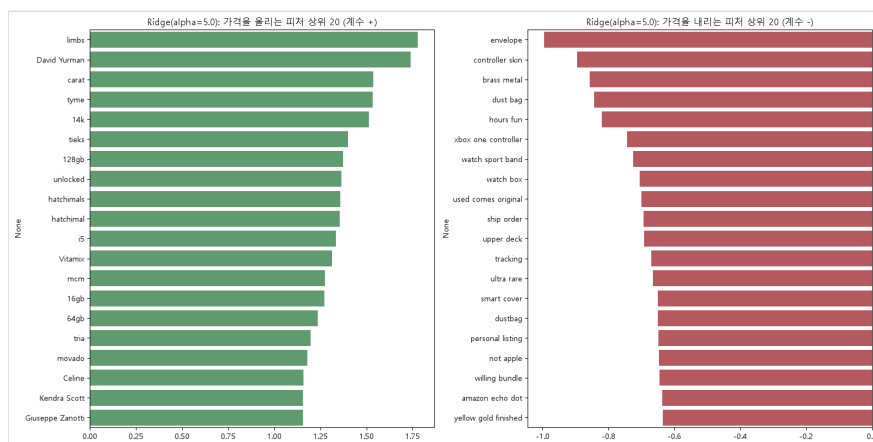
Ridge 그룹별 가중치 막대그래프

6개 컬럼 그룹의 가중치 합($\sum|\text{coef}|$)과 피쳐당 평균 가중치($\text{mean}|\text{coef}|$)를 나란히 비교한 막대그래프(용우). 왼쪽 축(합계)에서는 description/name이 압도하지만, 오른쪽 축(평균)으로 정규화하면 그룹 간 격차가 크게 줄어드는 것이 한눈에 드러난다.

가중치 합만 보면 피쳐 수가 압도적으로 많은 description·name이 크게 앞서지만, 피쳐 1개당 평균 가중치로 다시 보면

description (0.172)·name (0.199)·brand_name (0.196)이 비슷한 수준이고, 단일 피쳐인 shipping (0.185)도 이들과 맞먹는 평균 가중치를 갖는다. 반면 item_condition_id (0.087)만 뚜렷하게 낮는데, 이는 3.4절 상관계수 분석에서 item_condition_id와 $\log_{10}(\text{price})$ 의 상관계수가 -0.00으로 6개 수치형 피쳐 중 가장 낮았던 것과 정확히 일치하는 결과다 — 서로 다른 두 분석(상관계수, Ridge 가중치)이 같은 결론(상품 상태 등급 하나만으로는 가격을 거의 설명하지 못한다)에 독립적으로 도달한 셈이다.

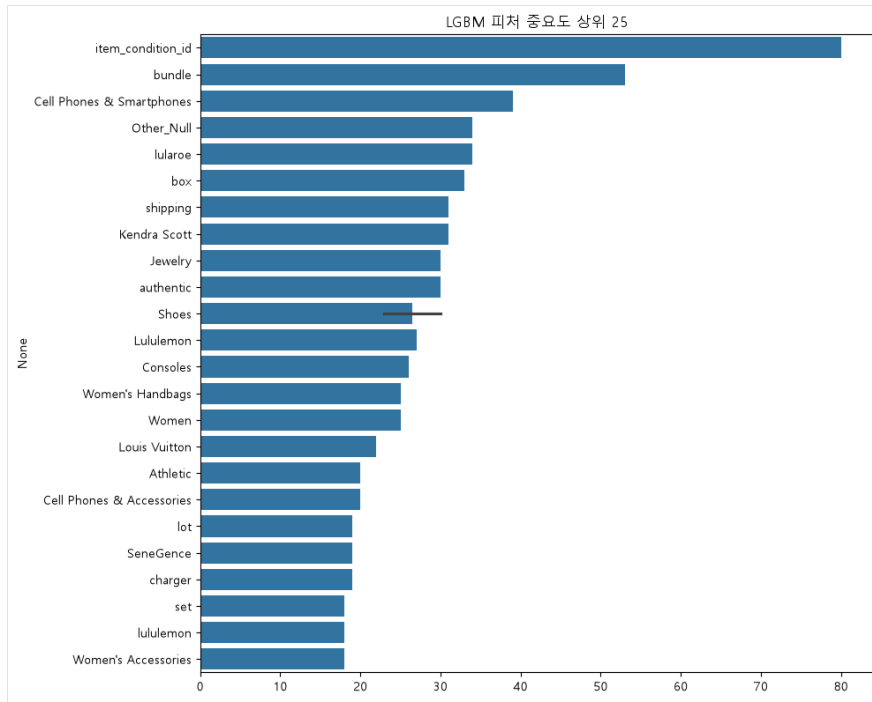
Ridge 회귀계수(유경). $\alpha=5.0$ 기준으로 가격을 가장 크게 올리는 피쳐는 limbs, David Yurman, carat, tyme, 14k 처럼고가 주얼리·브랜드·저장용량 관련 토큰이며, 가격을 가장 크게 낮추는 피쳐는 envelope, controller skin, brass metal, dust bag 처럼 저가 액세서리·소모품 관련 토큰이다. 이는 Ridge가 학습한 가중치가 실제 상품 카테고리의 가격 통념과 부합함을 보여준다. 14k (순금 함량 표기)처럼 숫자+단위가 결합된 토큰이 상위권에 오른 것은, 4.2절에서 확인한 “숫자 보존이 근소하게 유리했다”는 A/B 비교 결과의 구체적인 근거이기도 하다.



Ridge 상위 회귀계수

$\alpha=5.0$ Ridge 모델의 상위/하위 회귀계수(유경).

LightGBM 피쳐 중요도(유경). item_condition_id, bundle (묶음판매 토큰), Cell Phones & Smartphones, Other_Null (브랜드 결측), lularoe 순으로 중요도가 높게 나타나 용우의 결과와 상당 부분 겹친다 — 서로 다른 벡터화인코딩을 썼음에도 shipping / item_condition_id 같은 정형 변수와 특정 고빈도 브랜드·카테고리가 공통적으로 중요 피쳐로 확인된다는 점은 두 팀원의 결과가 교차 검증된 것으로 볼 수 있다. Other_Null (브랜드 결측을 나타내는 토큰)이 상위권에 오른 것도 주목할 만한데, 이는 3.2절에서 확인한 “결측 개수가 많을수록 평균 가격이 낮다”는 관찰(소현의 null_penalty 분석)이 유경의 LightGBM에서도 독립적으로 재확인된 것이다.



LGBM 피쳐 중요도 상위 25

LightGBM 피쳐 중요도 상위 25개(유경).

Ridge 회귀계수(진희) — 유경과의 교차검증. 진희는 `product_text` (name+brand_name+category_name 통합)·`item_description` 을 TF-IDF로 벡터화한 뒤, Ridge 학습이 끝난 계수를 컬럼명과 함께 데이터프레임으로 만들어 직접 정렬해봤다. 코드와 실제 상위 결과는 다음과 같다.

```
coef_df = pd.DataFrame({
    'feature': feature_names,
    'coef': ridge.coef_
})

# 가격을 가장 크게 낮추는 피쳐 20개
negative_features = coef_df.sort_values('coef', ascending=True).head(20)

# 가격을 가장 크게 올리는 피쳐 20개
positive_features = coef_df.sort_values('coef', ascending=False).head(20)
```

가격을 가장 크게 올리는 상위 피쳐는 `desc_14k` (2.45), `desc_10k` (1.91), `desc_128gb` (1.85), `desc_unlocked` (1.80), `desc_limbs` (1.80), `desc_64gb` (1.63), `desc_16gb` (1.61), `desc_32gb` (1.56)처럼 **금 순도(14k/10k)나 저장 용량(128gb/64gb/32gb/16gb) 같은 숫자+단위 조합 토큰이 대부분을 차지했다.** 가격을 가장 크게 낮추는 피쳐는 `product_dust bag` (-2.07), `product_charger` (-1.44), `product_sticker` (-1.44), `desc_envelope` (-1.20), `desc_controller skin` (-0.94)처럼 저가 액세서리·포장재 관련 토큰이었다.

이 결과를 6.2절 앞부분의 유경 결과와 나란히 놓으면 두 가지가 겹친다. 첫째, `limbs` 가 진희·유경 두 사람 모두의 Ridge 상위 양(+) 계수에 독립적으로 등장했다 — 서로 다른 노트북, 다른 피쳐 구성(진희는 `product_text` 통합, 유경은 원본 컬럼 분리)에서도 같은 토큰이 최상위로 뽑힌 것이다. 둘째, `envelope / controller skin` 계열의 저가 소모품 토큰도 두 사람의 하위(-) 계수 목록에 함께 등장했다. 셋째, 진희의 결과에서 `14k / 10k / 128gb` 같은 숫자+단위 토큰이 대거 상위권을 차지한 것은, 4.2절(3)에서 유경·용우가 A/B 테스트로 확인한 “숫자 보존이 근소하게 유리하다”는 결론에 **구체적으로 어떤 숫자가 가격 신호로 작동하는지**를 보여주는 직접적인 사례다.

용우·유경·진희 세 사람의 해석을 종합하면, `shipping / item_condition_id` 같은 소수의 정형 변수, `lularoe / limbs` 같은 특정 고빈도 브랜드·카테고리 토큰, 그리고 `14k / 128gb` 같은 숫자+단위 조합 토큰이 서로 다른 벡터화·인코딩 방식(용우: TF-IDF+원·핫, 유경:

CountVec/TfidfVec+LabelBinarizer, 진희: `product_text` 통합 TF-IDF)에서도 공통으로 상위권에 등장한다. 이는 세 사람이 독립적으로 구성한 피쳐셋임에도 모델이 실제로 의존하는 신호의 종류는 비슷하다는 것을 보여주며, 5장(Feature Engineering)에서 확인한 “텍스트 피쳐가 전체 차원의 90% 이상”이라는 구조적 특징과도 일관된다. 또한 용우의 Ridge 그룹별 가중치 분석과 3.4절의 상관계수 분석이 `item_condition_id` 에 대해 같은 결론(설명력이 거의 없음)에 독립적으로 도달했다는 점은, 서로 다른 팀원이 서로 다른 방법으로 검증했음에도 일관된 결과가 나왔다는 점에서 이 관찰의 신뢰도를 높여준다.

7. 하이퍼파라미터 최적화

7.1 Ridge alpha 튜닝

모든 팀원이 alpha를 바꿔가며 최적값을 탐색했다. 최적 alpha는 팀원마다 1~5 범위에서 같았는데(소현 3, 용우 3, 유경 5, 진희 1, 혜현 3~4), 이는 사용한 피쳐 개수·벡터화 방식(특히 `max_features`, `ngram` 범위)이 서로 달라 정규화에 필요한 강도도 함께 달라졌기 때문으로 해석된다.

소현은 다른 팀원들이 쓴 1~5 범위의 촘촘한 그리드 대신, `[0.01, 1000]`을 아우르는 훨씬 넓은 로그 스케일 그리드를 탐색했다. 의도는 "혹시 1~5보다 훨씬 작거나 훨씬 큰 alpha에서 더 나은 지점이 있는 건 아닌지"를 먼저 넓게 확인한 뒤 필요하면 좁혀가는 것이었다.

그리드를 선형이 아니라 로그 스케일(`0.01, 0.03, 0.1, 0.3, 1, 3, 10, 30, 100, 300, 1000`)로 구성한 이유도 명확하다 — Ridge의 정규화 항은 alpha가 커질수록 계수를 억제하는 효과가 급셈적으로 커지므로, 1, 2, 3, 4, 5 처럼 선형 간격으로는 자릿수가 다른 구간(예: 10 미만 vs 100 이상)의 효과 차이를 촘촘하게 비교할 수 없다. 자릿수마다 3개 지점(`x1`, `x3`, `x10`)을 고르게 배치하는 로그 그리드를 쓰면, 적은 개수(11개)의 실험만으로도 `0.01`부터 `1000`까지 5자릿수 전체 구간을 빠짐없이 훑을 수 있다는 것이 이 설계의 핵심이다.

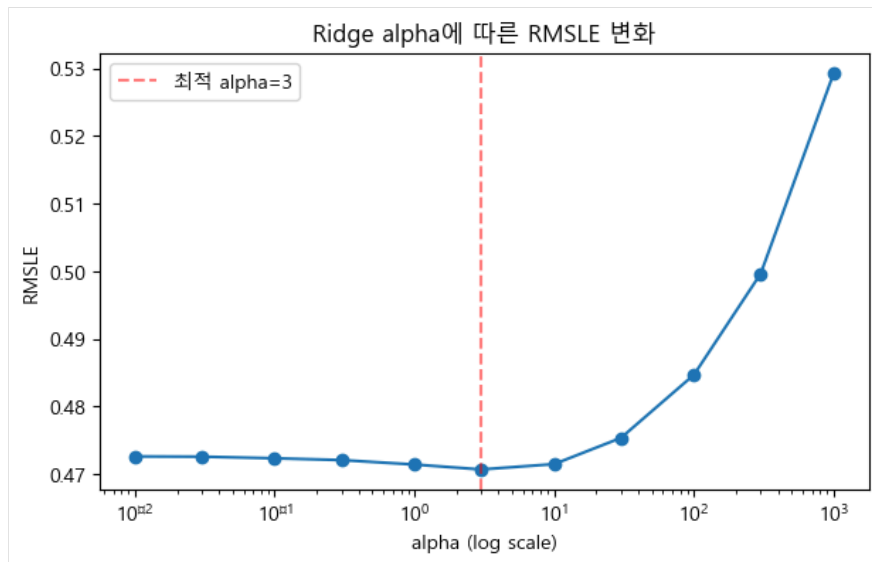
```
# alpha 그리드 탐색 (로그 스케일로 넓게: 과소규제~과대규제 양쪽 극단을 모두 포함)
alphas = [0.01, 0.03, 0.1, 0.3, 1, 3, 10, 30, 100, 300, 1000]
alpha_results = []

for a in alphas:
    ridge_a = Ridge(solver='lsqr', fit_intercept=False, alpha=a)
    ridge_a.fit(X_train_a, y_train_a)
    preds_a = ridge_a.predict(X_test_a)
    score = rmsle(y_test_a, preds_a)
    alpha_results.append((a, score))
    print(f'alpha={a:>7}: RMSLE={score:.5f}')

# 최적 alpha 확인 + 시각화
best_alpha, best_rmsle = min(alpha_results, key=lambda x: x[1])
```

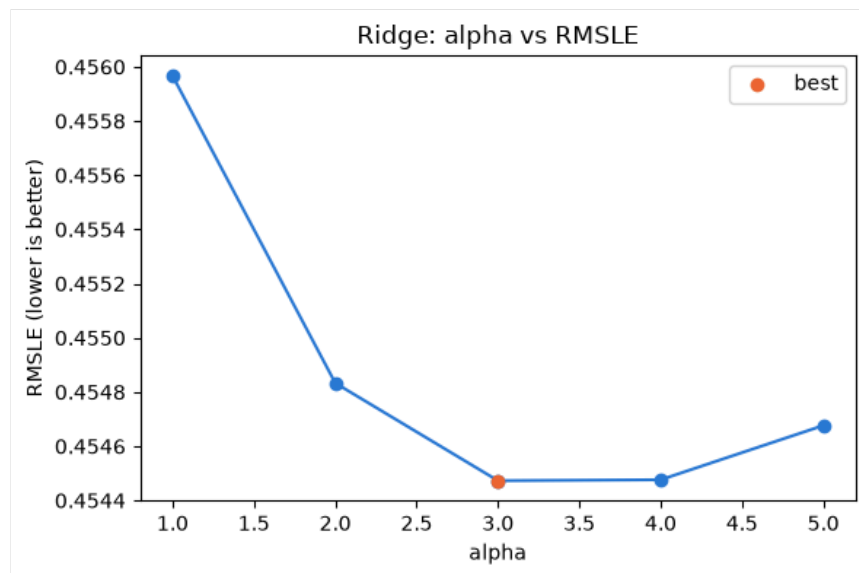
11개 값을 전부 실측한 결과, 극단값(0.01·1000) 모두 alpha=3보다 뚜렷이 나뉘었다 — 특히 alpha=1000에서는 RMSLE가 0.52934까지 치솟아, 1~5 범위 밖을 볼 필요가 없다는 것을 직접 확인했다(아래 그림의 U자형 곡선 참고). 이는 "다른 팀원들이 1~5 범위만 봐도 충분했다"는 것을 별도로 검증해 준 셈이기도 하다 — 좁은 범위만 탐색했다는 것이 탐색 부족이 아니라, 실제로 그 범위 밖에 더 나은 지점이 없었다는 것을 소현의 결과가 사후적으로 뒷받침한다.

팀원	최적 alpha	해당 alpha의 RMSLE	비교 alpha 구간에서의 RMSLE 변화
혜현	3~4	0.4593(설명 포함)	0.1(0.4685) → 3(0.4593) → 4(0.4591, 근소 최저) → 5(0.4591)
진희	2(스윙 기준)/1(최종 채택)	0.4343(스윙 시점 validation)	0.1(0.4558) → 0.5(0.4402) → 1(0.4358) → 2(0.4343, 스윙 최저) → 3(0.4351) → 5(0.4377)
유경	5	0.4560	1(0.4576) → 5(0.4560, 5개 중 최저, 완만한 개선)
소현	3	0.47067	alpha 0.01→1000 구간에서 0.47256(0.01) → 0.47067(3, 최저) → 0.52934(1000)로 U자형
용우	3	0.45447	1(0.45597) → 3(0.45447, 최저) → 5(0.45468)



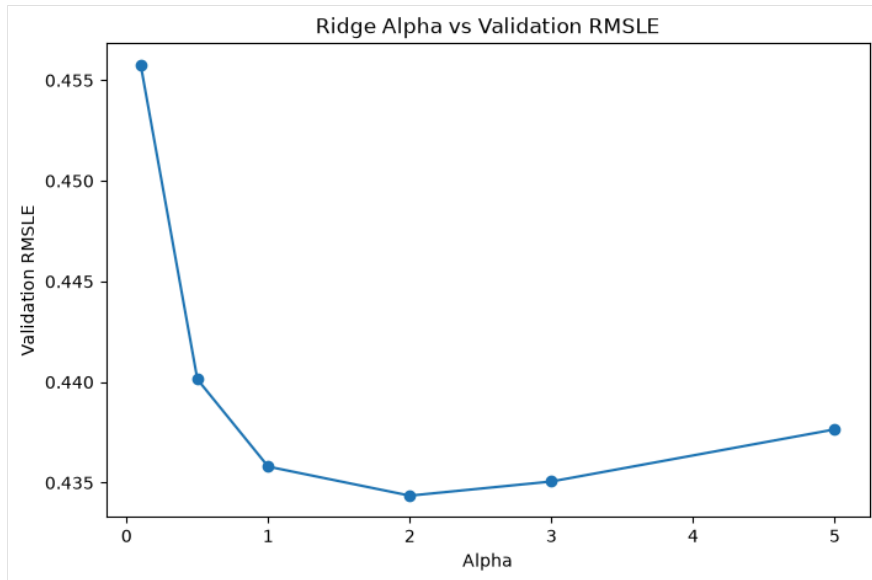
Ridge alpha 스윙(소현)

소현: Ridge alpha에 따른 RMSLE. alpha가 너무 작으면 과소 규제, 너무 크면(alpha=1000일 때 0.52934) 과도 규제, 텍스트 피쳐 계수가 눌러 성능이 나빠지는 U자형 곡선.



Ridge alpha 스윙(용우)

용우: Ridge alpha 스윙 결과, $\alpha = 3$ 에서 최적점(RMSLE 0.45447).



Ridge alpha 스윙(진희)

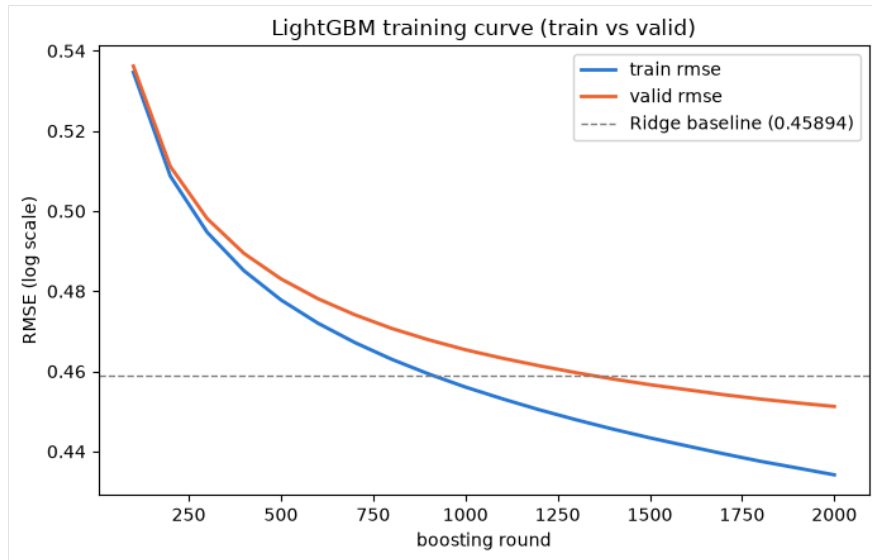
진희: alpha 0.1~5 구간의 validation RMSLE 곡선. alpha=2 부근에서 최저점(0.4343)을 찍고 이후 다시 완만히 상승하는 얇은 U자형으로, 소현·용우의 곡선보다 계곡이 훨씬 완만하다 — TF-IDF max_features 를 20만까지 넓게 잡은 만큼 alpha 변화에 대한 민감도 자체가 낮아진 것으로 해석된다. 이 스윙 이후 길이·단어수 파생 피처를 추가하고 StandardScaler로 스케일을 맞춘 최종 파이프라인에서는 alpha=1이 근소하게 더 낮게 나와(노트북 파일명 ridgefixfeature(0.4331)), 파생 피처 추가가 최적 alpha를 미세하게 이동시켰음을 보여준다.

유경의 alpha별 성능 곡선(모델 간 비교 포함)은 9.2절 그림을 함께 참고.

7.2 LightGBM 튜닝

라운드 수·정규화 강도에 따라 팀원 간 편차가 크게 나타났다. 소현·혜현처럼 early stopping과 라운드 수를 여러 단계로 늘려간 경우 RMSLE가 뚜렷하게 개선된 반면, 용우처럼 빠른 검증 모드(150라운드)로 실행한 경우 오히려 Ridge보다 낮은 성능을 보였다.

팀원	설정	RMSLE	비고
혜현	200라운드 기본값	0.4507	
용우	150라운드(FAST_LGBM_CHECK=True, early stop 미도달)	0.52045	Ridge(0.45597)보다 14.1% 낮은 성능 — 라운드 부족이 원인으로 추정
소현	OneHot+ 유니그램(v1), 200라운드	0.49220	Ridge(0.47067)보다 낮은 성능
유경	200라운드 기본값	0.5104	Ridge(0.4560)-LGBM 모두 대비 성능 낮음(튜닝 미실시)
혜현	1차 튜닝, 1000라운드, early stop 30	0.4422	
소현	LabelEncoder native categorical + n-gram(v2), 200라운드	0.47363	피처 표현 방식만 바뀌어 Ridge 근접
용우	(참고) 과거 2000라운드 전체 실행 기록	valid RMSE 0.45122까지 하락	Ridge baseline(0.45892)보다 근소 우위, 학습곡선 이미지로 별도 보존
혜현	2차 튜닝, 최대 3000라운드, lr 0.05, leaves 127	진행 로그상 2300라운드 시점 valid RMSE 0.4381	로그 마지막 구간까지 개선 지속 중
소현	신규 피처 추가 + 튜닝, 1200라운드 (early stop)	0.42881	
혜현	3차 튜닝, 최대 4000라운드, lr 0.03, leaves 255 + 정규화 (reg_alpha/lambda)	최종 0.42909	4명의 LGBM 단일 모델 중 최저 RMSLE 기록
소현	동일 설정, 2000라운드(체크포인트 이터 학습)	0.42388	1200라운드 대비 추가 개선



LightGBM 학습 곡선(용우)

과거 2000라운드 전체 실행 기록. train/valid RMSE가 완만하게 계속 하락하며 약 1300라운드 부근에서 Ridge baseline(0.45892)과 교차한다 — LightGBM이 라운드 수가 충분히 확보돼야 선형 모델을 능가할 수 있음을 보여준다.

7.1(Ridge alpha)과 7.2(LightGBM)를 함께 보면, alpha 튜닝의 개선폭은 대체로 0.2~2.0%(혜현 예외적으로 -2.0%)에 그친 반면 LightGBM 라운드·정규화 튜닝은 -1.2%(소현)~-4.8%(혜현)로 더 큰 폭의 개선을 보였다 — 정규화 강도 하나를 조정하는 것보다 학습 예산(라운드 수) 자체를 늘리는 쪽이 이 데이터에서는 더 유효했다는 뜻이다(8.4절 표에서 전체 개선폭을 다시 비교한다). 소현·혜현의 단계별 튜닝 서사와 FM_FTRL 학습 곡선은 9장(팀원별 실험 상세)에서 상세히 다룬다.

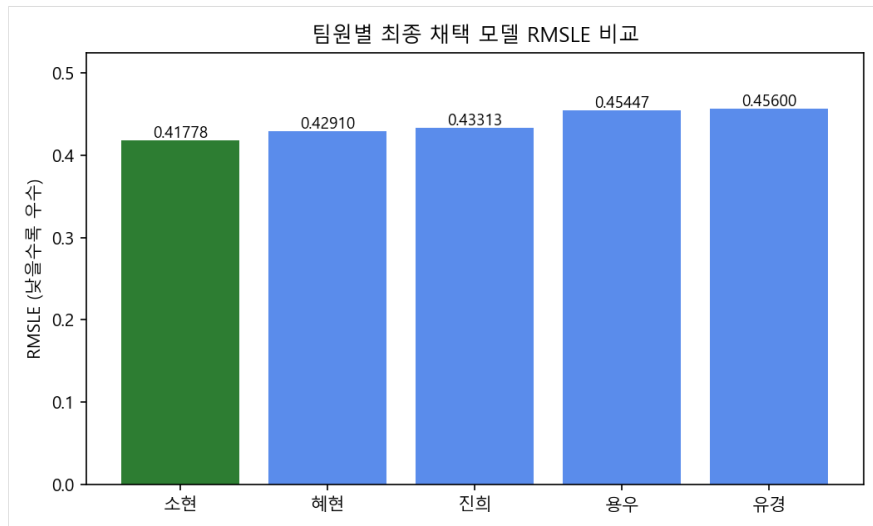
8. 모델 성능 비교 및 분석

이 장은 팀원 개인의 실험 서사가 아니라 **팀 전체를 가로지르는 비교와 분석**을 다룬다. 팀원별 세부 구현 과정은 9장(팀원별 실험 상세)에서 별도로 다룬다.

8.1 팀 전체 최종 성능 종합 비교

다섯 명 모두 동일한 문제(로그 변환된 `price`를 RMSLE로 평가, **낮을수록 우수**)를 풀었지만, 전처리·피처 구성·모델 선택 조합을 각자 다르게 가져갔고 그 결과 최종 RMSLE에도 뚜렷한 차이가 났다. 아래 표는 각자가 도달한 최종(채택) 모델과, 각자의 파이프라인 내부에서 측정한 베이스라인 대비 개선율을 정리한 것이다.

순위	담당자	최종 모델/양상물	최종 RMSLE	주요 피쳐 표현	핵심 튜닝/개선	자체 baseline 대비 개선율	비교 시 주의사항
1	소현	Ridge+LightGBM(2000라운드)+FM_FTRL(60epoch)가중 블렌드, leakage-free 2-stage 검증	0.41778	LGBM: 라벨인코딩+n-gram(v2) / Ridge-FM_FTRL: 원-핫+유니그램	신규 피쳐(target encoding 등)+FM_FTRL 직접구현+2단계 블렌딩	LGBM v1(0.49220) 대비 15.1%	5단계 순차 개선의 누적치(8.4절 참고)
2	혜현	LightGBM 단일(3차 정밀 튜닝)	0.4291	CountVec(name, 무제한)+TfidfVec(desc, ngram1-2)+LabelBinarizer	lr 0.03, num_leaves 255, L1/L2 정규화, 3단계 재튜닝	초기 Ridge baseline(0.4932) 대비 13.0%	Ridge+LGBM 양상물(0.4401)보다도 단일모델이 우수
3	진희	Ridge(alpha=1.0)+통합 product_text	0.43313	product_text/item_description 통합 TF-IDF(ngram1-2, 200k)	max_features 100k→200k, 길이-단어수 피쳐	TF-IDF 100k(0.438358) 대비 1.2%	카테고리 별도 인코딩 없이 텍스트 통합만으로 도달
4	유경	Ridge(alpha=5)	0.4560	LabelBinarizer(카테고리)+CountVec/TfidfVec(ngram1-3)	alpha 1→5 그리드	alpha=1(0.4576) 대비 0.35%	Lasso(0.6082)-LGBM 튜닝전(0.5104) 대비는 확실히 우수
5	용우	Ridge(alpha=3, 숫자보존)	0.45447	TfidfVec(ngram1-2)+OneHotEncoder	숫자 보존 정제 +alpha 1→3	숫자제거 +alpha1(0.45892) 대비 0.97%	LightGBM은 150라운드만 실행, 미완성 비교



팀원별 최종 RMSLE 비교

팀원별 최종 채택 모델의 RMSLE 비교(오름차순 = 성능 좋은 순, 낮을수록 우수). 소현이 5명 중 최저(최고 성능) RMSLE를 기록했다.

다섯 모델 모두 동일한 Mercari 데이터셋·동일한 RMSLE 지표를 사용했지만 정확한 train/test 분할 방식, 피쳐 캡(vocab 크기), 텍스트 정제 규칙이 서로 달라 **절대 수치를 팀원 간 1:1로 비교하는 것은 완전히 공정한 비교가 아니다**. “개선율” 얻은 각 팀원 자신의 파이프라인 내부에서 측정한 전/후 비교이며, 팀원 간 개선율끼리도 서로 다른 출발점·조건에서 계산된 것이라 직접 비교 대상이 아니다.

8.2 최고 성능 모델 분석

소현의 조합(LightGBM 2000라운드 + FM_FTRL 60epoch 블렌드)이 팀 내 최고 성능(0.41778)에 도달한 원인은 단일 요인이 아니라 다음 네 가지가 적된 결과로 볼 수 있다.

① **모델별로 분리된 피쳐 표현**. 하나의 공통 피쳐셋에 모든 모델을 억지로 맞추지 않고, LGBM은 라벨인코딩+n-gram, Ridge/FM_FTRL은 원-핫+유니그램을 각자에게 유리한 형태 그대로 사용했다(9.5절).

② **단일 모델 대비 블렌딩의 실제 개선폭**. 튜닝된 LGBM 단독(0.42388) → 블렌드(0.41778)로 약 1.4%(절대 0.0061) 추가 개선됐다. 이는 혜현의 사례(이미 잘 튜닝된 LGBM 단일모델 0.4291이 그보다 앞서 만든 Ridge+LGBM 양상물 0.4401보다 우수)와 대조적이다 — 혜현의 경우는 “튜닝이 덜 된 상태의 양상물은 역효과”를, 소현의 경우는 “개별 모델을 최대로 끌어올린 뒤의 이중 모델 블렌딩은 추가 효과”를 보여줘, 두 사례가 상호 보완적인 시사점을 제공한다(8.3절에서 더 다룬다).

③ **Ridge 가중치가 0으로 수렴한 의미.** valid에서 찾은 최적 블랜드 가중치(ridge=0, lgbm=0.65, fm=0.35)에서 Ridge의 가중치가 0이 됐다는 것은, 초기 시점(alpha 튜닝 단계, 7.1절)에는 Ridge가 LGBM보다 우수했지만 LGBM을 제대로 튜닝한 뒤로는 Ridge가 더 이상 새로운 정보를 보태지 못했다는 뜻이다 — Ridge와 (튜닝된) LGBM+FM_FTRL이 학습하는 신호가 상당 부분 겹친다는 의미로 해석할 수 있다. 반대로 LGBM(트리 기반 비선형 상호작용)과 FM_FTRL(선형+2차 상호작용을 명시적으로 모델링)은 65:35로 둘 다 유의미한 가중치를 유지했다. 다만 두 모델의 오차 상관관계를 직접 측정할 기록은 없어, “서로 다른 오차를 보완했다”는 것은 가중치 배분에서 유추한 해석이지 직접 검증된 사실은 아니라는 점을 밝혀둔다.

④ **leakage-free 2-stage 검증의 의미.** Stage 1(train→valid로 튜닝) → Stage 2(train+valid→test 단 한 번 평가) 구조는, RMSLE 0.41778이 하이퍼파라미터-블랜드 가중치를 test에 맞춰 고른 결과가 아니라는 신뢰성을 보장한다. 이는 나머지 4명이 공통으로 지킨 “train에만 fit” 원칙(4.1절)보다 한 단계 더 엄격한 절차다.

다른 팀원 최종 결과와의 실제 격차는, 2위 혜현(0.4291) 대비 약 2.6%(절대 0.0113), 5위 용우(0.45447) 대비 약 8.1% 낮다(8.1절 표 참고).

8.3 모델 계열별 성능 분석

팀원 순서가 아니라 모델 계열을 가로질러 정리하면 다음과 같다.

Ridge — 왜 여러 팀원에게 안정적인 베이스라인이었는가. 5명 모두 Ridge를 최초 베이스라인으로 채택했고 RMSLE는 0.43~0.47 구간에 몰려 있었다(8.1절 표). 6.1-7.1절에서 확인했듯 8만~15만 차원의 고차원 희소 텍스트-원-핫 피쳐 위에서, L2 정규화가 다수의 약한 신호(단어-브랜드-카테고리 더미)에 계수를 분산시켜 과적합 없이 안정적으로 수렴하기 때문으로 해석된다.

Lasso — 왜 유경의 실험에서 크게 과소적합했는가. 유경의 Lasso(alpha=0.001)는 RMSLE 0.6082로 Ridge(0.4560)·LGBM(0.5104) 모두보다 낮은 성능을 보였다. 유경은 이 결과를 그냥 넘기지 않고, 표본 25만 행(train 20만/test 5만)으로 재현한 축소 실험을 통해 원인을 다섯 갈래로 나눠 직접 진단했다.

원인	진단 내용
① L1의 구조적 특성	가격 신호가 <code>new / nwt / 브랜드명/n-gram</code> 처럼 수만 개의 “약한 신호”에 분산되어 있는데, Lasso(L1)는 83,486개 피쳐 중 152개(0.2%)만 남기고 나머지를 정확히 0으로 만들어 유효 신호 대부분을 함께 버렸다. Ridge(L2)는 계수를 0으로 만들지 않고 축소만 하므로 이 구조와 더 잘 맞는다.
② alpha 스케일 불일치	Ridge 목적함수는 $\sum w^2$, Lasso 목적함수는 $\sum w $ 를 정규화항으로 쓰므로 같은 alpha 값이라도 규제 강도가 전혀 다르다. <code>alpha=0.001</code> 이 L1 기준으로는 8만여 개의 약한 상관 피쳐 대부분을 0으로 밀어버릴 만큼 강했다.
③ 수렴 실패 여부(직접 확인)	<code>n_iter=788 < max_iter=2000</code> 이고 <code>ConvergenceWarning</code> 도 없었으며, <code>max_iter=10000</code> 으로 올려도 RMSLE 0.6131-계수 152개로 결과가 완전히 동일했다 — 즉 “덜 학습돼서”가 아니라 L1 해 자체가 그렇게 수렴한 것이다.
④ n-gram 다중공선성	“new” / “brand new” / “new with tags” / “nwt”처럼 서로 강하게 상관된 n-gram 피쳐가 많은데, L1은 상관된 무리에서 하나만 남기고 나머지를 0으로 만들며 그 선택도 표본에 따라 흔들린다. L2는 무리 전체에 가중치를 나눠 분산이 작고 일반화가 낮다.
⑤ LGBM(0.5104)과의 비교	LGBM은 <code>alpha=0.001</code> Lasso처럼 어휘를 통째로 버리지 않고(로그상 81,202개 피쳐 사용) 필요한 만큼만 분기에 사용하므로 Lasso보다는 낫지만, 튜닝 없는 기본 파라미터로는 Ridge를 넘지 못해 Ridge와 Lasso 사이에 위치했다.

축소 표본에서 Lasso(alpha=0.001)의 RMSLE(0.6131)는 Ridge(0.4950)보다 “train 평균만 예측하는 기준선”(0.7514)에 더 가까웠다 — 유경은 이를 과대적합이 아니라 **심한 과소적합**의 증거로 해석했고, “Lasso를 쓰려면 alpha를 훨씬 낮추거나 ElasticNet을 쓰는 편이 낫지만, 이 데이터에서는 Ridge가 가장 자연스럽고 단순한 좋은 기준선”이라는 결론을 내렸다.

학습 시간 측면에서도 세 모델의 격차는 뚜렷했다. 같은 조건(148만 행 전체, 85,521차원)에서 실측한 학습+예측 소요 시간은 다음과 같다.

모델	RMSLE	학습+예측 소요시간
Ridge(alpha=1.0)	0.4576	94.7초
Ridge(alpha=2.0)	0.4567	74.6초
Ridge(alpha=3.0)	0.4563	64.0초
Ridge(alpha=4.0)	0.4561	57.6초
Ridge(alpha=5.0, 최적)	0.4560	54.1초
Lasso(alpha=0.001)	0.6082	528.6초

alpha가 커질수록 Ridge의 계산 비용이 오히려 줄어드는 경향(94.7초→54.1초)이 관찰되는데, 이는 규제가 강해질수록 `lsqr` 솔버의 반복 수렴이 더 빨리 끝나기 때문으로 보인다. Lasso는 좌표하강(coordinate descent) 방식이 8만여 개 피쳐를 훑는 비용 자체가 커 Ridge 대비 5~10배의 시간이 들었는데도 성능은 오히려 가장 나빴다는 점에서, 이 데이터셋에서는 Lasso가 “시간 대비 성능” 양쪽 모두에서 불리한 선택이었다.

LightGBM — 언제 Ridge를 앞질렀고 언제 뒤쳐졌는가. 7.2절 표를 계열 기준으로 다시 보면, 라운드 수와 정규화를 충분히 준 경우(소현 1200~2000라운드, 혜현 3단계 4000라운드)에는 각각 0.42388, 0.4291까지 내려가 Ridge를 앞질렀다. 반면 라운드가 부족하거나(용우 150라운드, 0.52045) 튜닝을 하지 않은 경우(유경 200라운드 기본값, 0.5104) 모두 Ridge보다 낮은 성능에 그쳤다. 즉 LightGBM 자체의 우열이 아니라 **학습 예산과 정규화를 얼마나 투입했는가**가 Ridge 대비 승패를 갈랐다.

앙상블이 항상 단일 모델보다 좋지는 않았던 이유. 혜현의 Ridge+LightGBM 가중 앙상블(0.4401)은 이후 LightGBM을 3차까지 정밀 튜닝한 단일 모델(0.4291)보다 오히려 못했다(9.4절). 앙상블 시점에 결합된 두 모델(Ridge 0.4593, LGBM 튜닝 전 0.4507) 자체가 아직 최선이 아니었기 때문으로, “덜 튜닝된 두 모델을 섞는 것”이 “하나를 끝까지 튜닝하는 것”보다 못할 수 있음을 보여준다.

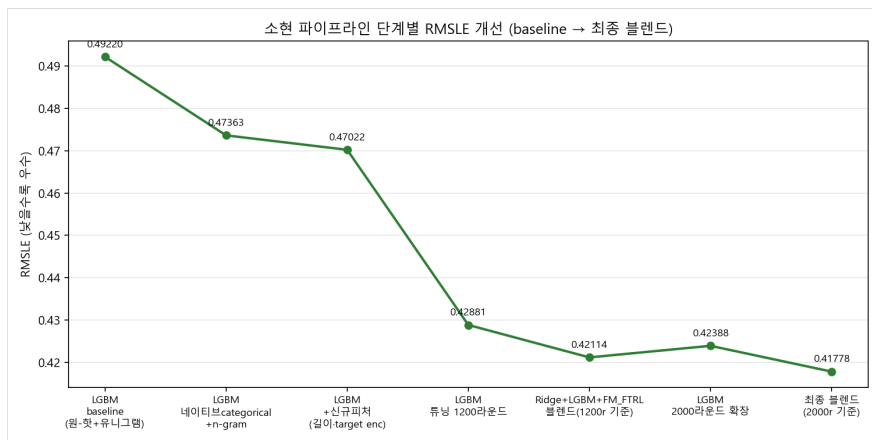
최종 블렌드가 추가로 개선한 근거. 소현의 경우는 다르다 — LightGBM을 이미 충분히 튜닝(0.42388)한 뒤에, 서로 다른 알고리즘 계열(FM_FTRL)을 추가로 블렌딩해 0.41778까지 개선했다(8.2절). 혜현의 사례와 함께 보면, “**튜닝이 덜 된 상태의 앙상블은 역효과를 낼 수 있지만, 개별 모델을 최대한 튜닝한 뒤의 이중 모델 블렌딩은 추가 개선 여지가 있다**”는 것이 두 사례에서 공통으로 확인된다.

8.4 최적화 및 Feature Engineering 효과 분석

각 팀원 파이프라인 내부에서 측정된 주요 개선 단계를 “변경 전 → 후 → 절대 개선폭 → 상대 개선율” 형태로 정리했다. **모든 수치는 각 팀원의 동일 조건 내부 전/후 비교이며, 팀원 간 절대 수치를 직접 비교하는 것이 아니다.** 개선폭의 크기만 놓고 보면 피쳐 표현 자체를 바꾼 단계(hstack 구성 교체, LightGBM 라운드+정규화 동시 확장)가 -3.8%~-4.8%로 가장 크고, 이미 어느 정도 튜닝된 상태에서 파라미터 하나만 미세 조정만 단계(alpha 튜닝, vocab 캡 제거)는 -0.06%~-2.0%로 상대적으로 작다. 이는 “무엇을 바꾸느냐”가 “얼마나 정교하게 튜닝하느냐”보다 개선폭에 더 크게 기여했다는 것을 시사한다.

개선 후(변경 후) RMSLE가 큰 값부터 작은 값 순으로 정렬했다.

항목	담당자	변경 전 → 후 (RMSLE)	절대 개선폭	상대 개선율
숫자 토큰 보존	유경	0.48787 → 0.48463	-0.00324	-0.7%
hstack 구성 교체	소현	0.49220 → 0.47363	-0.01857	-3.8%
vocab 캡 제거(참고)	소현	0.47320 → 0.47293	-0.00027	-0.06%(노이즈)
Ridge alpha 튜닝	소현	0.47140 → 0.47067	-0.00073	-0.2%
신규 피쳐(길이-TE)	소현	0.47320 → 0.47022	-0.00298	-0.6%
Ridge alpha 튜닝	혜현	0.4685 → 0.4593	-0.0092	-2.0%
Ridge alpha 튜닝	유경	0.4576 → 0.4560	-0.0016	-0.3%
숫자 토큰 보존	용우	0.45892 → 0.45597	-0.00295	-0.6%
Ridge alpha 튜닝	용우	0.45597 → 0.45447	-0.00150	-0.3%
TF-IDF vocab 확대	진희	0.438358 → 0.434032	-0.004326	-1.0%
길이·단어수 파생 피쳐	진희	0.434032 → 0.433132	-0.0009	-0.2%
LightGBM 라운드+정규화	혜현	0.4507 → 0.42909	-0.02161	-4.8%
LightGBM 라운드 확장	소현	0.42881 → 0.42388	-0.00493	-1.2%
FM_FTRL 블렌딩 추가	소현	0.42881 → 0.42114	-0.00767	-1.8%
블렌드+LGBM 재적용	소현	0.42114 → 0.41778	-0.00336	-0.8%



소현 파이프라인 단계별 개선

소현 파이프라인의 단계별 RMSLE 변화(baseline → 최종 블렌드, 낮을수록 우수). 피쳐 표현 교체와 라운드 확장이 가장 큰 개선폭을 보였다.

이 표에서 확인되는 패턴은, 피쳐 표현 방식 자체를 바꾸는 개선(hstack 구성 교체 -3.8%, LightGBM 정규화 튜닝 -4.8%)이 하이퍼파라미터 미세조정(alpha 튜닝 대부분 -0.2~2.0%, vocab 확대 -1.0% 내외)보다 개선폭이 크다는 것이다. 이는 8.5절 공통 인사이트로 다시 정리한다.

8.5 팀 전체 공통 인사이트

다섯 팀원의 실험을 종합해 Mercari 문제에서 반복적으로 확인된 공통 결론은 다음과 같다.

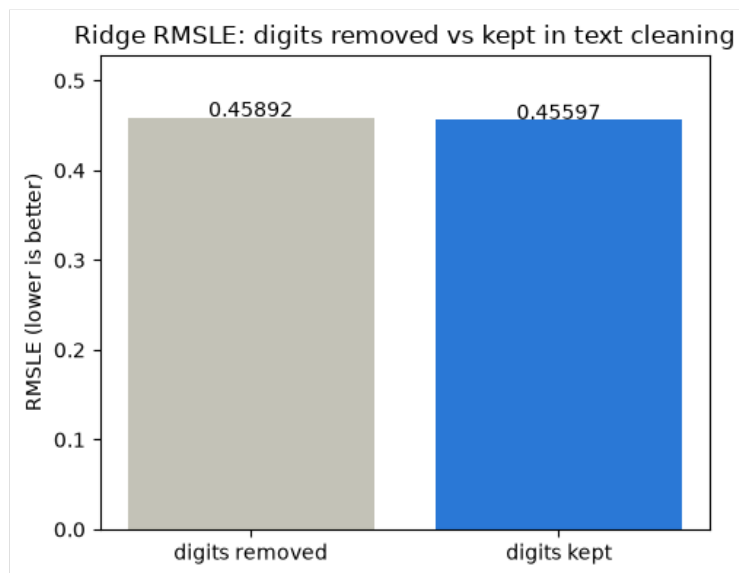
1. 고차원 희소 텍스트에서는 Ridge가 강한 베이斯拉인이다. 소현(0.47067 vs LGBM 튜닝전 0.49220), 용우(0.45447 vs LGBM 0.52045), 유경(0.4560 vs Lasso 0.6082/LGBM 0.5104) — 5명 중 4명에서 Ridge가 튜닝 전 LightGBM보다 우수했다(8.3절).
2. 가격 신호가 소수 피쳐가 아니라 다수의 희소 피쳐에 분산되어 있다. 유경의 Lasso 실험(83,486개 중 152개만 생존, 8.3절)과 소현의 vocab 실험(캡을 제거해도 성능이 거의 변하지 않음, 8.4절)이 독립적으로 이 가설을 뒷받침한다.
3. 숫자 토큰이 용량·모델명·사이즈 등 가격 정보를 담고 있다. 용우(0.45892→0.45597)와 유경(0.48787→0.48463)이 각자 독립적으로 숫자 보존이 근소하게 유리하다는 결론에 도달했다(8.4절).
4. TF-IDF vocabulary 확대의 효과는 점차 감소한다. 진희(100k→150k→200k, 개선폭 0.00257→0.00176로 축소)와 소현(vocab 캡을 아예 제거해도 0.47320→0.47293로 노이즈 수준)이 서로 다른 노트북에서 독립적으로 같은 결론에 도달했다(8.4절).
5. LightGBM은 충분한 학습 라운드와 적절한 정규화가 있어야 경쟁력이 생긴다. 소현(1200→2000라운드 확장)과 혜현(3단계 재튜닝, 4000라운드)은 충분한 예산을 투입해 Ridge를 앞질렀지만, 용우(150라운드)와 유경(기본값 미튜닝)은 그러지 못했다(8.3-8.4절).
6. 서로 다른 모델의 오차를 결합하면 블렌딩 효과가 발생하되, 결합 시점의 개별 모델 완성도가 중요하다. 소현(잘 튜닝된 LGBM+FM_FTRL, 8.2절)은 개선됐지만, 혜현(튜닝 전 Ridge+LGBM 앙상블, 9.4절)은 이후 LGBM 단일 튜닝보다 못했다(8.3절).
7. 고가 상품 구간이나 특정 카테고리에서 예측 오차가 더 커진다. 유경의 카테고리별 RMSLE 분석(Vintage & Collectibles-Electronics 등에서 RMSLE 높음, Women에서 최저 0.4232, 9.2절)과 진희의 실제-예측 산점도(고가 구간에서 예측이 낮게 편향, 9.3절)가 이를 보여준다.

9. 팀원별 실험 상세

8장이 팀 전체를 가로지르는 비교와 분석이었다면, 이 장은 각 팀원의 노트북에서 8장에 다 담기지 않은 고유한 실험 서사와 세부 구현을 정리한 부속 자료다.

9.1 용우 — 텍스트 정제 규칙과 alpha 튜닝 중심의 Ridge 파이프라인

용우는 name/description을 TF-IDF(ngram 1~2), 브랜드·카테고리를 원-핫 인코딩으로 처리해 85,529차원 피쳐를 구성했다. 핵심 실험은 텍스트 정제 시 숫자 보존 여부의 A/B 테스트로, 숫자를 보존한 버전이 RMSLE를 0.45892→0.45597로 개선시켰다(4.2절 유경과 교차검증).



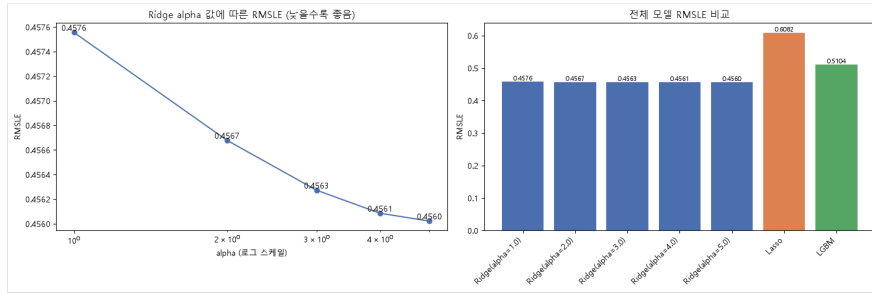
숫자 제거 vs 유지

사이즈·용량 정보를 담은 숫자를 정규식에서 지우지 않고 보존한 버전이 RMSLE를 0.00295 개선했다.

alpha 1~5 스윕으로 alpha=3에서 최적점(RMSLE 0.45447)을 찾았다. LightGBM은 150라운드 제한으로 0.52045에 그쳐 Ridge를 넘지 못했지만, 노트북에 하드코딩된 2000라운드 풀 학습 로그를 보면 0.45122까지 접근 가능했을 것으로 추정된다. 최종적으로 전체 파이프라인을 재사용 가능한 아티팩트로 저장해 새 상품 데이터에도 바로 적용할 수 있도록 설계한 것이 팀 내 유일한 실전 배포 지향 기여였다.

9.2 유경 — Ridge/Lasso/LGBM 3파전 비교와 오차 진단

유경은 브랜드-카테고리를 `LabelBinarizer`, name/description을 `CountVectorizer/TfidfVectorizer(ngram 1~3)`로 처리해 85,521차원 피처를 구성한 뒤 Ridge-Lasso-LGBM을 동일 조건에서 비교했다.



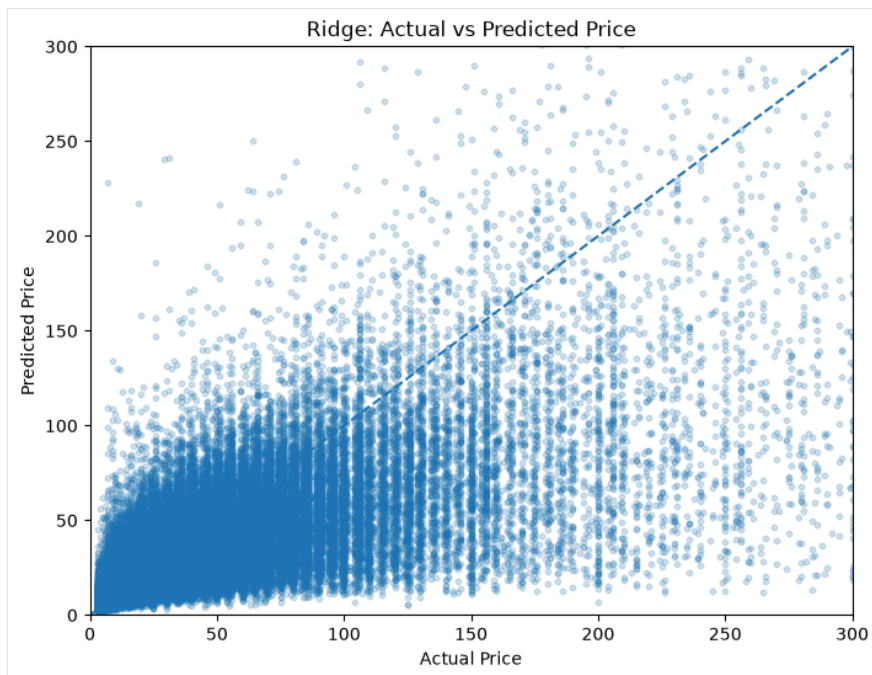
Ridge alpha 스윙 및 모델 비교

Ridge는 $\alpha=5.0$ 에서 RMSLE 0.4560으로 최저, Lasso(0.6082)는 LGBM(0.5104)보다도 크게 뒤처졌다.

Ridge는 $\alpha=5$ 에서 RMSLE 0.4560으로 최저를 기록한 반면, Lasso는 83,486개 피처 중 152개(0.2%)만 남기고 나머지를 0으로 만들어 과소적합에 빠졌다는 것을 직접 진단했다. 카테고리별 오차 분석에서는 표본이 적은 카테고리일수록 RMSLE가 높고 Women처럼 표본이 많은 카테고리는 가장 낮았다(0.4232) — 표본 크기가 예측 정확도에 미치는 영향을 팀에서 처음 정량화한 결과다.

9.3 진희 — product_text 통합과 TF-IDF 스케일 실험

진희(채택본 `ridgefixfeature(0.4331)`)는 name-brand_name-category_name을 하나의 `product_text`로 통합해 TF-IDF(ngram 1~2)를 적용했다 — 브랜드-카테고리가 겹쳐서도 상품명에서 유사 정보를 학습할 수 있다.



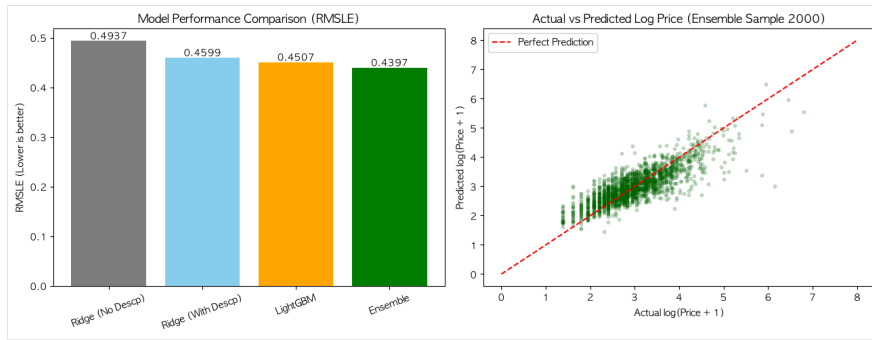
Ridge 실제값 vs 예측값

고가 구간으로 갈수록 예측이 실제보다 낮게 편향되는 경향이 관찰된다.

TF-IDF `max_features`를 100k→200k로 늘리며 RMSLE 0.438358→0.434032로 수확체감 개선을 확인했고, 길이-단어 수 파생 피처를 더해 최종 **0.43313**을 채택했다. 문자 단위 n-gram은 RMSLE 0.898로 크게 악화돼 채택하지 않았고, LGBM도 4000라운드까지 시도했으나 early stopping에 도달하지 못한 채 Ridge를 넘지 못했다 — 고차원 희소 피처에서는 정규화된 선형모델이 더 안정적이라는 결론이다.

9.4 혜현 — 단계적 튜닝으로 앙상블을 넘어선 LightGBM

혜현은 name에 `CountVectorizer`(무제한), item_description에 `TfidfVectorizer(max 50,000, ngram 1~2)`를 차등 적용하고 브랜드-카테고리는 `LabelBinarizer`로 인코딩했다. Ridge(0.4593)와 튜닝 전 LightGBM(0.4507)을 가장 평균 앙상블(0.45:0.55)해 RMSLE 0.4401을 달성한 뒤, LightGBM만 세 차례 정밀 튜닝했다.



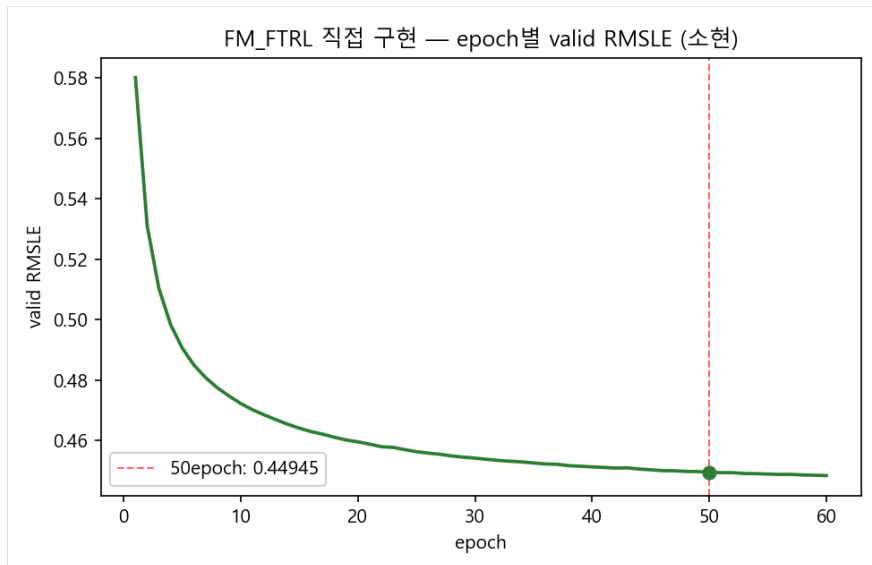
모델 RMSLE 비교 및 앙상블 산점도

3차 튜닝을 거친 LightGBM 단일 모델(0.4291)이 앞선 앙상블(0.4401)을 앞질렀다.

learning rate를 0.15→0.05→0.03으로 낮추고 num_leaves와 L1/L2 정규화를 단계적으로 강화해 RMSLE 0.4422→0.4353→**0.4291**까지 개선했다. 3차 튜닝을 거친 단일 LightGBM이 앞선 앙상블(0.4401)보다도 우수해, “덜 튜닝된 모델을 섞는 것”보다 “하나를 끝까지 튜닝하는 것”이 나올 수 있음을 보여준 사례다.

9.5 소현 — FM_FTRL 직접 구현과 leakage-free 2-stage 블렌딩

소현은 hstack 구성을 LGBM 네이티브 categorical+n-gram으로 바꿔 RMSLE를 0.49220→0.47363으로 개선한 뒤, 새 피처(5.6절)를 더했다. 핵심은 두 가지다. 첫째, 설치 불가능했던 wordbatch(FM_FTRL) 대신 FTRL-proximal+FM 상호작용항을 직접 구현했다.



FM_FTRL 직접 구현 — epoch별 valid RMSLE

60epoch 동안 valid RMSLE가 0.58017→0.44840까지 꾸준히 감소했다.

둘째, test set을 마지막 한 번만 쓰는 2단계 leakage-free 구조로 각 모델이 유리한 피처 표현을 그대로 쓰게 하고(Ridge/FM_FTRL: 원-핫+유니그램, LGBM: 라벨인코딩+n-gram) LGBM(0.42881)과 블렌딩(ridge=0, lgbm=0.65, fm=0.35)해 RMSLE 0.42114를, LGBM 2000라운드 확장 후 최종 **RMSLE 0.41778**을 달성했다 — baseline 대비 15.1% 개선이자 팀 전체 최고 성능이다.

10. 결론 및 회고

10.1 종합 결론

다섯 명이 각자 다른 전처리·피처·모델 조합으로 같은 문제를 풀었지만, 결과를 모아보면 공통된 결론에 자연스럽게 수렴했다 — 그 일곱 가지는 8.5절 (팀 전체 공통 인사이트)에서 팀원·수치·절 번호 근거와 함께 이미 정리했다.

검증 구조 설계(data leakage 방지) 자체도 하나의 성능 요소였다는 점은 8장에서 다루지 않았으므로 여기서 덧붙인다. 모든 팀원이 train에만 벡터라이저/인코더를 fit하는 원칙은 공통으로 지켰지만, 소현은 여기서 더 나아가 test set을 정말 마지막에 딱 한 번만 사용하는 2단계(train→valid 튜닝, train+valid→test 최종평가) 구조를 만들었다(9.5절). 이는 “성능이 좋아 보이는 것”과 “실제로 새 데이터에 일반화되는 것”을 구분하기 위한 장치였고, 유경이 느낀점에서 언급했듯 “좋아 보이는 것과 실제로 효과가 있는 것은 다르다”는 팀 전체의 문제의식과 맞닿아 있다.

종합하면, 이번 프로젝트에서 팀이 얻은 가장 큰 배움은 특정 알고리즘의 우열이 아니라 “문제의 데이터 구조(고차원 희소 텍스트)를 이해하고 그에 맞는 피쳐 표현과 검증 절차를 설계하는 것이, 어떤 단일 모델을 쓰느냐보다 결과에 훨씬 크게 기여한다”는 것이었다.

10.2 팀원별 느낀점 및 피드백

앞선 절이 다섯 명의 노트북을 하나의 흐름으로 종합한 것이라면, 이 절은 각 팀원이 프로젝트를 마치며 개인적으로 남긴 소감을 그대로 옮긴 것이다.

진희. 이번에 텍스트 전처리를 다양하게 시도하며 점수를 어떻게 높일지 계속 체크해왔다. 릿지나 LGBM처럼 비교적 단순한 모델을 썼기 때문에 오히려 하이퍼파라미터와 피쳐 구성 하나하나에 더 집중할 수 있었다. 그 결과 아주 작은 차이로 그친 실험도 있었고, 큰 차이로 효율을 확인한 실험도 있었다 — 예를 들어 단어 단위 대신 글자 단위(character n-gram) TF-IDF를 시도해봤지만 오히려 더 안 좋은 결과(RMSLE 0.898)를 가져왔다. 가장 가치 있었다고 생각하는 작업은 데이터 전처리였다. 전처리가 곧 학습이 잘되고 안 되고를 결정짓는다고 느꼈고, 그래서 name-brand-category를 통합한 product_text와 상품명만을 포함한 item_description을 만든 것을 가장 중요한 선택으로 꼽는다. 다만 그 성능을 실제로 최대한 끌어내는 데는 결국 max_features나 min_df 같은 모델 파라미터 조정이 함께 필요했고, 조금씩 조정할 때마다 미세하게 개선되는 과정을 반복하며 결국 이 두 축(피쳐 설계 + 파라미터 조정)에 집중할 수밖에 없었다. 팀원들과 서로의 노트북을 공유하고 설명하는 과정도 스스로를 되짚어보는 데 크게 도움이 됐다. 아쉬운 기억으로는, 글자 단위 TF-IDF를 체크하던 두 번째 파일이 저장되지 않아 유실된 일이 있었다 — 이후로는 중간 실험 결과를 더 자주 저장해두는 습관이 필요하다고 느꼈다.

용우. 데이터를 살펴보면 여러 가설을 세워 전처리하는 과정에서 생각할 거리가 많았다. 어떤 정답이 정해져 있는 문제가 아니었기에, 얻을 수 있는 정보를 최대한 끌어내며 진행했고, 그 과정에서 데이터의 형태에 따라 써야 하는 함수와 자료형이 달라진다는 것을 체감했다. 가장 크게 고민한 지점은 두 모델을 비교하며 마주한 선택이었다 — 빠르지만 점수가 낮은 모델을 쓸 것인가, 시간이 오래 걸리더라도 더 나아질 여지가 있는 모델에 시간을 투자할 것인가. 실무나 서비스 관점에서 생각해봤을 때 시간이 오래 걸리더라도 더 정확하게 예측해주는 모델을 선택하는 것이 낫겠다는 결론에 이르렀다. 이후에는 item_description 외에 상품명(name)에서도 브랜드 이름을 추출해 브랜드 사전에 추가로 매칭하는 방향을 시도해보고 있는데, 매칭되는 브랜드명이 늘어날수록 잘못된 브랜드를 잘못 가져와 오답이 늘어날 위험도 함께 커진다는 점을 유의하며 진행 중이다.

혜현. 이번 메르카리 프로젝트를 통해 비정형 텍스트 피쳐 엔지니어링과 대용량 데이터 파이프라인 최적화의 중요성을 깊이 체감했다. 상품명에는 CountVectorizer를, 본문 설명글에는 TfidfVectorizer(ngram 1~2)를 데이터 성격에 맞게 차별 적용해 문맥과 핵심 키워드를 효과적으로 피쳐화하려 했다. 특히 140만 건에 달하는 대규모 희소 데이터를 다루면서, train/test 분할 전에 벡터화를 해버리면 생길 수 있는 데이터 누수를 막기 위해 엄격한 사전 분할과 fit/transform 분리를 지켰다. 또한 모델링 전에 희소 행렬을 한 번만 결합하고 메모리를 즉시 반환하도록 설계해 연산 효율과 안정성을 높였다. 모델링 단계에서는 희소 행렬 연산에 최적화된 Ridge(solver='lsqr')로 먼저 기준 성능을 다진 뒤, LightGBM을 단계별로 정밀 튜닝해나갔다. 조기 종료와 피쳐-데이터 서브샘플링, L1/L2 복합 정규화를 순차적으로 적용하며 고차원 희소 피쳐의 과적합을 방지하고 비선형 패턴을 학습시켰고, 그 결과 3차 정밀 튜닝을 거친 LightGBM 단일 모델이 앞서 만든 Ridge+LightGBM 앙상블(0.4401)까지 넘어서는 최저 RMSLE(0.4291)를 달성했다. 이 과정에서 데이터 구조와 알고리즘 특성에 맞춘 하이퍼파라미터 최적화가 실제로 얼마나 큰 힘을 발휘하는지를 크게 실감할 수 있었다.

유경. 가장 크게 남은 배움은 “좋아 보이는 것”과 “실제로 효과가 있는 것”이 다르다는 점이었다. category_name을 몇 조각으로 나눌지, 정규식에서 숫자를 남길지 같은 사소한 보이는 선택도 직접 데이터로 비교해 보기 전에는 판단하기 어려웠다 — 실제로 카테고리 분리 방식은 결과에 거의 영향이 없었던 반면, 숫자 보존은 RMSLE를 꾸준히 약 0.7% 낮췄다. 모델 쪽에서는 화려한 모델보다 문제 구조에 맞는 모델이 중요하다는 것을 체감했다. 8만 개가 넘는 희소한 텍스트 피쳐에서는 계수를 조금씩 줄이는 Ridge가 가장 잘 맞았고, 반대로 Lasso는 규제가 세게 걸려 피쳐의 99% 이상을 0으로 만들어 버려 오히려 과소적합했다. LGBM도 튜닝 없이는 Ridge를 넘지 못했다. 지표를 다루면서 RMSLE가 “log를 씌운 상태에서의 RMSE”라는 것을 정확히 이해하는 것도 은근히 중요했다 — 타깃을 이미 log1p 했는데 여기에 또 log를 씌우면 이중 계산이 된다는 점을 정리하며, 지표 정의를 대충 넘기면 안 된다는 것을 배웠다. 아쉬운 점은 데이터가 커서 전체로 교차검증을 제대로 돌리지 못한 것, 그리고 텍스트 임베딩이나 모델 앙상블까지 시도하지 못한 것이다. 다만 잔차 분포와 카테고리별 오차를 시각화해 보니 어느 구간에서 모델이 약한지는 감을 잡을 수 있었다.

소현. 전처리를 마친 뒤 곧바로 하이퍼파라미터 튜닝에 매달리기보다, 좋은 feature vector를 만드는 데 집중하는 방향을 처음부터 의도적으로 택했다. “Garbage in, garbage out” — 아무리 튜닝을 정교하게 해도 feature 자체가 충분한 예측력을 갖지 못하면 한계가 뚜렷하다고 봤기 때문이다. 실제로 LGBM이 원본 sparse 표현(0.49220)에서 native categorical + n-gram(0.47363)으로 바꾼 것만으로 크게 개선된 것을 보고, 모델 하나를 정해 하이퍼파라미터를 극한까지 파는 대신 좋은 x_features를 만든 뒤 Ridge와 LGBM의 성능을 함께 확인하는 절차로 전략을 다시 짰다. Ridge는 선형모델이라 apple-iphone-128gb 같은 조건들을 단순히 더하는 역할만 할 수 있지만, 실제 가격은 “Apple×iPhone”처럼 조합에 따라 달라지므로 이 조합 효과를 잡으려면 LightGBM(트리 분기) 쪽이 유리하다는 것도 실험으로 확인했다. FM_FTRL을 캐글 공개 커널(Wordbatch, Anttip)에서 아이디어를 얻어 직접 구현하고, validation set에서 최적 불렌드 비율을 탐색해 최종 LGBM RMSLE 0.41778을 달성한 과정 전체가, 데이터 구조를 파악하는 데서 시작해 그에 맞는 feature 설계를 고민하는 것이 모델 파라미터 하나하나를 조정하는 것보다 근본적으로 중요하다는 확신으로 이어졌다.

결론

성과

4개 데이터셋을 거치며 머신러닝의 전체 흐름뿐 아니라, 데이터 전처리와 모델 학습 성능 개선 방법을 경험했다. 또한 결과 지표를 근거로 문제점을 다시 찾아 전처리·모델링을 개선해 나가는 방법과 이 과정을 실제로 어디에 활용할 수 있는지까지 살펴보았다. 다만 이렇게 얻은 방법이 '모든 데이터에 통하는 보편적 정답'은 아니며, 어떤 데이터냐에 따라 충분히 달라질 수 있다는 점도 함께 알게 됐다.

과정

많은 시간을 들인 만큼 많이 헤매기도 했다. 잘 정리된 자료를 참고하며 무엇을 배우고 적용할지 고민했고, 어떤 부분은 팀 전체가 통일하고 어떤 부분은 각자 차별점을 둘지 계속 논의했다. 결과가 좋으면 그 원인을 분석했고, 결과가 좋지 않으면 개선점을 다시 찾았다.

협업의 의미

개인 프로젝트였다면 한 사람의 판단대로 하나의 흐름만 남았을 것이다. 그러나 5명이 함께 고민하고 토의하며, 때로는 의견을 부딪히면서 만들어갔다. 그 과정에서 하나의 문제에도 여러 접근 방법이 있다는 것을 확인했고, 서로 다른 흐름을 비교하며 그 차이를 구분할 수 있었다. 혼자였다면 놓쳤을 지점을 서로 짚어가며 우리의 선택을 다시 검토했다. 우리가 찾던 것은 '정답'이 아니라 '최선의 답'이었다.

마무리

머신러닝에서 데이터가 모델의 경험이 되고 모델은 그 경험으로 새로운 데이터에 답하듯, 이번 프로젝트에서 겪은 시행착오와 고민도 우리에게 하나의 '데이터'가 됐다. 그래서 이번에 배운 것은 정해진 방법 하나를 적용하는 일이 아니라, 데이터와 결과를 근거로 끊임없이 판단하고 수정하며 더 나은 답을 찾아가는 과정 그 자체였다. 앞으로 비슷한 문제를 마주했을 때 이번 경험을 바탕으로 이전보다 더 나은 답을 찾아갈 수 있을 것이다.

후기

좋았던 점

지금까지 진행한 프로젝트는 대부분 개인 작업이었다. 하지만 이번 팀 프로젝트를 통해 협업과 분업의 중요성을 배울 수 있었고, 같은 데이터를 바라보는 서로 다른 시각을 접하고 공유할 수 있었다.

작업하는 과정에서 의견이 달라 부딪히는 순간도 있었지만, 오히려 그 차이 속에서 미처 생각하지 못했던 새로운 답을 찾기도 했다. 이러한 의견 차이는 단순한 갈등이라기보다 더 나은 방향으로 나아가게 해주는 촉진제가 되었다.

모든 팀원의 의욕이 높아 예정된 시간을 넘겨 작업하는 경우도 있었다. 그만큼 데이터와 결과에 대한 이야기를 많이 나눌 수 있었고, 함께 답을 찾아가는 과정 자체가 의미 있었다. 이 경험을 생각하면 앞으로의 팀 작업이 더 기대된다.

무엇보다 역할 분담이 확실했고, 각자 맡고 싶은 부분에 대해서도 충분한 소통이 이루어졌다. 덕분에 역할에 대한 큰 불만 없이 프로젝트를 재미있게 진행할 수 있었다.

아쉬웠던 점

통일성 면에서는 조금 아쉬움이 남는다. 우리 프로젝트의 컨셉 자체가 '수많은 시도'였던 만큼 다양한 접근을 보여줄 수 있다는 장점이 있었지만, 그만큼 보고서 전체의 흐름이 분산되는 부분도 있었다.

회의 과정에서도 아쉬움이 있었다. 일부 팀원의 의견이 많이 반영되면서, 충분히 말할 기회를 갖지 못한 팀원도 있었다고 생각한다. 모든 팀원의 의견을 조금 더 균형 있게 들었다면 더 다양한 이야기가 나올 수 있었을 것이다.

한 명씩 데이터셋을 맡아 진행하는 방식은 역할을 명확하게 나눌 수 있다는 점에서 좋았지만, 본인이 담당하지 않은 데이터셋에 대한 이해도는 상대적으로 낮을 수밖에 없었다. 그럼에도 서로의 진행 상황을 공유하고 적극적으로 참여하면서 이러한 한계를 최대한 보완하려고 했다.

또한 첫 번째 데이터셋을 담당할 팀원이 상대적으로 많은 시행착오를 겪었다는 점도 아쉽다. 뒤에 진행한 팀원들은 앞선 작업에서 얻은 시행착오와 피드백을 참고할 수 있었지만, 첫 번째 담당자는 참고할 선례가 없는 상태에서 시작해야 했다. 프로젝트 초반의 역할과 부담을 조금 더 배려하고 함께 나눴으면 좋았을 것이다.

앞으로 하고 싶은 것

다음 팀 프로젝트에서 가져가고 싶은 것은 두 가지다.

'재미'와 '구조'

물론 모든 과정이 재미있을 수는 없다. 결과가 나오지 않아 답답할 때도 있고, 같은 작업을 여러 번 반복해야 할 때도 있을 것이다. 하지만 과정이 힘들더라도 마지막에 보람 있는 성과로 이어진다면, 그 안에서 느끼는 재미는 자연스럽게 따라온다고 생각한다. 다음 프로젝트에서도 단순히 결과만 내는 것이 아니라, 함께 고민하고 답을 찾아가는 과정의 재미를 계속 가져가고 싶다.

두 번째는 구조다. 이번 프로젝트에서는 많은 시도를 하다 보니 가끔 전체적인 흐름이 중구난방해지는 경우가 있었다. 다음에는 처음부터 '가설 → 실험 목적 → 실험 → 결과 → 피드백'의 흐름을 조금 더 체계적으로 정리하고 싶다. 아무리 많은 시도를 하더라도 각각의 실험이 왜 필요한지, 무엇을 확인하기 위한 것인지 명확하게 기록해 방향을 잃지 않도록 할 것이다.

이번 프로젝트에서 얻은 '다양한 시도'라는 장점은 유지하되, 다음에는 그 시도들을 하나의 흐름으로 연결해보고 싶다.